DETECCIÓN DE FALLAS DE CORTOCIRCUITO EN DEVANADOS DE MOTORES DE INDUCCIÓN MEDIANTE ANÁLISIS DE FIRMA DE CORRIENTE (MCSA) E INTELIGENCIA ARTIFICIAL

TESIS PROFESIONAL COMO REQUISITO PARCIAL PARA OBTENER EL TÍTULO DE LICENCIADO EN INGENIERÍA MECATRÓNICA

PRESENTA:

Juan Emmanuel Luna Velázquez

DIRECTOR

José Enrique Hernández Díez

CODIRECTORES

Ernesto Bárcenas Bárcenas
Diego Rivelino Espinoza Trejo

Matehuala, S.L.P., octubre de 2026

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ
UNIDAD ACADÉMICA MULTIDISCIPLINARIA REGIÓN ALTIPLANO
INGENIERÍA MECATRÓNICA

COORDINACIÓN ACADÉMICA
REGIÓN ALTIPLANO

DETECCIÓN DE FALLAS DE CORTOCIRCUITO EN DEVANADOS DE MOTORES DE INDUCCIÓN MEDIANTE ANÁLISIS DE FIRMA DE CORRIENTE (MCSA) E INTELIGENCIA ARTIFICIAL

TESIS PROFESIONAL COMO REQUISITO PARCIAL PARA OBTENER EL TÍTULO DE LICENCIADO EN INGENIERÍA MECATRÓNICA

PRESENTA:
Juan Emmanuel Luna Velázquez

DIRECTOR
José Enrique Hernández Díez

CODIRECTORES
Ernesto Bárcenas Bárcenas
Diego Rivelino Espinoza Trejo

Matehuala, S.L.P., octubre de 2026

AGRADECIMIENTOS

Primeramente, y antes que nada, quiero agradecer a Dios por haberme permitido llegar a este punto de mi carrera y por darme la fuerza y la sabiduría durante todo este camino.

A mi director de tesis, el Dr. José Enrique Hernández Díez, y a mis codirectores, el Dr. Ernesto Bárcenas Bárcenas y el Dr. Diego Rivelino Espinoza Trejo, por su gran ayuda en todo este proceso: por su dirección, su asesoría, su acompañamiento y su apoyo.

A la Universidad Autónoma de San Luis Potosí, por prestar el equipo necesario para este trabajo.

A mis padres, Juliana y Manuel, por haber creído siempre en mí y no dejar de apoyarme; en especial a mi madre, por ser el pilar más importante de mi vida.

A mis hermanas, Ángeles y Brenda, por su apoyo y sus consejos durante toda mi carrera universitaria.

A mis sobrinos, Alejandro, Daena y Liam, por su cariño y por creer en mí.

A mi sobrina, Andrea, por ser mi compañía a lo largo de este camino.

A mi novia, Quetzali, por su amor, su paciencia y su apoyo incondicional, y por acompañarme en cada etapa de este proceso.

Y me agradezco a mí mismo por haber llegado hasta este punto.

RESUMEN

Este proyecto surge ante la necesidad de detectar de forma temprana y en línea los cortocircuitos entre espiras en el estator de los motores de inducción trifásicos, una falla que en su etapa inicial apenas altera la operación del motor, pero que evoluciona hacia daños severos. El objetivo general consistió en desarrollar un sistema de diagnóstico predictivo basado en el análisis de firma de corriente (MCSA), las componentes simétricas, el procesamiento digital de señales y modelos de inteligencia artificial. La metodología comprendió la construcción de un banco de pruebas con un motor ABB de 1.5 kW, un generador como carga e inyección controlada de fallas en la fase C con tres severidades, de aproximadamente 7.0, 14.3 y 21.6 % de las espiras; un firmware para el ESP32-S3 que adquiere seis canales y decide la alarma con una regla física sobre la corriente de secuencia negativa, y clasifica la severidad con umbrales aprendidos; una aplicación de escritorio con un asistente de mantenimiento local basado en recuperación de documentos (RAG), y validaciones con protocolos declarados antes de obtener los datos. Como principales resultados, la validación ciega en vivo detectó 10/10 fallas SEVERA y 10/10 INTERMEDIA a 75 % de carga y 59/60 fallas a 0 y 25 %, con una confirmación de unos 5 s; el clasificador de severidad acertó en 596 de 597 muestras dejando una carga fuera, y los modelos de aprendizaje automático evaluados para la detección no generalizaron entre cargas. La tasa de falsas alarmas en operación continua fue de 4.1 a 7.3 por hora, frente al criterio de una por hora. Se concluye que la detección con hardware de bajo costo es viable cuando se apoya en un indicador con fundamento físico y cada modelo se valida en las condiciones en que va a operar, y que reducir las falsas alarmas es el principal pendiente para una operación desatendida.

ABSTRACT

This project arises from the need for early, online detection of inter-turn short circuits in the stator of three-phase induction motors, a fault that barely affects motor operation at its onset but evolves into severe damage. The overall objective was to develop a predictive diagnostic system based on motor current signature analysis (MCSA), symmetrical components, digital signal processing and artificial intelligence models. The methodology comprised the construction of a test bench with a 1.5 kW ABB motor, a generator acting as load and controlled fault injection in phase C at three severity levels, of approximately 7.0, 14.3 and 21.6 % of the turns; ESP32-S3 firmware that acquires six channels, raises the alarm with a physical rule on the negative-sequence current and classifies the severity with learned thresholds; a desktop application with a local maintenance assistant based on retrieval-augmented generation (RAG); and validations under protocols declared before the data were collected. As main results, the live blind validation detected 10/10 severe and 10/10 intermediate faults at 75 % load and 59/60 faults at 0 and 25 % load, with a confirmation time of about 5 s; the severity classifier was correct in 596 of 597 samples with leave-one-load-out validation, and the machine learning models evaluated for detection did not generalize across loads. The false alarm rate in continuous operation was 4.1 to 7.3 per hour, against a criterion of one per hour. It is concluded that detection with low-cost hardware is feasible when it relies on a physically grounded indicator and each model is validated under the conditions in which it will operate, and that reducing false alarms is the main remaining task for unattended operation.

ÍNDICE

AGRADECIMIENTOS I
RESUMEN II
ÍNDICE III
ÍNDICE DE FIGURAS VI
ÍNDICE DE TABLAS X

CAPÍTULO 1. INTRODUCCIÓN 1
1.1 ANTECEDENTES 1
1.2 DELIMITACIÓN DEL PROBLEMA 2
1.3 JUSTIFICACIÓN 3
1.4 HIPÓTESIS 4
1.5 OBJETIVOS 4
1.6 LÍMITES Y ALCANCES 5

CAPÍTULO 2. MARCO TEÓRICO 7
2.1 EL MOTOR DE INDUCCIÓN TRIFÁSICO 7
2.2 FALLAS EN MOTORES DE INDUCCIÓN 8
2.2.1 FALLAS ENTRE ESPIRAS 8
2.3 ANÁLISIS DE FIRMA DE CORRIENTE (MCSA) 9
2.4 TRANSFORMADA RÁPIDA DE FOURIER Y ESTIMACIÓN ESPECTRAL 9
2.5 COMPONENTES SIMÉTRICAS DE FORTESCUE 10
2.6 DETECCIÓN SECUENCIAL DE CAMBIOS 11
2.7 INTELIGENCIA ARTIFICIAL Y APRENDIZAJE AUTOMÁTICO 12
2.7.1 RANDOM FOREST 12
2.7.2 XGBOOST 12
2.7.3 VALIDACIÓN Y CALIBRACIÓN DE MODELOS 12
2.8 MODELOS DE LENGUAJE Y GENERACIÓN AUMENTADA POR RECUPERACIÓN 13
2.8.1 MODELOS DE LENGUAJE 13
2.8.2 GENERACIÓN AUMENTADA POR RECUPERACIÓN (RAG) 13
2.9 SENSADO DE CORRIENTE Y VOLTAJE 14
2.10 SISTEMAS EMBEBIDOS: ESP32-S3 14
2.11 ESTADO DEL ARTE 15

CAPÍTULO 3. METODOLOGÍA 16
3.1 MATERIALES Y MÉTODOS 16
3.1.1 RECURSOS Y FACTIBILIDAD 16
3.1.2 COMPONENTES ELECTRÓNICOS 18
3.1.3 MATERIALES Y EQUIPO DEL BANCO 19
3.1.4 SOFTWARE Y ENTORNO DE DESARROLLO 22

3.1.5 EQUIPOS DE LABORATORIO Y HERRAMIENTAS 23
3.2 DESCRIPCIÓN DE LA APLICACIÓN DE LA METODOLOGÍA PROPUESTA 23
3.2.1 DISEÑO POR BLOQUES 23
3.2.2 TÉCNICAS DE INVESTIGACIÓN Y VALIDACIÓN 24
3.2.3 PASOS PARA LOGRAR LA APLICACIÓN 25
3.3 DESARROLLO DE LA METODOLOGÍA 26
3.3.1 BANCO DE PRUEBAS 26
3.3.1.1 MOTOR, CARGA Y NIVELES DE OPERACIÓN 26
3.3.1.2 CARACTERIZACIÓN DEL DEVANADO Y DEFINICIÓN DE LAS CLASES DE FALLA 27
3.3.1.3 SISTEMA DE INYECCIÓN DE FALLAS 28
3.3.1.4 ACONDICIONAMIENTO Y CALIBRACIÓN DE SENSORES 29
3.3.1.5 CONEXIONADO E INTEGRIDAD DE LA SEÑAL 30
3.3.2 ADQUISICIÓN Y PROCESAMIENTO DE SEÑALES 33
3.3.2.1 MUESTREO 33
3.3.2.2 CARACTERÍSTICAS MCSA 33
3.3.2.3 COMPONENTES SIMÉTRICAS Y FIRMA DE SECUENCIA NEGATIVA 34
3.3.2.4 CONJUNTOS DE DATOS 34
3.3.3 MODELOS DE INTELIGENCIA ARTIFICIAL 35
3.3.3.1 ETAPA EXPLORATORIA 35
3.3.3.2 CLASIFICADOR DE SEVERIDAD 35
3.3.3.3 ESTUDIOS DE COMPARACIÓN PARA LA DETECCIÓN 35
3.3.4 FIRMWARE ESP32-S3 36
3.3.5 APLICACIÓN DE ESCRITORIO MOTORGUARD-ITF 40
3.3.5.1 MOTIVACIÓN Y REQUISITOS 40
3.3.5.2 ARQUITECTURA 41
3.3.5.3 SECCIONES DE LA INTERFAZ 41
3.3.5.4 SEGURIDAD DE LA INYECCIÓN 44
3.3.5.5 ESTABILIDAD Y VERIFICACIÓN 47
3.3.5.6 EMPAQUETADO 47
3.3.5.7 RECORRIDO DE OPERACIÓN 48
3.3.6 ASISTENTE DE MANTENIMIENTO CON RECUPERACIÓN DE DOCUMENTOS (RAG) 58
3.3.6.1 OBJETIVO Y ALCANCE 58
3.3.6.2 FUENTES DOCUMENTALES 58
3.3.6.3 INGESTA Y RECUPERACIÓN 59
3.3.6.4 MODELO DE LENGUAJE LOCAL 59
3.3.6.5 CONSULTA AUTOMÁTICA AL ALARMAR 61
3.3.6.6 MODELO EXTERNO OPCIONAL 61
3.3.6.7 PROTOCOLO DE EVALUACIÓN 61
3.3.6.8 RECORRIDO DEL ASISTENTE 62
3.3.7 INTEGRACIÓN DEL SISTEMA 68

CAPÍTULO 4. RESULTADOS 70
4.1 ANÁLISIS DE RESULTADOS 70
4.1.1 CALIBRACIÓN Y CARACTERIZACIÓN DE SENSORES 70
4.1.2 CARACTERIZACIÓN DEL DEVANADO Y DEL LAZO DE INYECCIÓN 70
4.1.3 ANTECEDENTES: ETAPA EXPLORATORIA (JULIO A PRINCIPIOS DE OCTUBRE DE 2026) 71
4.1.3.1 CLASIFICADORES SOBRE CARACTERÍSTICAS MCSA 71
4.1.3.2 ENSAYOS DE DETECCIÓN DE SEPTIEMBRE 73
4.1.3.3 RUIDO POR CONEXIONES Y PRIMERA FIRMA DE LA FALLA (1 DE OCTUBRE) 77
4.1.4 FIRMA DE LA FALLA POR CLASE Y CARGA 79
4.1.5 CLASIFICADOR DE SEVERIDAD 81
4.1.6 DETECTOR DIRECCIONAL EMBEBIDO 82
4.1.7 VALIDACIÓN CIEGA EN VIVO 82
4.1.8 PRUEBA LARGA SIN FALLA 84
4.1.9 COMPARACIÓN CON APRENDIZAJE AUTOMÁTICO PARA LA DETECCIÓN 85
4.1.9.1 ESTUDIO 1: REGLA FRENTE A MODELOS EN OPERACIÓN CONTINUA 85
4.1.9.2 ESTUDIO 2: GENERALIZACIÓN ENTRE CARGAS 87
4.1.9.3 VALIDACIÓN CIEGA 2: MODELO CONGELADO FRENTE A LA REGLA CON DATOS NUEVOS 87
4.1.10 RUIDO DE LA FIRMA Y VARIANTES DEL DETECTOR 89
4.1.11 APLICACIÓN DE ESCRITORIO Y ASISTENTE RAG 91
4.1.12 CUMPLIMIENTO DE OBJETIVOS E IMPACTO 95
4.2 DISCUSIÓN DE RESULTADOS 97

CAPÍTULO 5. CONCLUSIONES 99
5.1 CUMPLIMIENTO DE LOS OBJETIVOS Y DE LA HIPÓTESIS 99
5.2 EL PAPEL DE LA INTELIGENCIA ARTIFICIAL 100
5.3 LIMITACIONES 101
5.4 TRABAJO FUTURO 102

REFERENCIAS 104

ANEXOS 108
ANEXO A. FIRMWARE DEL ESP32-S3 108
ANEXO B. ADQUISICIÓN Y PROTOCOLO DE ENSAYO 114
ANEXO C. MODELOS DE INTELIGENCIA ARTIFICIAL Y CLASIFICADOR DE SEVERIDAD 119
ANEXO D. ANÁLISIS DE LAS SESIONES DE OCTUBRE 123
ANEXO E. APLICACIÓN DE ESCRITORIO Y ASISTENTE RAG 125

ÍNDICE DE FIGURAS

Figura 1. Representación de las componentes simétricas de un sistema trifásico: secuencias positiva, negativa y cero. **10**

Figura 2. Banco de pruebas en el Laboratorio de Mecatrónica con sus elementos principales señalados: motor y generador acoplados, resistencias de carga, variac, relevador de estado sólido, instrumentación y computadora de supervisión. **17**

Figura 3. Relevador de estado sólido Fotek SSR-25DA del lazo de inyección de falla, montado sobre su base. **18**

Figura 4. Placa de datos del motor ABB M1AA 145T-D4. ..... **19**

Figura 5. Motor ABB con las derivaciones del devanado accesibles en la caja de bornes. **20**

Figura 6. Placa de datos del generador Baldor CDP3585 usado como carga. **20**

Figura 7. Resistencias de potencia del banco de carga conectadas a la salida del generador. **21**

Figura 8. Banco de pruebas: motor ABB M1AA 145T-D4 acoplado al generador Baldor CDP3585, instrumentación sobre protoboard y computadora de adquisición. **22**

Figura 9. Entorno de desarrollo en Ubuntu 26.04: Antigravity CLI con Gemini 3.8 Flash (izquierda) y Claude Code con Claude Opus 5.5 (derecha). Se ocultó el correo electrónico de la cuenta. **23**

Figura 10. Diagrama de bloques del sistema final: banco de pruebas, firmware con detección y severidad, y aplicación de escritorio con el asistente de mantenimiento. **24**

Figura 11. Circuito de potencia del banco de pruebas: red, variac, sensores de corriente en serie y de voltaje entre fase y neutro, motor en doble estrella y generador de corriente directa con su resistencia de carga. **27**

Figura 12. Derivaciones de la fase C y lazo de inyección de falla: fusible, resistencia limitadora y lado de carga del relevador entre las dos derivaciones de la clase seleccionada. La corriente del lazo es la medida con la pinza durante la captura del conjunto de datos. **28**

Figura 13. Circuito de activación del relevador de estado sólido: el pin D5 (GPIO 8) satura el transistor 2N2222, que cierra el lado de control del SSR-25DA alimentado desde VBUS. **29**

Figura 14. Acondicionamiento de las señales: (a) canal de corriente con el ACS712-30A y el divisor resistivo; (b) canal de voltaje con el ZMPT101B. A la derecha, la asignación de los seis canales a los pines, GPIO y convertidores del ESP32-S3. ..... **30**

Figura 15. Conexiones del Arduino Nano ESP32: entradas de los sensores, alimentación desde VBUS y 3V3, salida al circuito del relevador, bus SPI de la pantalla TFT y enlace USB con la aplicación. **31**

Figura 16. Montaje de la instrumentación sobre protoboard: sensores ZMPT101B y ACS712, Arduino Nano ESP32 y relevador de estado sólido. **32**

Figura 17. Protoboard con los tres módulos ZMPT101B de voltaje y el Arduino Nano ESP32. ..... **32**

Figura 18. Comparación de forma de onda y espectro de la corriente de la fase C entre motor sano y con falla LEVE (sistema de un canal, julio de 2026). **33**

Figura 19. Procesamiento de un bloque en el firmware 299a943: muestreo, fasores, componentes simétricas, firma de secuencia negativa y detector direccional con severidad. **37**

Figura 20. Estados del motor y del detector direccional en el firmware 299a943: arranque, calentamiento, vigilancia, alarma enclavada con su severidad y liberación. **38**

Figura 21. Pantalla TFT del sistema en el banco: (a) motor apagado (OFF); (b) calentamiento tras el arranque; (c) estado sano sin inyección (Conf 0 %); (d) durante una inyección, con el primer bloque sobre el umbral (Conf 33 %); (e) alarma LEVE confirmada (Conf 100 %). **40**

Figura 22. Sección Monitoreo de la aplicación con el motor sano, durante una sesión con el banco del 16 de octubre de 2026. **42**

Figura 23. Plano de la firma en la sección Monitoreo: firma z, referencia, cambio Δz, dirección de falla de −81.4° con su umbral y anillos de severidad. **42**

Figura 24. Sección Dashboard con las consultas de lectura del firmware: osciloscopio de las corrientes crudas y espectro de la fase vigilada. **43**

Figura 25. Sección Adquisición: captura de pares SANO/FALLA con los parámetros del protocolo de la tesis. **44**

Figura 26. Diálogo de confirmación previo a una serie de inyección, con las casillas obligatorias sobre la seguridad del lazo y las derivaciones conectadas (modo de demostración). **45**

Figura 27. Secuencia de un pulso de inyección: bloqueos y confirmación en la aplicación, verificaciones y protecciones del firmware, y respuesta del banco y del detector. **46**

Figura 28. Aviso de instancia única: solo una ventana de la aplicación puede usar el microcontrolador; las reproducciones de registros sí pueden abrirse varias. **47**

Figura 29. Ventana «Acerca de» de la aplicación: versión 1.0.0, firmware compatible 299a943 y licencias de los componentes de terceros. **48**

Figura 30. Sección Monitoreo con el motor sano, en modo de demostración: tramas reales del banco (sesión INTER a 75 % de carga, 9 de octubre de 2026) reproducidas por un simulador del firmware; los comandos no llegan a ningún hardware. **49**

Figura 31. Sección Dashboard con todos los campos de la trama, las gráficas de corrientes y de desbalance de secuencia negativa y la tarjeta de consultas a petición (modo de demostración). **50**

Figura 32. Gráficas del Dashboard después de pulsar dos veces el botón de acercamiento: unos 35 s de historial, con los bloques individuales de 1.70 s (modo de demostración). **50**

Figura 33. Consulta del osciloscopio en el Dashboard: 2048 muestras por fase de las corrientes crudas, sin calibrar (volcado real del 6 de octubre de 2026 reproducido en el modo de demostración). **51**

Figura 34. Sección Monitoreo 1.5 s después de aceptar un pulso: relevador encendido y detector todavía en SANO. «F1(LEVE)» es el nombre que el firmware da a su salida del relevador, no la clase de la falla (modo de demostración). **52**

Figura 35. Panel de inyección: (a) durante el pulso, con «Inyectar» bloqueado porque el firmware reporta el relevador encendido; (b) después de dos pulsos, bloqueado por el enfriamiento, con la tabla de pulsos (modo de demostración). **52**

Figura 36. Alarma INTER confirmada y enclavada en la sección Monitoreo: proyección P sobre el umbral, bloques con alarma en naranja y botón «Liberar alarma» (modo de demostración). **53**

Figura 37. Plano de la firma durante la alarma: estela de z, referencia, Δz, dirección de falla, base congelada, anillo de severidad, fasores de secuencia y comparación de la P del firmware con la recalculada por la aplicación (modo de demostración). **54**

Figura 38. Tabla de eventos de alarma con dos alarmas INTER liberadas por el firmware (modo de demostración). **54**

Figura 39. Alarma INTER mantenida enclavada para mostrar el botón «Liberar alarma»; P y ML ya son bajos porque la demostración repite la última trama con alarma (modo de demostración). **55**

Figura 40. Diálogo «Liberar alarma»: reinicio del detector con la advertencia de que una falla real no volvería a detectarse y la casilla de confirmación obligatoria (modo de demostración). **55**
Figura 41. Sección Monitoreo después de liberar la alarma: el detector vuelve a SANO (modo de demostración). **56**
Figura 42. Sección Adquisición durante una captura de pares SANO/FALLA: parámetros, avance y consola con los mensajes del protocolo de la tesis (modo de demostración). **57**
Figura 43. Sección Adquisición con la captura terminada: total de pares, descartes y ruta del archivo generado (modo de demostración). **57**
Figura 44. Diálogo de ajustes: carpeta de registros y registro interno de la aplicación (modo de demostración). **58**
Figura 45. Sección Asistente en tema oscuro: modelo local preparado, fuentes disponibles y conversación. **60**
Figura 46. Configuración del modelo externo por API: proveedor Anthropic, protocolo nativo, clave guardada en el llavero sin mostrarse y aviso de los datos que salen de la computadora. **61**
Figura 47. Sección Asistente lista con el modelo local: aviso de apoyo, tarjeta de la alarma sin alarma, tarjeta del modelo y preguntas sugeridas (asistente real; modo de demostración). **62**
Figura 48. Respuesta del modelo local a una pregunta libre: (a) mientras escribe; (b) terminada, con sus citas y la lista de fuentes. Dos de los puntos aplican métodos de caracterización del banco como pruebas de confirmación (asistente real; modo de demostración; tiempos ilustrativos). **63**
Figura 49. Tarjeta del fragmento citado: texto original del documento técnico del banco y botón para abrir el documento en esa página (asistente real; modo de demostración). **64**
Figura 50. Recomendación del modelo local ante una alarma INTER: (a) mientras escribe; (b) terminada, con la seguridad primero y las citas a las secciones de falsas alarmas y limitaciones de la tesis (asistente real; alarma del modo de demostración; tiempos ilustrativos). **65**
Figura 51. Tarjeta del modelo con el modelo externo activo (Claude Sonnet 5.5) y el selector del esfuerzo de razonamiento (asistente real; modo de demostración). **66**
Figura 52. Respuesta del modelo externo a la misma pregunta libre: (a) mientras escribe; (b) terminada, con una precaución marcada «sin fuente» (asistente real; modo de demostración; tiempos ilustrativos). **67**
Figura 53. Recomendación del modelo externo ante la alarma INTER: precauciones, confiabilidad de la alarma, pruebas sugeridas y comparación de impulsos, todas con cita (asistente real; alarma del modo de demostración; tiempo ilustrativo). **68**
Figura 54. Matriz de confusión del primer modelo XGBoost con datos reales (SANO/LEVE, carga baja, julio de 2026). **72**
Figura 55. Evaluación del Random Forest binario con validación cruzada agrupada en la campaña del 11 al 15 de septiembre: curva ROC, curva de precisión y sensibilidad, diagrama de fiabilidad y distribución de probabilidades calibradas. **73**
Figura 56. Resultado de cada corrida de inyección de la falla LEVE de septiembre: detecciones nuevas y ciclos que iniciaron con la alarma ya activa. A la derecha, detecciones nuevas sobre ciclos que iniciaron en estado sano. **75**
Figura 57. Estadístico CUSUM en dos corridas consecutivas de septiembre a 75 % de carga. Las franjas indican los pulsos de inyección. **76**
Figura 58. Ensayo de puente manual directo del 29 de septiembre: residuo de secuencia negativa (arriba) y estadístico CUSUM (abajo). Las franjas indican los intervalos en que el detector indicó falla LEVE. **77**

Figura 59. Proyección del residuo de secuencia negativa durante diez minutos de base sana, con las conexiones de la instrumentación degradadas (arriba) y rehechas (abajo), 1 de octubre de 2026. ..... **78**

Figura 60. Cambio del residuo de secuencia negativa producido por cada uno de diez pulsos de falla LEVE (puntos) y su valor medio (flecha), 1 de octubre de 2026, firmware anterior a la recalibración. **79**

Figura 61. Estadístico del detector direccional de escalón evaluado fuera de línea sobre la sesión de diez pulsos LEVE del 1 de octubre. Las franjas indican los pulsos de inyección y la barra inferior el estado de alarma. **79**

Figura 62. Cambio de la firma de secuencia negativa Δz de cada par en el plano complejo, por clase y carga. La línea discontinua indica la dirección de falla de −81.4° y la punteada el umbral de detección P = 0.40 pp. **81**

Figura 63. Compromiso entre falsas alarmas por hora y detección de la falla LEVE en el estudio 1, para la regla de Fortescue y el Random Forest. Las estrellas indican los puntos de operación declarados. **86**

Figura 64. Cambio relativo de |I1| durante el bloque con falla, por clase y carga (mediana y rango intercuartil). En vacío, la falla LEVE prácticamente no modifica |I1|. **86**

Figura 65. Detección de la falla LEVE por carga, con intervalos de confianza de Wilson: (a) pares de bloques del estudio 2; (b) registro continuo en vivo, regla del firmware frente al Random Forest congelado entrenado a 75 %. **88**

Figura 66. Falsas alarmas por hora de operación sana, con intervalos de confianza de 95 %, en el estudio 1 y en la validación ciega 2. La línea punteada indica el criterio de una falsa alarma por hora. **89**

Figura 67. Sección Monitoreo de la aplicación durante la sesión del 16 de octubre de 2026: alarma LEVE enclavada desde las 14:13:34 (arriba, a los 78 s) que escaló a INTERMEDIA con los pulsos siguientes (abajo, a los 153 s). **92**

Figura 68. Proyección de la firma en la sesión del 16 de octubre a las 19:28: tres pulsos con derivaciones INTERMEDIA, clasificados SEVERA (4.61–4.75 pp), seguidos de tres pulsos con derivaciones SEVERA (9.46–9.88 pp). **92**

Figura 69. Respuesta del asistente local (Qwen3.5-4B) a una pregunta sobre la construcción de la firma, no cubierta por los documentos: cada punto se marca «sin fuente» y se listan los fragmentos consultados. **94**

Figura 70. Sección Asistente durante la reproducción de un registro real del banco (sesión INTER al 75 % del 9 de octubre de 2026). El asistente local (Qwen3.5-4B) respondió de forma automática a dos alarmas INTER confirmadas por el firmware (2.25 y 2.53 pp), con el 100 % de las líneas citadas. Las horas de las tarjetas son las de la reproducción; el panel de alarma ya muestra la alarma liberada. **95**

ÍNDICE DE TABLAS

Tabla 1. Datos nominales del motor bajo prueba. **7**
Tabla 2. Enfoques representativos para la detección de fallas entre espiras y su relación con este trabajo. **15**
Tabla 3. Electrónica adquirida para el proyecto (registro de compras). **17**
Tabla 4. Niveles de carga del banco de pruebas. **26**
Tabla 5. Resistencia entre terminales de la fase C medida con el LCR-819. ..... **27**
Tabla 6. Voltaje inducido entre terminales con el motor en marcha y fracción de espiras correspondiente, referida al voltaje de fase de 123 V. **27**
Tabla 7. Calibración del canal de la fase C contra osciloscopio (julio de 2026). **30**
Tabla 8. Conjuntos de datos experimentales construidos durante el proyecto. **34**
Tabla 9. Documentos de la colección del asistente y partes indexadas. **58**
Tabla 10. Corriente del lazo de falla medida con pinza durante la captura del conjunto de datos multiclase (6 al 8 de octubre de 2026). **71**
Tabla 11. Resultados de los clasificadores de la etapa exploratoria (julio a septiembre de 2026). **71**
Tabla 12. Matriz de confusión del modelo XGBoost de cuatro clases (filas: clase real; columnas: clase predicha). **72**
Tabla 13. Desempeño del Random Forest binario de septiembre según el esquema de partición. **72**
Tabla 14. Corridas de inyección de la falla LEVE del 28 y 29 de septiembre de 2026. ..... **74**
Tabla 15. Efecto del estado de las conexiones de la instrumentación (registros del 1 de octubre de 2026, carga media). **77**
Tabla 16. Pares y firma de la falla por clase y carga (bloque principal de 597 pares, 6 al 8 de octubre de 2026). **80**
Tabla 17. Resultados del clasificador de severidad con validación dejando una carga fuera (V1) y con la repetición de otra sesión (V2). **82**
Tabla 18. Matriz de confusión del modelo M1 con validación dejando una carga fuera (filas: clase real; columnas: clase predicha). **82**
Tabla 19. Validación ciega en vivo con el firmware final a 75 % de carga (8 y 9 de octubre de 2026). **83**
Tabla 20. Bloques de validación ciega LEVE a 75 % anteriores al firmware final (6 y 7 de octubre de 2026). **83**
Tabla 21. Validación ciega en vivo con el firmware final a 0 y 25 % de carga (15 de octubre de 2026). **84**
Tabla 22. Prueba larga sin falla del 12 de octubre de 2026 (firmware final, lazo desconectado). **84**
Tabla 23. Estudio 1: detectores evaluados dejando un grupo fuera (13 grupos, 2.76 h de motor sano). **85**
Tabla 24. Estudio 2: generalización entre cargas con pares de bloques (607 pares). **87**
Tabla 25. Validación ciega 2, hipótesis H1: falsas alarmas en 4.906 h de operación sana. **87**
Tabla 26. Validación ciega 2, hipótesis H2: detección de la falla LEVE a baja carga. **87**
Tabla 27. Validación ciega 2, resultados secundarios (4.906 h de operación sana). **88**
Tabla 28. Desviación de Allan (pp) de la proyección de la firma con el motor sano, por carga y tiempo de promediado τ. **90**
Tabla 29. Variantes del detector evaluadas fuera de línea (rejilla de 1904 configuraciones, selección dejando un grupo fuera). **90**
Tabla 30. Paquetes de instalación de la aplicación probados el 17 de octubre de 2026. ..... **91**

Tabla 31. Tiempo de la recomendación del asistente local (Qwen3.5-4B con el prefijo del mensaje precalentado) al confirmarse una alarma. **93**
Tabla 32. Comparación preliminar de los modelos locales (13 de octubre de 2026, borrador de 23 preguntas, calificación propuesta). **93**
Tabla 33. Evaluación del asistente con el set de prueba 2 (24 preguntas, 17 de octubre de 2026, calificación propuesta por Claude). **94**
Tabla 34. Comparativa de los objetivos específicos planteados y los resultados obtenidos. **96**
Tabla 35. Objetivos específicos del registro frente a lo realizado. **96**
Tabla 36. Trabajo futuro priorizado para reducir las falsas alarmas del detector. **102**

CAPÍTULO 1. INTRODUCCIÓN

El motor de inducción trifásico es la máquina que mueve la mayor parte de los procesos industriales: bombas, compresores, ventiladores, bandas transportadoras y máquinas herramienta dependen de él. Su robustez lo ha convertido en un elemento al que pocas veces se le presta atención hasta que falla. Cuando eso ocurre, el paro no programado detiene la producción y, con frecuencia, termina en el rebobinado o el reemplazo del equipo. Entre las fallas posibles, el cortocircuito entre espiras de un mismo devanado del estator es especialmente delicado: en sus primeras etapas apenas modifica el comportamiento del motor, pero la corriente que circula por la espira dañada genera calor localizado que degrada el aislamiento vecino y hace avanzar la falla hasta comprometer la fase completa.

Este capítulo plantea el contexto y la motivación del trabajo. Primero se presenta un recorrido por el diagnóstico de fallas en motores de inducción, desde la inspección periódica fuera de línea hasta el análisis de la firma de corriente del motor (MCSA, *Motor Current Signature Analysis*), la incorporación del aprendizaje automático y, más recientemente, de los modelos de lenguaje como apoyo al mantenimiento. A partir de ese contexto se delimita el problema: detectar en línea, con hardware de bajo costo, un cortocircuito entre espiras en etapa temprana, distinguiéndolo de las perturbaciones normales de la red eléctrica, de la carga mecánica y de la propia instrumentación.

Para atender esa necesidad se propone un sistema embebido basado en el microcontrolador ESP32-S3, que adquiere las corrientes y los voltajes de las tres fases del motor y calcula en tiempo real indicadores espectrales y de componentes simétricas. La inteligencia artificial se integra en un enfoque de tres etapas. La **detección** la decide una regla física sobre la corriente de secuencia negativa, cuyos parámetros se estiman de los datos del banco. La **severidad** la estima un clasificador aprendido y embebido en el microcontrolador. La **recomendación** de mantenimiento la propone un asistente basado en un modelo de lenguaje local con recuperación de documentos (RAG, *Retrieval-Augmented Generation*), integrado en una aplicación de escritorio. La detección no se confió a un modelo aprendido porque los modelos evaluados para esa tarea no generalizaron entre niveles de carga, como se muestra en el Capítulo 4. El capítulo cierra con la justificación, la hipótesis, los objetivos y los límites y alcances del proyecto.

1.1 ANTECEDENTES

Los sistemas accionados por motores eléctricos consumen más del 40 % de la electricidad que se genera en el mundo (Waide y Brunner, 2011), y dentro de ellos el motor de inducción de jaula de ardilla es la tecnología dominante por su sencillez constructiva, su bajo costo y su capacidad de operar durante años con mantenimiento mínimo. Esa misma presencia hace que su confiabilidad tenga un impacto directo en la continuidad de los procesos productivos.

Los estudios de confiabilidad realizados en la industria clasifican las fallas del motor de inducción en tres grupos principales: las de rodamientos, las del estator y las del rotor. Las fallas del estator, asociadas en su mayoría al deterioro del aislamiento de los devanados, figuran de manera consistente entre las causas principales (Bonnett y Soukup, 1992; Thorsen y Dalva, 1995). Un mecanismo frecuente es el cortocircuito entre espiras (ITF, *inter-turn fault*): por esfuerzos térmicos, eléctricos, mecánicos o ambientales, el esmalte que separa dos vueltas contiguas de una bobina pierde su capacidad dieléctrica y entre ellas se forma un camino de conducción. La espira en cortocircuito se comporta como el secundario de un transformador cuyo primario es el resto del devanado; por ella circula una corriente elevada que calienta la zona y acelera la propagación del daño hacia espiras vecinas, hacia otras bobinas o hacia la carcasa (Nandi, Toliyat y Li, 2005).

Históricamente, el estado del aislamiento se evaluó con pruebas fuera de línea, como la medición de resistencia de aislamiento o el índice de polarización, que exigen detener y desconectar el motor. Estas pruebas son útiles en mantenimientos programados, pero no detectan una falla que se inicia entre dos inspecciones. Para cubrir ese hueco se desarrollaron técnicas de monitoreo en línea basadas en variables que pueden medirse sin intervenir la máquina: vibración, flujo axial, temperatura y, sobre todo, la corriente de alimentación.

El análisis de la firma de corriente del motor se consolidó como una de las técnicas más difundidas porque solo requiere sensores en los conductores de alimentación, lejos de la máquina. Thomson y Fenger (2001) sistematizaron su aplicación industrial y mostraron que distintas fallas dejan huellas características en el espectro de la corriente, como bandas laterales alrededor de la frecuencia de la red cuya posición depende del deslizamiento del motor. En el caso específico de las fallas entre espiras, la asimetría que introduce el cortocircuito en el devanado se refleja además en un desbalance de las corrientes de las tres fases, que puede cuantificarse mediante la componente de secuencia negativa de la teoría de componentes simétricas de Fortescue (1918); trabajos recientes la siguen usando como indicador directo de esta falla (Ruzimov et al., 2025). La dificultad práctica de este enfoque es que el motor sano también produce corriente de secuencia negativa, por el desbalance de los voltajes de la red y por sus propias asimetrías constructivas, de modo que el diagnóstico debe distinguir el cambio que introduce la falla de ese nivel de base.

En los últimos años, el diagnóstico de fallas incorporó técnicas de aprendizaje automático. En lugar de fijar a mano un umbral sobre un indicador, se entrena un clasificador con ejemplos etiquetados de motor sano y con falla para que aprenda la frontera de decisión a partir de varias características simultáneas. Los ensambles de árboles de decisión, como Random Forest (Breiman, 2001) y XGBoost (Chen y Guestrin, 2016), son populares en este tipo de problemas tabulares por su buen desempeño con conjuntos de datos moderados y por la posibilidad de interpretar la importancia de cada variable. Al mismo tiempo, la literatura advierte que los resultados de estos clasificadores dependen fuertemente de cómo se construyen y validan los datos: cuando las muestras de entrenamiento y de prueba provienen de las mismas sesiones de captura, la exactitud reportada puede sobrestimar el desempeño real ante condiciones nuevas (Kaufman, Rosset y Perlich, 2011).

Más recientemente, los modelos de lenguaje de gran tamaño abrieron otra vía para apoyar el mantenimiento: responder en lenguaje natural preguntas sobre inspección y reparación. Su limitación es que pueden generar afirmaciones fluidas pero no respaldadas por ninguna fuente (Ji et al., 2023). La generación aumentada por recuperación atiende ese problema entregando al modelo, antes de responder, los fragmentos pertinentes de una colección de documentos, de modo que la respuesta pueda citar su origen (Lewis et al., 2020). Los modelos de pesos abiertos y los motores de inferencia optimizados permiten, además, ejecutar ese asistente en una computadora personal, sin enviar datos a un servicio externo (Yang et al., 2025; Gerganov, s. f.).

Paralelamente, la disponibilidad de microcontroladores de bajo costo con capacidad de procesamiento digital de señales, como la familia ESP32 de Espressif, abrió la posibilidad de ejecutar la adquisición, el análisis espectral y la decisión directamente junto al motor, sin depender de una computadora ni de equipos de adquisición comerciales (Espressif Systems, 2026). Este trabajo se ubica en esa convergencia: MCSA, componentes simétricas, aprendizaje automático, modelos de lenguaje y sistemas embebidos aplicados a la detección temprana de cortocircuitos entre espiras.

1.2 DELIMITACIÓN DEL PROBLEMA

En la mayoría de las pequeñas y medianas industrias, el mantenimiento de los motores de inducción sigue siendo correctivo o, en el mejor de los casos, preventivo con inspecciones periódicas. Existen sistemas comerciales de monitoreo de condición en línea, pero requieren instrumentación especializada y personal capacitado, por lo que su implementación suele justificarse primero en máquinas de gran potencia o en procesos

críticos. Para los motores de baja y mediana potencia, que son la mayoría, el resultado es que un cortocircuito entre espiras suele descubrirse cuando ya produjo un disparo de protecciones o un daño mayor.

Detectar la falla en su etapa temprana es técnicamente difícil por tres razones. La primera es la magnitud de la señal: cuando solo una pequeña fracción del devanado está afectada, los cambios en la corriente de fase son pequeños y comparables con el ruido de medición. La segunda es la presencia de perturbaciones que imitan la falla: el desbalance de voltaje de la red, las variaciones de la carga mecánica y la deriva térmica del motor modifican los mismos indicadores en los que se apoya el diagnóstico. La tercera es la propia instrumentación de bajo costo: sensores de efecto Hall y transformadores de voltaje de propósito general, con su acondicionamiento y sus conexiones, introducen errores y ruido que, si no se controlan, dominan sobre la firma de la falla.

A estas dificultades se suman dos problemas metodológicos. El primero es que un detector que se ajusta y evalúa con pocos ensayos cortos puede mostrar una tasa de acierto muy alta y, sin embargo, producir falsas alarmas frecuentes al operar de forma continua; para que una herramienta de mantenimiento predictivo sea útil, debe demostrar tanto su capacidad de detectar la falla como su capacidad de permanecer en silencio cuando el motor está sano durante periodos prolongados. El segundo es que un modelo aprendido puede apoyarse en variables que acompañan a la falla solo en las condiciones con que se entrenó, como el nivel de corriente a una carga dada, y fallar en las demás; por eso cada modelo debe validarse en las condiciones en que va a operar, con protocolos fijados antes de obtener los datos.

El problema que aborda este trabajo es, por lo tanto, el siguiente: desarrollar y evaluar experimentalmente un sistema embebido de bajo costo que detecte en línea un cortocircuito entre espiras en el estator de un motor de inducción trifásico, en distintos niveles de severidad y de carga, que estime su severidad, que distinga la falla de las perturbaciones normales y que mantenga una tasa de falsas alarmas controlada durante la operación sana, y complementarlo con una aplicación que muestre el diagnóstico y proponga acciones de mantenimiento citando sus fuentes. El estudio se realiza sobre un motor ABB M1AA 145T-D4 de 1.5 kW, conectado en doble estrella (220-YY) a la red de 220 V, con derivaciones (taps) en el devanado de la fase C que permiten provocar fallas controladas de forma reversible.

1.3 JUSTIFICACIÓN

Justificación técnica. La detección temprana de una falla entre espiras permite intervenir el motor antes de que el daño se propague, cuando la reparación todavía es acotada y puede programarse. Una técnica basada en la medición de corriente y voltaje es no invasiva: no requiere modificar el motor ni detenerlo, y los sensores pueden instalarse en el tablero de alimentación. La corriente de secuencia negativa tiene una relación física directa con la asimetría que introduce la falla, lo que permite construir un detector cuya decisión puede explicarse y verificarse, y reservar el aprendizaje automático para tareas en las que su generalización puede comprobarse, como la estimación de la severidad. Ejecutar la detección y la severidad en un microcontrolador junto al motor elimina la dependencia de una computadora para el diagnóstico y facilita su replicación; la computadora solo se usa para supervisar y para el asistente de mantenimiento, que no interviene en la decisión de la alarma.

Justificación económica. El prototipo se construye con componentes de propósito general y bajo costo: un Arduino Nano ESP32, sensores de corriente ACS712-30A, sensores de voltaje ZMPT101B, una pantalla TFT y relevadores de estado sólido para la inyección de fallas en el banco. El costo de la electrónica adquirida para el proyecto fue de $1,807 MXN; el motor, el generador de carga y el variac son préstamo del laboratorio. El software, incluido el modelo de lenguaje del asistente, es libre o de uso gratuito y se ejecuta en la computadora del proyecto. Este orden de magnitud hace viable considerar el monitoreo continuo de motores que hoy no justifican un sistema comercial.

Justificación académica. El trabajo integra áreas centrales de la ingeniería mecatrónica: máquinas eléctricas, instrumentación, procesamiento digital de señales, sistemas embebidos de tiempo real, aprendizaje automático y desarrollo de software. Además de los resultados positivos, documenta los resultados negativos y la forma en que se obtuvieron: la caracterización experimental de las derivaciones del motor, que no coincidían con la documentación disponible; la identificación de las conexiones de la instrumentación como fuente dominante de ruido; la falta de generalización entre cargas de los modelos de detección, demostrada con protocolos declarados, y una tasa de falsas alarmas que no cumple el criterio fijado. Ese registro, junto con el código y los datos del banco, constituye una base replicable para trabajos posteriores en la Unidad Académica Multidisciplinaria Región Altiplano.

Justificación social. Una herramienta accesible de mantenimiento predictivo contribuye a reducir paros no planificados, a extender la vida útil de los equipos y a mejorar la seguridad en el entorno de trabajo. Un asistente que responde en español con base en manuales y guías de mantenimiento citados acerca esa información al personal técnico. Su beneficio potencial se dirige a las pequeñas y medianas empresas del sector manufacturero, al personal técnico de mantenimiento industrial y a estudiantes e investigadores de las áreas de mecatrónica e inteligencia artificial.

1.4 HIPÓTESIS

Es posible detectar en línea un cortocircuito entre espiras en el estator de un motor de inducción trifásico, desde una severidad de aproximadamente el 7 % de las espiras de una fase, mediante un sistema embebido de bajo costo que estima el residuo de la corriente de secuencia negativa compensado contra el desbalance de la red y lo evalúa con una regla de decisión secuencial, sin producir falsas alarmas durante la operación sana del motor en distintos niveles de carga.

Se postula, además, que una vez detectada la falla, las variables físicas del detector (la magnitud y la dirección del residuo de secuencia negativa y la variación de corriente en cada fase) contienen la información suficiente para que un clasificador de aprendizaje automático estime el nivel de severidad.

1.5 OBJETIVOS

OBJETIVO GENERAL

Desarrollar un sistema de diagnóstico predictivo para la detección temprana de cortocircuitos entre espiras en el estator de motores de inducción trifásicos, mediante el análisis de firma de corriente (MCSA), las componentes simétricas, el procesamiento digital de señales y modelos de inteligencia artificial, con validación experimental en un banco de pruebas.

OBJETIVOS ESPECÍFICOS

- Construir y caracterizar un banco de pruebas con un motor de inducción trifásico y un generador como carga, con un sistema de inyección controlada de cortocircuitos entre espiras en tres niveles de severidad.

- Implementar en un microcontrolador ESP32-S3 la adquisición sincronizada de las corrientes y los voltajes de las tres fases, y el cálculo en tiempo real de características MCSA y del residuo de secuencia negativa.
- Implementar un detector de falla y evaluar experimentalmente su tasa de detección, su latencia y su tasa de falsas alarmas en distintos niveles de carga.
- Evaluar modelos de aprendizaje automático (XGBoost y Random Forest) para la detección y desarrollar un clasificador de severidad sobre las variables físicas del detector.
- Desarrollar una aplicación de monitoreo en tiempo real del estado del motor, con un asistente de mantenimiento que proponga acciones citando sus fuentes.

1.6 LÍMITES Y ALCANCES

ALCANCES

El proyecto comprende el desarrollo de un prototipo funcional de diagnóstico, desde la construcción del banco de pruebas hasta su evaluación experimental, con los siguientes puntos:

- **Banco de pruebas:** motor ABB M1AA 145T-D4 (1.5 kW, 220 V en doble estrella, 5.8 A, 1725 rpm) acoplado a un generador Baldor CDP3585 que alimenta bancos resistivos, con cuatro niveles de carga nominal (0, 25, 50 y 75 %). Inyección de fallas en la fase C mediante un relevador de estado sólido y una resistencia limitadora de 5 Ω, en tres niveles de severidad caracterizados experimentalmente: LEVE (≈ 7.0 %), INTERMEDIA (≈ 14.3 %) y SEVERA (≈ 21.6 % de las espiras de la fase).
- **Instrumentación y firmware:** adquisición de seis canales (tres corrientes y tres voltajes) con un Arduino Nano ESP32; análisis espectral, cálculo de componentes simétricas, detector direccional con enclavamiento de la alarma, clasificación embebida de la severidad, protecciones del lazo de inyección y visualización local en una pantalla TFT.
- **Inteligencia artificial:** evaluación de XGBoost y Random Forest para la detección, con protocolos declarados antes de obtener los datos; clasificador de severidad entrenado y validado dejando una carga fuera y embebido en el firmware; y asistente de mantenimiento con un modelo de lenguaje local y recuperación sobre manuales, guías y artículos de libre acceso.
- **Aplicación:** aplicación de escritorio MotorGuard-ITF para Ubuntu, que recibe la telemetría del microcontrolador por puerto serie, muestra el estado del motor y la firma de la falla en tiempo real, registra las alarmas, controla las inyecciones del banco con confirmación humana y aloja al asistente.
- **Evaluación experimental:** validaciones ciegas en vivo con inyecciones de falla en orden aleatorio, pruebas prolongadas con el motor sano para medir las falsas alarmas, estudios comparativos entre la regla y los modelos aprendidos, y evaluación del asistente con un conjunto de preguntas independiente.

LÍMITES

- **Un solo motor y fallas inducidas:** los resultados corresponden a un motor y a cortocircuitos provocados a través de derivaciones fijas con una resistencia limitadora en serie. No se estudian fallas por degradación natural del aislamiento ni su evolución en el tiempo.

- **Una fase:** las fallas se inyectan únicamente en la fase C. Las derivaciones disponibles en la fase A quedan documentadas, pero no se ensayan.
- **Niveles de carga nominales y fijos:** los niveles de carga se establecen con resistencias en el generador, sin medición de par en el eje. El generador no permite cambiar la carga con el motor girando, por lo que no se evalúan escalones de carga en caliente. La operación al 100 % de carga se excluye porque el banco resistivo no puede disipar de forma segura la potencia requerida.
- **Condiciones de laboratorio:** el banco opera con la red de la unidad académica (≈ 123 V por fase) a través de un variac. No se evalúan ambientes industriales con ruido electromagnético severo ni accionamientos con variador de frecuencia.
- **Instrumentación en protoboard:** la instrumentación se monta en protoboard; no se construye una placa soldada.
- **Comunicación:** la telemetría se transmite por USB serie. La conectividad inalámbrica se deshabilitó porque interfiere con el convertidor analógico-digital que usan dos de los canales de voltaje.
- **Asistente como apoyo:** el asistente propone inspecciones y acciones de mantenimiento con citas a sus fuentes, pero no diagnostica: la alarma y la severidad las decide el firmware.
- **Fuera del alcance:** el servidor de telemetría con base de datos para varios motores (API REST, PostgreSQL/TimescaleDB) y la versión de la aplicación para Windows se plantean como trabajo futuro.

CAPÍTULO 2. MARCO TEÓRICO

Este capítulo reúne los fundamentos necesarios para comprender el funcionamiento del sistema de diagnóstico propuesto. Se parte del principio de operación del motor de inducción trifásico y de las variables que describen su estado, como la velocidad síncrona y el deslizamiento, para después revisar los tipos de falla que lo afectan, con énfasis en el cortocircuito entre espiras del estator y en el mecanismo por el cual una falla incipiente evoluciona hacia una falla severa.

Posteriormente se presentan las herramientas de análisis que sostienen el diagnóstico: el análisis de firma de corriente, la transformada discreta de Fourier y su cálculo eficiente, la teoría de componentes simétricas de Fortescue y la detección secuencial de cambios mediante la suma acumulada (CUSUM). Cada una aporta una pieza distinta: el análisis espectral identifica la huella de la falla en la frecuencia, las componentes simétricas cuantifican la asimetría entre fases y la regla secuencial decide cuándo esa evidencia es suficiente para declarar una alarma.

El capítulo continúa con los conceptos de aprendizaje automático empleados en el proyecto, incluidos los ensambles de árboles de decisión, la validación de modelos y la calibración de probabilidades, y con los fundamentos de los modelos de lenguaje y de la generación aumentada por recuperación, en los que se basa el asistente de mantenimiento. Concluye con la descripción de los sensores de corriente y voltaje, del microcontrolador ESP32-S3 y con una revisión del estado del arte en la detección de fallas entre espiras.

2.1 EL MOTOR DE INDUCCIÓN TRIFÁSICO

El motor de inducción trifásico está formado por un estator, que aloja tres devanados desplazados 120° eléctricos entre sí, y un rotor, que en la construcción de jaula de ardilla consiste en barras conductoras unidas en sus extremos por anillos de cortocircuito. Al alimentar el estator con un sistema trifásico de voltajes se establece un campo magnético giratorio cuya velocidad, llamada velocidad síncrona, depende de la frecuencia de la red y del número de polos de la máquina:

$$n_s = \frac{120 f_1}{P} \tag{1}$$

donde n_s es la velocidad síncrona en revoluciones por minuto, f_1 es la frecuencia de alimentación en hertz y P es el número de polos. El campo giratorio induce corrientes en las barras del rotor y la interacción entre ambas produce el par electromagnético. Para que exista inducción, el rotor debe girar a una velocidad n menor que la síncrona; la diferencia relativa entre ambas se denomina deslizamiento:

$$s = \frac{n_s - n}{n_s} \tag{2}$$

El deslizamiento aumenta con la carga mecánica y es un parámetro central del diagnóstico, porque determina la posición de varias de las componentes espectrales que aparecen en la corriente del estator. El motor empleado en este trabajo es un ABB M1AA 145T-D4 de cuatro polos; con alimentación de 60 Hz su velocidad síncrona es de 1800 rpm y, a la velocidad nominal de placa de 1725 rpm, el deslizamiento nominal resulta de aproximadamente 4.2 %. La Tabla 1 resume sus datos nominales.

Tabla 1. Datos nominales del motor bajo prueba.

Parámetro	Valor
Modelo	ABB M1AA 145T-D4
Potencia nominal	1.5 kW (2 HP)

Parámetro	Valor
Voltaje y conexión	220 V, doble estrella (220-YY)
Corriente nominal	5.8 A
Velocidad nominal	1725 rpm
Frecuencia	60 Hz
Número de polos	4

Cuando el motor está sano y la red es simétrica, las tres corrientes de fase tienen la misma amplitud y están desfasadas 120° entre sí. Cualquier asimetría, ya sea de la alimentación o de la propia máquina, rompe ese equilibrio y modifica tanto la forma de onda como la relación entre las fases. Este hecho es la base de las técnicas de diagnóstico que se describen en las secciones siguientes.

2.2 FALLAS EN MOTORES DE INDUCCIÓN

Las fallas del motor de inducción se agrupan habitualmente en tres categorías: fallas de rodamientos, fallas del estator y fallas del rotor, además de un grupo menor asociado a excentricidades, desalineación y problemas de la carga acoplada. Los estudios de confiabilidad realizados en la industria muestran que las fallas de rodamientos y las del estator concentran la mayor parte de los casos, aunque la proporción exacta varía según el sector, la potencia de los motores y las condiciones de operación (Bonnett y Soukup, 1992; Thorsen y Dalva, 1995).

Las fallas del estator están ligadas principalmente al sistema de aislamiento de los devanados. El aislamiento se degrada por esfuerzos térmicos (sobrecargas, ciclos de arranque, ventilación deficiente), eléctricos (sobretensiones y transitorios de conmutación), mecánicos (vibración y movimiento de las bobinas durante los arranques) y ambientales (humedad, contaminación y agentes químicos). La evolución típica de la falla comienza con un deterioro localizado del esmalte de los conductores y puede terminar en un cortocircuito entre fases o entre una fase y la carcasa (Siddique, Yadava y Singh, 2005; Tallam et al., 2007).

2.2.1 FALLAS ENTRE ESPIRAS

El cortocircuito entre espiras (ITF, *inter-turn fault*) ocurre cuando se pierde el aislamiento entre dos o más vueltas de una misma bobina. Desde el punto de vista eléctrico, el grupo de espiras en cortocircuito se comporta como el devanado secundario de un autotransformador cuyo primario es el resto de la fase: el flujo del entrehierro induce en ese lazo un voltaje proporcional al número de espiras afectadas y, como la impedancia del lazo es muy baja, circula por él una corriente que puede superar varias veces la corriente nominal de la fase (Tallam, Habetler y Harley, 2002; Arkan, Kostic-Perovic y Unsworth, 2005).

Esa corriente tiene dos consecuencias. La primera es térmica: el calor generado en un volumen pequeño degrada el aislamiento vecino y propaga la falla hacia más espiras, hacia otras bobinas y finalmente hacia tierra, por lo que el tiempo entre una falla incipiente y una falla catastrófica puede ser corto. La segunda es electromagnética: el lazo en cortocircuito produce una fuerza magnetomotriz que se opone a la de la fase afectada, de modo que el devanado deja de ser simétrico. Esa asimetría es la que permite detectar la falla desde los terminales del motor, ya sea por la aparición de componentes espectrales características o por el desbalance que provoca entre las corrientes de las tres fases.

En un banco de pruebas, la falla puede reproducirse de forma controlada y reversible si el motor dispone de derivaciones (*taps*) que dan acceso a puntos intermedios del devanado. Al unir dos derivaciones a través de una

resistencia limitadora se crea un lazo equivalente a un cortocircuito entre las espiras comprendidas entre ellas; la resistencia limita la corriente del lazo para proteger la máquina y permite repetir el ensayo sin dañar el aislamiento.

2.3 ANÁLISIS DE FIRMA DE CORRIENTE (MCSA)

El análisis de firma de corriente del motor (MCSA, *Motor Current Signature Analysis*) consiste en medir la corriente de alimentación durante la operación normal y examinar su contenido espectral en busca de componentes asociadas a fallas específicas. Su principal ventaja es que no requiere sensores en la máquina: basta con transductores de corriente en los conductores de alimentación, que pueden instalarse en el tablero de control (Thomson y Fenger, 2001; Benbouzid, 2000).

En la literatura se han identificado familias de frecuencias características para distintas fallas. Las asimetrías del rotor, como las barras rotas, producen bandas laterales alrededor de la frecuencia de alimentación en

$$f_b = f_1\,(1 \pm 2ks), \qquad k = 1, 2, 3, \ldots \tag{3}$$

cuya separación respecto de la fundamental crece con el deslizamiento. Para los cortocircuitos entre espiras del estator se han propuesto componentes en frecuencias que dependen del número de pares de polos y de la velocidad del rotor, y que en la práctica suelen tener amplitudes pequeñas y quedar cerca de otras componentes de la corriente (Thomson y Fenger, 2001; Nandi, Toliyat y Li, 2005). Por esa razón, muchos trabajos complementan el análisis espectral con indicadores del desbalance entre fases, como la componente de secuencia negativa que se describe en la sección 2.5.

Una consecuencia práctica de la dependencia con el deslizamiento es que, con el motor en vacío, s es muy pequeño y las bandas laterales quedan prácticamente encimadas sobre la fundamental, por lo que resultan difíciles de resolver. A medida que aumenta la carga, el deslizamiento crece y las bandas se separan. Por ello, los ensayos de diagnóstico por MCSA se realizan habitualmente en varios niveles de carga.

2.4 TRANSFORMADA RÁPIDA DE FOURIER Y ESTIMACIÓN ESPECTRAL

Para obtener el espectro de una señal muestreada se emplea la transformada discreta de Fourier (DFT). Dada una secuencia de N muestras x[n] tomadas con frecuencia de muestreo f_s, la DFT se define como

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \qquad k = 0, 1, \ldots, N-1 \tag{4}$$

y cada índice k corresponde a la frecuencia $k \cdot f_s/N$. La separación entre líneas espectrales, o resolución en frecuencia, es por lo tanto

$$\Delta f = \frac{f_s}{N} \tag{5}$$

Una resolución fina exige registros largos: con f_s = 2403.85 Hz y N = 4096 muestras, como en el firmware de este proyecto, cada bloque abarca 1.70 s y la resolución es de 0.59 Hz. La transformada rápida de Fourier (FFT) es el conjunto de algoritmos que calculan la DFT con un número de operaciones proporcional a N log N en lugar de N^2, lo que hace viable su ejecución en un microcontrolador (Cooley y Tukey, 1965).

Cuando la duración del registro no contiene un número entero de periodos de la señal, la energía de cada componente se reparte entre líneas vecinas, fenómeno conocido como fuga espectral. Para reducirla, la señal se multiplica antes de la transformada por una ventana que atenúa sus extremos; la ventana de Hann es una elección habitual por su buen compromiso entre ancho del lóbulo principal y atenuación de los lóbulos laterales (Harris, 1978). La ventana, sin embargo, no elimina el error de posición: la frecuencia verdadera de una componente rara vez coincide con una línea de la DFT. Los algoritmos de interpolación espectral, como el propuesto por Grandke (1983) para señales ponderadas con ventana, estiman la frecuencia y la amplitud reales a partir de las dos líneas de mayor magnitud alrededor del pico, con una precisión muy superior a la resolución Δf. En este trabajo, esa interpolación se utiliza para estimar la frecuencia fundamental de la red en cada bloque.

2.5 COMPONENTES SIMÉTRICAS DE FORTESCUE

Fortescue (1918) demostró que cualquier conjunto de tres fasores desbalanceados puede descomponerse de manera única en la suma de tres conjuntos balanceados: uno de secuencia positiva, con el mismo orden de fases que el sistema original; uno de secuencia negativa, con el orden de fases invertido; y uno de secuencia cero, formado por tres fasores iguales (Figura 1). Con el operador $a = e^{j120°}$, las componentes de las corrientes de fase I_a, I_b e I_c se calculan como

$$I_0 = \frac{1}{3}(I_a + I_b + I_c) \tag{6}$$

$$I_1 = \frac{1}{3}(I_a + a\,I_b + a^2 I_c) \tag{7}$$

$$I_2 = \frac{1}{3}(I_a + a^2 I_b + a\,I_c) \tag{8}$$

y de forma análoga se obtienen las componentes de voltaje V_0, V_1 y V_2. En un motor conectado en estrella sin neutro, como la doble estrella del motor de este trabajo, la componente de secuencia cero de la corriente es nula.

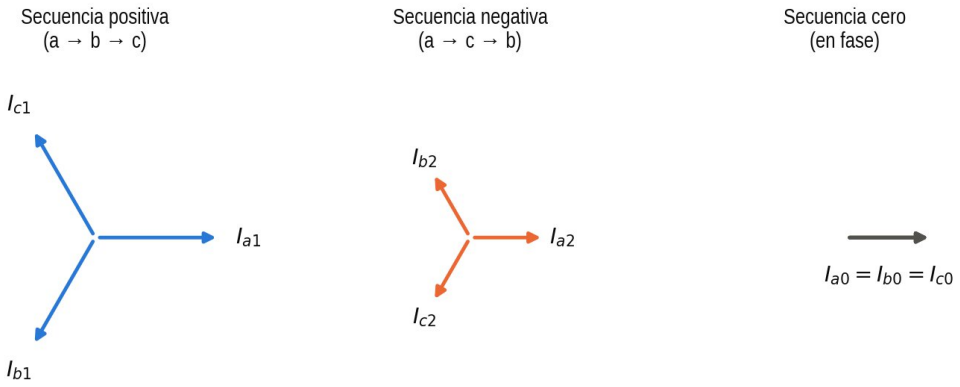


Figura 1. *Representación de las componentes simétricas de un sistema trifásico: secuencias positiva, negativa y cero.*

En un motor sano alimentado por una red perfectamente simétrica, solo existe secuencia positiva. En la práctica aparecen dos fuentes de corriente de secuencia negativa. La primera es externa: la red nunca es

perfectamente simétrica, y un voltaje de secuencia negativa V_2 produce una corriente proporcional a través de la admitancia de secuencia negativa del motor, Y_2. La segunda es interna: la asimetría constructiva propia de la máquina y, sobre todo, una falla en el devanado. Un cortocircuito entre espiras genera una corriente de secuencia negativa adicional cuya magnitud crece con la severidad de la falla (Kliman et al., 1996; Arkan, Kostic-Perovic y Unsworth, 2001; Bouzid y Champenois, 2013).

Para que el indicador responda a la falla y no a la red, la corriente de secuencia negativa se compensa restando la parte explicada por el desbalance de voltaje y la asimetría intrínseca medida con el motor sano. El residuo resultante, normalizado con la corriente de secuencia positiva para independizarlo de la carga, puede escribirse como

$$e_2 = \frac{|I_2 - Y_2 V_2 - I_{2,0}|}{|I_1|} \tag{9}$$

donde $I_{2,0}$ representa la asimetría constructiva del motor sano. Los parámetros Y_2 e $I_{2,0}$ se estiman durante una etapa de comisionamiento con el motor en buen estado. Como el residuo es un fasor, su cambio respecto de la condición de referencia tiene magnitud y dirección; la dirección depende de la fase afectada, lo que permite, en principio, distinguir una falla del estator de otras perturbaciones que modifican la magnitud del residuo en direcciones diferentes.

2.6 DETECCIÓN SECUENCIAL DE CAMBIOS

Un indicador como el residuo de secuencia negativa fluctúa por el ruido de medición, por la variación de la red y por la carga. Compararlo bloque a bloque contra un umbral fijo obliga a elegir entre dos males: un umbral bajo detecta pronto, pero produce falsas alarmas frecuentes; un umbral alto reduce las falsas alarmas, pero retrasa o impide la detección de fallas pequeñas. La detección secuencial de cambios resuelve ese compromiso acumulando evidencia en el tiempo.

El procedimiento de suma acumulada (CUSUM), propuesto por Page (1954), actualiza en cada bloque un estadístico de la forma

$$S_t = \max(0,\ S_{t-1} + z_t - K) \tag{10}$$

donde z_t es la evidencia del bloque t a favor de la hipótesis de falla, normalmente un logaritmo de razón de verosimilitudes o una transformación monótona de él, y K es un término de deriva que mantiene al estadístico cerca de cero mientras el proceso está en control. La alarma se declara cuando S_t supera un umbral H. El desempeño de la regla se describe con dos cantidades: la longitud media de corrida en control, ARL_0, que es el número medio de bloques entre falsas alarmas cuando no hay falla, y la longitud media de corrida fuera de control, ARL_1, que es el retardo medio de detección una vez que la falla aparece. Elevar H aumenta ARL_0 a costa de un ARL_1 mayor (Lorden, 1971). La variante de Page-Hinkley aplica la misma idea a la detección de un cambio en la media de una señal (Hinkley, 1971).

Una regla secuencial más sencilla, y la que usa el detector final de este trabajo, es la de persistencia: se declara la alarma cuando el indicador supera un umbral durante k bloques consecutivos. Si los bloques fueran independientes y la probabilidad de superar el umbral con el motor sano fuera p, la probabilidad de una racha de k bloques sería del orden de p^k, de modo que la persistencia reduce mucho las falsas alarmas a costa de un retardo de k bloques. Esa estimación deja de valer cuando el ruido está correlacionado entre bloques consecutivos, porque las rachas se vuelven mucho más probables de lo que predice el cálculo con bloques independientes.

Una propiedad importante para este proyecto es que ARL_0 depende de la distribución de z_t en condición sana. Si la evidencia se calcula a partir de un indicador cuyo ruido real es mayor que el supuesto en el diseño, la tasa de falsas alarmas observada será mayor que la prevista. Por ello, ARL_0 debe verificarse experimentalmente con registros prolongados del motor sano, y no solo calcularse.

Para caracterizar un ruido que no es blanco se emplea la desviación de Allan, propuesta originalmente para la estabilidad de los patrones de frecuencia (Allan, 1966). Se calcula a partir de las diferencias entre promedios consecutivos de la señal sobre ventanas de duración τ, y su variación con τ distingue el tipo de ruido: con ruido blanco disminuye en proporción a $\tau^{-1/2}$, con ruido de parpadeo se mantiene aproximadamente constante y con deriva aumenta (Riley, 2008). En este trabajo indica cuánto puede reducirse el ruido de la firma promediando bloques o alargando la referencia.

2.7 INTELIGENCIA ARTIFICIAL Y APRENDIZAJE AUTOMÁTICO

El aprendizaje automático supervisado construye, a partir de ejemplos etiquetados, una función que asigna a cada vector de características una clase o una probabilidad. En el diagnóstico de fallas, las características suelen ser magnitudes espectrales y estadísticas de las señales medidas, y las clases corresponden al estado del motor. Los árboles de decisión dividen recursivamente el espacio de características mediante umbrales sobre una variable a la vez, de forma que cada hoja agrupa ejemplos de una misma clase. Un árbol aislado es fácil de interpretar, pero muy sensible a pequeñas variaciones de los datos.

2.7.1 RANDOM FOREST

Random Forest combina muchos árboles entrenados sobre muestras aleatorias con reemplazo del conjunto de entrenamiento y, en cada división, sobre un subconjunto aleatorio de variables. La predicción del ensamble es el voto mayoritario o el promedio de las probabilidades de los árboles. La aleatorización reduce la correlación entre árboles y, con ello, la varianza del modelo, sin aumentar sustancialmente su sesgo (Breiman, 2001). Por su estructura de reglas de comparación, un Random Forest entrenado puede traducirse a código en lenguaje C y ejecutarse en un microcontrolador.

2.7.2 XGBOOST

XGBoost es una implementación del método de *gradient boosting* con árboles: los árboles se entrenan de manera secuencial y cada uno corrige los errores residuales de los anteriores, minimizando una función objetivo que combina la pérdida de entrenamiento con un término de regularización que penaliza la complejidad de los árboles. Incorpora además técnicas para tratar datos dispersos y para paralelizar el entrenamiento (Chen y Guestrin, 2016). En problemas con datos tabulares de tamaño moderado suele alcanzar una exactitud alta con poco ajuste de hiperparámetros.

2.7.3 VALIDACIÓN Y CALIBRACIÓN DE MODELOS

El desempeño de un clasificador se estima con datos que no participaron en el entrenamiento. Las métricas habituales son la exactitud (proporción de aciertos), la precisión (proporción de alarmas que corresponden a fallas reales), la sensibilidad o *recall* (proporción de fallas detectadas), la puntuación F1, que es la media armónica de las dos anteriores, y el área bajo la curva ROC (AUC), que resume la capacidad de separar las

clases para todos los umbrales posibles. En la validación cruzada de k pliegues, los datos se dividen en k partes y el modelo se entrena k veces, reservando cada vez una parte distinta para la prueba (Pedregosa et al., 2011).

Estas estimaciones solo son confiables si los datos de prueba son realmente independientes de los de entrenamiento. Cuando las muestras se toman como ventanas consecutivas de una misma sesión de captura, las ventanas vecinas son muy parecidas entre sí; si una partición aleatoria las reparte entre entrenamiento y prueba, el modelo puede reconocer la sesión en lugar de la falla. A este problema se le conoce como fuga de información (Kaufman, Rosset y Perlich, 2011). La práctica recomendada es la validación agrupada, en la que todas las muestras de un mismo grupo (por ejemplo, una sesión o un día de captura) quedan del mismo lado de la partición (Roberts et al., 2017). Una prueba aún más exigente consiste en entrenar con una campaña de captura y evaluar con otra independiente.

Por último, muchos clasificadores producen puntuaciones que no son probabilidades bien calibradas. La calibración de Platt ajusta una función sigmoide sobre la salida del modelo para que la probabilidad estimada coincida con la frecuencia observada de la clase (Platt, 1999; Niculescu-Mizil y Caruana, 2005). Una probabilidad calibrada es útil cuando la salida del clasificador alimenta otra etapa de decisión, como un detector secuencial; la calibración, sin embargo, solo es válida para la variable y la distribución de datos con las que se ajustó.

2.8 MODELOS DE LENGUAJE Y GENERACIÓN AUMENTADA POR RECUPERACIÓN

2.8.1 MODELOS DE LENGUAJE

Los modelos de lenguaje de gran tamaño son redes neuronales entrenadas para predecir el siguiente fragmento de texto (*token*) a partir de los anteriores. La arquitectura dominante es el Transformer, que sustituye la recurrencia por mecanismos de atención que relacionan cada posición del texto con todas las demás (Vaswani et al., 2017). Entrenados con grandes colecciones de texto y ajustados para seguir instrucciones, estos modelos pueden redactar, resumir y responder preguntas en lenguaje natural. Los modelos de pesos abiertos, como la familia Qwen3 (Yang et al., 2025), pueden ejecutarse en una computadora personal; para reducir su memoria y su tiempo de cálculo se cuantizan, es decir, se representan sus parámetros con 4 u 8 bits en lugar de 16, y se ejecutan con motores de inferencia optimizados para procesador, como llama.cpp (Gerganov, s. f.).

Su principal limitación para una aplicación técnica es que pueden generar afirmaciones fluidas pero falsas o no respaldadas por ninguna fuente, fenómeno conocido como alucinación (Ji et al., 2023). En un contexto de mantenimiento, una recomendación inventada puede ser peligrosa, por lo que el modelo no debe responder solo con lo que aprendió en su entrenamiento.

2.8.2 GENERACIÓN AUMENTADA POR RECUPERACIÓN (RAG)

La generación aumentada por recuperación combina el modelo de lenguaje con una búsqueda en una colección de documentos: antes de responder, el sistema recupera los fragmentos más pertinentes para la pregunta y se los entrega al modelo como contexto, junto con la instrucción de basar su respuesta en ellos (Lewis et al., 2020). Así, la respuesta puede citar su fuente, y el conocimiento del sistema se actualiza cambiando los documentos, sin reentrenar el modelo.

La recuperación puede ser semántica o léxica. En la recuperación semántica, cada fragmento y cada pregunta se representan como un vector (*embedding*) calculado por un modelo entrenado para que textos de significado parecido tengan vectores cercanos, y la pertinencia se mide con la similitud coseno entre vectores; los modelos multilingües, como Qwen3-Embedding (Zhang et al., 2025), permiten relacionar una pregunta en

español con un documento en inglés. En la recuperación léxica, la pertinencia se calcula con la frecuencia de los términos de la pregunta en cada fragmento, ponderada por su rareza en la colección; el método BM25 es el más difundido de esta familia (Robertson y Zaragoza, 2009). Ambos enfoques fallan en casos distintos, por lo que suelen combinarse; la fusión de rangos recíprocos (RRF) suma, para cada fragmento, el inverso de su posición en cada ordenamiento más una constante, y ha mostrado superar a métodos de combinación más elaborados (Cormack, Clarke y Buettcher, 2009).

2.9 SENSADO DE CORRIENTE Y VOLTAJE

El sensor ACS712 de Allegro MicroSystems es un circuito integrado que mide corriente por efecto Hall: la corriente atraviesa un conductor de cobre interno de baja resistencia y el campo magnético que genera es detectado por un elemento Hall, aislado galvánicamente del circuito de potencia. La salida es un voltaje proporcional a la corriente, centrado en la mitad del voltaje de alimentación; en la versión de ±30 A la sensibilidad es de 66 mV/A con alimentación de 5 V. La salida es ratiométrica, es decir, tanto el voltaje de reposo como la sensibilidad son proporcionales al voltaje de alimentación del sensor (Allegro MicroSystems, 2024). Esta propiedad implica que cualquier variación o ruido en la alimentación o en la referencia de tierra aparece directamente en la medición, por lo que la calidad de las conexiones y de la alimentación forma parte de la cadena de medición.

El ZMPT101B es un transformador de voltaje miniatura de tipo corriente, que se conecta al voltaje de la red a través de una resistencia en serie con su primario y entrega en el secundario una corriente proporcional, convertida a voltaje mediante una resistencia de carga y, en los módulos comerciales, amplificada por un amplificador operacional con ganancia ajustable mediante un potenciómetro (Qingxian Zeming Langxi Electronic, s. f.). Proporciona aislamiento galvánico entre la red y el circuito de adquisición, y su salida se superpone a un nivel de reposo para que pueda leerse con un convertidor analógico-digital unipolar.

2.10 SISTEMAS EMBEBIDOS: ESP32-S3

El ESP32-S3 de Espressif Systems es un sistema en chip con dos núcleos Xtensa LX7 de 32 bits que operan hasta 240 MHz, 512 KB de memoria SRAM interna, conectividad Wi-Fi y Bluetooth LE, y dos convertidores analógico-digitales de aproximaciones sucesivas de 12 bits (ADC1 y ADC2). El ADC2 comparte recursos con el subsistema de radio, de modo que su uso está restringido cuando el Wi-Fi está activo (Espressif Systems, 2026). Su entorno de desarrollo incorpora el sistema operativo de tiempo real FreeRTOS, que permite asignar tareas a núcleos específicos con prioridades definidas, y temporizadores de hardware para disparar el muestreo a intervalos exactos.

La tarjeta Arduino Nano ESP32 integra el ESP32-S3, a través del módulo NORA-W106 de u-blox, en el formato compacto Nano, con acceso a sus entradas analógicas y digitales y programación mediante el entorno Arduino (Arduino, s. f.). La combinación de capacidad de cómputo, memoria y bajo costo permite ejecutar en la misma tarjeta la adquisición de seis canales, la FFT, el cálculo de componentes simétricas, la regla de decisión y la actualización de una pantalla local.

2.11 ESTADO DEL ARTE

La detección de fallas del estator en motores de inducción ha sido objeto de numerosas revisiones. Siddique, Yadava y Singh (2005) y Tallam et al. (2007) recopilaron las técnicas de monitoreo del estator, desde las pruebas de aislamiento fuera de línea hasta los métodos en línea basados en corriente, voltaje, impedancia de secuencia negativa y flujo axial. Nandi, Toliyat y Li (2005), Bellini et al. (2008) y Henao et al. (2014) ampliaron esa revisión a todas las fallas de la máquina y documentaron la tendencia hacia técnicas de procesamiento de señales más elaboradas y hacia el uso de inteligencia artificial. Gangsar y Tiwari (2020) revisaron específicamente las técnicas basadas en señales, incluidos los métodos de aprendizaje automático. La Tabla 2 resume los enfoques más relacionados con este trabajo.

Tabla 2. Enfoques representativos para la detección de fallas entre espiras y su relación con este trabajo.

Referencia	Enfoque	Relación con este trabajo
Thomson y Fenger (2001); Benbouzid (2000)	Análisis espectral de la corriente (MCSA)	Base de las características espectrales de la etapa exploratoria
Kliman et al. (1996)	Corriente de secuencia negativa compensada para detectar fallas entre espiras en línea	Fundamento del residuo de secuencia negativa del detector
Arkan, Kostic-Perovic y Unsworth (2001)	Impedancia de secuencia negativa	Compensación del desbalance de la red mediante Y_2
Bouzid y Champenois (2013)	Expresiones de las componentes simétricas bajo falla del estator	Interpretación de la dirección del fasor del residuo
Gangsar y Tiwari (2020); Henao et al. (2014)	Revisión de técnicas basadas en señales e inteligencia artificial	Contexto de la etapa de aprendizaje automático
Ruzimov et al. (2025)	Detección de fallas entre espiras con la corriente de secuencia negativa en motores alimentados por inversor	Antecedente directo del indicador; forma parte de los documentos del asistente
Chen et al. (2023)	Detección de fallas entre espiras con la energía de la envolvente de la corriente de arranque	Enfoque alternativo, basado en el arranque; forma parte de los documentos del asistente
Page (1954); Lorden (1971)	Detección secuencial de cambios (CUSUM)	Regla de decisión de las versiones anteriores del detector
Lewis et al. (2020)	Generación aumentada por recuperación	Base del asistente de mantenimiento

En conjunto, la literatura coincide en que el desbalance de la red y la asimetría constructiva del motor son los principales obstáculos para detectar fallas pequeñas a partir de la secuencia negativa, y en que su compensación es indispensable. También advierte que los clasificadores de aprendizaje automático requieren conjuntos de datos representativos y una validación cuidadosa para que su desempeño se mantenga fuera de las condiciones de entrenamiento. La contribución de este trabajo se ubica en la integración de ambos enfoques en un sistema embebido de bajo costo, en su evaluación experimental, incluida la medición de falsas alarmas en operación sana prolongada y la comparación con protocolo declarado entre una regla física y modelos de aprendizaje automático, y en la incorporación de un asistente de mantenimiento local que cita sus fuentes.

CAPÍTULO 3. METODOLOGÍA

En este capítulo se describe cómo se desarrolló el sistema de diagnóstico, desde la evaluación de los recursos disponibles hasta la integración de un prototipo funcional. El trabajo se sustenta en un banco de pruebas construido alrededor de un motor de inducción ABB de 1.5 kW acoplado a un generador que actúa como carga, en un sistema de adquisición basado en el microcontrolador ESP32-S3 y en una aplicación de escritorio que supervisa el sistema e incorpora un asistente de mantenimiento con inteligencia artificial generativa.

El sistema final se organiza en tres etapas. La **detección** la realiza en el microcontrolador una regla física sobre la corriente de secuencia negativa; la **severidad**, un clasificador aprendido y embebido en el mismo firmware; y la **recomendación de mantenimiento**, un asistente con recuperación de documentos (RAG) que se ejecuta en la computadora. Se llegó a esa arquitectura de forma iterativa: las derivaciones del devanado no correspondían a la documentación disponible, los primeros clasificadores entrenados con datos reales no generalizaban entre campañas de captura, el ruido de la instrumentación dependía de la calidad de las conexiones y los modelos de aprendizaje automático para la detección aprendían atajos ligados a la carga. Se documentan esas decisiones y su justificación, porque explican el diseño final.

El capítulo se organiza en tres partes. La primera presenta los materiales, componentes, software y equipo de laboratorio. La segunda describe la metodología propuesta, el diseño por bloques y las técnicas de validación, incluidos los protocolos declarados antes de obtener los datos. La tercera detalla el desarrollo del banco de pruebas, de la adquisición y el procesamiento de señales, de los modelos de inteligencia artificial, del firmware, de la aplicación de escritorio, del asistente de mantenimiento y de su integración.

3.1 MATERIALES Y MÉTODOS

3.1.1 RECURSOS Y FACTIBILIDAD

La realización del proyecto fue factible gracias a la combinación de recursos humanos, técnicos y económicos. En el aspecto humano, el proyecto se apoyó en la formación en ingeniería mecatrónica para el diseño electrónico, la programación de sistemas embebidos, el desarrollo de software y el análisis de datos. En el aspecto técnico, el Laboratorio de Mecatrónica de la Unidad Académica Multidisciplinaria Región Altiplano proporcionó en préstamo el motor de inducción, el generador de carga, el variac trifásico y los instrumentos de medición (Figura 2). En el aspecto económico, la electrónica de adquisición e inyección de fallas se adquirió con recursos propios; la Tabla 3 detalla las compras registradas. El software del sistema, incluido el modelo de lenguaje del asistente, es libre o de uso gratuito y se ejecuta en la computadora del proyecto.

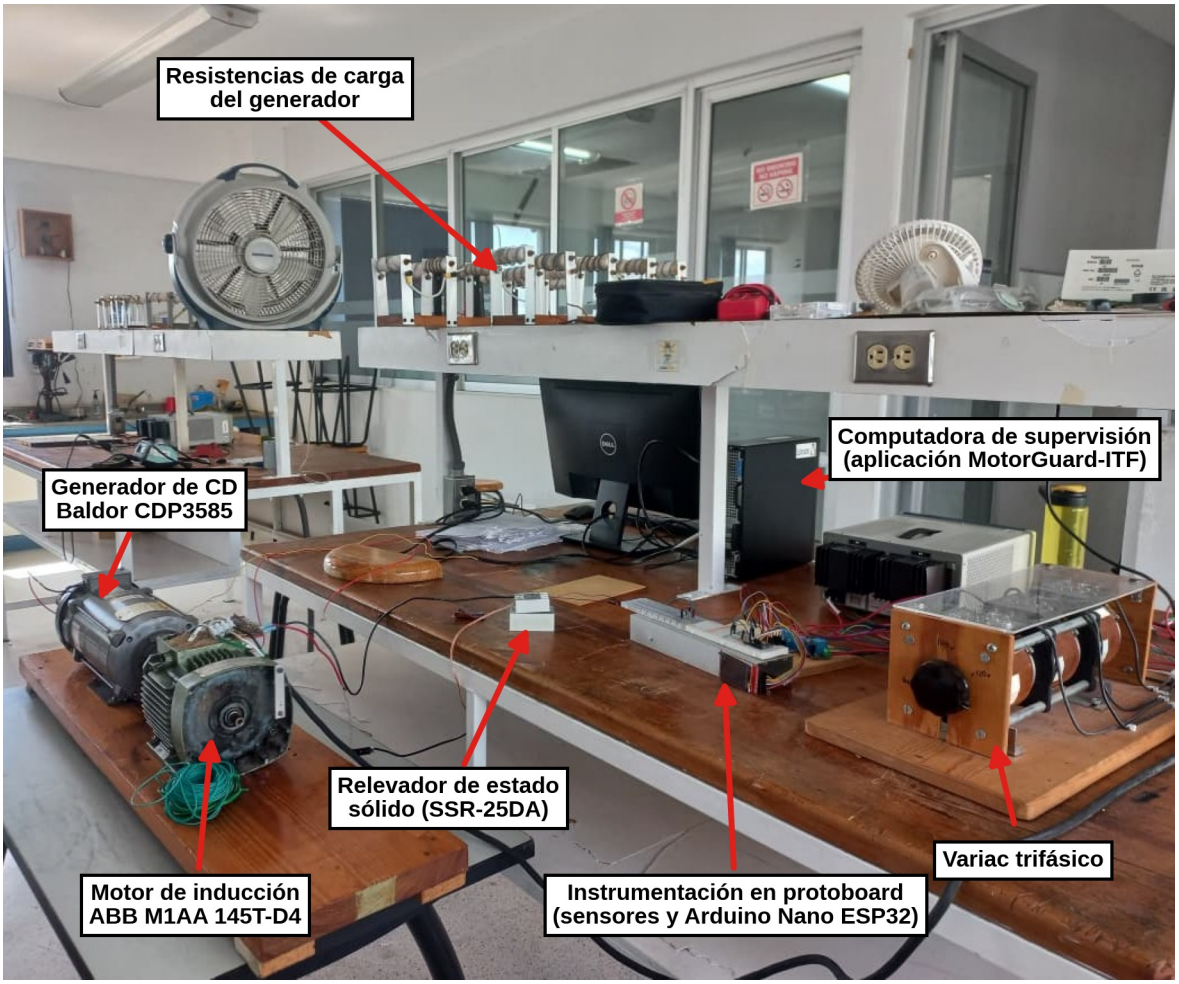


Figura 2. *Banco de pruebas en el Laboratorio de Mecatrónica con sus elementos principales señalados: motor y generador acoplados, resistencias de carga, variac, relevador de estado sólido, instrumentación y computadora de supervisión.*

Tabla 3. Electrónica adquirida para el proyecto (registro de compras).

Componente	Cantidad	Precio unitario (MXN)	Total (MXN)
Relevador de estado sólido SSR-25DA	3	119	357
Arduino Nano ESP32	1	347	347
Sensor de corriente ACS712	4	98	392
Sensor de voltaje ZMPT101B	4	67	268
Pantalla TFT	1	265	265
Cables de conexión tipo jumper	2	89	178
Total			**1,807**

El registro no incluye componentes menores, como resistencias, fusibles, transistores y la protoboard; aun así, el orden de magnitud confirma que la electrónica del sistema puede construirse con un presupuesto reducido. La hoja de compras registra además dos meses de la suscripción Claude Pro (349 MXN cada uno, 698 MXN), que se incluyen en el costo del proyecto: 1,807 MXN de la electrónica más 698 MXN de la suscripción dan un total de 2,505 MXN.

3.1.2 COMPONENTES ELECTRÓNICOS

Arduino Nano ESP32. Es la unidad de procesamiento del sistema. Integra el ESP32-S3 con dos núcleos Xtensa LX7 a 240 MHz y 512 KB de SRAM (Espressif Systems, 2026; Arduino, s. f.). En el proyecto ejecuta el muestreo de seis canales analógicos, el procesamiento espectral, el cálculo de componentes simétricas, el detector de falla, el clasificador de severidad, el control del lazo de inyección y la actualización de la pantalla.

Sensores de corriente ACS712-30A. Se emplea un sensor por fase. Con alimentación de 5 V su sensibilidad es de 66 mV/A y su salida en reposo es de 2.5 V (Allegro MicroSystems, 2024). Como las entradas del ESP32-S3 admiten como máximo 3.3 V, la salida de cada sensor pasa por un divisor resistivo de 10 kΩ y 20 kΩ que la reduce a dos tercios de su valor.

Sensores de voltaje ZMPT101B. Se emplea un módulo por fase, alimentado a 3.3 V y conectado directamente a la entrada analógica, sin divisor. La ganancia de cada módulo se ajusta con su potenciómetro.

Relevador de estado sólido Fotek SSR-25DA. Conmuta el lazo de inyección de falla en el devanado (Figura 3). Se controla con un voltaje de corriente directa y conmuta cargas de corriente alterna (Fotek Controls, s. f.). Como la salida del microcontrolador es de 3.3 V, la entrada del relevador se activa a través de un transistor NPN 2N2222 con resistencia de base de 1 kΩ, conectado al pin D5 de la tarjeta (GPIO 8).

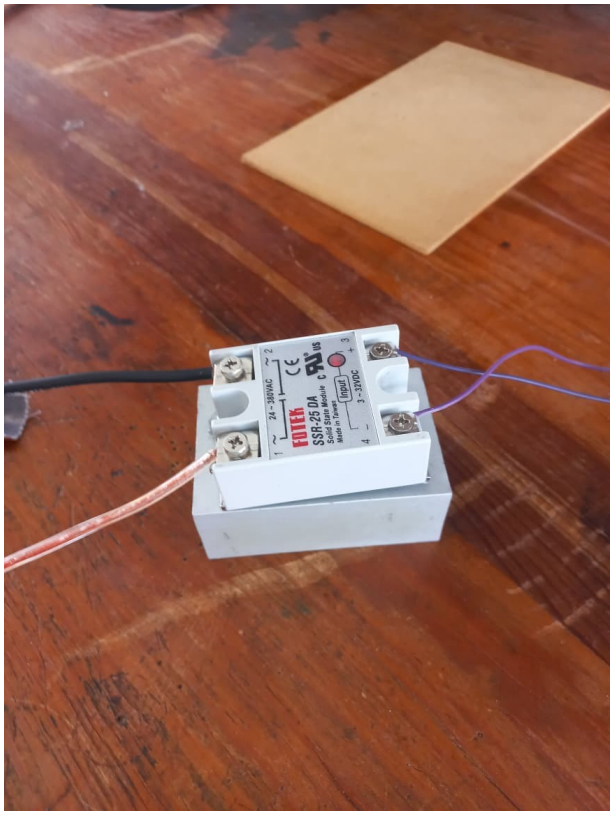


Figura 3. *Relevador de estado sólido Fotek SSR-25DA del lazo de inyección de falla, montado sobre su base.*

Resistencia limitadora y fusibles. La resistencia limitadora, en serie con el lazo de inyección, es de 5 Ω y aproximadamente 150 W. El lazo se protege con un fusible de 5 A en las clases LEVE e INTERMEDIA y con uno de 10 A de acción rápida en la clase SEVERA.

Pantalla TFT ILI9341. Pantalla de 2.8 pulgadas y 240 × 320 píxeles con interfaz SPI, que muestra localmente el estado del motor, las corrientes, los voltajes y el estado del detector.

3.1.3 MATERIALES Y EQUIPO DEL BANCO

Motor bajo prueba. Motor de inducción trifásico ABB M1AA 145T-D4 (Tabla 1 y Figura 4). Además de los datos de la Tabla 1, su placa indica una eficiencia de 81.5 %, un factor de potencia de 0.83, factor de servicio de 1.15, aislamiento clase F, grado de protección IP55 y diseño NEMA B. Para 220 V la placa indica la conexión en doble estrella (220-YY), que es la utilizada en el banco. El motor cuenta con una caja de bornes de 12 terminales que da acceso a derivaciones de los devanados: los terminales 1 a 4 corresponden a la fase C, del 5 al 8 a la fase A y del 9 al 12 a la fase B (Figura 5).



Figura 4. *Placa de datos del motor ABB M1AA 145T-D4.*

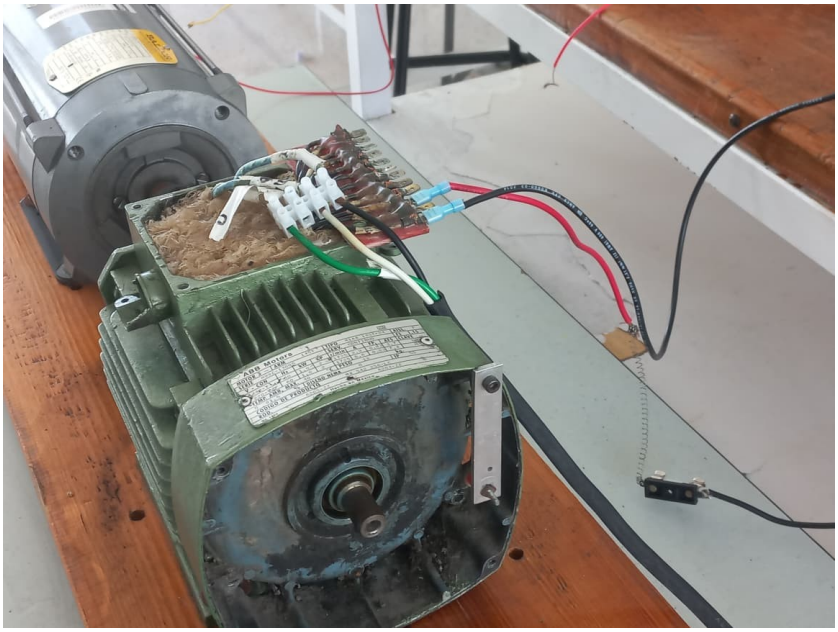


Figura 5. *Motor ABB con las derivaciones del devanado accesibles en la caja de bornes.*

Carga mecánica. Generador de corriente directa Baldor CDP3585 acoplado mecánicamente al eje del motor (Figura 6). Su placa indica 180 V y 9.6 A en la armadura y 1750 rpm, y se tomó una eficiencia de 0.86, dato del autor. La potencia que entrega el generador se disipa en resistencias de potencia conectadas a su salida (Figura 7), de modo que el par que demanda al motor se ajusta con el valor de la resistencia.



Figura 6. *Placa de datos del generador Baldor CDP3585 usado como carga.*



Figura 7. *Resistencias de potencia del banco de carga conectadas a la salida del generador.*

Resistencias de carga. Tres configuraciones definen los niveles de carga nominal del banco, además de la operación en vacío: 75 Ω para el nivel bajo (25 %), 33 Ω para el nivel medio (50 %) y dos resistencias de 50 Ω en paralelo (25 Ω) para el nivel alto (75 %).

Variac trifásico. Autotransformador variable de 3.5 kVA entre la red y el motor, que permite energizar el banco de forma gradual. La red de la unidad académica entrega aproximadamente 123 V por fase.

La Figura 8 muestra el banco armado: el motor ABB acoplado al generador Baldor, la instrumentación montada sobre protoboard junto al motor y la computadora que recibe la telemetría.

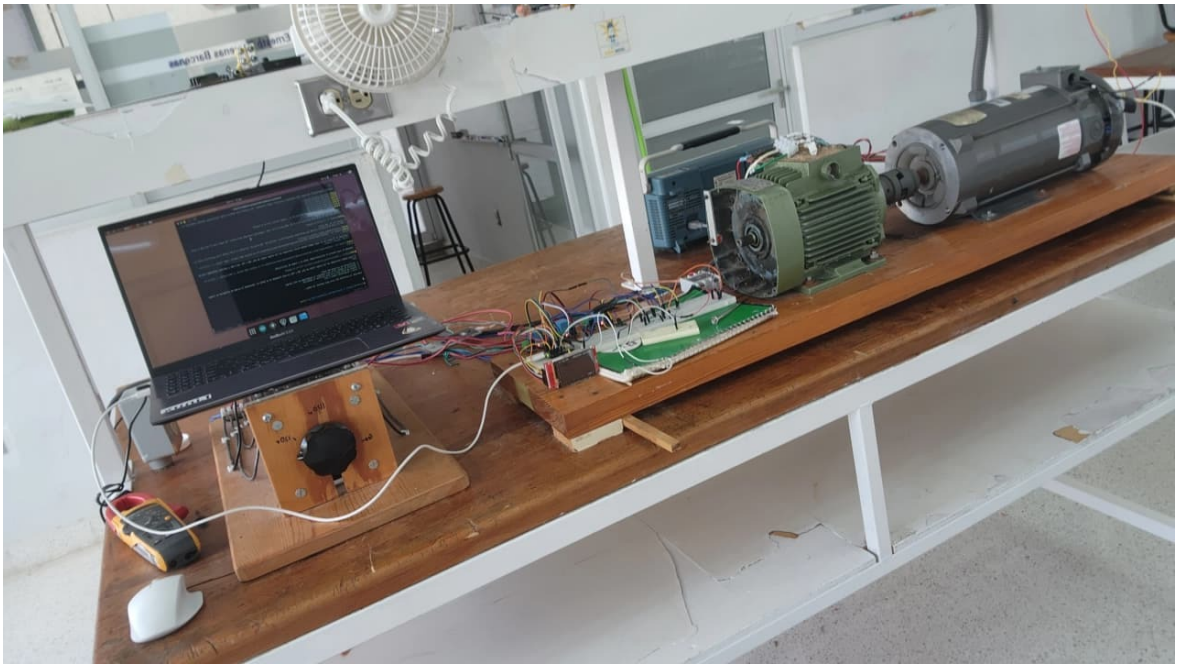


Figura 8. *Banco de pruebas: motor ABB M1AA 145T-D4 acoplado al generador Baldor CDP3585, instrumentación sobre protoboard y computadora de adquisición.*

3.1.4 SOFTWARE Y ENTORNO DE DESARROLLO

Sistema operativo. Todo el desarrollo se realizó en Linux, en la distribución Ubuntu 26.04 LTS, que se utilizó tanto para programar el microcontrolador como para la adquisición de datos, el análisis, el desarrollo de la aplicación y la redacción.

Arduino IDE 2, arduino-cli y núcleo ESP32. El firmware se escribió en C++ con el entorno Arduino IDE 2 y se compiló y cargó también desde la terminal con arduino-cli 1.5.1, utilizando el núcleo para la tarjeta Arduino Nano ESP32 y el sistema operativo de tiempo real FreeRTOS para la planificación de tareas.

Python 3.12. Lenguaje utilizado para la captura de datos por puerto serie, el análisis de las sesiones de ensayo, el entrenamiento de modelos y la generación de figuras, con las bibliotecas NumPy, SciPy, pandas, scikit-learn (Pedregosa et al., 2011), XGBoost (Chen y Guestrin, 2016), m2cgen, Matplotlib y pySerial.

Aplicación de escritorio. Se desarrolló en Python con PySide6 (Qt) y PyQtGraph, con pySerial para el puerto serie, pypdf para leer los documentos del asistente y keyring para guardar claves en el llavero del sistema. Se empaquetó para Debian y Ubuntu como paquete `.deb`, en una versión que usa las bibliotecas de la distribución y otra autónoma construida con PyInstaller.

Asistente de mantenimiento. El modelo de lenguaje local se ejecuta con el motor de inferencia llama.cpp (Gerganov, s. f.), con el modelo Qwen3.5-4B cuantizado a 4 bits (Qwen Team, s. f.-a) y el modelo de *embeddings* Qwen3-Embedding-0.6B (Zhang et al., 2025; Qwen Team, s. f.-b).

Herramientas de apoyo. Control de versiones con Git, elaboración de diagramas con draw.io y conversión de documentos con el editor ONLYOFFICE.

Asistentes de inteligencia artificial generativa. Durante el proyecto se utilizaron dos asistentes de programación basados en modelos de lenguaje, ejecutados desde la terminal: Antigravity CLI (versión 1.2.16) con el modelo Gemini 3.8 Flash y Claude Code (versión 2.1.289) con el modelo Claude Opus 5.5 (Figura 9). Se emplearon como apoyo para escribir y depurar el firmware, los *scripts* de Python y la aplicación, para analizar

registros de ensayo y para la redacción de reportes y de este documento. El diseño experimental, la operación del banco, las mediciones y las decisiones técnicas son del autor, quien es responsable del contenido. La experiencia del proyecto mostró que estas herramientas requieren verificación: algunos reportes intermedios generados con su ayuda contenían datos incorrectos, como la potencia del motor o el modelo del sensor de corriente, y conclusiones más favorables que las que sostenían los registros. Por ello, todas las cifras, tablas y figuras de esta tesis se obtuvieron o se comprobaron directamente a partir de los registros experimentales originales, con *scripts* que permiten regenerarlas (Anexos A–D).

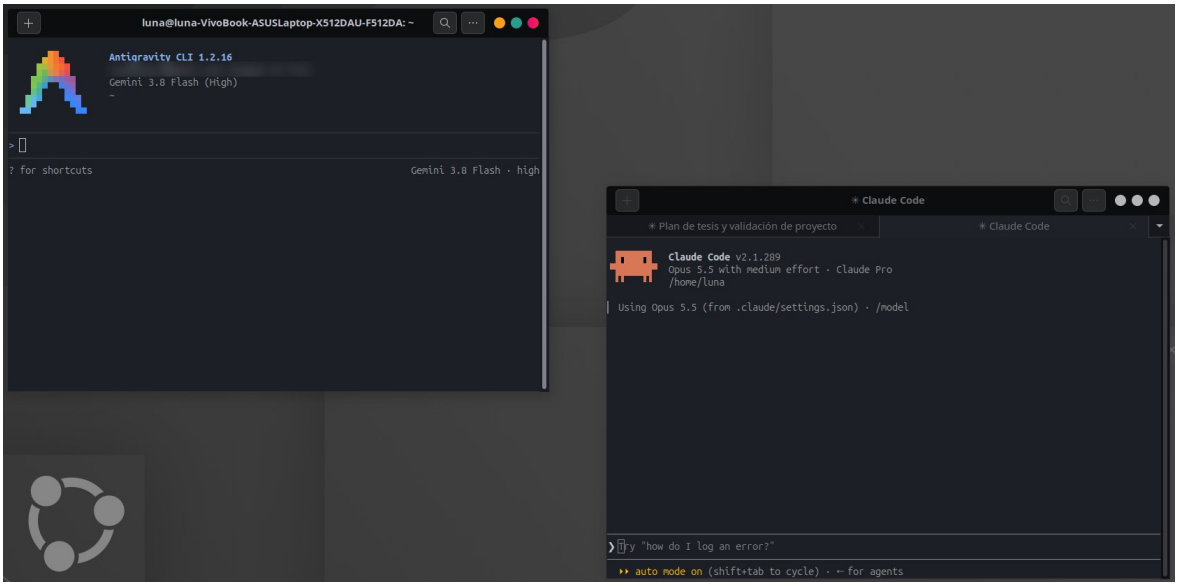


Figura 9. *Entorno de desarrollo en Ubuntu 26.04: Antigravity CLI con Gemini 3.8 Flash (izquierda) y Claude Code con Claude Opus 5.5 (derecha). Se ocultó el correo electrónico de la cuenta.*

3.1.5 EQUIPOS DE LABORATORIO Y HERRAMIENTAS

- **Osciloscopios Tektronix TDS-1002CEDU y MSO 2014B:** referencia para calibrar y verificar los canales de corriente y voltaje del sistema.
- **Sonda diferencial de voltaje Tektronix P5200A y pinza de corriente Tektronix A622:** permiten observar en el osciloscopio el voltaje y la corriente de potencia con aislamiento.
- **Medidor LCR GW Instek LCR-819:** medición de baja resistencia entre las derivaciones del devanado.
- **Multímetro de verdadero valor eficaz y pinza amperimétrica:** verificación de voltajes, continuidad, resistencias y corriente del lazo de inyección en cada sesión.

3.2 DESCRIPCIÓN DE LA APLICACIÓN DE LA METODOLOGÍA PROPUESTA

3.2.1 DISEÑO POR BLOQUES

El sistema se diseñó por bloques funcionales que se desarrollaron y verificaron por separado antes de integrarse (Figura 10). El bloque de potencia comprende la red, el variac, el motor y el generador con su carga

resistiva. El bloque de inyección conecta, a través del relevador de estado sólido y la resistencia limitadora, dos derivaciones de la fase C del motor. El bloque de instrumentación mide las tres corrientes y los tres voltajes de alimentación del motor. El bloque de procesamiento, ejecutado en el ESP32-S3, convierte esas señales en la firma de secuencia negativa y en las dos primeras etapas de la decisión: la detección y la severidad. Por último, el bloque de supervisión, en la computadora, muestra y registra lo que reporta el firmware, controla la inyección con bloqueos de seguridad y genera la recomendación de mantenimiento, que es la tercera etapa.

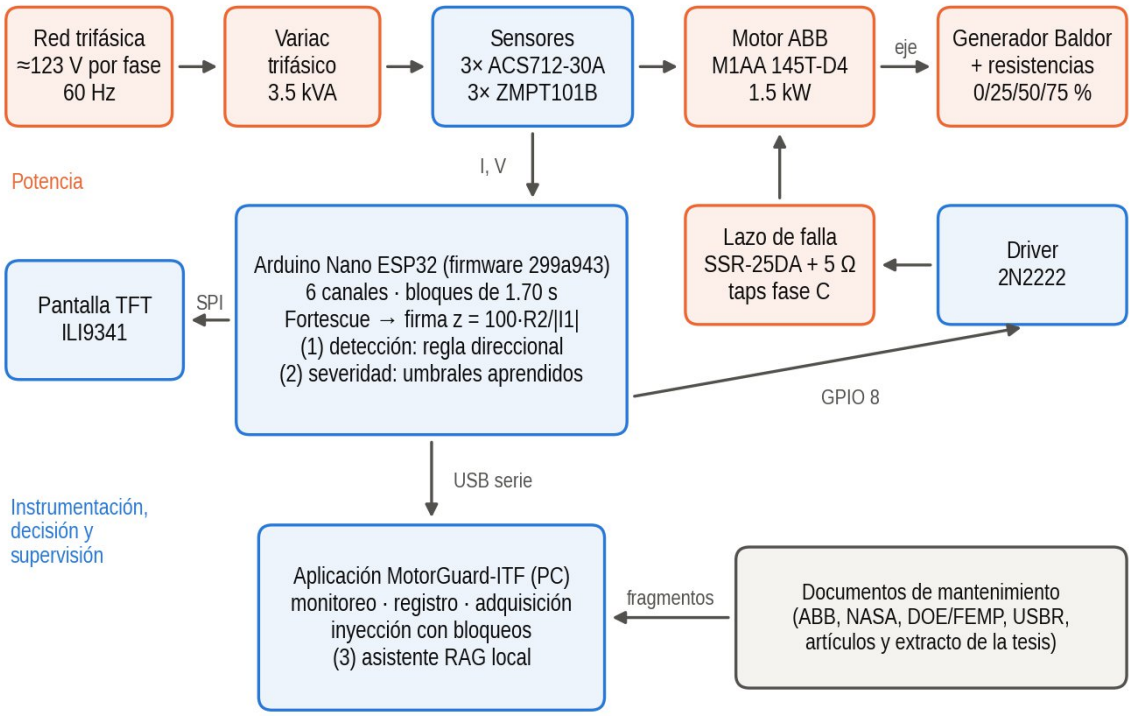


Figura 10. *Diagrama de bloques del sistema final: banco de pruebas, firmware con detección y severidad, y aplicación de escritorio con el asistente de mantenimiento.*

La elección de este método obedeció a dos razones. La primera es que cada bloque tiene sus propios modos de falla y su propio procedimiento de verificación, por lo que separarlos permitió aislar los problemas. La segunda es que la cadena de medición debía validarse antes que el algoritmo de diagnóstico, ya que cualquier error de instrumentación se propaga a los indicadores y puede confundirse con una falla del motor. Por la misma razón, la aplicación y el asistente se diseñaron para no intervenir en la decisión: el diagnóstico es el del firmware validado.

3.2.2 TÉCNICAS DE INVESTIGACIÓN Y VALIDACIÓN

Protocolos declarados. Los estudios que sustentan las conclusiones principales se planearon por escrito antes de obtener o analizar los datos. Cada protocolo fija las hipótesis, los datos, el análisis y el criterio de éxito, y se registró en el control de versiones antes de correrlo, de modo que el identificador del *commit* sirve como evidencia de su fecha. Cuando las condiciones del banco obligaron a cambiar algo, el cambio se registró como

enmienda, también antes de usar los datos afectados. Los análisis que se hicieron después de ver los datos se reportan como **post hoc** y se presentan solo como exploración o como hipótesis para validar después.

Las técnicas empleadas fueron:

- **Calibración contra instrumento de referencia:** comparación de las lecturas del ESP32-S3 con un osciloscopio y sondas aisladas.
- **Caracterización del devanado:** medición de resistencias con el LCR-819 y de los voltajes inducidos entre derivaciones con el motor en marcha, para establecer la fracción de espiras de cada clase de falla.
- **Pares entrelazados:** captura del conjunto de datos alternando un bloque con el motor sano y uno con la falla, para que cada firma se mida contra una referencia sana inmediata.
- **Validación dejando una carga fuera y validación agrupada:** el modelo se entrena sin datos de una carga, o de un grupo de sesión, y se prueba en ella, para evitar la fuga de información entre entrenamiento y prueba.
- **Equivalencia entre la computadora y el firmware:** el detector se reimplementa en Python sobre los fasores crudos de la telemetría y se compara bloque a bloque con la decisión del microcontrolador.
- **Validación ciega en vivo:** un plan aleatorio de pruebas reales y simuladas, generado con una semilla registrada, cuyo orden no se decide durante la sesión.
- **Prueba larga sin falla:** registros prolongados con el motor sano y el lazo desconectado, para medir la tasa real de falsas alarmas por hora.
- **Comparación con modelos congelados:** los modelos de aprendizaje automático se congelan, con su huella SHA verificada, antes de evaluarlos con datos nuevos.
- **Prueba de provocación:** verificación de la integridad de las conexiones de la instrumentación antes de cada sesión.
- **Pruebas automáticas y de resistencia del software:** pruebas unitarias y de integración de la aplicación, contra registros reales y contra un simulador del firmware, y pruebas de operación prolongada con criterios declarados.
- **Evaluación del asistente con un conjunto de preguntas independiente:** el resultado del asistente se mide con preguntas que no se usaron para ajustarlo.
- **Trazabilidad de resultados:** cada cifra reportada se recalculó con *scripts* a partir de los registros originales, en lugar de tomarla de reportes intermedios.

3.2.3 PASOS PARA LOGRAR LA APLICACIÓN

Fase 1: Banco de pruebas. Montaje del motor, del generador y de las resistencias de carga; acondicionamiento de los sensores; construcción del lazo de inyección; caracterización de las derivaciones del devanado y definición de los niveles de carga.

Fase 2: Adquisición y procesamiento. Implementación del muestreo por temporizador de hardware, de la FFT, de las características MCSA y de las componentes simétricas; construcción de los conjuntos de datos experimentales.

Fase 3: Inteligencia artificial. Etapa exploratoria con clasificadores sobre características MCSA (julio a septiembre); clasificador de severidad sobre la firma de secuencia negativa (octubre); estudios de comparación entre la regla física y modelos de aprendizaje automático para la detección.

Fase 4: Firmware del ESP32-S3. Integración en tiempo real de la adquisición, el procesamiento, el detector direccional, el enclavamiento, la severidad embebida, las protecciones del lazo de inyección y la pantalla.

Fase 5: Aplicación de escritorio. Desarrollo de la aplicación de supervisión, registro, adquisición e inyección controlada, con sus pruebas y su empaquetado.

Fase 6: Asistente de mantenimiento. Selección de las fuentes documentales, construcción del índice, integración del modelo de lenguaje local y evaluación.

Fase 7: Validación experimental. Validación ciega en vivo a 0, 25 y 75 % de carga, prueba larga sin falla, estudios de comparación con aprendizaje automático y prueba de la aplicación con el banco. El servidor de telemetría con base de datos, previsto en el plan inicial, queda como trabajo futuro.

3.3 DESARROLLO DE LA METODOLOGÍA

3.3.1 BANCO DE PRUEBAS

3.3.1.1 MOTOR, CARGA Y NIVELES DE OPERACIÓN

El motor se alimenta desde la red a través del variac y acciona el generador Baldor, cuya salida se conecta a las resistencias de carga. Inicialmente se planearon cuatro niveles de carga: 25, 50, 75 y 100 % de la potencia nominal. Durante la preparación del nivel alto, una resistencia de 25 Ω y 700 W conectada sola a la salida del generador se puso al rojo vivo y el variac se sobrecargó, por lo que el ensayo se interrumpió de inmediato. La medición posterior de 163.5 V en la salida del generador permitió estimar que esa resistencia disipaba alrededor de 1.07 kW, muy por encima de su capacidad.

A partir de ese incidente se adoptaron dos medidas. La primera fue repartir la potencia del nivel alto entre dos resistencias de 50 Ω en paralelo, que equivalen a los 25 Ω requeridos con la mitad de la potencia en cada elemento. La segunda fue excluir el nivel de 100 %, ya que, con la eficiencia de 0.86 del generador, operar el motor a su potencia nominal de 1.5 kW requeriría disipar del orden de 1.29 kW en el banco resistivo, por encima del límite que el banco demostró soportar de forma segura. La operación en vacío (0 %) se incorporó en octubre como cuarto nivel. La Tabla 4 resume los niveles de operación empleados.

Tabla 4. Niveles de carga del banco de pruebas.

Nivel	Carga nominal	Resistencia en el generador
Vacío	0 %	Sin carga eléctrica
Bajo	25 %	75 Ω
Medio	50 %	33 Ω
Alto	75 %	2 × 50 Ω en paralelo (25 Ω)

Los porcentajes de la Tabla 4 son valores nominales: el par en el eje no se midió, y la potencia que realmente demanda cada resistencia depende del voltaje del generador, que a su vez varía con el ajuste del variac y con la velocidad del conjunto. El generador no permite cambiar la carga con el motor girando, de modo que cada nivel se ensaya en un tramo separado y los escalones de carga en caliente no pudieron evaluarse.

La Figura 11 muestra el circuito de potencia del banco y la ubicación de los sensores. Cada línea del variac pasa por un sensor ACS712-30A antes de llegar al motor, y cada sensor ZMPT101B mide el voltaje entre su fase y el neutro del variac. El neutro solo sirve como referencia para esos sensores: el motor, conectado en doble estrella, no se conecta a él. El generador Baldor CDP3585 es de imán permanente: su placa indica el tipo PM3548P y deja en blanco los datos del campo (Figura 6).

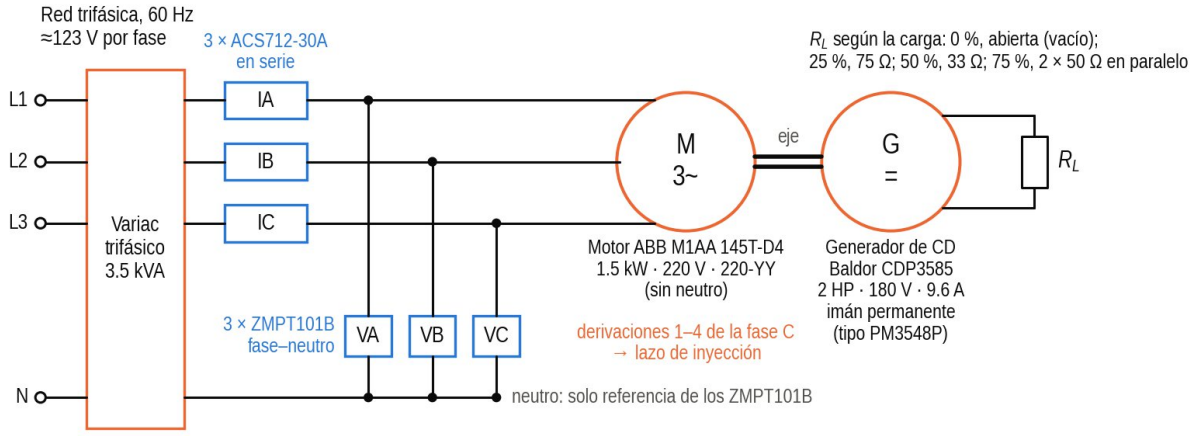


Figura 11. *Circuito de potencia del banco de pruebas: red, variac, sensores de corriente en serie y de voltaje entre fase y neutro, motor en doble estrella y generador de corriente directa con su resistencia de carga.*

3.3.1.2 CARACTERIZACIÓN DEL DEVANADO Y DEFINICIÓN DE LAS CLASES DE FALLA

La documentación disponible del motor indicaba derivaciones correspondientes al 5, 11 y 20 % del devanado. Las primeras pruebas de inyección no produjeron los cambios esperados en la corriente, lo que motivó una caracterización experimental. En una primera etapa se midió con el LCR-819 la resistencia entre pares de terminales de la fase C (Tabla 5). La sección de menor resistencia resultó ser la comprendida entre los terminales 1 y 2, y no la que indicaba la documentación. Además, algunas resistencias medidas no eran aditivas, de modo que este método no bastaba para establecer con certeza la fracción de espiras de cada sección.

Tabla 5. Resistencia entre terminales de la fase C medida con el LCR-819.

Par de terminales	Resistencia (Ω)
1 ↔ 2	0.193
2 ↔ 4	0.452
1 ↔ 4	0.795
2 ↔ 3	1.045
3 ↔ 4	1.600
1 ↔ 3	1.660

Por ello se empleó un segundo método, basado en el voltaje inducido. Con el motor en marcha, cada sección del devanado se comporta como un tramo de la bobina de la fase, y el voltaje que aparece entre dos terminales es proporcional al número de espiras comprendidas entre ellos. Se midieron con multímetro los voltajes entre todos los pares de terminales de las fases C y A (Tabla 6). En ambas fases, las tres secciones consecutivas tienen voltajes prácticamente iguales y sus sumas coinciden con el voltaje entre los terminales extremos.

Tabla 6. Voltaje inducido entre terminales con el motor en marcha y fracción de espiras correspondiente, referida al voltaje de fase de 123 V.

Fase C	Voltaje (V)	Espiras (%)	Fase A	Voltaje (V)	Espiras (%)
1 ↔ 2	8.6	7.0	5 ↔ 6	9.4	7.6
2 ↔ 3	8.6	7.0	6 ↔ 7	8.5	6.9
3 ↔ 4	8.6	7.0	7 ↔ 8	8.5	6.9
1 ↔ 3	17.6	14.3	5 ↔ 7	18.2	14.8

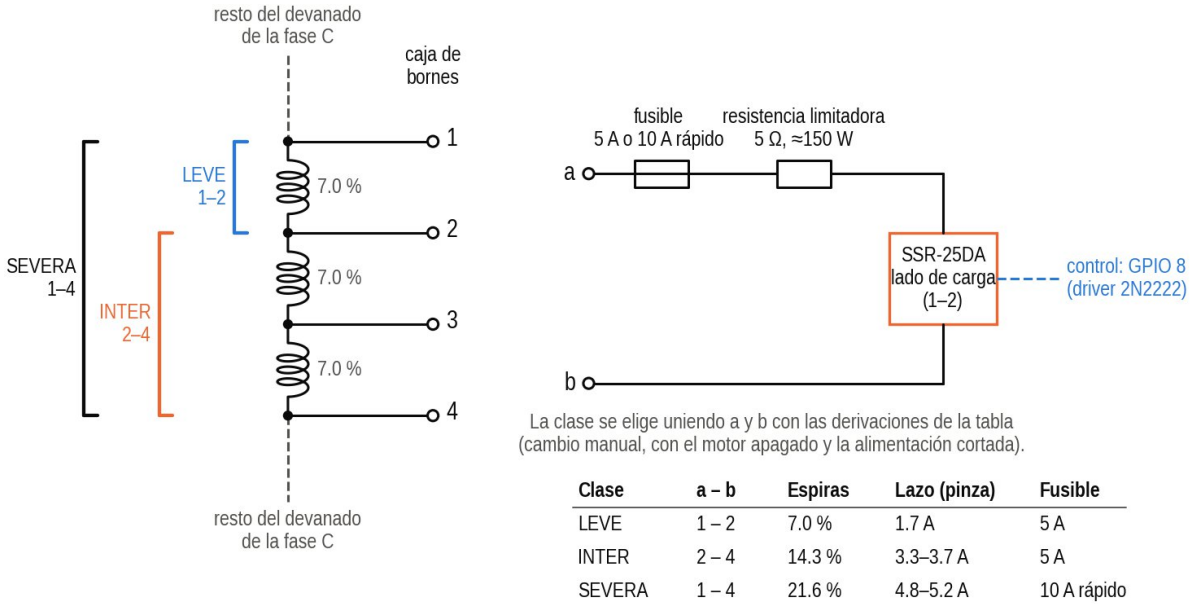
Fase C	Voltaje (V)	Espiras (%)	Fase A	Voltaje (V)	Espiras (%)
2 ↔ 4	17.6	14.3	6 ↔ 8	17.4	14.1
1 ↔ 4	26.6	21.6	5 ↔ 8	27.0	22.0

Con base en estas mediciones se definieron tres clases de falla, todas en la fase C para que la progresión de severidad fuera anidada y se observara en la misma fase: LEVE entre los terminales 1 y 2 (una sección, ≈ 7.0 % de las espiras), INTERMEDIA entre los terminales 2 y 4 (dos secciones, ≈ 14.3 %) y SEVERA entre los terminales 1 y 4 (tres secciones, ≈ 21.6 %). La fracción de espiras se calculó como el cociente entre el voltaje medido y el voltaje de fase de 123 V, con el supuesto de que el voltaje inducido es proporcional al número de espiras. Las derivaciones de la fase A quedan caracterizadas para trabajos futuros.

3.3.1.3 SISTEMA DE INYECCIÓN DE FALLAS

El lazo de inyección conecta en serie el relevador de estado sólido, la resistencia limitadora, el fusible y las dos derivaciones de la clase de falla que se desea reproducir. Al activarse el relevador, el lazo cierra un camino de baja impedancia en paralelo con las espiras seleccionadas, que emula el cortocircuito entre ellas. El diseño del lazo evolucionó durante el proyecto: la resistencia limitadora pasó de 15 Ω y 20 W en el diseño inicial a 10 Ω y 20 W en las campañas de julio a septiembre, y a 5 Ω y aproximadamente 150 W en la configuración final. La corriente del lazo de cada clase se midió con la pinza durante la captura del conjunto de datos (Tabla 10 del Capítulo 4).

La Figura 12 representa las derivaciones de la fase C y el lazo de inyección. Las tres secciones entre los terminales 1 y 4 tienen la misma fracción de espiras (≈ 7.0 %), de modo que las clases quedan anidadas: la LEVE abarca una sección, la INTERMEDIA dos y la SEVERA tres. La clase se cambia reconectando a mano los extremos del lazo, con el motor apagado y la alimentación cortada.



Clase	a – b	Espiras	Lazo (pinza)	Fusible
LEVE	1 – 2	7.0 %	1.7 A	5 A
INTER	2 – 4	14.3 %	3.3–3.7 A	5 A
SEVERA	1 – 4	21.6 %	4.8–5.2 A	10 A rápido

Figura 12. *Derivaciones de la fase C y lazo de inyección de falla: fusible, resistencia limitadora y lado de carga del relevador entre las dos derivaciones de la clase seleccionada. La corriente del lazo es la medida con la pinza durante la captura del conjunto de datos.*

Durante la puesta en marcha se detectó que el primer relevador estaba dañado: su lado de control respondía, pero el elemento de potencia no conducía, y solo circulaban unos 30 mA debidos a la red de protección interna del dispositivo. En octubre, la corriente del lazo INTERMEDIA bajó de forma progresiva de 3.7 a 3.1 A hasta que se sustituyó el relevador por el de repuesto, y la resistencia limitadora se soltó en dos capturas de la clase SEVERA. Estos episodios dejaron tres prácticas: medir la corriente del lazo con la pinza al inicio, a la mitad y al final de cada sesión, montar el relevador sobre un disipador y usar terminales de ojillo.

El firmware protege el devanado y la resistencia con tres límites: el relevador no puede permanecer encendido más de 12 s continuos, debe transcurrir un tiempo mínimo de 2 s entre activaciones y la inyección se aborta si la corriente de la fase supera 8 A. La aplicación de escritorio añade sus propios bloqueos, más estrictos (sección 3.3.5).

El relevador se activa con un transistor 2N2222 en configuración de interruptor del lado de tierra (Figura 13). La entrada de control del relevador se alimenta con los 5 V del pin VBUS de la tarjeta, y el pin D5 (GPIO 8) satura el transistor a través de la resistencia de base de 1 kΩ. El relevador no tiene una red de amortiguamiento RC en su salida.

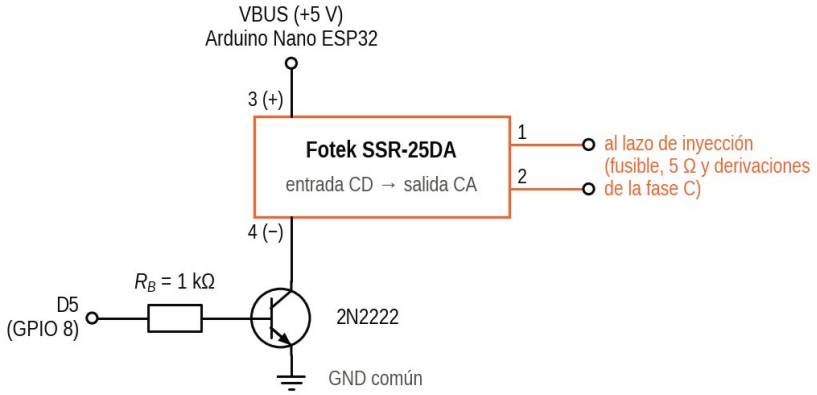
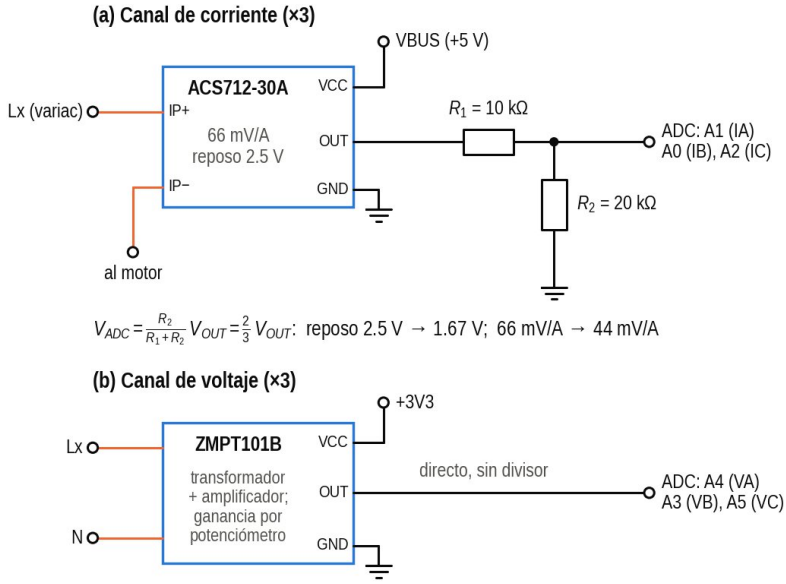


Figura 13. *Circuito de activación del relevador de estado sólido: el pin D5 (GPIO 8) satura el transistor 2N2222, que cierra el lado de control del SSR-25DA alimentado desde VBUS.*

3.3.1.4 ACONDICIONAMIENTO Y CALIBRACIÓN DE SENSORES

Los sensores se probaron primero de forma individual y después como sistema trifásico. En junio de 2026, el sistema de seis canales se verificó con el osciloscopio TDS-1002CEDU sobre una carga resistiva de 100 Ω, con lecturas de 125 V, 1.25 A y factor de potencia de 0.998. En julio, el canal de la fase C se calibró contra el osciloscopio MSO 2014B con la sonda P5200A y la pinza A622, con el motor acoplado al generador sin carga eléctrica (Tabla 7).

La Figura 14 muestra el acondicionamiento de los dos tipos de canal. Los tres sensores ACS712-30A se alimentan con los 5 V del pin VBUS y su salida pasa por el divisor de 10 kΩ y 20 kΩ, que la lleva al intervalo del ADC: el reposo de 2.5 V queda en 1.67 V y la sensibilidad de 66 mV/A en 44 mV/A. Los ZMPT101B se alimentan a 3.3 V y entran al ADC sin divisor. Dos de los canales de voltaje (A4 y A5) están en el ADC2 del ESP32-S3, que no puede usarse con el radio encendido (Espressif Systems, 2026); por eso el Wi-Fi permanece deshabilitado.



Canal	Pin	GPIO	ADC
VA	A4	11	ADC2
VB	A3	4	ADC1
VC	A5	12	ADC2
IA	A1	2	ADC1
IB	A0	1	ADC1
IC	A2	3	ADC1

ADC de 12 bits, atenuación de 11 dB. VA y VC están en el ADC2, que no funciona con el radio encendido: el Wi-Fi está deshabilitado.

Figura 14. *Acondicionamiento de las señales: (a) canal de corriente con el ACS712-30A y el divisor resistivo; (b) canal de voltaje con el ZMPT101B. A la derecha, la asignación de los seis canales a los pines, GPIO y convertidores del ESP32-S3.*

Tabla 7. Calibración del canal de la fase C contra osciloscopio (julio de 2026).

Magnitud	Osciloscopio	ESP32-S3	Error
Voltaje eficaz	124 V	128.87 V	+3.9 %
Corriente eficaz	3.92 A	4.004 A	+2.1 %

El 6 de octubre se recalibraron las corrientes de los tres canales (firmware `2dff915`); todas las capturas y validaciones del sistema final se hicieron con esa calibración. Durante las capturas, el canal de corriente de la fase C perdió ganancia dos veces por conexiones (16 % por la protoboard y 12 % por el contacto del divisor), y en ambos casos se corrigió la conexión sin recalibrar; el segundo contacto se soldó y volvió a coincidir con la pinza. La verificación de los seis canales contra osciloscopio con el firmware final no se realizó antes de la entrega y se reporta como limitación (Capítulo 5).

3.3.1.5 CONEXIONADO E INTEGRIDAD DE LA SEÑAL

La instrumentación se montó en una protoboard con cables de conexión tipo Dupont (Figuras 16 y 17). El análisis de las sesiones del 1 de octubre de 2026 mostró que la calidad de esas conexiones era la principal fuente de ruido del indicador de falla (sección 4.1.3.3): el sensor ACS712 tiene salida ratiométrica, de modo que una resistencia de contacto variable en su alimentación o en su referencia de tierra se traduce directamente en variaciones de la corriente medida. Se adoptaron dos medidas: rehacer las conexiones en una protoboard mejorada con cable rígido y rieles de tierra separados, y ejecutar una prueba de provocación antes de cada sesión para detectar conexiones defectuosas. No se construyó una placa soldada: todos los resultados de esta tesis se obtuvieron con la instrumentación en protoboard, y el montaje soldado queda como trabajo futuro (Capítulo 5).

La Figura 15 resume las conexiones del Arduino Nano ESP32 con la numeración de su serigrafía y el GPIO correspondiente, con el que se compila el firmware (opción de numeración por GPIO): las seis entradas

analógicas, la salida del relevador, el bus SPI de la pantalla, que se alimenta a 3.3 V, y el puerto USB hacia la computadora.

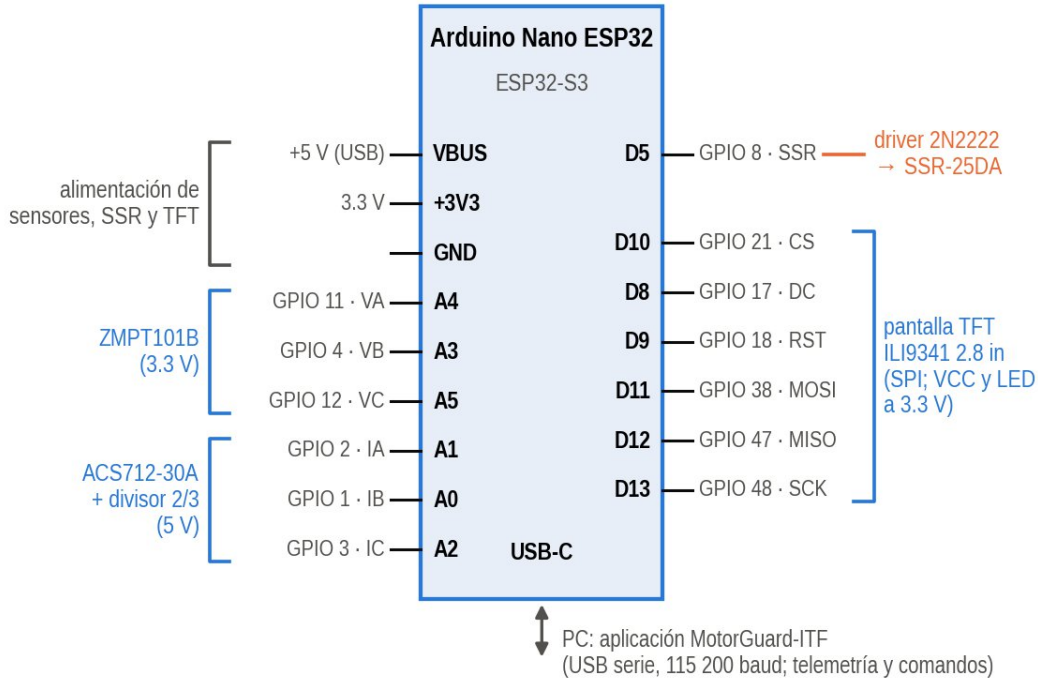


Figura 15. *Conexiones del Arduino Nano ESP32: entradas de los sensores, alimentación desde VBUS y 3V3, salida al circuito del relevador, bus SPI de la pantalla TFT y enlace USB con la aplicación.*

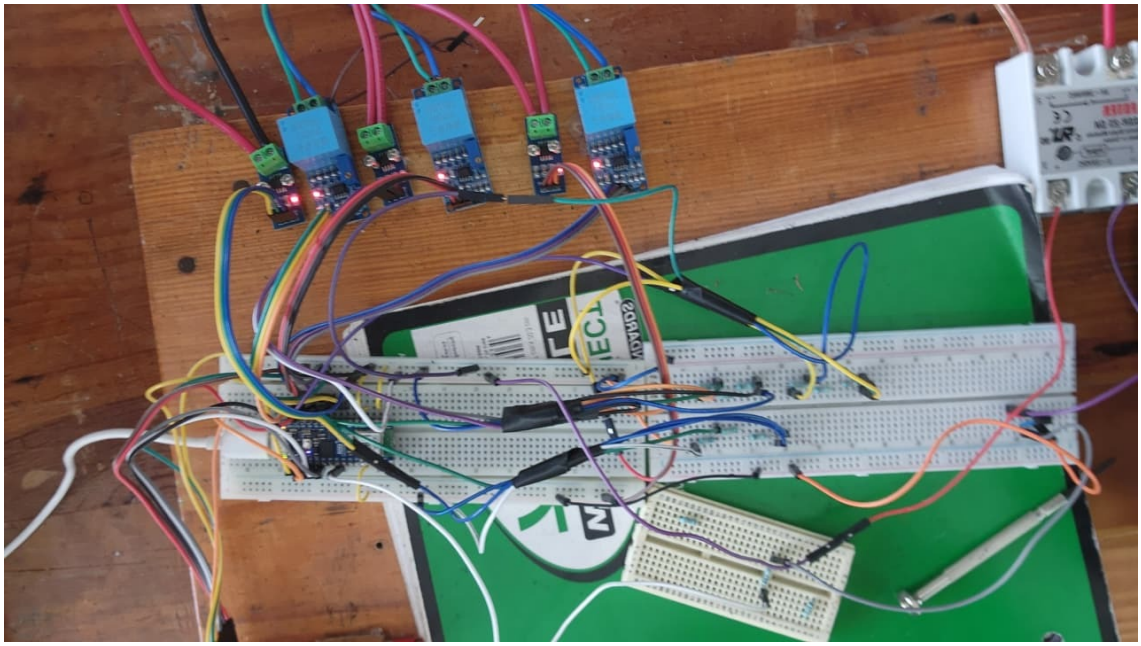


Figura 16. *Montaje de la instrumentación sobre protoboard: sensores ZMPT101B y ACS712, Arduino Nano ESP32 y relevador de estado sólido.*

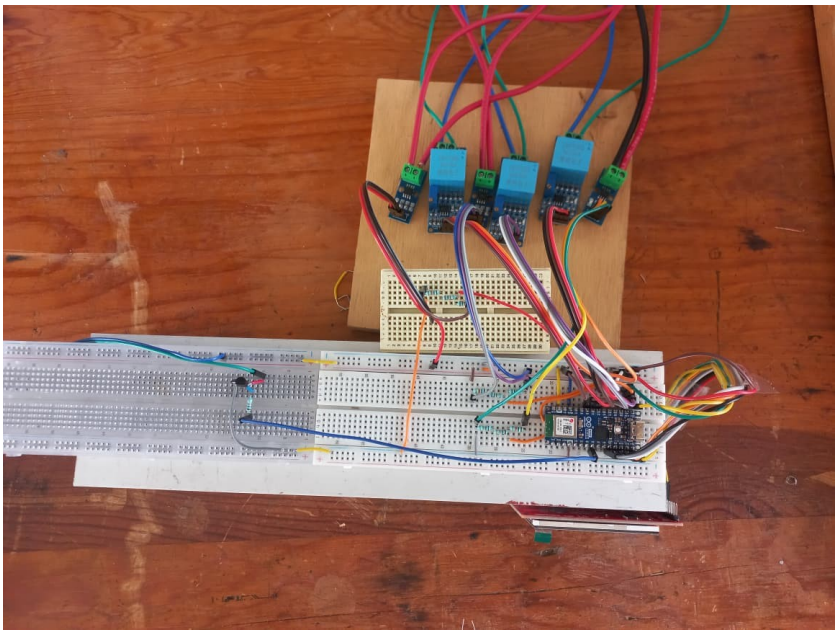


Figura 17. *Protoboard con los tres módulos ZMPT101B de voltaje y el Arduino Nano ESP32.*

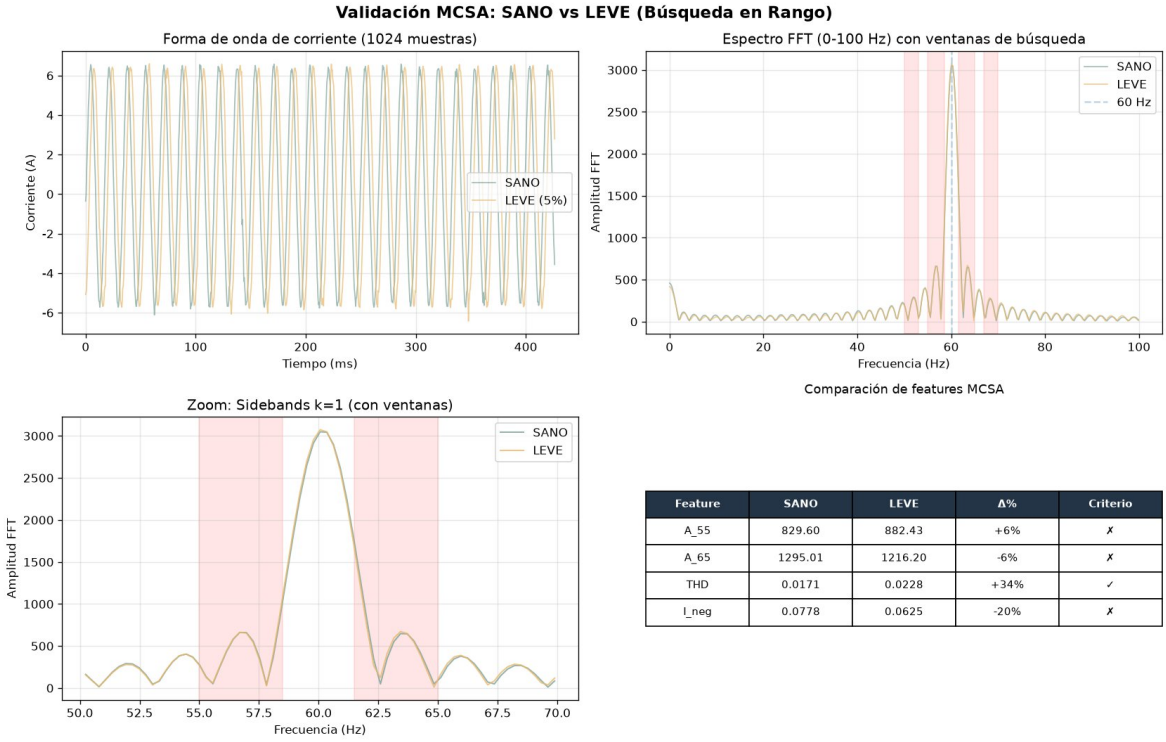
3.3.2 ADQUISICIÓN Y PROCESAMIENTO DE SEÑALES

3.3.2.1 MUESTREO

El muestreo se dispara con un temporizador de hardware cada 416 µs, lo que equivale a una frecuencia de muestreo de 2403.85 Hz por canal. Una tarea de FreeRTOS asignada a uno de los núcleos lee en cada disparo los seis canales y los almacena en un búfer doble de 4096 muestras por canal: mientras un bloque se llena, el otro se procesa, de modo que la adquisición no se interrumpe. Cada bloque abarca 1.70 s, lo que da una resolución espectral de 0.59 Hz. El Wi-Fi del microcontrolador se mantiene deshabilitado por el uso del ADC2 (sección 3.3.1.4).

3.3.2.2 CARACTERÍSTICAS MCSA

Sobre la corriente de la fase C se calcula la FFT de 4096 puntos con ventana de Hann. De su espectro se extraen la amplitud de la componente fundamental y las amplitudes máximas en ventanas de búsqueda a ambos lados de la fundamental, en las regiones donde aparecen las bandas laterales $f_1(1 \pm 2s)$, además de la distorsión armónica total (THD). La frecuencia fundamental se estima en cada bloque mediante interpolación espectral (Grandke, 1983). En el sistema final, estas características se conservan en la telemetría como información y para los estudios de comparación, pero no intervienen en la decisión. La Figura 18 muestra un ejemplo de la comparación espectral entre motor sano y con falla LEVE obtenida con la versión de un solo canal del sistema.



Feature	SANO	LEVE	Δ%	Criterio
A_55	829.60	882.43	+6%	✗
A_65	1295.01	1216.20	-6%	✗
THD	0.0171	0.0228	+34%	✓
I_neg	0.0778	0.0625	-20%	✗

Figura 18. *Comparación de forma de onda y espectro de la corriente de la fase C entre motor sano y con falla LEVE (sistema de un canal, julio de 2026).*

3.3.2.3 COMPONENTES SIMÉTRICAS Y FIRMA DE SECUENCIA NEGATIVA

En cada bloque se calculan los fasores de la componente fundamental de las tres corrientes y los tres voltajes, y a partir de ellos las componentes de secuencia positiva y negativa (sección 2.5). El residuo de secuencia negativa R_2 se normaliza con la magnitud de la corriente de secuencia positiva para obtener la firma

$$z = 100\frac{R_2}{|I_1|} \tag{11}$$

expresada en puntos porcentuales (pp) de $|I_1|$ y con su ángulo medido respecto de $\arg(V_1)$. La versión inicial del detector utilizaba solo la magnitud del residuo, y el análisis de las sesiones del 1 de octubre mostró que esa magnitud dependía de la dirección del desplazamiento de I_2, de modo que una misma falla podía aumentar o disminuir el indicador. El sistema final trabaja con el cambio vectorial de la firma, Δz, respecto de una referencia sana reciente, y con su proyección sobre la dirección característica de la falla (sección 3.3.4). En el conjunto de datos multiclase, el cambio de la firma producido por cada par se expresa como $\Delta R_2 = (R_{2,\text{falla}} - R_{2,\text{sano}})/|I_1|$.

3.3.2.4 CONJUNTOS DE DATOS

Durante el proyecto se construyeron varios conjuntos de datos, cada uno con el *pipeline* vigente en su momento (Tabla 8). Los de julio a septiembre se capturaron con una calibración y características que no son compatibles con el firmware final; documentan la etapa exploratoria de inteligencia artificial.

Tabla 8. Conjuntos de datos experimentales construidos durante el proyecto.

Periodo	Contenido	Muestras	Uso
Julio 2026	SANO y LEVE, un canal, carga baja	199	Primer XGBoost con datos reales
28 ago – 3 sep 2026	SANO, LEVE, INTERMEDIA y SEVERA, tres niveles de carga	3160	XGBoost de cuatro clases
11 – 15 sep 2026	SANO y LEVE intercalados, tres niveles de carga	900	Random Forest binario, validación agrupada
23 – 24 sep 2026	SANO y LEVE intercalados, tres niveles de carga	900	Prueba de transferencia entre campañas
1 oct 2026	Registros continuos de base y pulsos LEVE, carga media	—	Ruido de base y primera firma
6 – 8 oct 2026	**Pares entrelazados LEVE, INTERMEDIA y SEVERA, cuatro cargas**	**597 pares (607 con la repetición y los descartes)**	**Firma por clase y carga, clasificador de severidad y estudio 2**
6 – 16 oct 2026	Registros continuos de las validaciones en vivo y de la prueba larga	—	Validación ciega, falsas alarmas, estudio 1 y validación ciega 2

El conjunto de datos multiclase de octubre se capturó con el protocolo de pares entrelazados del *script* de captura multiclase (versión 4.2): cada par consta de un bloque con el motor sano seguido de un bloque con el relevador encendido, para una sola severidad por grupo de derivaciones y para las cuatro cargas, con la calibración del 6 de octubre. Las exclusiones (pares sin inyección por apertura del lazo o resistencia suelta) y los incidentes de instrumentación se documentaron en el propio registro, y los originales se conservaron en un directorio aparte. En la captura del 11 de septiembre se detectó además que un archivo de datos había sido modificado al abrirse y guardarse con una hoja de cálculo; se restauró desde su respaldo y se estableció la regla de manipular los archivos de datos únicamente con herramientas de texto o con Python.

3.3.3 MODELOS DE INTELIGENCIA ARTIFICIAL

La inteligencia artificial se incorporó en tres momentos. Primero, en una etapa exploratoria se evaluaron clasificadores entrenados con características MCSA para decidir directamente el estado del motor. Segundo, en octubre se desarrolló el clasificador de severidad, que es la etapa de aprendizaje automático del sistema final. Tercero, se realizaron estudios con protocolo declarado para decidir si un modelo debía sustituir a la regla física en la detección. La recomendación de mantenimiento con inteligencia artificial generativa se describe en la sección 3.3.6.

3.3.3.1 ETAPA EXPLORATORIA

Antes de disponer del banco completo se desarrolló un simulador en Python que genera corrientes trifásicas con las componentes espectrales esperadas para cada clase de falla, ruido, deriva y desbalance de fase, y con 3200 muestras sintéticas se entrenaron modelos XGBoost y Random Forest para construir el *pipeline* de características, entrenamiento y exportación a código C. Con el banco operativo se entrenaron modelos XGBoost con datos reales, primero binarios (julio) y después de cuatro clases con 3160 muestras de nueve sesiones (septiembre), con partición aleatoria. Para el microcontrolador se eligió un Random Forest binario de 45 árboles con profundidad máxima de 6 y cinco características normalizadas, que se tradujo a C con la biblioteca m2cgen y luego a una tabla de nodos de 12 bytes en la memoria de programa (18.5 KB, con una diferencia máxima de 1.0×10^{-7} frente a Python). Su evaluación con partición aleatoria, validación agrupada (Roberts et al., 2017) y transferencia entre las campañas de septiembre se presenta en la sección 4.1.3. Al revisar la construcción de los conjuntos se identificaron condiciones que favorecen que el modelo aprenda a reconocer la sesión además de la falla (Kaufman, Rosset y Perlich, 2011), y una auditoría de septiembre encontró que el filtro temporal de entonces dependía del estado del relevador de inyección, es decir, de la propia etiqueta; esa dependencia se eliminó.

3.3.3.2 CLASIFICADOR DE SEVERIDAD

El clasificador de severidad es la segunda etapa del sistema: clasifica solo pares con falla, porque la detección es la etapa previa. Su diseño se declaró antes de ver los resultados (*commit* `dcff8fe`) y compara tres modelos:

- **M1 (modelo primario):** dos umbrales sobre la magnitud $|\Delta R_2|$; cada umbral es el punto medio entre clases vecinas, o el de máxima exactitud si se traslapan.
- **M2:** regresión logística sobre $[|\Delta R_2|, |I_1|]$.
- **M3:** árbol de decisión de profundidad máxima 3 sobre $[|\Delta R_2|$, proyección, $|I_1|]$.

La validación principal (V1) consiste en dejar una carga fuera: se entrena con tres cargas y se prueba con la cuarta, rotando las cuatro. La validación V2 entrena con el bloque principal y prueba con una repetición de la clase LEVE capturada en otra sesión. Se eligió M1 para embeberlo en el firmware por ser el más simple de implementar y de explicar; sus umbrales finales, entrenados con todo el bloque, son de 1.626 pp entre LEVE e INTERMEDIA y de 4.471 pp entre INTERMEDIA y SEVERA. Los resultados se presentan en la sección 4.1.5.

3.3.3.3 ESTUDIOS DE COMPARACIÓN PARA LA DETECCIÓN

Para decidir si la detección debía hacerla un modelo de aprendizaje automático se realizaron tres estudios con protocolo declarado, cuyos resultados se presentan en la sección 4.1.9:

1. **Estudio 1 (commit `459bee9`):** la regla del firmware frente a un Random Forest y un modelo de *gradient boosting* sobre características de Fortescue, con registros continuos de 13 grupos (los 4

tramos de la prueba larga y 9 sesiones en vivo a 75 %), evaluados dejando un grupo fuera. El criterio declarado exigía intervalos de confianza de 95 % sin traslape.

2. **Estudio 2 (commit `61c09f1`, enmienda `dd93b67`):** generalización entre cargas con los 607 pares del conjunto multiclase, entrenando a 75 % y probando en las otras cargas (E1) o dejando una carga fuera (E2), para la regla, el Random Forest de Fortescue y un Random Forest con características MCSA absolutas y relativas.
3. **Validación ciega 2 (commit `54ecef3`, enmienda `769cce9`):** el Random Forest del estudio 1, congelado y verificado por su huella SHA, frente a la regla, con datos nuevos y continuos a 0, 25, 50 y 75 %. Las hipótesis declaradas fueron H1 (el modelo reduce las falsas alarmas) y H2 (el modelo conserva la detección de LEVE a baja carga), y el análisis se corrió una sola vez después del último registro.

Además, el ruido de la firma y 1904 variantes del detector se analizaron fuera de línea con un protocolo declarado (*commit* `0518dd4`), con la desviación de Allan y una rejilla de configuraciones evaluadas dejando un grupo fuera.

3.3.4 FIRMWARE ESP32-S3

El firmware se desarrolló de manera iterativa. Las primeras versiones, de un solo canal, resolvieron problemas de asignación de pines, filtrado y calibración, e incorporaron un servidor web con punto de acceso Wi-Fi para controlar el banco. Las versiones posteriores ampliaron la adquisición a seis canales, añadieron la FFT de alta resolución, las componentes simétricas, el CUSUM y la pantalla TFT, y desactivaron el Wi-Fi por su interferencia con el ADC2. El 7 de octubre se embebió el detector direccional como decisión principal (*commit* `81657f4`) y el 8 de octubre se agregaron el enclavamiento y la severidad (*commit* `299a943`), que es la versión final usada en todas las validaciones y que no se modificó después. En cada bloque de 1.70 s el firmware ejecuta las siguientes etapas:

- **Detección de motor encendido:** el motor se considera en operación cuando las tres fases presentan corriente de 60 Hz durante varios bloques consecutivos; al encenderse inicia un periodo de calentamiento de 60 s sin decisiones.
- **Procesamiento:** valores eficaces, FFT y características MCSA, fasores, componentes simétricas y firma z.
- **Detección direccional:** la firma se compara con una referencia móvil formada por la mediana, componente por componente, de los bloques entre 11 y 6 s atrás; el cambio Δz se proyecta sobre la dirección de falla de −81.4°, y la alarma se confirma cuando la proyección P supera 0.40 pp de $|I_1|$ durante 3 bloques consecutivos (unos 5 s).
- **Enclavamiento:** al confirmar la alarma se congela la referencia previa a la falla, para que una falla sostenida no se absorba en la referencia móvil; la alarma se libera cuando la proyección contra esa base cae por debajo de max(0.20 pp, 0.35 veces la proyección de confirmación) durante 3 bloques.
- **Severidad:** la mediana de $|\Delta z|$ de los 3 bloques de confirmación se compara con los umbrales de 1.626 y 4.471 pp; mientras la alarma sigue enclavada, la severidad solo puede aumentar, porque una falla entre espiras no se repara sola.
- **Salidas:** actualización de la pantalla, telemetría por USB y protecciones del lazo de inyección.

La Figura 19 resume el procesamiento que el firmware aplica a cada bloque de 1.70 s, desde el muestreo hasta la decisión, y la Figura 20, los estados del motor y del detector con las condiciones de cada transición. El CUSUM y el Random Forest espectral heredados se siguen calculando, pero solo se informan en la telemetría.

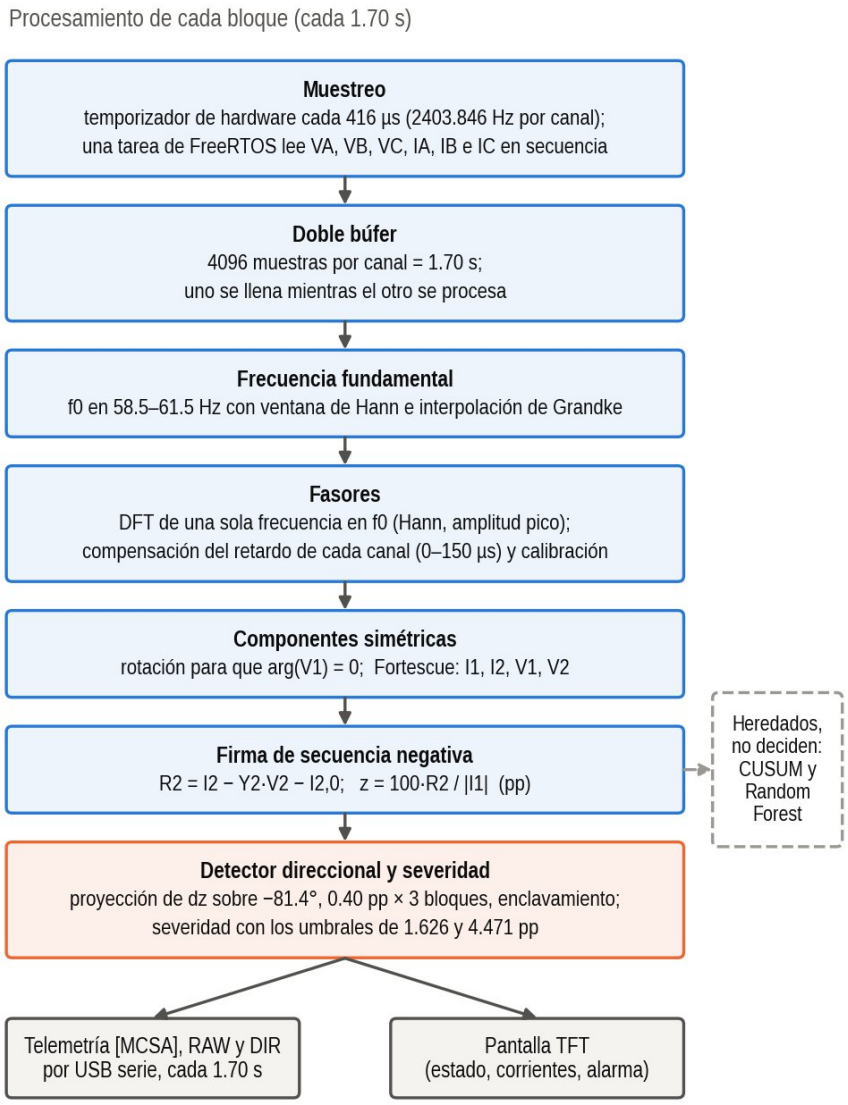


Figura 19. *Procesamiento de un bloque en el firmware 299a943: muestreo, fasores, componentes simétricas, firma de secuencia negativa y detector direccional con severidad.*

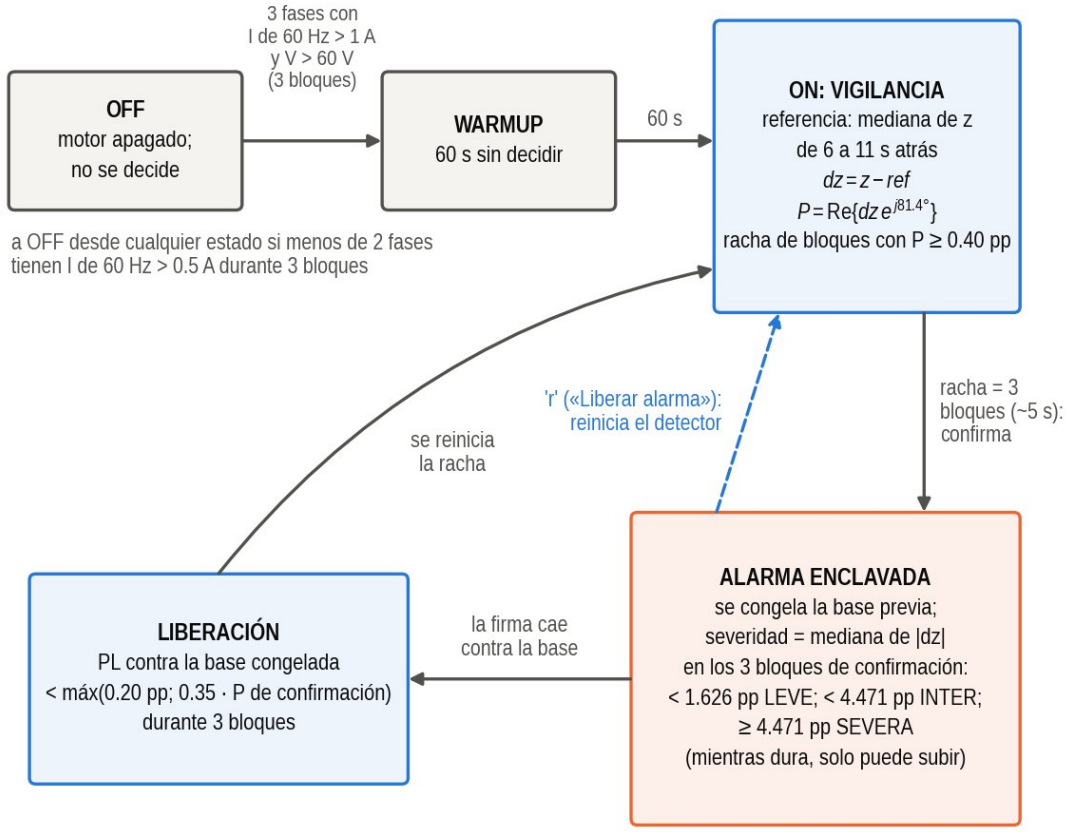


Figura 20. *Estados del motor y del detector direccional en el firmware 299a943: arranque, calentamiento, vigilancia, alarma enclavada con su severidad y liberación.*

El Listado 1 reproduce la función del detector direccional. Los parámetros se ajustaron con el bloque LEVE del 6 de octubre y el propio código advierte que no deben modificarse sin repetir la validación ciega. El detector no utiliza ninguna variable del estado del relevador; esa independencia se verificó con pruebas automáticas y con los ensayos de puente manual. Las constantes del módulo, su llamada desde el lazo principal y las protecciones del relevador se reproducen en el Anexo A, y el clasificador de severidad que fijó los umbrales, en el Anexo C.

Listado 1. *Función del detector direccional con enclavamiento y severidad en el firmware final (firmware/PRUEBA_MODELO_1/detector_dir.cpp, líneas 34–123).*

```
int dir_step(DetectorDir* st, uint32_t t_ms, float r2_re, float r2_im, float i1_mag) {
    if (i1_mag < 1e-6f) return st->sev;
    float z_re = r2_re / i1_mag * 100.0f;
    float z_im = r2_im / i1_mag * 100.0f;

    // Referencia con los bloques previos dentro de [t - 11 s, t - 6 s]
    float ref_re[DIR_HIST_N], ref_im[DIR_HIST_N];
    int m = 0;
    for (int i = 0; i < st->n; i++) {
        uint32_t edad = t_ms - st->t_ms[i];
        if (edad <= DIR_REF_DESDE_MS && edad >= DIR_REF_HASTA_MS) {
            ref_re[m] = st->z_re[i];
            ref_im[m] = st->z_im[i];
            m++;
```

```
        }
    }

    // Agregar el bloque actual y descartar los de mas de 30 s
    int k = 0;
    for (int i = 0; i < st->n; i++) {
        if (t_ms - st->t_ms[i] <= DIR_HIST_MS) {
            st->t_ms[k] = st->t_ms[i]; st->z_re[k] = st->z_re[i]; st->z_im[k] = st->z_im[i]; k++;
        }
    }
    if (k == DIR_HIST_N) {  // historial lleno: se descarta el mas antiguo
        memmove(st->t_ms, st->t_ms + 1, (k - 1) * sizeof(uint32_t));
        memmove(st->z_re, st->z_re + 1, (k - 1) * sizeof(float));
        memmove(st->z_im, st->z_im + 1, (k - 1) * sizeof(float));
        k--;
    }
    st->t_ms[k] = t_ms; st->z_re[k] = z_re; st->z_im[k] = z_im;
    st->n = k + 1;

    // Detector con referencia movil (sin referencia suficiente no se modifica la racha)
    st->ref_ok = (m >= 2);
    float rre = 0.0f, rim = 0.0f;
    if (st->ref_ok) {
        rre = mediana(ref_re, m);
        rim = mediana(ref_im, m);
        st->d_re = z_re - rre;
        st->d_im = z_im - rim;
        st->proy = proyectar(st->d_re, st->d_im);
        st->racha = (st->proy >= DIR_UMBRAL) ? st->racha + 1 : 0;
        empujar(st->mag3, sqrtf(st->d_re * st->d_re + st->d_im * st->d_im));
    }

    if (!st->confirmada) {
        // Confirmacion: se congela la referencia pre-falla y se clasifica la severidad
        if (st->ref_ok && st->racha >= DIR_PERSISTENCIA) {
            float tmp[DIR_PERSISTENCIA];
            memcpy(tmp, st->mag3, sizeof(tmp));
            st->confirmada = 1;
            st->base_re = rre;
            st->base_im = rim;
            st->proy_conf = st->proy;
            st->proy_lat = st->proy;
            st->mag_lat = mediana(tmp, DIR_PERSISTENCIA);
            st->sev = clasificar_sev(st->mag_lat);
            memset(st->maglat3, 0, sizeof(st->maglat3));
            st->nlat = 0;
            st->bajos = 0;
        }
        return st->sev;
    }

    // Alarma enclavada: firma contra la base congelada
    float dl_re = z_re - st->base_re, dl_im = z_im - st->base_im;
    st->proy_lat = proyectar(dl_re, dl_im);
    st->mag_lat = sqrtf(dl_re * dl_re + dl_im * dl_im);
    empujar(st->maglat3, st->mag_lat);
    if (++st->nlat >= DIR_PERSISTENCIA) {
        // La severidad solo escala (una falla entre espiras no se repara sola)
        float tmp[DIR_PERSISTENCIA];
        memcpy(tmp, st->maglat3, sizeof(tmp));
        int s = clasificar_sev(mediana(tmp, DIR_PERSISTENCIA));
        if (s > st->sev) st->sev = s;
    }

    float nivel = DIR_LIBERA_FRAC * st->proy_conf;
    if (nivel < 0.5f * DIR_UMBRAL) nivel = 0.5f * DIR_UMBRAL;
    st->bajos = (st->proy_lat < nivel) ? st->bajos + 1 : 0;
    if (st->bajos >= DIR_PERSISTENCIA) {
        // Falla retirada: se libera la alarma; la racha se reinicia para no re-disparar
        st->confirmada = 0;
```

```
        st->sev = 0;
        st->racha = 0;
        st->bajos = 0;
    }
    return st->sev;
}
```

La telemetría de cada bloque es una línea de texto que incluye el estado del motor y del relevador, los voltajes y corrientes por fase, los fasores de secuencia y el campo del detector direccional, con la proyección P, la confirmación C, la referencia, la severidad, la magnitud de la firma ML y la proyección enclavada PL. El firmware atiende además comandos por el puerto serie: encender (`1`) y apagar (`0`) el relevador, reiniciar el detector (`r`), consultas de solo lectura de la forma de onda (`W`), del espectro (`f`) y de la distorsión (`s`), y comandos de servicio, como el borrado de la memoria no volátil, que la aplicación no envía nunca.

La pantalla TFT muestra el estado del sistema en el propio banco: el motor apagado, el calentamiento tras el arranque, el estado sano y las alarmas con su severidad, junto con las corrientes y voltajes de la fase vigilada (Figura 21). El indicador «Conf» es el avance de la persistencia del detector: 0 % sin bloques sobre el umbral, 33 % y 67 % con uno y dos bloques consecutivos, y 100 % con la alarma confirmada y enclavada. Durante una inyección, la pantalla pasa así del estado sano con 0 % a la alarma con 100 % en unos 5 s.

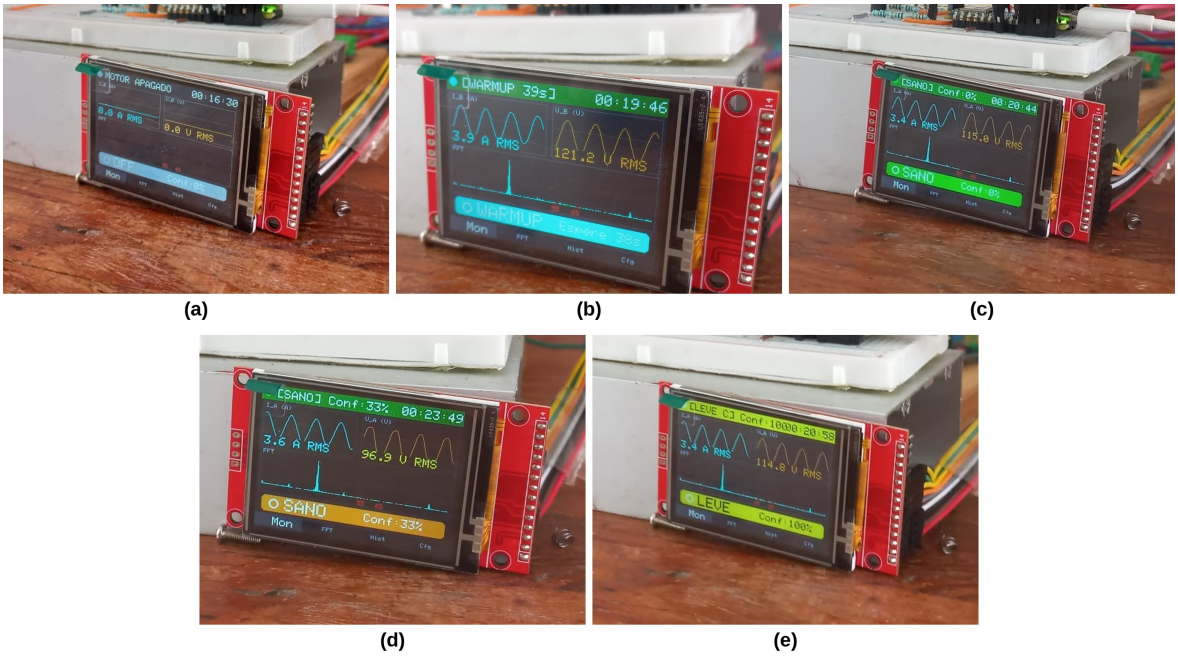


Figura 21. *Pantalla TFT del sistema en el banco: (a) motor apagado (OFF); (b) calentamiento tras el arranque; (c) estado sano sin inyección (Conf 0 %); (d) durante una inyección, con el primer bloque sobre el umbral (Conf 33 %); (e) alarma LEVE confirmada (Conf 100 %).*

3.3.5 APLICACIÓN DE ESCRITORIO MOTORGUARD-ITF

3.3.5.1 MOTIVACIÓN Y REQUISITOS

La aplicación de escritorio supervisa el sistema desde una computadora conectada por USB al microcontrolador. La versión anterior, desarrollada en septiembre, se descartó porque sintetizaba formas de onda

a partir de los valores eficaces, es decir, mostraba datos que no se habían medido; permitía activar la inyección sin confirmación; enviaba comandos que el firmware no tiene; no apagaba el relevador al cerrarse; terminaba a la fuerza al programa que tuviera abierto el puerto, y su grabador escribía dentro del directorio de los conjuntos de datos. La versión nueva se escribió desde cero entre el 10 y el 17 de octubre de 2026, en una rama propia del control de versiones con 84 *commits*, después de que el director aprobara el 10 de octubre incluir la aplicación y el asistente en el alcance de la tesis.

Los requisitos de la nueva versión fueron cuatro. Primero, **no diagnosticar**: la aplicación muestra y registra lo que reporta el firmware final, que no se modificó, y cualquier cálculo propio solo sirve para dibujar. Segundo, **seguridad de la inyección**: ningún pulso sin confirmación humana, con bloqueos y con apagado garantizado. Tercero, **estabilidad en sesiones largas**. Cuarto, **reproducibilidad**: capturar datos con los mismos protocolos y formatos que los *scripts* de la tesis y poder revisar sesiones grabadas sin el hardware. La licencia es GPL-3.0 y la versión entregada es la 1.0.0 (Figura 29).

3.3.5.2 ARQUITECTURA

La aplicación está escrita en Python con PySide6 (Qt) y PyQtGraph y se organiza en módulos independientes:

- **Fuentes de datos (`telemetry/`):** el puerto serie, leído en un hilo propio con reconexión automática y sin quitarle el puerto a otro programa, y la reproducción de registros seriales reales con su ritmo original, que permite revisar sesiones sin el hardware. Ambas entregan los datos con la misma interfaz.
- **Intérprete de la telemetría (`telemetry/parser.py`):** convierte cada línea del firmware `299a943` en una trama con el estado del motor y del relevador, los voltajes y corrientes por fase, los fasores de secuencia y el campo del detector direccional. Solo acepta ese firmware; los registros de versiones anteriores no traen la severidad.
- **Registro de alarmas (`telemetry/alarmas.py`):** detecta los flancos de la confirmación C y registra cada alarma con su inicio, fin, duración, severidad máxima y magnitud, en un archivo CSV por sesión.
- **Control de la inyección (`inyeccion/control.py`):** una máquina de estados pura, sin dependencia de la interfaz, que decide cuándo puede enviarse cada comando al relevador.
- **Adquisición (`adquisicion/`):** captura de conjuntos de datos con los protocolos de la tesis.
- **Asistente (`rag/`):** índice de documentos, recuperación y comunicación con el modelo de lenguaje (sección 3.3.6). Este módulo no tiene acceso al puerto serie.
- **Interfaz (`ui/`):** ventanas, gráficas y temas, con un estilo inspirado en Material 3, tema claro, oscuro o del sistema, y tipografías Figtree y JetBrains Mono de licencia libre.

3.3.5.3 SECCIONES DE LA INTERFAZ

La interfaz se organiza en cuatro secciones, accesibles desde una barra lateral, más los ajustes y la ventana «Acerca de».

Monitoreo. Muestra el estado del detector embebido, con su severidad y la magnitud de la firma; los voltajes y corrientes por fase; la proyección P en el tiempo con el umbral de 0.40 pp, y las tablas de alarmas y de pulsos (Figura 22). Incluye el **plano de la firma**, que dibuja en el plano complejo el método del detector: la firma z, la referencia móvil, Δz, la dirección de falla de −81.4°, el umbral y, con la alarma enclavada, la base congelada y los anillos de severidad de 1.626 y 4.471 pp (Figura 23). El plano solo recalcula la geometría para dibujarla; las decisiones mostradas son siempre las del firmware.

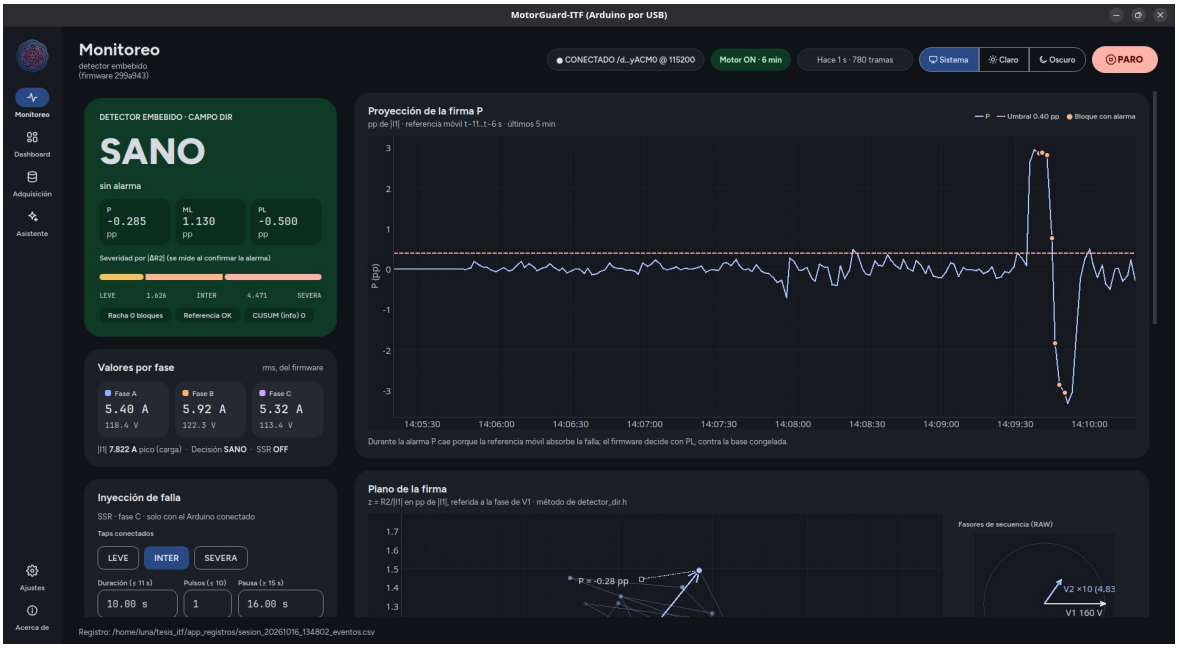


Figura 22. *Sección Monitoreo de la aplicación con el motor sano, durante una sesión con el banco del 16 de octubre de 2026.*

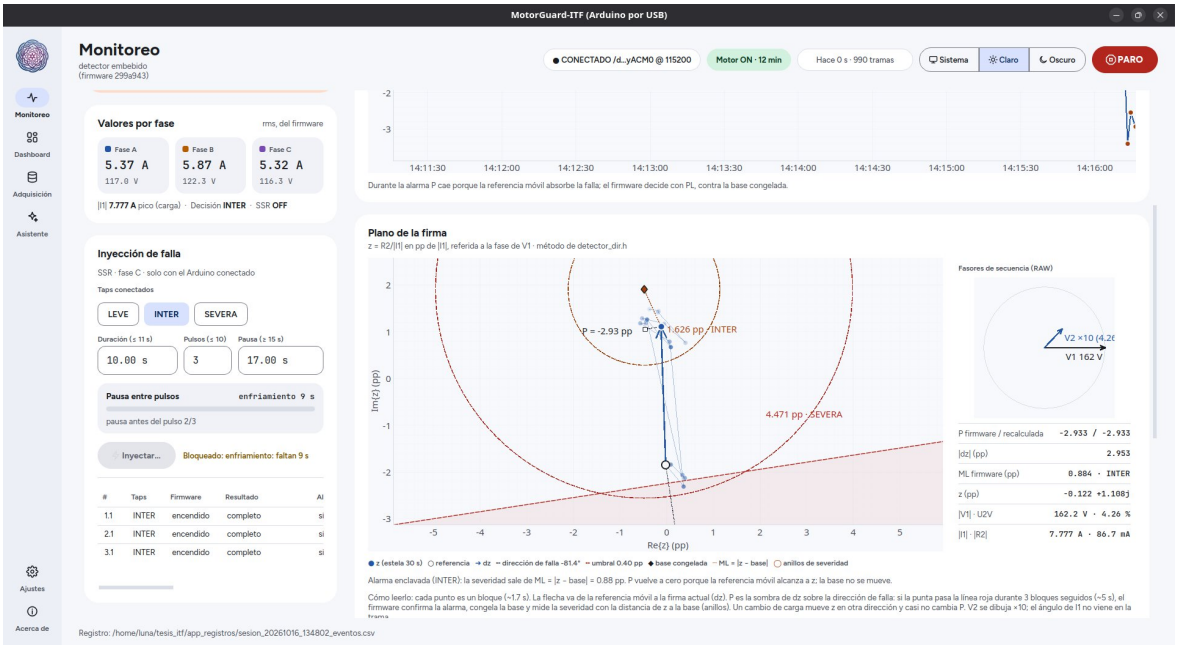


Figura 23. *Plano de la firma en la sección Monitoreo: firma z, referencia, cambio Δz, dirección de falla de −81.4° con su umbral y anillos de severidad.*

Dashboard. Presenta todos los campos de la trama, las gráficas de corrientes por fase y del desbalance de secuencia negativa, y, solo a petición del usuario, las consultas de lectura del firmware: el osciloscopio de corrientes crudas (`w`), el espectro de 50 a 70 Hz (`f`) y la distorsión (`s`) (Figura 24). Mientras el firmware imprime una consulta, su ciclo se detiene: un volcado real de la forma de onda tardó 0.78 s y retrasó 0.85 s la trama

siguiente, sin perder bloques. Por eso estas consultas no se usan en sesiones de validación y quedan bloqueadas durante un pulso.



Figura 24. *Sección Dashboard con las consultas de lectura del firmware: osciloscopio de las corrientes crudas y espectro de la fase vigilada.*

Adquisición. Captura conjuntos de datos nuevos para trabajos posteriores con los mismos protocolos y formatos que los *scripts* de la tesis: pares SANO/FALLA, como el *script* de captura multiclase, y base sin falla, como el de la prueba larga (Figura 25). Por decisión del autor, usa un enfriamiento mayor entre pulsos (al menos 15 s, o 20 s con SEVERA, en lugar de 8 s) y bloques de 10 pulsos con una nueva confirmación cada uno, y guarda los archivos en un directorio propio, nunca en los de los conjuntos de datos o de validación.

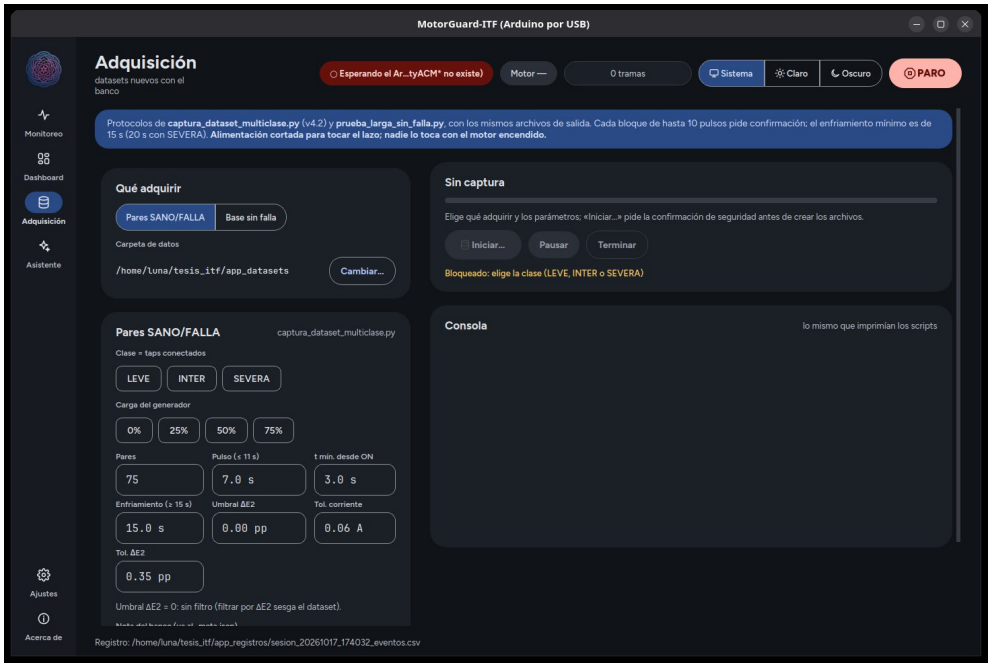


Figura 25. *Sección Adquisición: captura de pares SANO/FALLA con los parámetros del protocolo de la tesis.*

Asistente. Recomendaciones de mantenimiento con citas a sus fuentes (sección 3.3.6).

3.3.5.4 SEGURIDAD DE LA INYECCIÓN

La aplicación solo puede enviar al microcontrolador los comandos de encender (`1`) y apagar (`0`) el relevador, las tres consultas de lectura y, desde el 16 de octubre, el reinicio del detector (`r`). Nunca envía los comandos de servicio del firmware, como los que borran la memoria no volátil. Cada encendido pasa por la máquina de estados del Listado 2, que lo impide si no hay conexión o telemetría reciente, si el motor no está encendido o está en calentamiento, si el firmware reporta el relevador encendido, si hay un pulso en curso o si no ha transcurrido el enfriamiento. El bloqueo durante el calentamiento lo pone la aplicación, porque el firmware sí acepta el encendido en ese estado. Además, cada pulso dura como máximo 11 s, hay un número máximo de pulsos por serie y cada pulso o serie exige una confirmación humana en un diálogo con casillas sobre las derivaciones conectadas y la seguridad del lazo (Figura 26).

Listado 2. *Condiciones que bloquean el encendido del relevador en la aplicación (app/motorguard_dashboard/inyeccion/control.py, líneas 195–211).*

```
def bloqueo(self, ahora: float, taps: Optional[str] = None, con_enfriamiento: bool = True) ->
Optional[str]:
    """Motivo por el que no se puede encender el SSR ahora, o None."""
    if not self.conectado:
        return "sin conexión con el Arduino"
    t = self.ultima_trama
    if t is None or t.t is None or ahora - t.t > TRAMA_RECIENTE_S:
        return f"sin telemetría reciente (> {TRAMA_RECIENTE_S:g} s)"
    if t.motor != "ON":
        return f"motor en {t.motor}: solo se inyecta con el motor en ON (fuera de WARMUP)"
    if t.ssr != "OFF":
        return f"el firmware reporta el SSR encendido ({t.ssr})"
    if self.estado in (self.ESPERA_ENCENDIDO, self.ENCENDIDO, self.ESPERA_APAGADO):
        return "hay un pulso en curso"
    resta = self.enfriamiento_restante(ahora, taps) if con_enfriamiento else 0.0
```

```
    if resta > 0:
        return f"enfriamiento: faltan {resta:.0f} s"
    return None
```



Figura 26. *Diálogo de confirmación previo a una serie de inyección, con las casillas obligatorias sobre la seguridad del lazo y las derivaciones conectadas (modo de demostración).*

El botón de **paro** está siempre visible en la barra superior, y la aplicación envía el apagado al abrir el puerto, al cerrarse, al perder la conexión y si el firmware no confirma un apagado. Cada pulso se registra en un archivo CSV con la duración pedida, la confirmación y el tiempo encendido que reporta el firmware, el resultado, la latencia de la alarma y la firma observada.

La Figura 27 muestra la secuencia de un pulso de inyección entre la aplicación, el firmware y el banco, con los bloqueos y protecciones que actúan en cada lado.

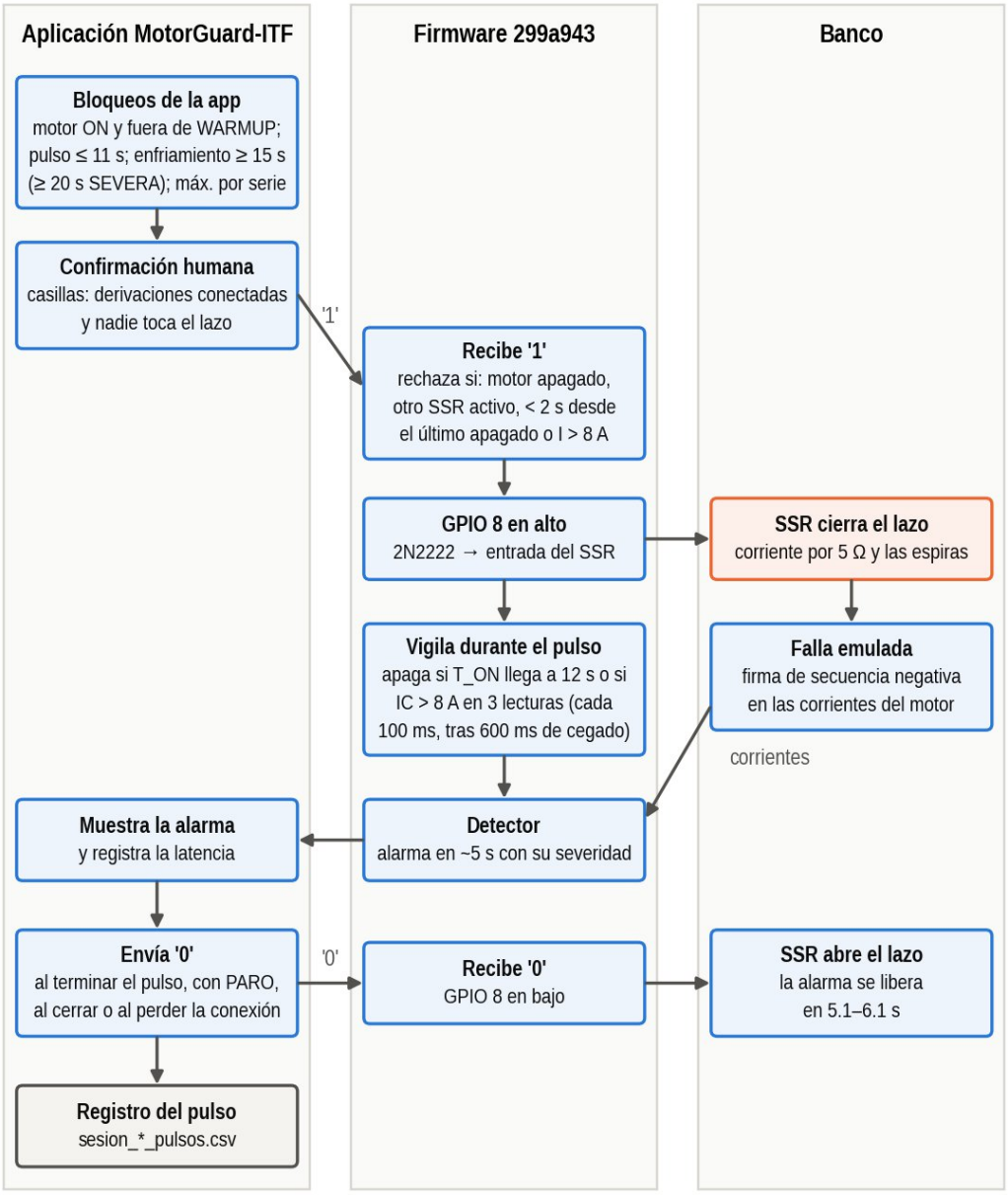


Figura 27. *Secuencia de un pulso de inyección: bloqueos y confirmación en la aplicación, verificaciones y protecciones del firmware, y respuesta del banco y del detector.*

El botón **«Liberar alarma»** envía el reinicio del detector. Se agregó, con la aprobación del autor, después de que el 16 de octubre una alarma quedara enclavada 567 s durante una sesión con el banco (sección 4.1.11). Solo aparece con una alarma enclavada, el motor encendido y sin pulso, serie ni captura en curso, y exige marcar una casilla que advierte que, si la falla es real y persiste, el detector no volverá a alarmar, porque la nueva referencia ya la incluye. La alarma se cierra en el registro como liberada manualmente. El botón se probó con el simulador y con el microcontrolador real (prueba del autor, confirmada el 17 de octubre de 2026).

3.3.5.5 ESTABILIDAD Y VERIFICACIÓN

Para sesiones largas, la aplicación tiene un registro interno con rotación, hilos que se reinician ante errores, tablas acotadas en memoria (1000 alarmas y 200 pulsos), una sola instancia con el microcontrolador (Figura 28), rutas configurables y un aviso si el usuario no pertenece al grupo con permiso de acceso al puerto serie.



Figura 28. *Aviso de instancia única: solo una ventana de la aplicación puede usar el microcontrolador; las reproducciones de registros sí pueden abrirse varias.*

La verificación se hizo en cuatro niveles. El primero son 119 funciones de prueba automática que cubren, entre otras cosas, el intérprete contra registros reales, los bloqueos de la inyección y el envío del apagado al cerrar o desconectar (con un puerto virtual y un simulador del firmware que solo se usa en pruebas), la equivalencia de los formatos de adquisición con los de los *scripts*, el plano de la firma contra la decisión del firmware, el asistente con un servidor de modelo simulado y el contraste del texto. El segundo son pruebas de resistencia acelerada, reproduciendo registros reales sin pausas durante el equivalente de 24 y 48 h. El tercero es una prueba de resistencia en tiempo real de 4 h contra el simulador, con criterios declarados de memoria, errores, tramas recibidas y pulsos terminados. El cuarto es un protocolo de prueba en vivo con el banco (*commit* `5227223`), con pasos para el monitoreo, un pulso INTER verificado con la pinza, el paro y el cierre seguro. Los resultados de los cuatro niveles se presentan en la sección 4.1.11.

3.3.5.6 EMPAQUETADO

La aplicación se distribuye en dos paquetes `.deb`: uno ligero, que usa las bibliotecas de la distribución, y uno autónomo construido con PyInstaller, que incluye su propio intérprete. Ninguno de los dos incluye los modelos de lenguaje ni los documentos del asistente: la función «Preparar asistente» descarga el motor de inferencia, los modelos y los documentos de sus fuentes oficiales y verifica sus huellas SHA-256. La ventana «Acerca de» identifica la versión, el firmware compatible, el autor y las licencias de todos los componentes (Figura 29). Se decidió no desarrollar una versión para Windows, porque no aporta resultados a la tesis; queda como trabajo futuro.

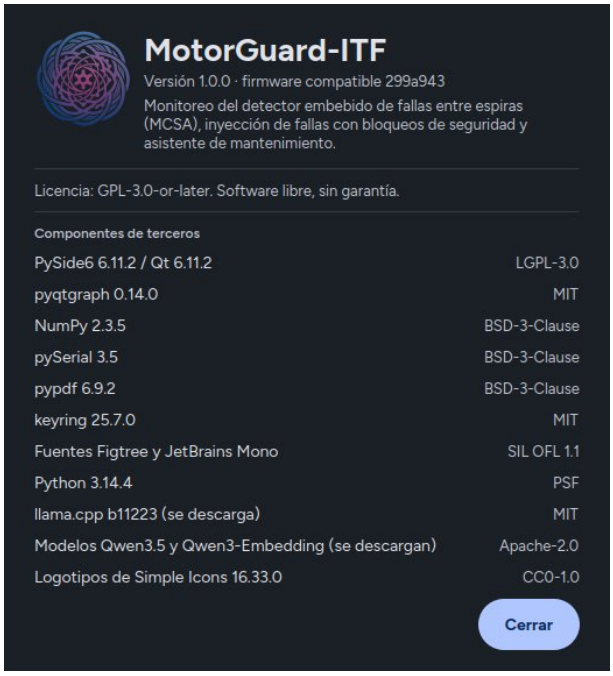


Figura 29. *Ventana «Acerca de» de la aplicación: versión 1.0.0, firmware compatible 299a943 y licencias de los componentes de terceros.*

3.3.5.7 RECORRIDO DE OPERACIÓN

Para documentar el funcionamiento de la aplicación sin depender del banco, el 29 de octubre de 2026 se agregó un modo de demostración (opción `--demo`, *commit* `a0072b8`). En ese modo, un simulador del firmware, que solo se usa en pruebas y no se incluye en los paquetes, reproduce a su ritmo real, un bloque cada 1.70 s, las tramas del registro serial de la sesión INTER a 75 % de carga del 9 de octubre: con el relevador apagado repite las tramas sanas, y cada orden de encendido reproduce uno de los 10 episodios reales de inyección de ese registro, con su alarma, su enclavamiento y su liberación. Los comandos no llegan a ningún hardware, la conexión es un puerto virtual y la barra superior muestra siempre la etiqueta «DEMO · simulador del firmware». Lo simulado es, por lo tanto, la conexión y el instante de cada inyección; los valores que se muestran son los del banco. Las figuras de esta sección se tomaron en ese modo con el código del 29 de octubre (*commit* `d265bc9`), que difiere de la versión 1.0.0 entregada solo en tres ajustes de presentación.

Monitoreo con el motor sano. Al conectarse, la sección Monitoreo (Figura 30) muestra en la tarjeta del detector la decisión del firmware, las variables del campo direccional P, ML y PL, la barra de severidad con los umbrales de 1.626 y 4.471 pp, y los indicadores de racha, referencia y CUSUM informativo. Debajo están los valores eficaces por fase, $|I_1|$, el estado del relevador y el panel de inyección, y a la derecha, la gráfica de la proyección P de los últimos 5 min con el umbral de 0.40 pp y el plano de la firma. Las caídas de P de hasta unos −2.5 pp que se ven en la gráfica no son ruido del motor sano: son tramas del registro posteriores a alarmas, en las que la referencia móvil ya había absorbido la falla, y que la demostración repite como reposo.

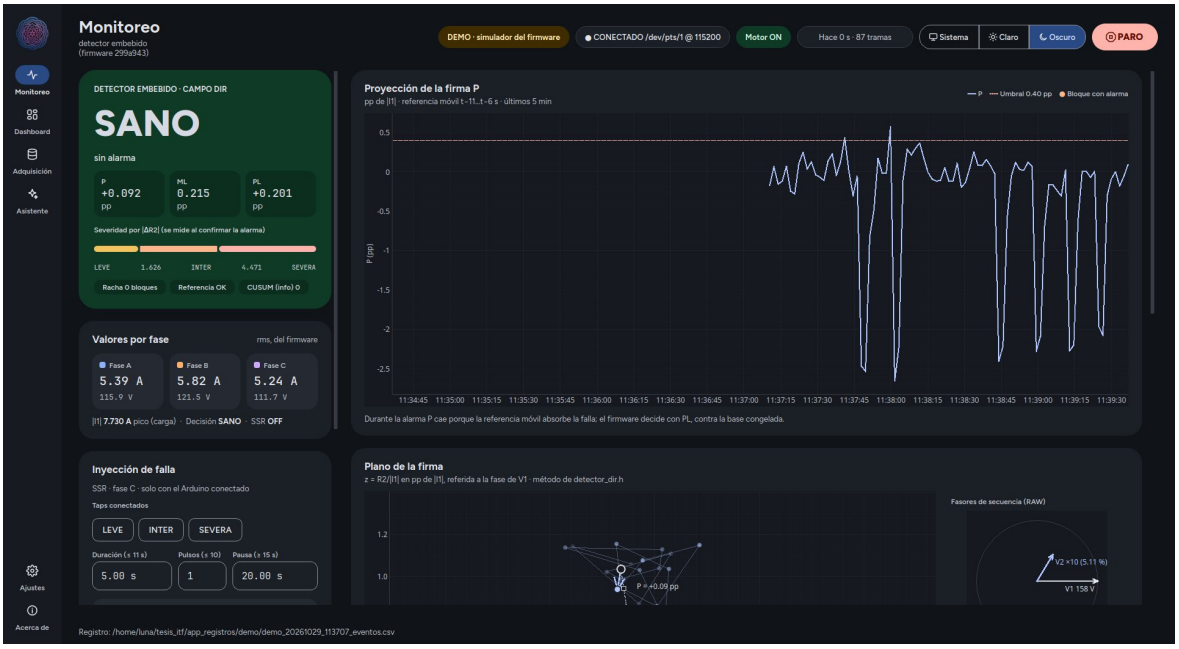


Figura 30. *Sección Monitoreo con el motor sano, en modo de demostración: tramas reales del banco (sesión INTER a 75 % de carga, 9 de octubre de 2026) reproducidas por un simulador del firmware; los comandos no llegan a ningún hardware.*

Dashboard. La Figura 31 muestra con datos la sección Dashboard descrita en la sección 3.3.5.3, con las tablas de las características MCSA, de las componentes de Fortescue, del CUSUM anterior y del análisis MCSA de alta resolución. Cada gráfica tiene botones de zoom en el tiempo: «+» muestra la mitad del intervalo, «−» el doble y «Ajustar» vuelve a todo el historial, siempre pegado al dato más reciente; con dos pulsaciones de «+», la ventana pasa de unos 2.5 min a unos 35 s y se distinguen los bloques individuales de 1.70 s (Figura 32). La Figura 33 muestra la respuesta a la consulta del osciloscopio: 2048 muestras por fase (852 ms a 2403.85 Hz) de las corrientes crudas, sin calibrar, que en este caso son el volcado real del 6 de octubre. El espectro y la distorsión no se muestran, porque el simulador no reproduce esas respuestas.

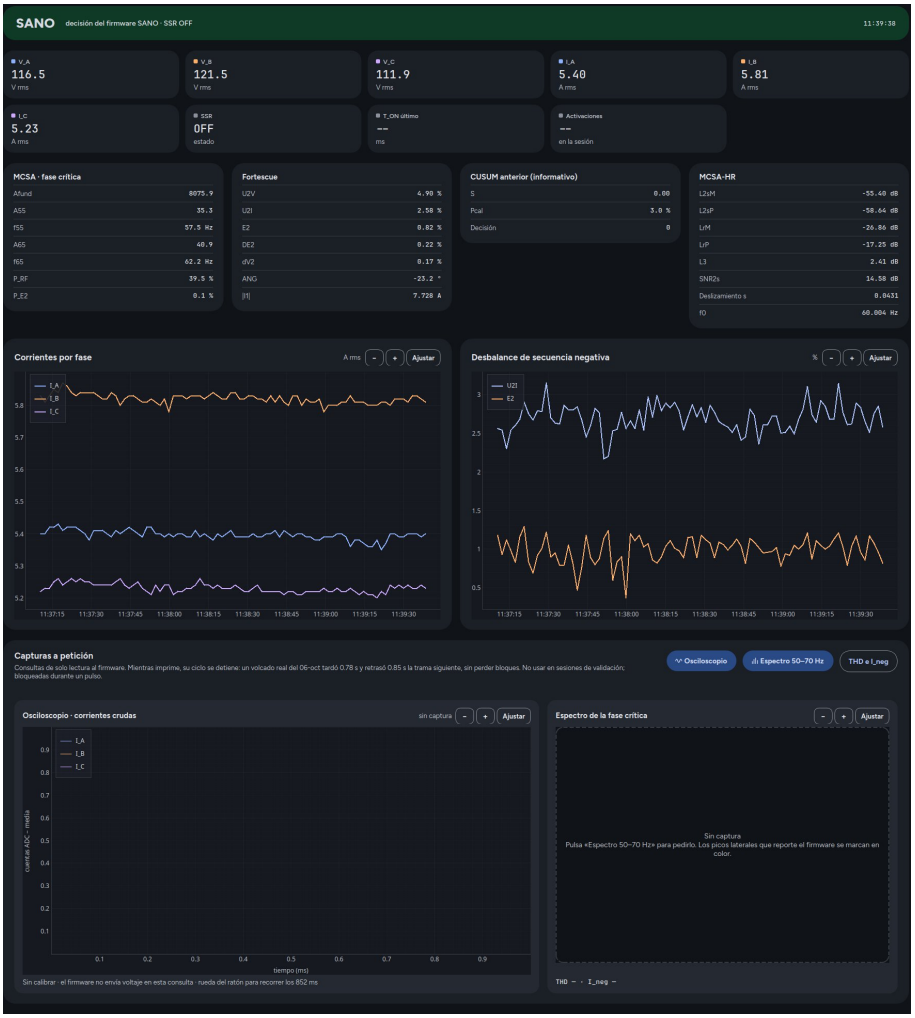


Figura 31. *Sección Dashboard con todos los campos de la trama, las gráficas de corrientes y de desbalance de secuencia negativa y la tarjeta de consultas a petición (modo de demostración).*



Figura 32. *Gráficas del Dashboard después de pulsar dos veces el botón de acercamiento: unos 35 s de historial, con los bloques individuales de 1.70 s (modo de demostración).*

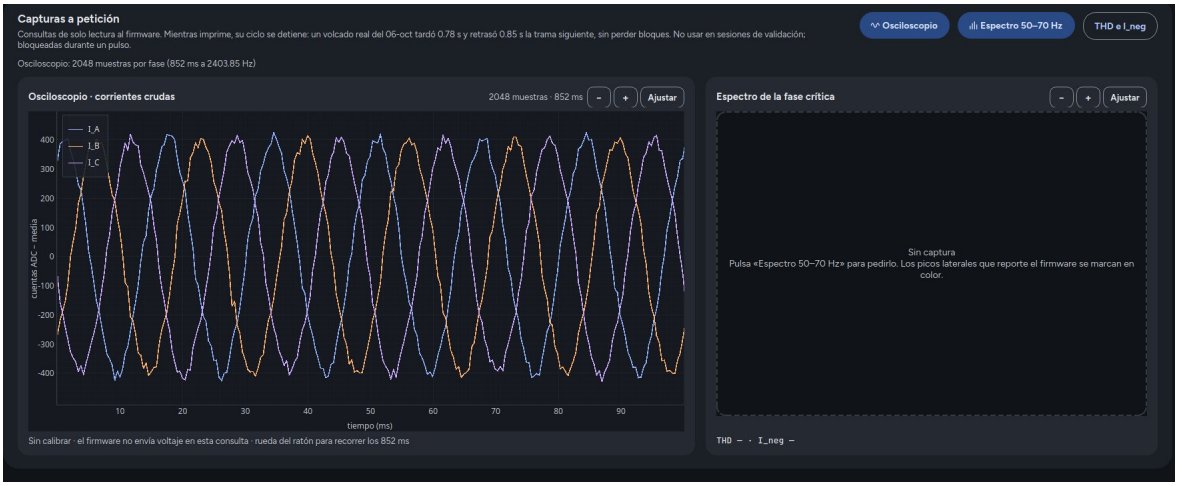


Figura 33. *Consulta del osciloscopio en el Dashboard: 2048 muestras por fase de las corrientes crudas, sin calibrar (volcado real del 6 de octubre de 2026 reproducido en el modo de demostración).*

Inyección de un pulso. Para inyectar, el operador elige en el panel los taps conectados, la duración, el número de pulsos y la pausa, y pulsa «Inyectar». La aplicación abre el diálogo de confirmación (Figura 26), que resume la serie y exige marcar las tres casillas de seguridad; si el motor ya estaba encendido al abrir la aplicación, agrega el aviso de esperar unos 5 min después del arranque. El mismo diálogo aparece al inicio de cada bloque de una captura de adquisición. Durante el pulso (Figura 34), la tarjeta de valores indica el relevador encendido y el detector sigue en SANO, porque la alarma necesita 3 bloques. La etiqueta «F1(LEVE)» es el nombre que el firmware 299a943 da a su salida del relevador, cualquiera que sea la clase conectada, y no indica la clase de la falla. El panel de inyección (Figura 35) muestra el avance del pulso y mantiene bloqueado el botón mientras el firmware reporta el relevador encendido; al terminar la serie lo mantiene bloqueado durante el enfriamiento. La tabla de pulsos registra, para cada uno, los taps, el estado que reporta el firmware, el resultado, si hubo alarma y su severidad.

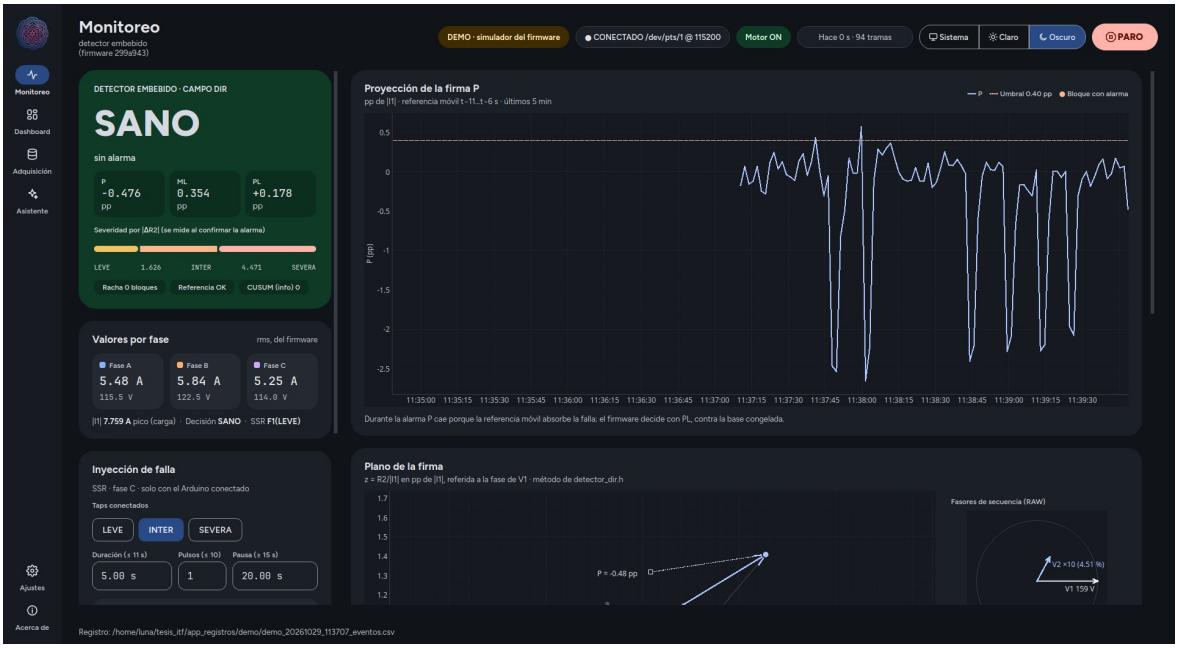


Figura 34. *Sección Monitoreo 1.5 s después de aceptar un pulso: relevador encendido y detector todavía en SANO. «F1(LEVE)» es el nombre que el firmware da a su salida del relevador, no la clase de la falla (modo de demostración).*



Figura 35. *Panel de inyección: (a) durante el pulso, con «Inyectar» bloqueado porque el firmware reporta el relevador encendido; (b) después de dos pulsos, bloqueado por el enfriamiento, con la tabla de pulsos (modo de demostración).*

Alarma y plano de la firma. Unos 5 s después del inicio del pulso, el firmware confirma la alarma: la tarjeta del detector cambia a INTER, con el indicador ENCLAVADA, la hora de inicio y el botón «Liberar alarma», y la

gráfica de P marca en naranja los bloques con alarma (Figura 36). El plano de la firma (Figura 37) dibuja la estela de z, la referencia, el cambio Δz, la dirección de falla de −81.4° con el umbral, la base congelada y el anillo de severidad, junto con los fasores de secuencia y una tabla con los valores del firmware y los recalculados. En esta captura, la P recalculada por la aplicación (+2.405 pp) no coincide con la del firmware (+2.653 pp), porque la aplicación fecha los bloques al recibirlos y el firmware con su propio reloj; la aplicación lo advierte en el mismo plano, y la decisión mostrada es siempre la del firmware. Cada alarma queda en la tabla de eventos (Figura 38) con su inicio, fin, duración, severidad inicial y máxima, ML máxima, $|I_1|$ y el motivo de fin; en la demostración, las dos alarmas INTER duraron 10.2 y 13.6 s y las liberó el firmware.

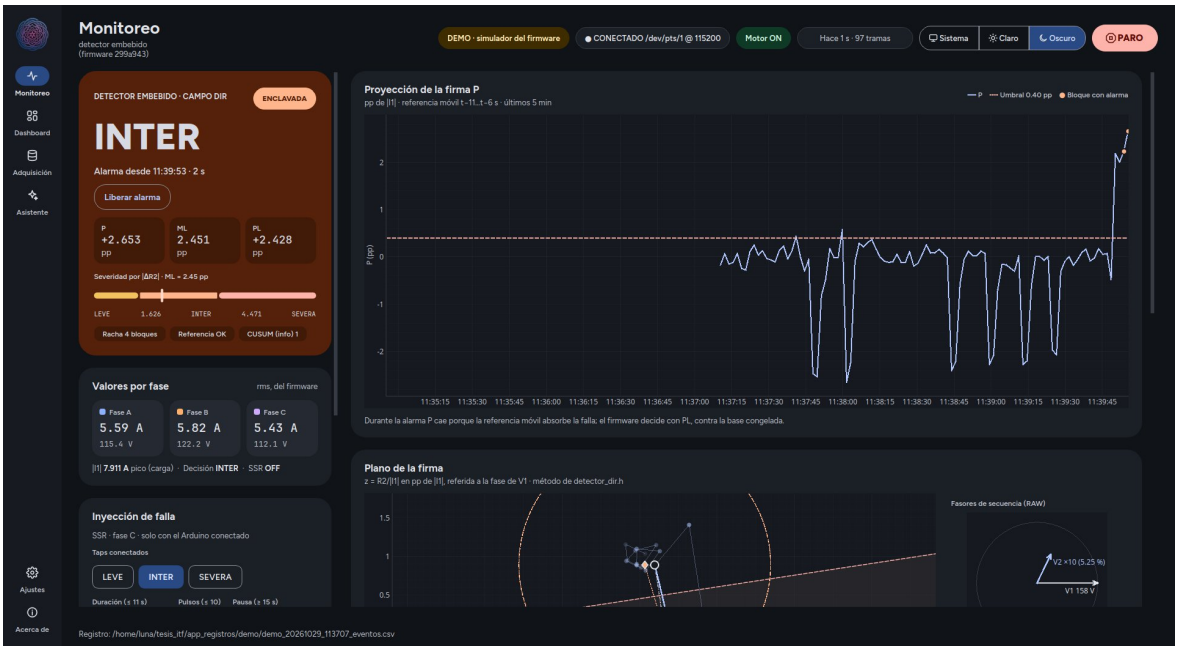


Figura 36. *Alarma INTER confirmada y enclavada en la sección Monitoreo: proyección P sobre el umbral, bloques con alarma en naranja y botón «Liberar alarma» (modo de demostración).*

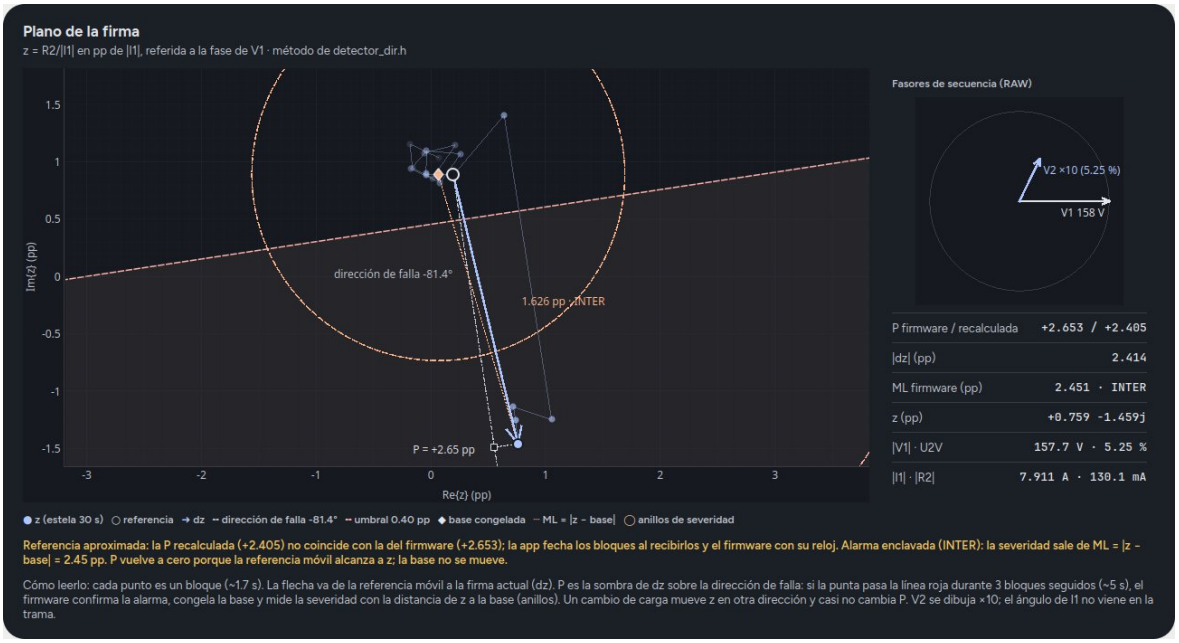


Figura 37. *Plano de la firma durante la alarma: estela de z, referencia, Δz, dirección de falla, base congelada, anillo de severidad, fasores de secuencia y comparación de la P del firmware con la recalculada por la aplicación (modo de demostración).*

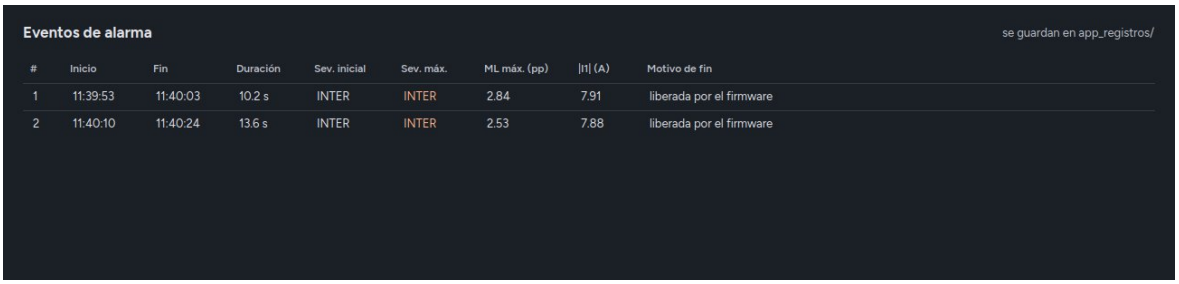


Figura 38. *Tabla de eventos de alarma con dos alarmas INTER liberadas por el firmware (modo de demostración).*

Liberación manual de una alarma. La Figura 39 muestra una alarma que no se libera sola. Para poder capturar el botón, la demostración mantuvo enclavada la alarma repitiendo la última trama con alarma del episodio, por lo que P y ML ya son bajos; la figura representa casos como la alarma enclavada 567 s del 16 de octubre (sección 4.1.11), no una alarma recién confirmada. Al pulsar «Liberar alarma» aparece el diálogo de la Figura 40, que explica que el detector rehará su referencia con el motor tal como está (unos 8–11 s sin decisión), advierte que una falla real no volvería a detectarse y exige marcar la casilla de confirmación. Después del reinicio, el detector vuelve a SANO (Figura 41).

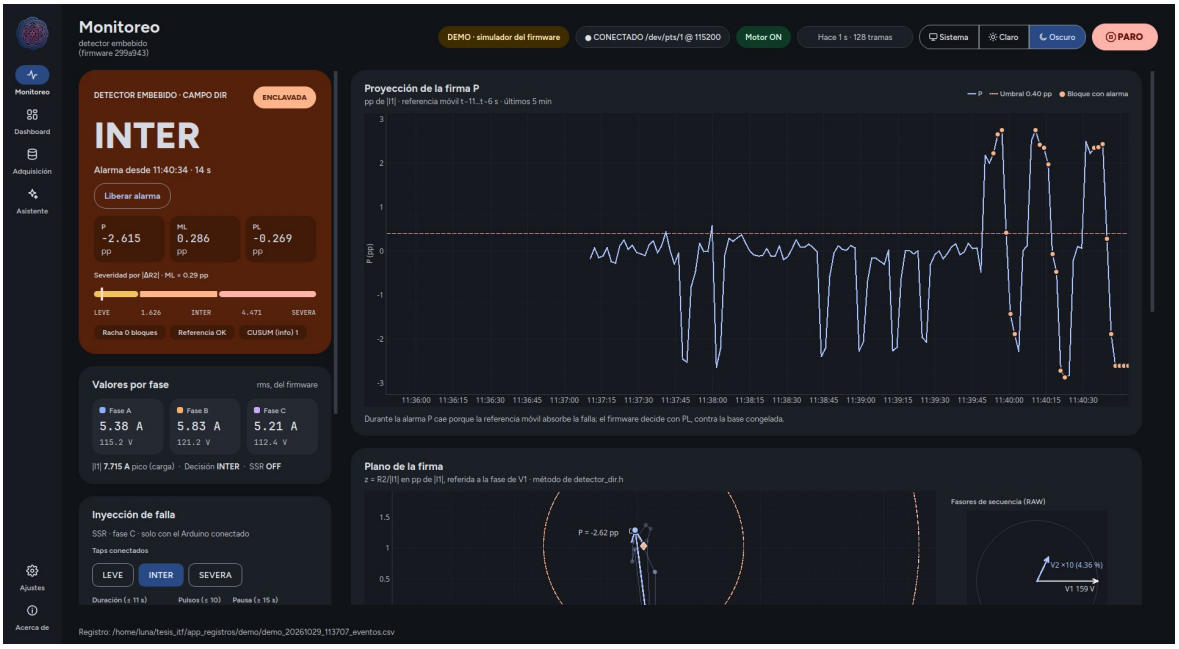


Figura 39. *Alarma INTER mantenida enclavada para mostrar el botón «Liberar alarma»; P y ML ya son bajos porque la demostración repite la última trama con alarma (modo de demostración).*



Figura 40. *Diálogo «Liberar alarma»: reinicio del detector con la advertencia de que una falla real no volvería a detectarse y la casilla de confirmación obligatoria (modo de demostración).*

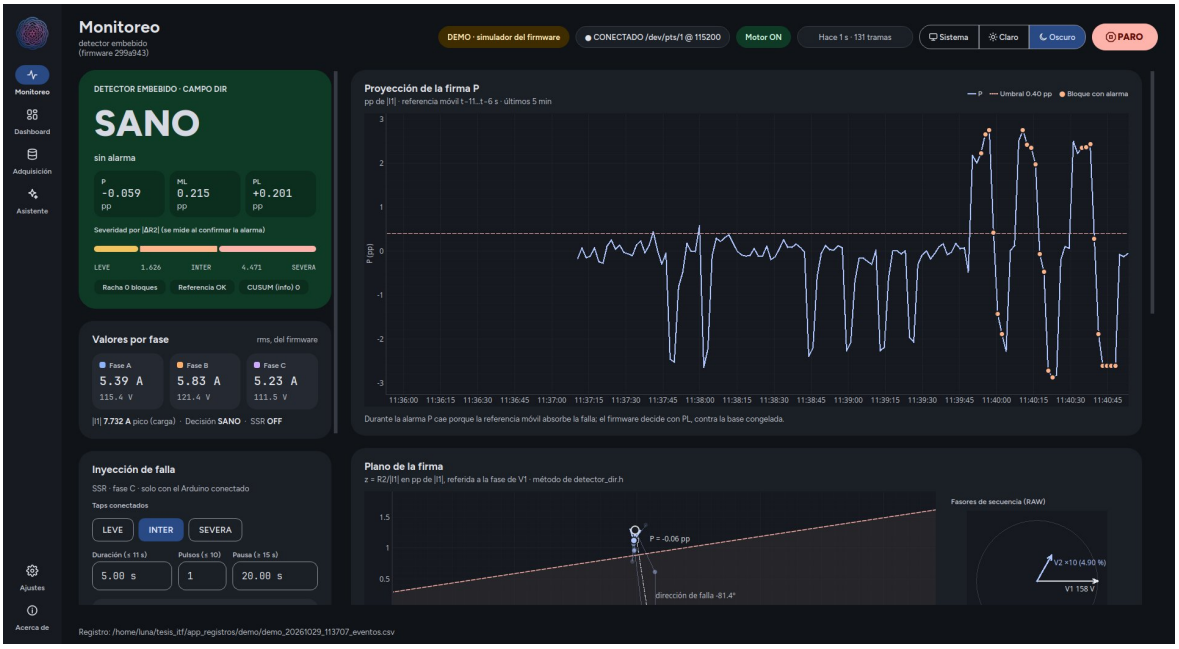


Figura 41. *Sección Monitoreo después de liberar la alarma: el detector vuelve a SANO (modo de demostración).*

Adquisición. En la sección Adquisición, el operador elige el protocolo (pares SANO/FALLA o base sin falla), la carpeta de datos, la clase, la carga y los parámetros de la captura. Cada bloque de pulsos pide la confirmación del mismo diálogo de seguridad (Figura 26), con el resumen del bloque en lugar del de la serie. Durante la captura (Figura 42), la aplicación muestra el avance, los descartes y el último par, y la consola imprime los mismos mensajes que el *script* de captura multiclase (sección 3.3.5.3). Al terminar (Figura 43), indica el total de pares y la ruta del archivo CSV generado. La carga de 75 % de la figura es la que eligió el operador en la demostración y coincide con la del registro reproducido.

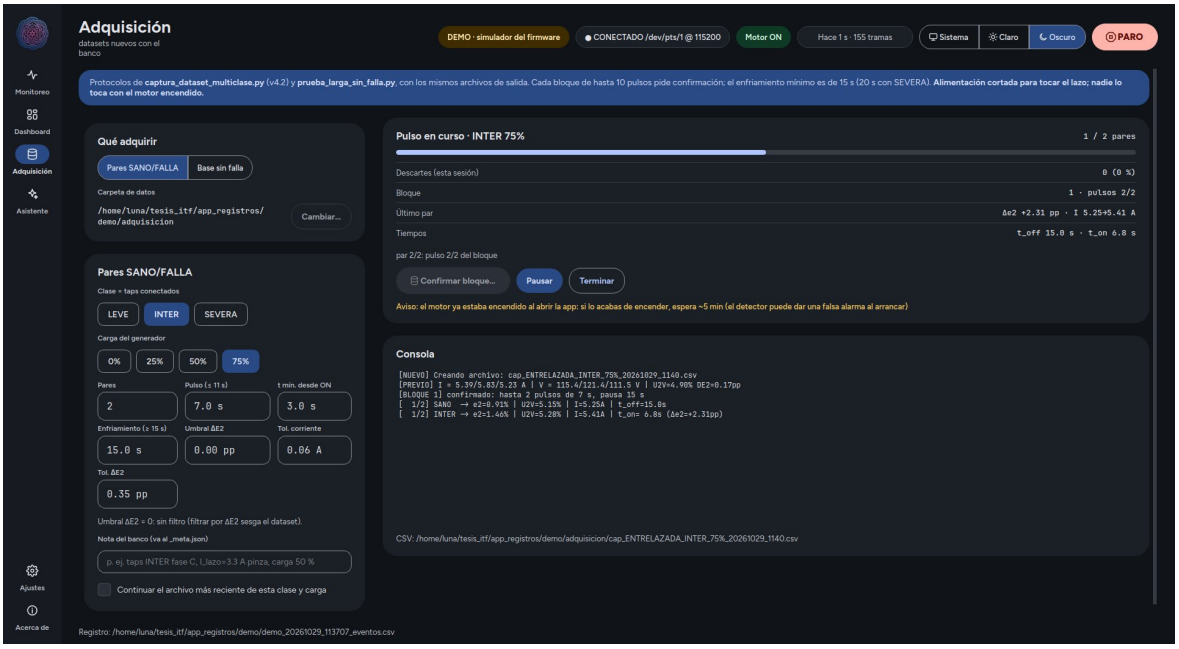


Figura 42. *Sección Adquisición durante una captura de pares SANO/FALLA: parámetros, avance y consola con los mensajes del protocolo de la tesis (modo de demostración).*

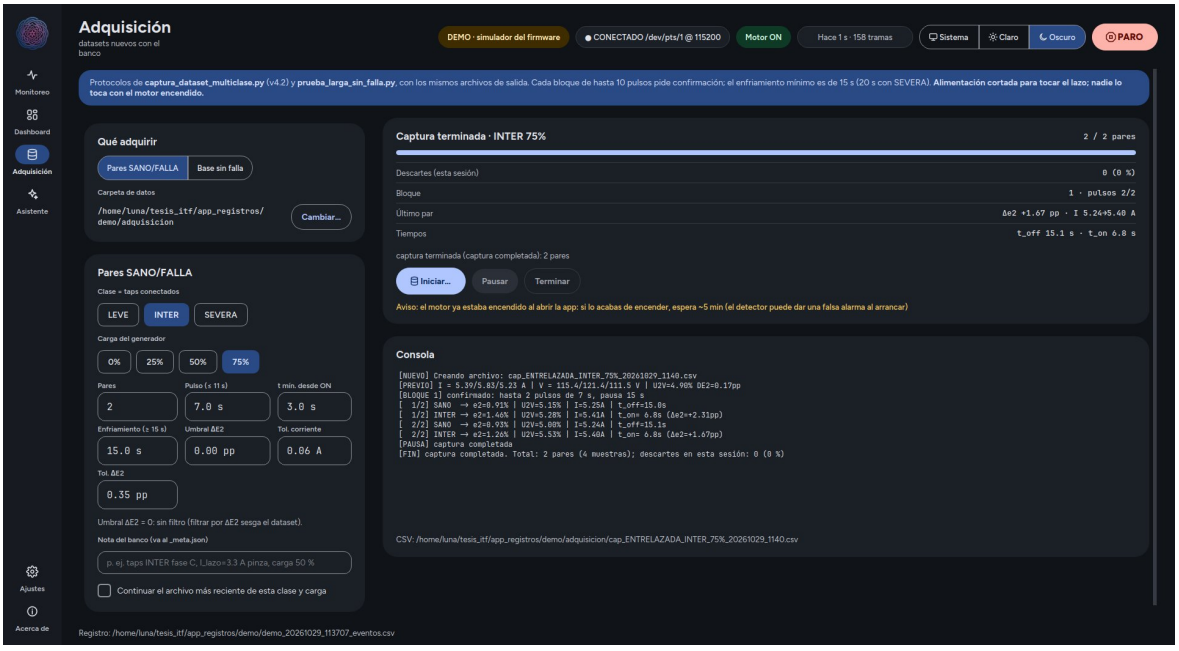


Figura 43. *Sección Adquisición con la captura terminada: total de pares, descartes y ruta del archivo generado (modo de demostración).*

Ajustes. El diálogo de ajustes (Figura 44) permite cambiar la carpeta de los registros, con efecto a partir de la sesión siguiente, y abrir el registro interno de la aplicación. En la figura aparece la carpeta de la demostración; en una sesión normal los registros van a la carpeta principal de la aplicación.

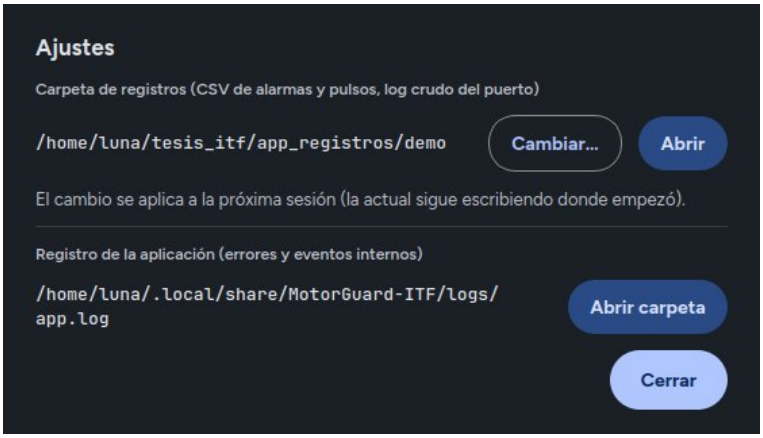


Figura 44. *Diálogo de ajustes: carpeta de registros y registro interno de la aplicación (modo de demostración).*

3.3.6 ASISTENTE DE MANTENIMIENTO CON RECUPERACIÓN DE DOCUMENTOS (RAG)

3.3.6.1 OBJETIVO Y ALCANCE

El detector embebido indica que hay una posible falla entre espiras y su severidad, pero no qué hacer después. El asistente responde esa pregunta con las fuentes de mantenimiento disponibles. Es un **apoyo, no un diagnóstico**: el diagnóstico es el del detector validado, y el asistente solo propone inspecciones y acciones de mantenimiento citando el documento y la página de cada una. Se usa la técnica de generación aumentada por recuperación (sección 2.8): antes de redactar, el sistema busca en una colección local los fragmentos más parecidos a la pregunta y se los entrega al modelo de lenguaje con la instrucción de responder solo con ellos.

3.3.6.2 FUENTES DOCUMENTALES

Solo se usaron documentos de uso legal, descargados de su fuente oficial y registrados con su dirección, fecha, huella SHA-256 y licencia (Tabla 9): el manual del fabricante del motor (ABB, 2022), la guía de mantenimiento centrado en confiabilidad de la NASA (National Aeronautics and Space Administration, 2008), la guía de buenas prácticas de operación y mantenimiento del DOE/FEMP (Sullivan et al., 2010), tres volúmenes de instrucciones técnicas del U.S. Bureau of Reclamation (1991a, 1991b, 2026), dos artículos de acceso abierto (Ruzimov et al., 2025; Chen et al., 2023) y dos documentos propios de esta tesis. De cada documento se indexan solo los capítulos pertinentes: el resto trata de equipos ajenos al problema y solo agregaría ruido a la búsqueda.

Tabla 9. Documentos de la colección del asistente y partes indexadas.

Documento	Partes indexadas	Uso
ABB, manual de motores de baja tensión 3GZF500730-85 Rev H (2022)	Sección en español (págs. 95–119)	Solo uso local; no se redistribuye
NASA, *Reliability-Centered Maintenance Guide* (2008)	Caps. 5–7 (págs. 47–136)	Programa RCM y técnicas predictivas, incluido el MCSA
DOE/FEMP, *Operations & Maintenance Best Practices Guide*, Release 3.0 (2010)	Caps. 5–6 y 9.10	Tipos de mantenimiento, análisis de motores
USBR, FIST 3-1 (1991)	Completo	Pruebas del aislamiento
USBR, FIST 3-4 (1991)	Completo	Mantener secos los devanados
USBR, FIST 4-1B (2026)	Secc. 1, 21 y 23	Frecuencias de mantenimiento de motores
Ruzimov et al. (2025)	Completo, sin referencias	Acceso abierto, CC BY 4.0

Documento	Partes indexadas	Uso
Chen et al. (2023)	Completo, sin referencias	Acceso abierto, CC BY 4.0
Extracto de resultados de esta tesis	Completo	Significado de las severidades, validación y limitaciones
Documento técnico del banco de pruebas (36 secciones)	Completo	Descripción del banco y del método

Los dos artículos se descargaron de su copia en PubMed Central, porque el sitio de la editorial rechaza las descargas automáticas. El manual de ABB no se puede redistribuir: no se incluye en el control de versiones ni en los paquetes, y cada usuario lo descarga del sitio del fabricante. La colección final tiene 936 fragmentos.

3.3.6.3 INGESTA Y RECUPERACIÓN

El texto se extrae página por página y se divide en fragmentos de unos 900 caracteres que **nunca cruzan de página**, para que cada cita apunte a una sola. Se eliminan los encabezados y pies repetidos, los índices y las páginas en otros idiomas. La búsqueda es híbrida y combina dos ordenamientos por fusión de rangos recíprocos (Cormack, Clarke y Buettcher, 2009), con la constante K = 15:

- una búsqueda semántica por similitud coseno entre *embeddings* del modelo multilingüe Qwen3-Embedding-0.6B (Zhang et al., 2025), que relaciona preguntas en español con documentos en inglés;
- una búsqueda léxica BM25 (Robertson y Zaragoza, 2009), ampliada con un glosario técnico español-inglés.

Para que la respuesta no dependa de un solo documento, se toman como máximo 3 fragmentos por documento. Si el fragmento más parecido a la pregunta tiene una similitud coseno menor que 0.49, el asistente responde que no hay fuente **sin consultar al modelo de lenguaje**. Ese umbral se fijó con preguntas de desarrollo distintas de las de la evaluación: las preguntas fuera de tema alcanzaron como máximo 0.412 y las pertinentes no bajaron de 0.564, y se tomó el punto medio.

3.3.6.4 MODELO DE LENGUAJE LOCAL

El modelo de lenguaje se ejecuta en la misma computadora, sin conexión a internet, con el servidor del motor de inferencia llama.cpp (build b11223, licencia MIT), que la aplicación lanza como proceso hijo de baja prioridad y que solo escucha en la dirección local. Se usa el modelo Qwen3.5-4B cuantizado a 4 bits (Q4_K_M, licencia Apache 2.0), con el modo de razonamiento desactivado; el de 2B parámetros queda como respaldo para equipos más lentos, y los *embeddings* se calculan con Qwen3-Embedding-0.6B cuantizado a 8 bits. El autor eligió el modelo de 4B el 13 de octubre porque, en la evaluación preliminar, cometió menos errores que el de 2B (sección 4.1.11). El criterio declarado antes de medir fue responder una consulta de alarma en 90 s como máximo.

Las indicaciones del sistema (Listado 3) le exigen al modelo responder solo con los fragmentos numerados, terminar cada punto con su cita, decir cuando los documentos no cubren la pregunta, no inventar mediciones, no afirmar que el motor tiene o no una falla, poner primero las precauciones de seguridad y no aplicar al motor vigilado las cifras del banco de la tesis. Esta última regla se agregó porque, en la evaluación preliminar, el modelo aplicaba al motor vigilado datos del banco, como el porcentaje de espiras de una severidad o la fase con falla; la regla no alcanza a los manuales ni a los artículos, cuyas cifras se atribuyen a esos documentos. La función de búsqueda híbrida y el registro de alarmas de la aplicación se incluyen en el Anexo E.

Listado 3. *Indicaciones del sistema que recibe el modelo de lenguaje del asistente (app/motorguard_dashboard/rag/asistente.py, líneas 37–47).*

```
SISTEMA = """Eres el asistente de mantenimiento de MotorGuard-ITF. Ayudas al personal técnico a
decidir qué inspeccionar y qué mantenimiento hacer en un motor de inducción trifásico cuando un
detector embebido reporta una posible falla entre espiras del devanado del estator.

Reglas obligatorias:
1. Usa SOLO la información de los fragmentos numerados. No agregues conocimiento propio.
2. Cada punto termina con la cita del fragmento que lo respalda entre corchetes, por ejemplo [2]. Si
se apoya en varios, cítalos todos: [1][3].
3. Si los fragmentos no responden la pregunta, escribe solo: "Los documentos disponibles no cubren
esto." No completes con suposiciones.
4. No inventes mediciones ni valores. Los únicos valores medidos son los de "Valores medidos"; no
los cambies.
5. No afirmes que el motor tiene o no tiene una falla: el diagnóstico es del detector; tú das apoyo
para la inspección y el mantenimiento.
6. Si un fragmento da una precaución de seguridad (desenergizar, bloquear, descargar el devanado),
ponla primero.
7. Los fragmentos de "Resultados validados de la tesis" y del "Documento técnico del banco de
pruebas" describen el banco de pruebas de la tesis, no el motor vigilado: no apliques al motor
vigilado sus porcentajes de espiras, su fase con falla ni sus otras cifras. Ante una alarma, úsalos
solo para decir qué tan confiable es. Esta regla no aplica a los manuales, guías ni artículos: sus
cifras son de esos documentos y se atribuyen a ellos, no al banco.
8. Responde en español, en una lista de 3 a 4 puntos breves que empiecen con "- ", sin introducción
ni despedida. Traduce al español lo que esté en inglés."""
```

La aplicación revisa cada respuesta: marca las citas a fragmentos inexistentes y muestra en gris, como «sin fuente», las líneas sin cita; cada cita se puede abrir para ver el fragmento original y el documento en la página citada (Figura 45). Una prueba automática verifica que el módulo del asistente no tiene acceso al puerto serie.

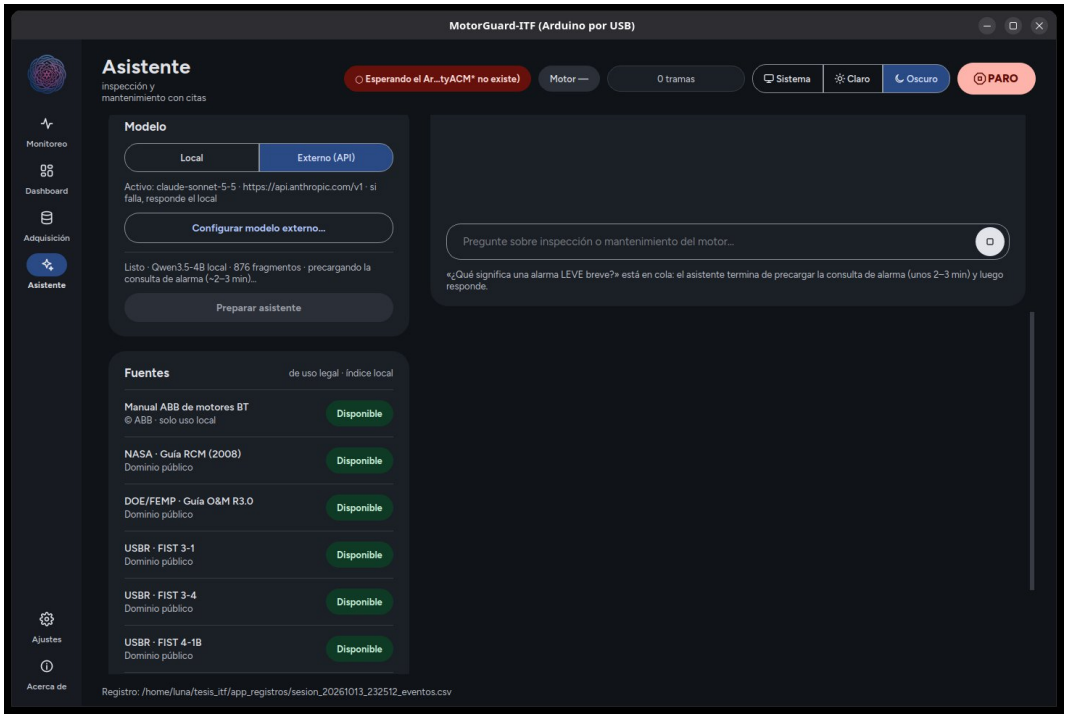


Figura 45. *Sección Asistente en tema oscuro: modelo local preparado, fuentes disponibles y conversación.*

3.3.6.5 CONSULTA AUTOMÁTICA AL ALARMAR

Al confirmarse una alarma, el asistente genera las recomendaciones una vez por evento. No lo hace si la alarma la provocó el operador (una serie de inyección, una captura o un pulso en los últimos 30 s), y la consulta automática se puede desactivar. El contexto de la alarma lleva solo valores de la trama: severidad, magnitud de la firma, $|I_1|$, corrientes por fase, tiempo con alarma y decisión del firmware. A los fragmentos recuperados se agregan siempre las secciones de falsas alarmas y de limitaciones del extracto de la tesis, para que la recomendación recuerde que una alarma LEVE breve es compatible con una falsa alarma del motor sano. Como la consulta de una alarma usa una búsqueda fija, el principio del mensaje es igual en todas las alarmas; la aplicación lo procesa una vez al arrancar (precalentado) y, al alarmar, reutiliza ese cálculo y solo procesa los valores medidos. La alarma del detector no espera al asistente: la emite el firmware y la aplicación la muestra en cuanto llega la trama.

3.3.6.6 MODELO EXTERNO OPCIONAL

Opcionalmente, el usuario puede configurar un modelo externo por API de cualquier proveedor (Figura 46). La aplicación admite dos protocolos: el compatible con la API de OpenAI, que usan la mayoría de los proveedores y los servidores locales, y el nativo de Anthropic. La clave se guarda en el llavero del sistema, nunca en archivos ni en registros; antes del primer uso se avisa qué datos salen de la computadora, incluidos los fragmentos del manual de ABB; si el servicio falla, responde el modelo local, y cada respuesta muestra el nombre del modelo que la generó. El modelo local sigue siendo el sistema principal de la tesis.

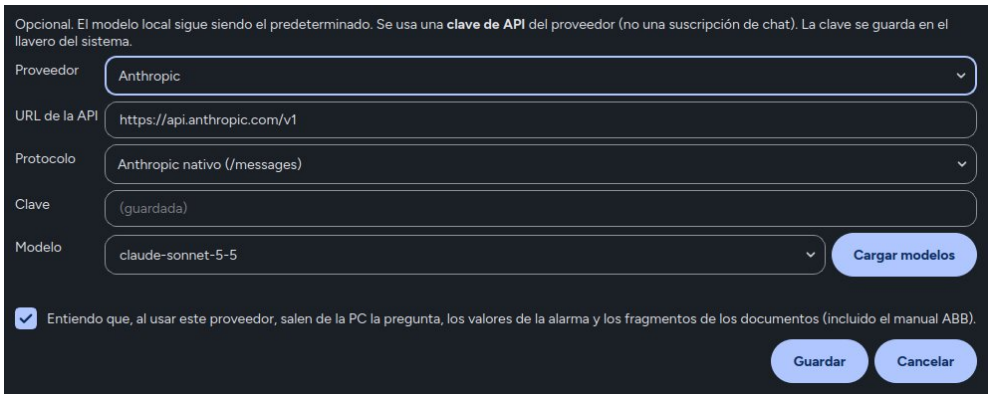


Figura 46. *Configuración del modelo externo por API: proveedor Anthropic, protocolo nativo, clave guardada en el llavero sin mostrarse y aviso de los datos que salen de la computadora.*

3.3.6.7 PROTOCOLO DE EVALUACIÓN

El asistente se evalúa con preguntas que tienen una respuesta esperada y la página que la contiene, e incluyen preguntas sin respuesta en los documentos. Las métricas automáticas son el *recall@5* (la página esperada está entre los cinco fragmentos recuperados), si la respuesta cita la página esperada y la abstención en las preguntas sin fuente. Las métricas calificadas a mano son la corrección de la respuesta y la presencia de afirmaciones inventadas. Como la recuperación se ajustó con un primer conjunto de preguntas, el resultado del asistente se mide con un segundo conjunto independiente (set 2) de 24 preguntas, redactado sin correr el sistema y evaluado una sola vez con el sistema congelado, comparando el modelo local con un modelo externo que recibe los mismos fragmentos. Los resultados se presentan en la sección 4.1.11.

3.3.6.8 RECORRIDO DEL ASISTENTE

Las figuras de esta sección se tomaron en el mismo modo de demostración, pero el asistente es el real: el modelo Qwen3.5-4B local, sin precarga de la consulta de alarma, con el índice de 936 fragmentos y, cuando se indica, Claude Sonnet 5.5 por la API de Anthropic con esfuerzo bajo. Solo la alarma proviene de la demostración. Los tiempos de respuesta que aparecen en las capturas son ilustrativos y no son mediciones declaradas; las cifras reportables son las de la sección 4.1.11.

Asistente listo. La sección Asistente (Figura 47) muestra el aviso «Apoyo, no diagnóstico»; la tarjeta de la alarma, con los valores medidos de la trama, el botón «Recomendar para esta alarma», deshabilitado mientras no hay alarma, y las opciones de recomendar al confirmarse una alarma y de precargar la consulta; la tarjeta del modelo; el estado del índice, y la conversación vacía con preguntas sugeridas.

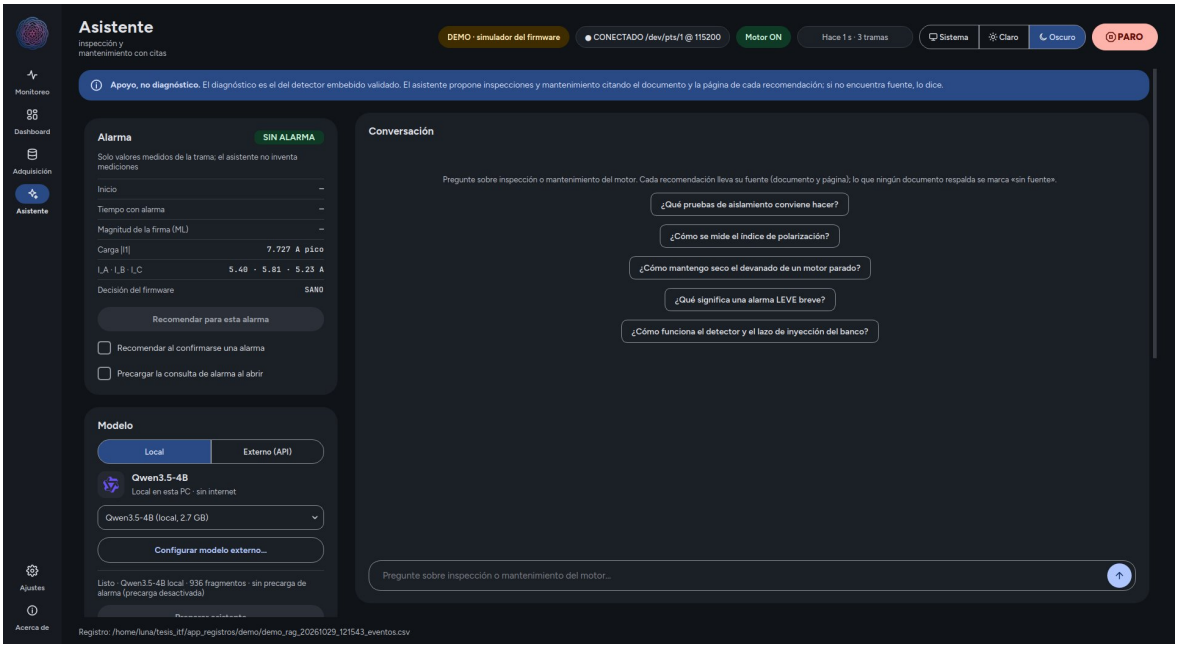
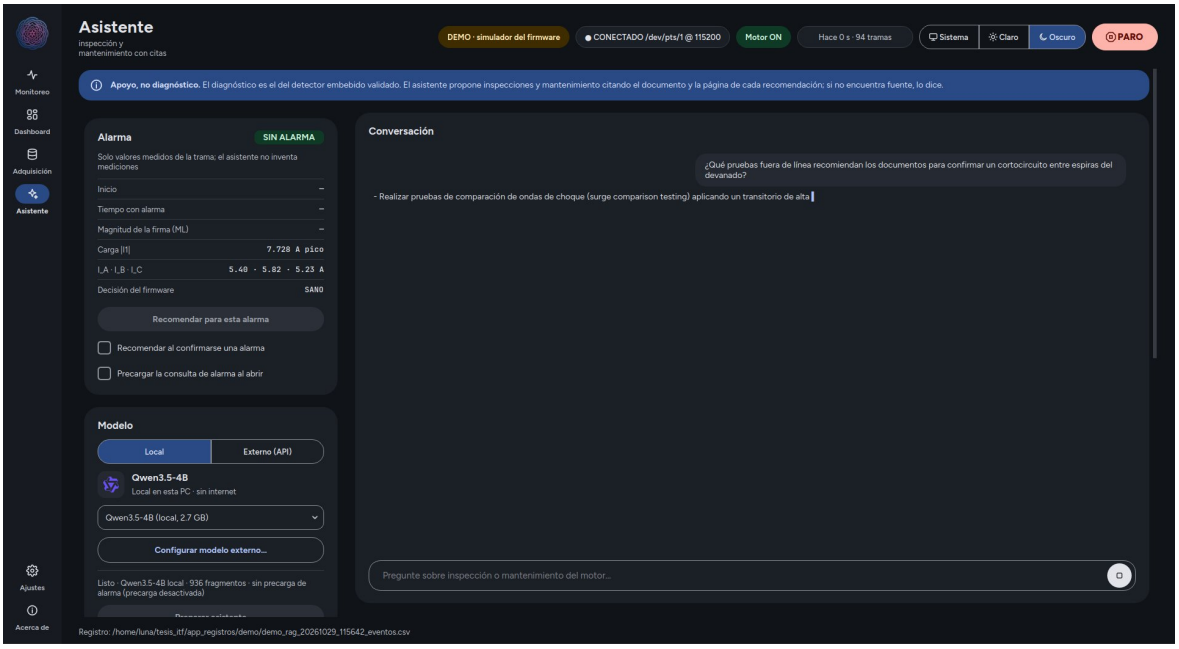
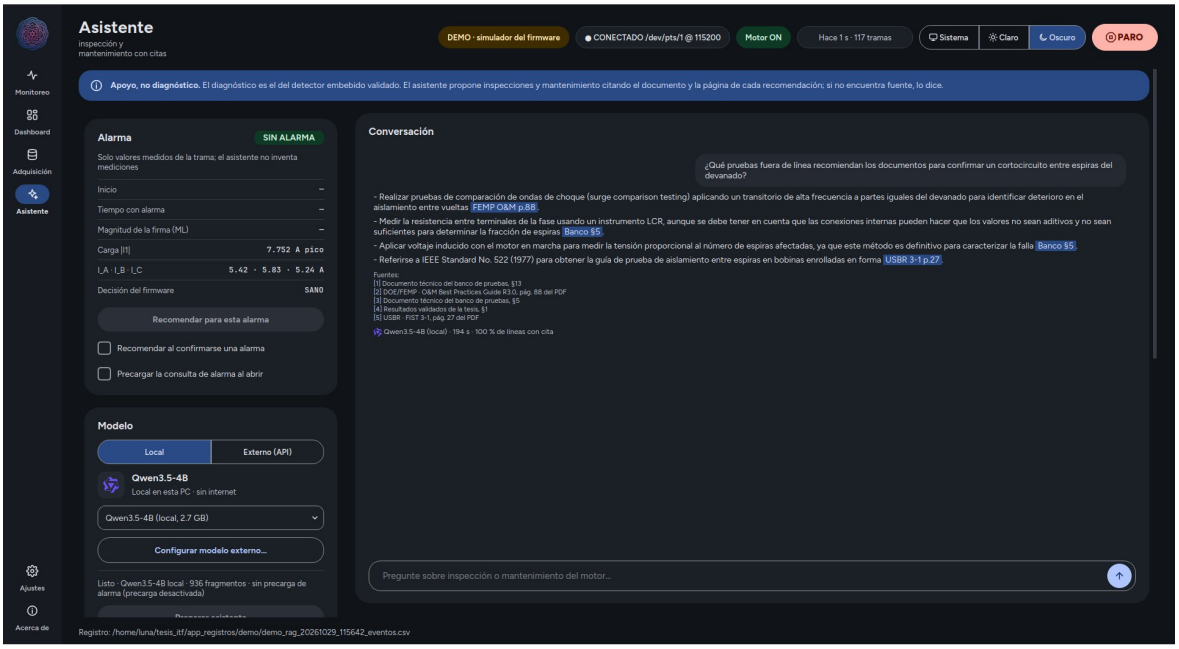


Figura 47. *Sección Asistente lista con el modelo local: aviso de apoyo, tarjeta de la alarma sin alarma, tarjeta del modelo y preguntas sugeridas (asistente real; modo de demostración).*

Pregunta libre. Ante la pregunta «¿Qué pruebas fuera de línea recomiendan los documentos para confirmar un cortocircuito entre espiras del devanado?», que no pertenece a ningún set de evaluación, el modelo local escribe la respuesta en vivo y termina cada punto con su cita (Figura 48). Todas sus líneas llevan cita, pero dos de los cuatro puntos proponen como pruebas la medición de la resistencia entre terminales y del voltaje inducido, que el documento técnico del banco describe como métodos para caracterizar sus derivaciones (Banco §5), no para confirmar una falla en un motor en servicio. Es otro ejemplo de que una línea citada puede aplicar mal su fuente. Cada cita se abre en la tarjeta del fragmento citado (Figura 49), con el texto original y un botón para abrir el documento en esa página.



(a)



(b)

Figura 48. *Respuesta del modelo local a una pregunta libre: (a) mientras escribe; (b) terminada, con sus citas y la lista de fuentes. Dos de los puntos aplican métodos de caracterización del banco como pruebas de confirmación (asistente real; modo de demostración; tiempos ilustrativos).*

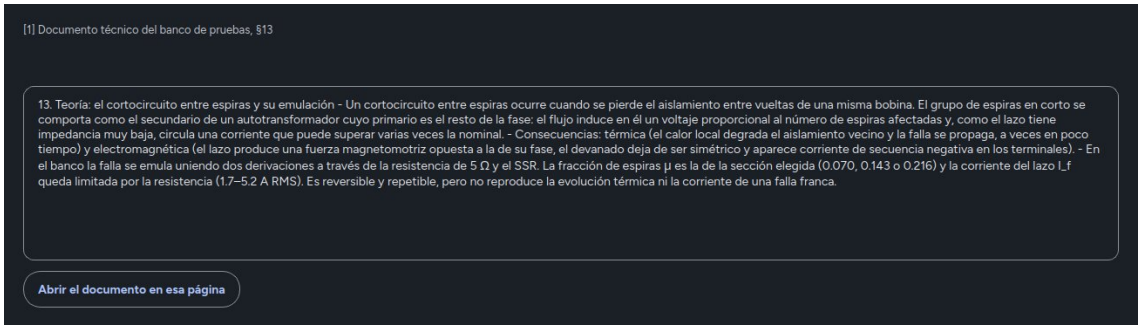
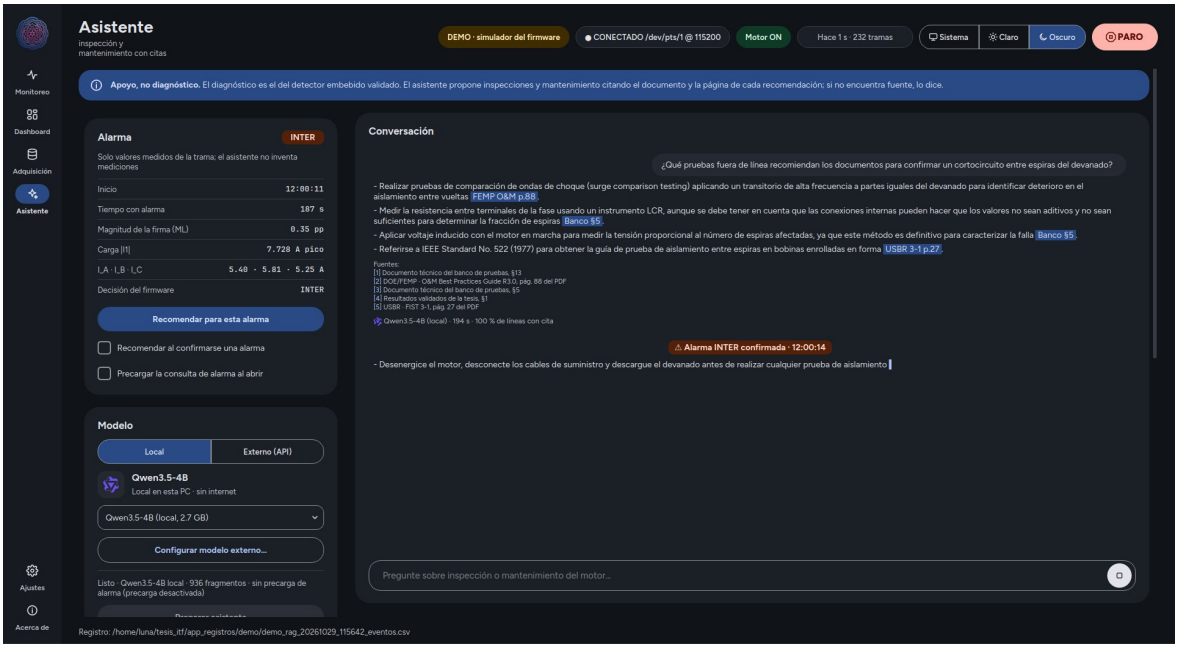
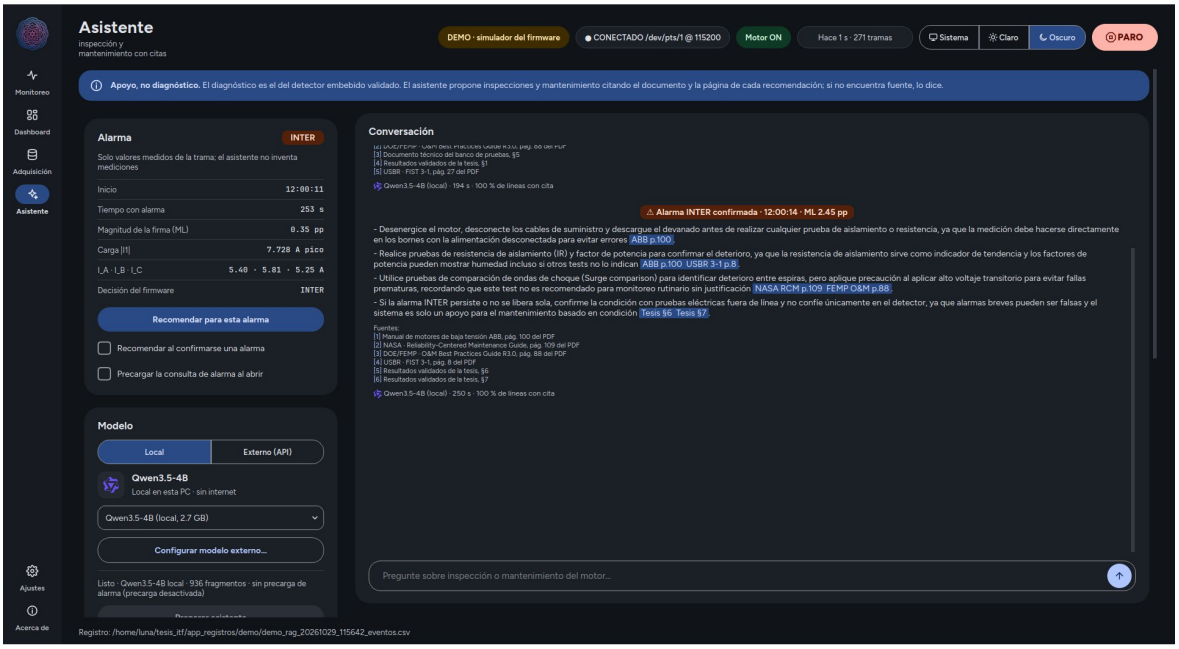


Figura 49. *Tarjeta del fragmento citado: texto original del documento técnico del banco y botón para abrir el documento en esa página (asistente real; modo de demostración).*

Recomendación ante una alarma. Con una alarma INTER confirmada, el botón «Recomendar para esta alarma» genera la recomendación con los valores de la trama (Figura 50). La respuesta pone primero la desenergización del motor, propone pruebas de aislamiento y de comparación de ondas de choque con sus precauciones y recuerda, con las secciones de falsas alarmas y de limitaciones de la tesis, que la alarma debe confirmarse con pruebas fuera de línea. En la tarjeta, ML ya es de 0.35 pp y el tiempo con alarma de 253 s, porque la alarma se mantuvo enclavada mientras el modelo respondía; el valor con el que se pidió la recomendación es el del aviso de la conversación (ML 2.45 pp).



(a)



(b)

Figura 50. *Recomendación del modelo local ante una alarma INTER: (a) mientras escribe; (b) terminada, con la seguridad primero y las citas a las secciones de falsas alarmas y limitaciones de la tesis (asistente real; alarma del modo de demostración; tiempos ilustrativos).*

Modelo externo. El modelo externo se configura en el diálogo de la Figura 46 (sección 3.3.6.6). Con el modelo externo activo (Figura 51) aparece el selector del esfuerzo de razonamiento y la nota de que, si el servicio falla, responde el modelo local. Ante la misma pregunta libre (Figura 52), el modelo externo respondió con tres

puntos citados y una precaución previa marcada «sin fuente», porque los fragmentos no traían pasos de seguridad para esas pruebas: así se ve en la práctica la marca de las líneas sin respaldo. Su recomendación ante la alarma (Figura 53) sigue el mismo orden: precauciones, confiabilidad de la alarma con las secciones de la tesis, pruebas sugeridas y la comparación de impulsos con precaución. En este caso, el contexto enviado ya llevaba ML = 0.35 pp, porque correspondía a la trama enclavada repetida; con la alarma recién confirmada sería del orden de 2.4–2.8 pp.

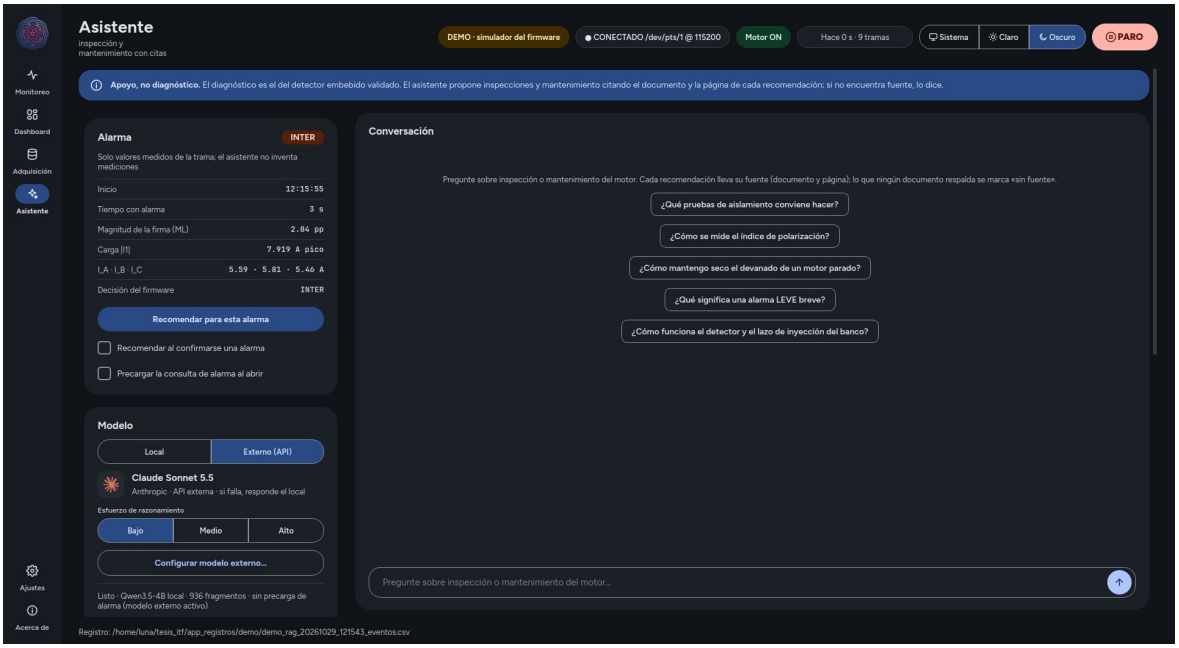
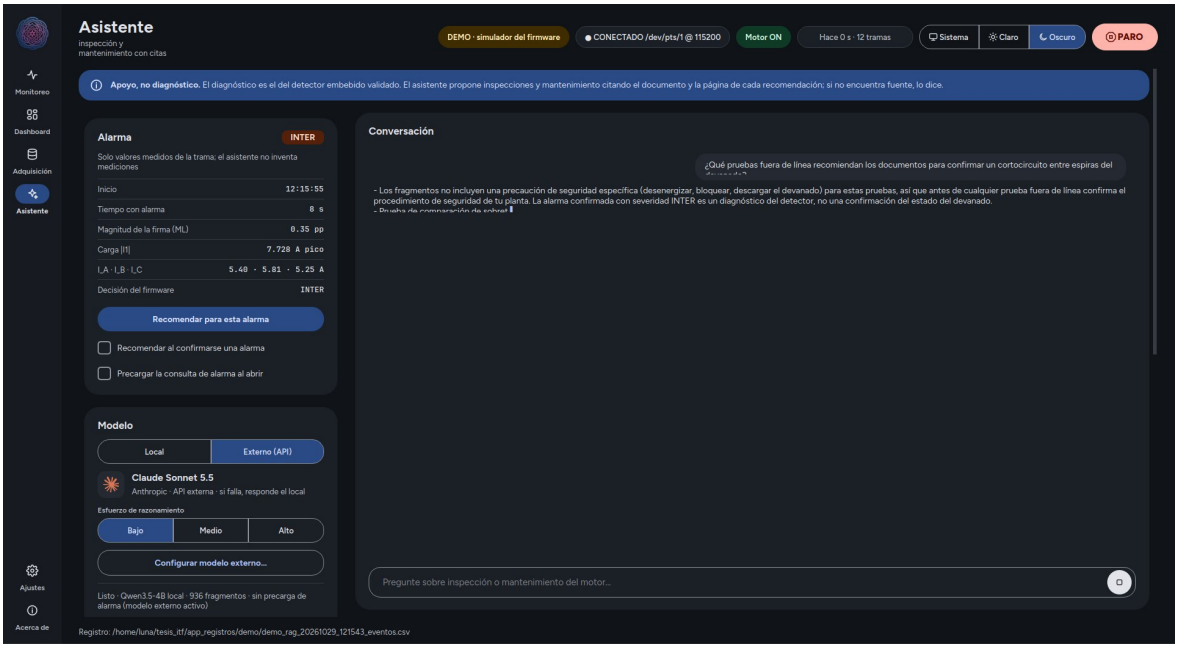
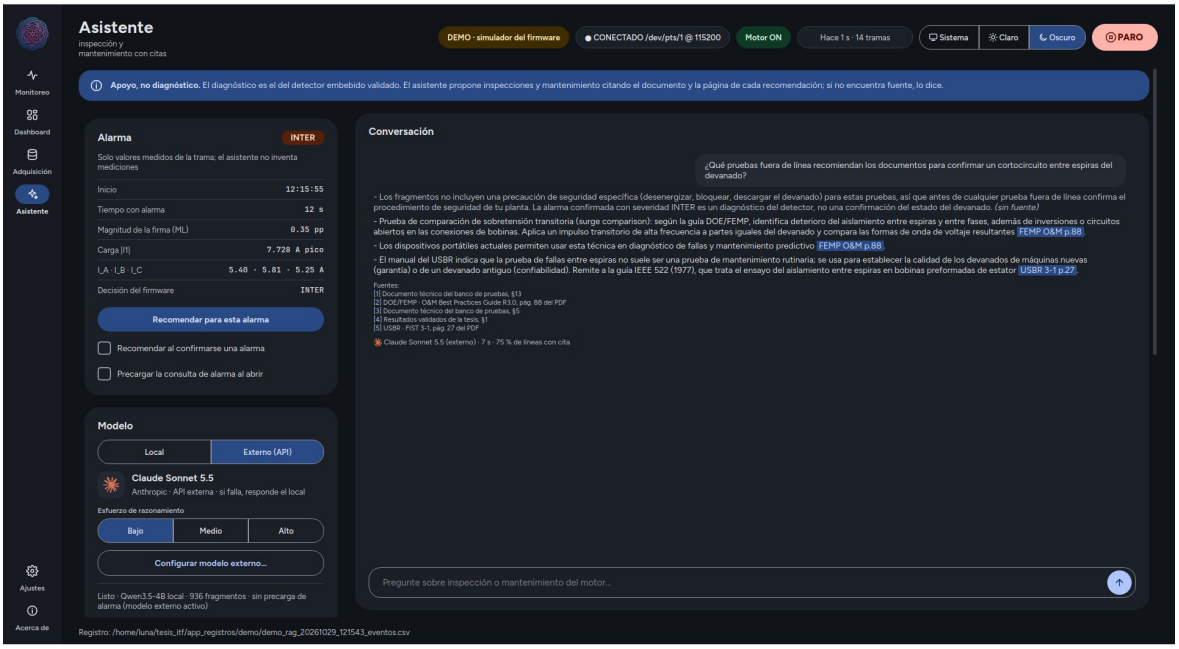


Figura 51. *Tarjeta del modelo con el modelo externo activo (Claude Sonnet 5.5) y el selector del esfuerzo de razonamiento (asistente real; modo de demostración).*



(a)



(b)

Figura 52. *Respuesta del modelo externo a la misma pregunta libre: (a) mientras escribe; (b) terminada, con una precaución marcada «sin fuente» (asistente real; modo de demostración; tiempos ilustrativos).*

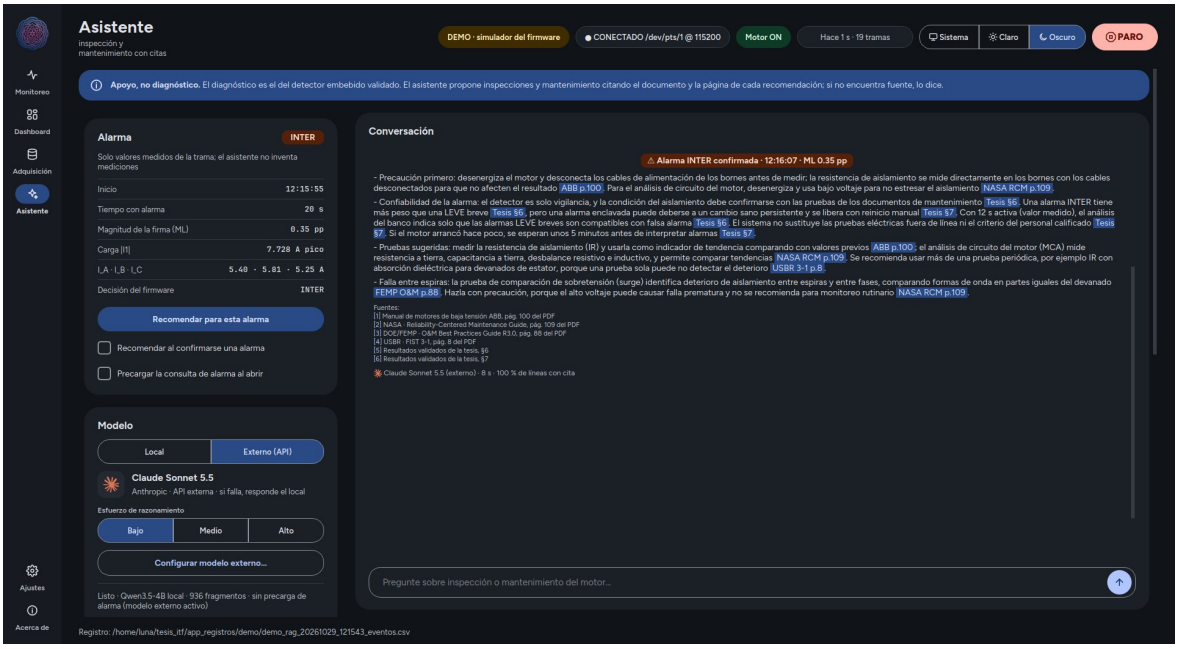


Figura 53. *Recomendación del modelo externo ante la alarma INTER: precauciones, confiabilidad de la alarma, pruebas sugeridas y comparación de impulsos, todas con cita (asistente real; alarma del modo de demostración; tiempo ilustrativo).*

3.3.7 INTEGRACIÓN DEL SISTEMA

La operación integrada sigue un protocolo de sesión definido a partir de la experiencia de los ensayos:

1. **Prueba de provocación:** con el motor detenido y luego en marcha, se perturban deliberadamente las conexiones de la instrumentación y se verifica que las lecturas permanezcan estables.
2. **Arranque:** se energiza el motor con el nivel de carga seleccionado y se espera alrededor de 5 min, más que el calentamiento de 60 s del firmware, porque al arrancar puede aparecer una falsa alarma (sección 4.1.7).
3. **Pulso de prueba:** antes de cada sesión con falla se hace un pulso aislado, fuera del análisis, para verificar con la pinza la corriente del lazo.
4. **Base sana y pulsos:** se registra un periodo sin falla y después se ejecutan los pulsos del plan, registrando de forma continua toda la telemetría.
5. **Análisis:** los registros se procesan fuera de línea con *scripts* de Python que calculan la detección, la latencia, las falsas alarmas y la firma de cada pulso, y la concordancia con la decisión del firmware.

El Listado 4 muestra la parte del *script* de registro de sesiones que implementa el paso 4; los *scripts* completos se incluyen en los Anexos B y D. Este protocolo permite que cada resultado del Capítulo 4 pueda regenerarse a partir de los registros originales.

Listado 4. *Base sana y tren de pulsos en el script de registro de sesiones (adquisicion/sesion_larga_base_pulsos.py, líneas 108–126).*

```
fase = "BASE"
print(f"{time.time() - t0:7.1f}s BASE {args.base_min} min (SSR OFF)", flush=True)
t_base = time.time()
while time.time() - t_base < args.base_min * 60:
    leer(min(300, args.base_min * 60 - (time.time() - t_base)))
    print(f"{time.time() - t0:7.1f}s [BASE] progreso {(time.time() - t_base) /
60:.1f}/{args.base_min} min", flush=True)
```

```
for k in range(1, args.pulsos + 1):
    fase = f"P{k}_ON"
    s.reset_input_buffer()
    s.write(b"1\n")
    s.flush()
    leer(args.t_on, eco=False)
    s.write(b"0\n")
    s.flush()
    fase = f"P{k}_OFF"
    leer(args.t_off, eco=False)
    print(f"{time.time() - t0:7.1f}s pulso {k}/{args.pulsos} completado", flush=True)
print("FIN SESION", flush=True)
```

CAPÍTULO 4. RESULTADOS

En este capítulo se presentan y analizan los resultados obtenidos con el banco de pruebas, el sistema de diagnóstico, la aplicación de escritorio y el asistente de mantenimiento hasta el 17 de octubre de 2026, fecha de los últimos ensayos; las figuras del modo de demostración de la aplicación (secciones 3.3.5.7 y 3.3.6.8) se tomaron el 29 de octubre. El sistema final se organiza en tres etapas. La primera es la detección, a cargo de una regla física sobre la corriente de secuencia negativa calculada en el microcontrolador. La segunda es la estimación de la severidad, a cargo de un clasificador aprendido y embebido en el firmware. La tercera es la recomendación de mantenimiento, a cargo de un asistente de inteligencia artificial generativa con recuperación de documentos (RAG) integrado en la aplicación.

El capítulo sigue el orden de esa arquitectura. Primero se resumen la caracterización de la instrumentación y del devanado, y los antecedentes de julio a principios de octubre que llevaron al diseño final. Después se presentan la firma de la falla por clase y carga, el clasificador de severidad, el detector embebido y su validación ciega en vivo, la prueba larga sin falla y los estudios que comparan la regla física con modelos de aprendizaje automático. Siguen el análisis del ruido de la firma, los resultados de la aplicación y del asistente, y la evaluación del cumplimiento de los objetivos.

Cada resultado se clasifica según su grado de evidencia. Se denomina **validado** al resultado medido con el sistema final y respaldado por registros en archivo; **preliminar** al que se obtuvo con una versión anterior, de forma incompleta o con una limitación declarada; y **pendiente** al que todavía no se ha obtenido. Se distingue además entre los análisis cuyo protocolo se declaró antes de obtener los datos, con el registro de control de versiones como evidencia de su fecha, y los análisis **post hoc**, que se hicieron después de ver los datos y se presentan solo como exploración. El capítulo cierra con la discusión de los hallazgos.

4.1 ANÁLISIS DE RESULTADOS

4.1.1 CALIBRACIÓN Y CARACTERIZACIÓN DE SENSORES

La calibración del canal de la fase C contra el osciloscopio (Tabla 7) dio errores de +3.9 % en voltaje y +2.1 % en corriente, dentro del intervalo esperado considerando la tolerancia del instrumento de referencia y las fluctuaciones de la red. La verificación previa del sistema trifásico sobre una carga resistiva arrojó un factor de potencia de 0.998, coherente con una carga puramente resistiva. Ambos resultados están **validados** para las versiones del sistema en que se obtuvieron.

El 6 de octubre se recalibraron las corrientes de los tres canales (firmware `2dff915`), y todas las capturas y validaciones posteriores se hicieron con esa calibración. Durante las capturas se registraron dos incidentes de instrumentación en el canal de corriente de la fase C: el 6 de octubre hacia las 16:40, el canal perdió 16 % de ganancia por una conexión de la protoboard, que se corrigió a las 16:57 sin recalibrar; y el 8 de octubre hacia las 13:25 perdió alrededor de 12 % por el contacto del divisor resistivo, que se soldó y volvió a coincidir con la pinza amperimétrica sin recalibrar. Además, desde las 16:10 del 6 de octubre los voltajes de las fases A y B mostraron desviaciones de −5 % y +3 % frente al multímetro. Esto no afecta a la firma de la falla, que se calcula con las corrientes, pero sí a las columnas de voltaje de esas capturas.

4.1.2 CARACTERIZACIÓN DEL DEVANADO Y DEL LAZO DE INYECCIÓN

Las mediciones de voltaje inducido (Tabla 6) mostraron que cada fase con derivaciones está dividida en tres secciones de voltaje prácticamente igual, de 8.5 a 9.4 V, cuyas sumas coinciden con el voltaje entre los

terminales extremos: 26.6 V en la fase C y 27.0 V en la fase A. Referidas al voltaje de fase de 123 V, las clases de falla de la fase C corresponden a ≃ 7.0 % (LEVE, terminales 1 y 2), ≃ 14.3 % (INTERMEDIA, terminales 2 y 4) y ≃ 21.6 % (SEVERA, terminales 1 y 4) de las espiras. Estos valores difieren de los de la documentación original del motor (5, 11 y 20 %) y, sobre todo, de la ubicación de las derivaciones que esta indicaba. El resultado está **validado**, con el supuesto de proporcionalidad entre voltaje inducido y número de espiras. Las resistencias medidas con el LCR-819 (Tabla 5) confirmaron que la sección de menor resistencia está entre los terminales 1 y 2, pero no resultaron aditivas, por lo que no se emplearon para definir las clases.

La corriente del lazo de falla se midió con pinza amperimétrica durante la captura del conjunto de datos multiclase, con la resistencia limitadora de 5 Ω (Tabla 10).

Tabla 10. Corriente del lazo de falla medida con pinza durante la captura del conjunto de datos multiclase (6 al 8 de octubre de 2026).

Clase	Corriente del lazo	Observación
LEVE	1.7 A	
INTERMEDIA	3.6–3.7 A a 0 y 25 % de carga; 3.3 A a 50 y 75 %	El valor bajó de forma progresiva de 3.7 a 3.1 A hasta que se sustituyó el relevador por el de repuesto
SEVERA	4.8–5.2 A	Con fusible de 10 A de acción rápida

El lazo de inyección resultó ser la parte menos estable del banco. Sus contactos (relevador, resistencia limitadora y derivaciones) se degradaron varias veces durante las capturas y las validaciones, y fueron la causa de los bloques de validación anulados o contaminados que se describen más adelante; las prácticas que se adoptaron por ello se describen en la sección 3.3.1.3.

4.1.3 ANTECEDENTES: ETAPA EXPLORATORIA (JULIO A PRINCIPIOS DE OCTUBRE DE 2026)

Esta sección resume los resultados obtenidos antes del sistema final. Todos son **preliminares** o se presentan como antecedente: se obtuvieron con *pipelines*, calibraciones o versiones del detector anteriores al firmware final, y su valor principal es que explican por qué el sistema final quedó como quedó.

4.1.3.1 CLASIFICADORES SOBRE CARACTERÍSTICAS MCSA

La Tabla 11 resume los clasificadores entrenados durante la etapa exploratoria.

Tabla 11. Resultados de los clasificadores de la etapa exploratoria (julio a septiembre de 2026).

Modelo y datos	Muestras	Partición	Resultado
XGBoost, datos sintéticos, 4 clases	3200	Aleatoria 80/20	Exactitud de prueba 99.84 %
Random Forest, datos sintéticos con ruido y deriva	45 (prueba)	Conjunto independiente	Exactitud 68.9 %
XGBoost, datos reales, SANO/LEVE, carga baja (julio)	199	Aleatoria 80/20 y validación cruzada de 5 pliegues	Exactitud de prueba 82.5 %; validación cruzada 66.4 % ± 14.0 %
XGBoost, datos reales, SANO/LEVE, cargas media y alta	800	Aleatoria 80/20	Exactitud de prueba 97.5 %
XGBoost, datos reales, 4 clases, tres cargas	3160	Aleatoria 80/20	Exactitud de prueba 96.68 %

La Figura 54 muestra la matriz de confusión del primer modelo XGBoost entrenado con datos reales, en julio, con 40 muestras de prueba: 16 de 20 ventanas sanas y 17 de 20 con falla LEVE se clasificaron correctamente.

La caída de 99.84 % con datos sintéticos a 82.5 % con datos reales es la primera indicación de que el desempeño en condiciones controladas no se traslada directamente al banco.

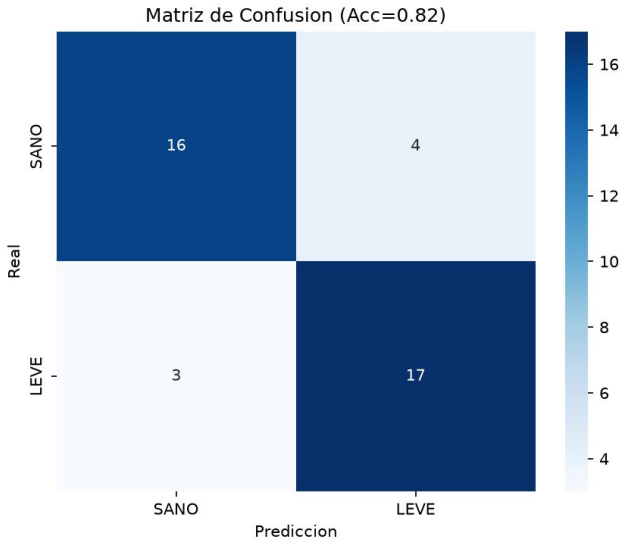


Figura 54. *Matriz de confusión del primer modelo XGBoost con datos reales (SANO/LEVE, carga baja, julio de 2026).*

El modelo de cuatro clases alcanzó una exactitud de 96.68 % sobre 632 muestras de prueba, con la matriz de confusión de la Tabla 12. Cada clase de falla se capturó en sesiones distintas, con diferencias sistemáticas de voltaje entre niveles de carga, y la partición aleatoria mezcló ventanas consecutivas de una misma sesión. Por ello, esta exactitud mide la capacidad de distinguir sesiones de captura, además de fallas, y no debe interpretarse como el desempeño esperado en operación.

Tabla 12. Matriz de confusión del modelo XGBoost de cuatro clases (filas: clase real; columnas: clase predicha).

Real \ Predicha	SANO	LEVE	INTERMEDIA	SEVERA
SANO	235	1	2	2
LEVE	2	155	0	3
INTERMEDIA	4	1	109	1
SEVERA	2	3	0	112

Para cuantificar ese efecto, el Random Forest binario que se embebió entonces en el microcontrolador se evaluó con tres esquemas de partición (Tabla 13). Dentro de la campaña del 11 al 15 de septiembre, la exactitud se mantuvo en torno a 96 % tanto con partición aleatoria como con validación agrupada; sin embargo, al entrenar con esa campaña y evaluar con la del 23 al 24 de septiembre, la exactitud cayó a 49.78 %, equivalente al azar, y en sentido inverso a 5.22 %.

Tabla 13. Desempeño del Random Forest binario de septiembre según el esquema de partición.

Conjunto	Partición aleatoria: exactitud (AUC)	Validación agrupada: exactitud (AUC)	Transferencia a la otra campaña: exactitud (AUC)
11–15 sep (900 muestras)	95.78 % (0.994)	96.00 % (0.995)	49.78 % (0.514)
23–24 sep (900 muestras)	65.00 % (0.696)	66.67 % (0.692)	5.22 % (0.028)

La Figura 55 muestra, para la campaña del 11 al 15 de septiembre con validación agrupada, la curva ROC, la curva de precisión y sensibilidad, el diagrama de fiabilidad antes y después de la calibración de Platt y la distribución de las probabilidades calibradas. Estas curvas describen el desempeño dentro de una misma campaña; no reflejan la caída observada al transferir el modelo. El estudio de ablación sobre esa campaña mostró que las dos características de bandas laterales bastaban para obtener 95.67 %, mientras que la corriente de secuencia negativa sin compensar, por sí sola, alcanzaba 64.67 %.

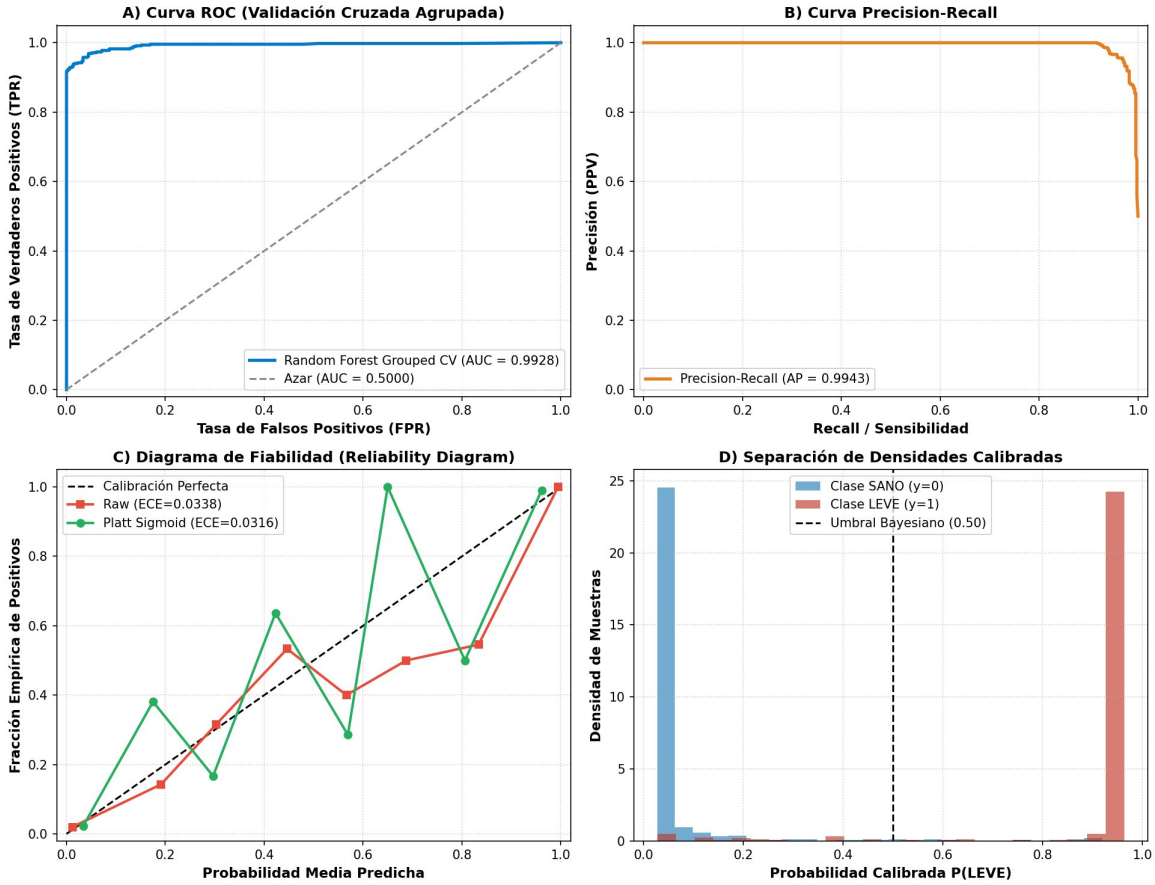


Figura 55. *Evaluación del Random Forest binario con validación cruzada agrupada en la campaña del 11 al 15 de septiembre: curva ROC, curva de precisión y sensibilidad, diagrama de fiabilidad y distribución de probabilidades calibradas.*

4.1.3.2 ENSAYOS DE DETECCIÓN DE SEPTIEMBRE

Entre el 28 y el 29 de septiembre se realizaron doce corridas de inyección de la falla LEVE con el detector de secuencia negativa y CUSUM de entonces, mientras el firmware se modificaba entre corridas. La Tabla 14 presenta todas las corridas. La columna de ciclos detectados aplica el criterio del *script* de ensayo (estado LEVE o estadístico CUSUM ≥ 5 en algún momento del pulso); la última columna informa solo las detecciones nuevas, es decir, los ciclos que empezaron con el detector en estado sano.

Tabla 14. Corridas de inyección de la falla LEVE del 28 y 29 de septiembre de 2026.

Fecha y hora	Carga nominal	Ciclos	Detectados (criterio del *script*)	Con alarma activa al iniciar	Detecciones nuevas
28-sep 13:45	Sin registro (I = 4.50 A)	10	8	0	8 de 10
28-sep 14:03	25 % (I = 4.44 A)	10	10	2	8 de 8
28-sep 14:13	50 %	10	10	4	6 de 6
28-sep 14:30	75 %	6	6	5	1 de 1
28-sep 14:42	75 %	10	4	4	0 de 6
28-sep 15:13	75 %	10	5	3	2 de 7
29-sep 11:27	75 %	10	10	8	2 de 2
29-sep 11:48	75 %	10	0	0	0 de 10
29-sep 11:55	75 %	10	6	1	5 de 9
29-sep 12:02	75 %	10	7	0	7 de 10
29-sep 12:24	50 %	10	8	1	7 de 9
29-sep 12:34	25 %	10	3	3	0 de 7
Total		**116**	**77**	**31**	**46 de 85 (54 %)**

Las dos primeras corridas no registran la carga en su archivo. La de las 14:03 aparece como de 25 % de carga en un reporte del mismo 28 de septiembre; la de las 13:45 no tiene registro, aunque su corriente media, cercana a la de la corrida de 25 % del 29 de septiembre (4.42 A), sugiere el mismo nivel. Las mejores corridas de cada carga suman 30 ciclos detectados de 30, que es el resultado reportado inicialmente para esta campaña (F1 = 1.000). El análisis de todos los registros cambia esa lectura: en 31 de los 116 ciclos la alarma ya estaba activa antes del pulso, y de los 85 ciclos que comenzaron en estado sano el detector identificó la falla en 46 (54 %) (Figuras 56 y 57). Estos ensayos tampoco incluyeron periodos prolongados de motor sano para medir las falsas alarmas.

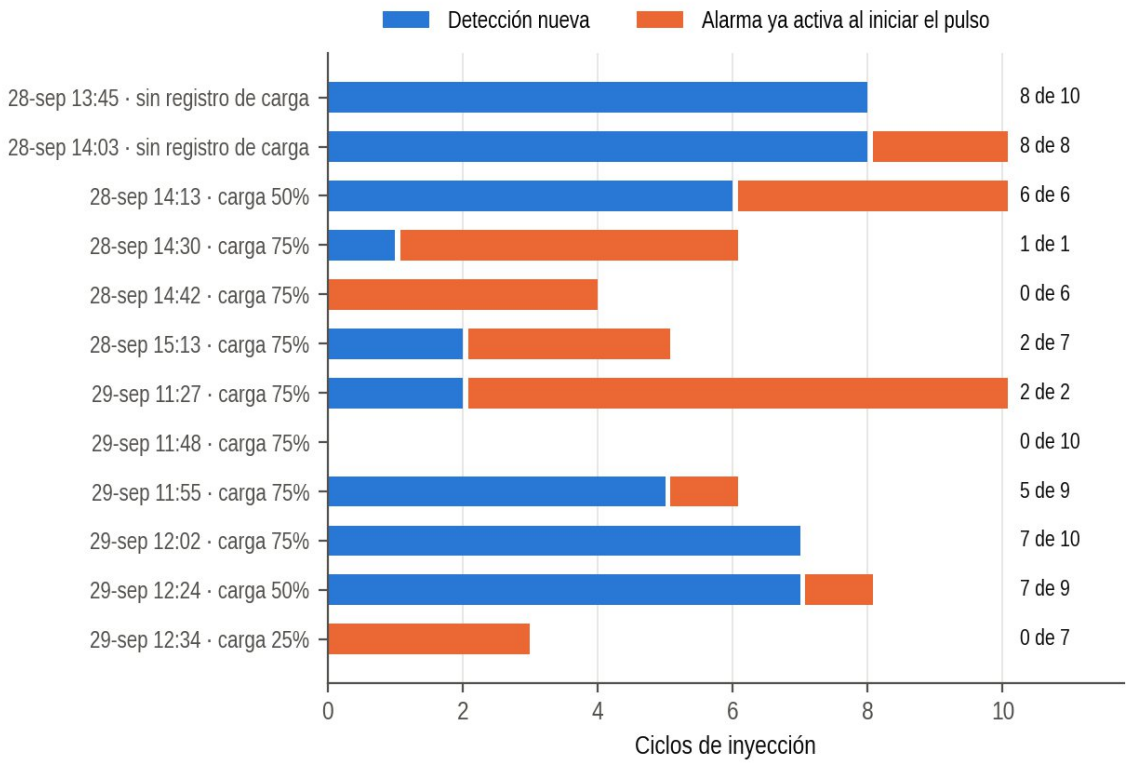


Figura 56. *Resultado de cada corrida de inyección de la falla LEVE de septiembre: detecciones nuevas y ciclos que iniciaron con la alarma ya activa. A la derecha, detecciones nuevas sobre ciclos que iniciaron en estado sano.*

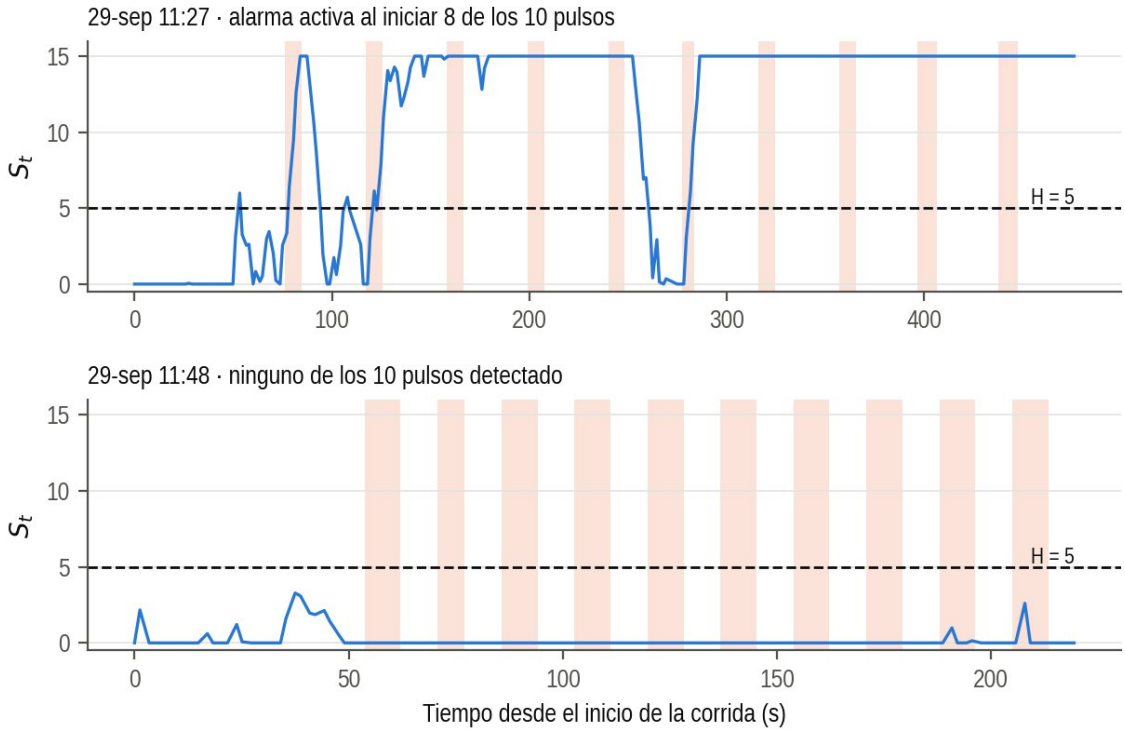


Figura 57. *Estadístico CUSUM en dos corridas consecutivas de septiembre a 75 % de carga. Las franjas indican los pulsos de inyección.*

El 29 de septiembre se realizó además un ensayo de puente manual, en el que las derivaciones de la fase C se unieron a mano sin intervención del relevador (Figura 58). En un registro continuo de 118 s, el detector cambió a LEVE en dos ocasiones (a los 53.2 s y a los 86.1 s) y regresó a SANO después de cada contacto (a los 62.2 s y a los 92.9 s). El ensayo confirmó que la detección no depende de ninguna señal del actuador; las latencias no pueden calcularse porque el instante del contacto manual no quedó registrado.

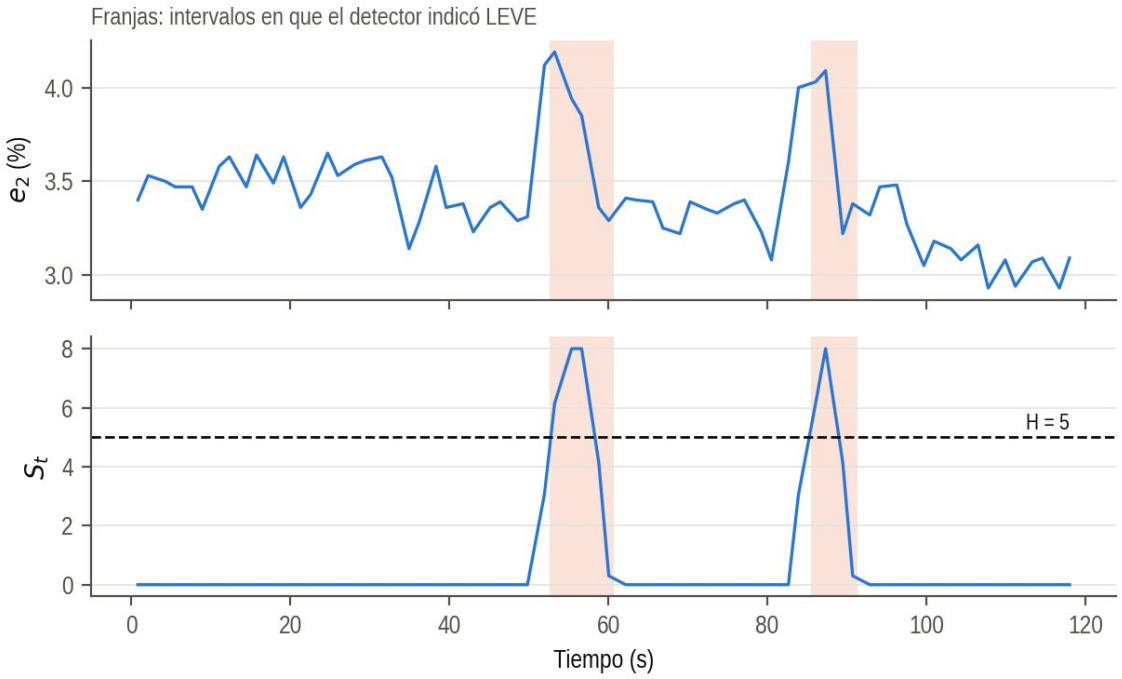


Figura 58. *Ensayo de puente manual directo del 29 de septiembre: residuo de secuencia negativa (arriba) y estadístico CUSUM (abajo). Las franjas indican los intervalos en que el detector indicó falla LEVE.*

4.1.3.3 RUIDO POR CONEXIONES Y PRIMERA FIRMA DE LA FALLA (1 DE OCTUBRE)

El 1 de octubre se registró una sesión de 30 min con el motor sano a carga media. El detector del firmware de entonces produjo 28 falsas alarmas, casi una por minuto, y al simular sobre ese registro el detector de septiembre se obtuvieron 17. El análisis mostró que las corrientes de las fases B y C fluctuaban entre 1.5 y 2.0 %, frente a 0.4 % en la fase A, y que al reinsertar los cables de los sensores, sin tocar los de potencia, las fluctuaciones de los tres canales bajaban a 0.2–0.4 % (Tabla 15 y Figura 59). La causa es coherente con el principio de operación del ACS712, cuya salida es ratiométrica respecto de su alimentación. Este hallazgo llevó a rehacer el conexionado y a establecer la prueba de provocación antes de cada sesión.

Tabla 15. Efecto del estado de las conexiones de la instrumentación (registros del 1 de octubre de 2026, carga media).

Indicador	Conexiones degradadas	Conexiones rehechas
Fluctuación relativa de corriente (fases B y C)	1.5–2.0 %	0.2–0.4 %
Desviación estándar de la proyección del residuo	0.58 pp	0.22 pp
Falsas alarmas del firmware en base sana	28 en 30 min	0 en 10 min
Detección de la falla LEVE	12–17 % (semisintética)	8 de 8 pulsos reales (detector de escalón fuera de línea)

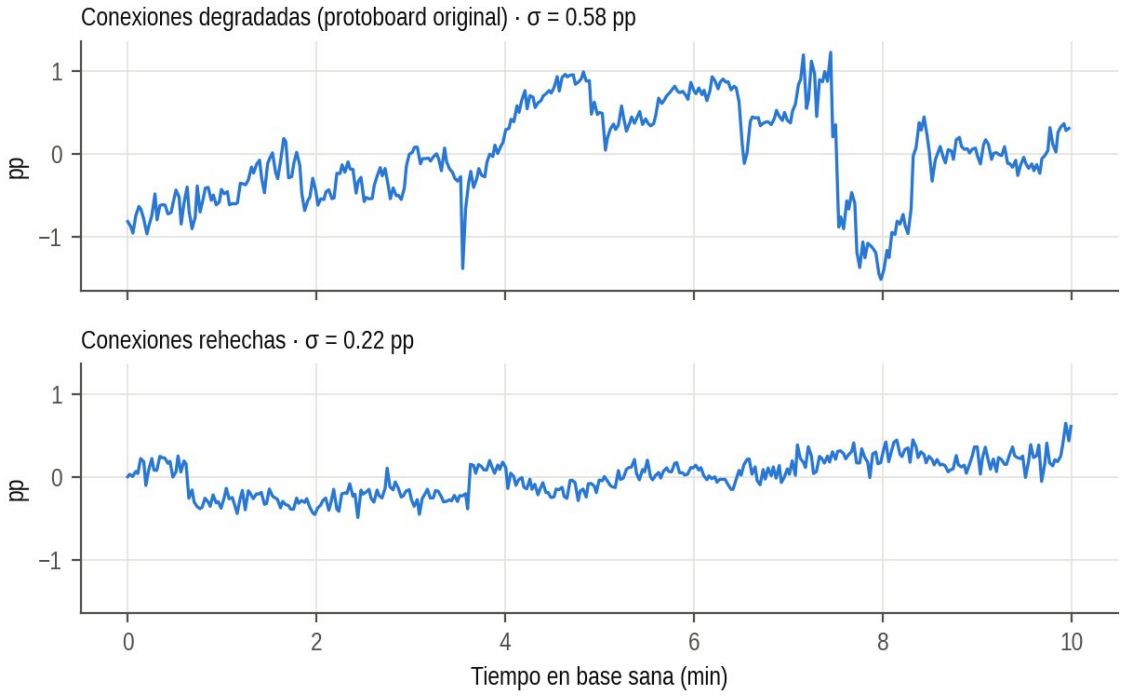


Figura 59. *Proyección del residuo de secuencia negativa durante diez minutos de base sana, con las conexiones de la instrumentación degradadas (arriba) y rehechas (abajo), 1 de octubre de 2026.*

Con las conexiones rehechas se midió por primera vez una firma reproducible de la falla LEVE: diez pulsos produjeron un cambio del residuo de 0.74 ± 0.14 puntos porcentuales en una dirección de 160.6° ± 7.3° respecto del voltaje de secuencia positiva (Figura 60). Un detector direccional de escalón evaluado fuera de línea sobre ese registro detectó los ocho primeros pulsos con una latencia media de 2.8 s (Figura 61). Este resultado dio origen al detector direccional del firmware final. La dirección medida ese día no coincide con la del conjunto de datos de octubre (−81.4°, sección 4.1.4), que se capturó después del 6 de octubre. El cambio ocurrió entre el 5 y el 6 de octubre, cuando se reubicó la electrónica fuera de la zona de vibración, se reconectaron los seis sensores y se recalibraron las corrientes; el primer pulso LEVE después de esos cambios ya dio −78.6°. No se debe a la referencia angular del firmware, que es la misma en ambas versiones, ni lo explica la recalibración por sí sola, que solo cambió las ganancias de corriente por factores de 0.92 a 1.12. No se identificó el mecanismo exacto. Por eso, la firma del 1 de octubre se presenta solo como antecedente.

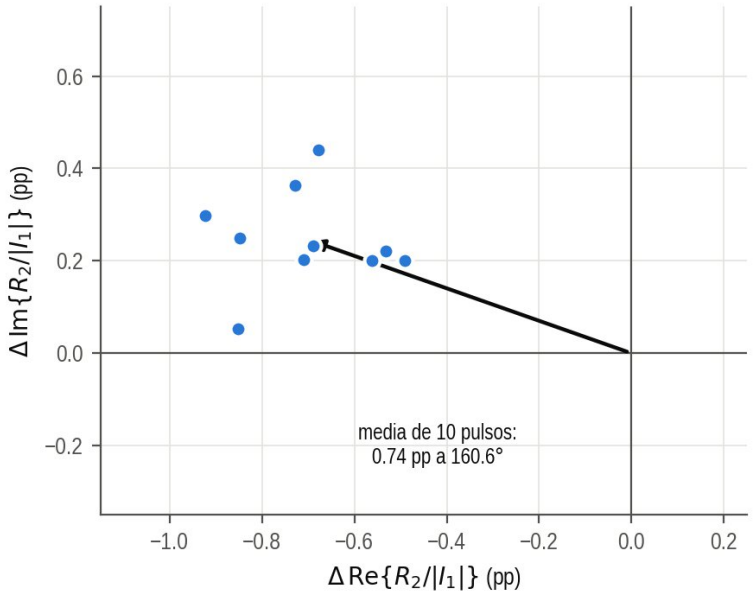


Figura 60. *Cambio del residuo de secuencia negativa producido por cada uno de diez pulsos de falla LEVE (puntos) y su valor medio (flecha), 1 de octubre de 2026, firmware anterior a la recalibración.*

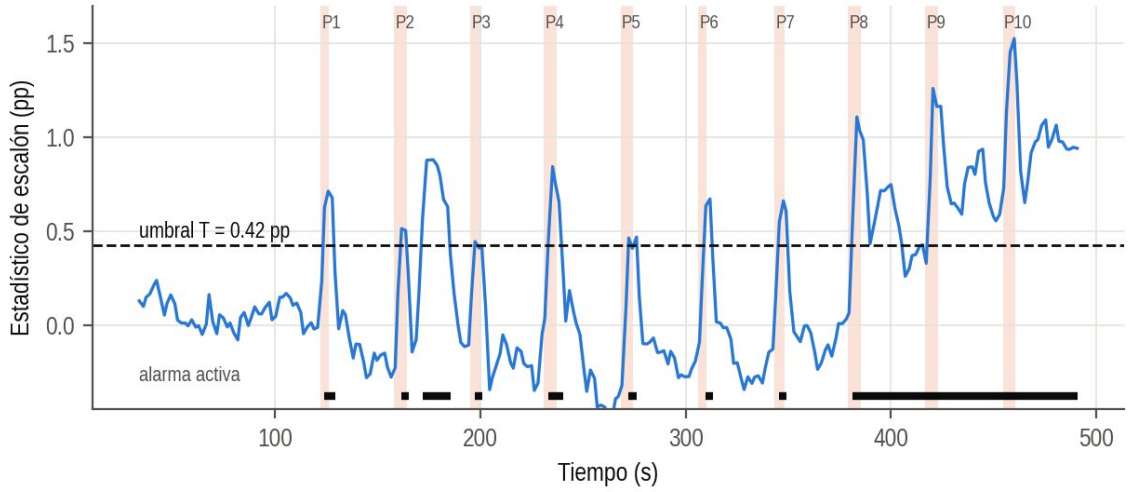


Figura 61. *Estadístico del detector direccional de escalón evaluado fuera de línea sobre la sesión de diez pulsos LEVE del 1 de octubre. Las franjas indican los pulsos de inyección y la barra inferior el estado de alarma.*

4.1.4 FIRMA DE LA FALLA POR CLASE Y CARGA

Entre el 6 y el 8 de octubre se capturó el conjunto de datos multiclase con el protocolo de pares entrelazados: cada par consta de un bloque con el motor sano seguido de un bloque con el relevador encendido. La variable de

la firma es el cambio del residuo de secuencia negativa normalizado, $\Delta R_2 = (R_{2,falla} - R_{2,sano})/|I_1|$, en puntos porcentuales (pp) de $|I_1|$, con su dirección medida respecto de $\arg(V_1)$. La Tabla 16 resume los 597 pares del bloque principal. El resultado está **validado**.

Tabla 16. Pares y firma de la falla por clase y carga (bloque principal de 597 pares, 6 al 8 de octubre de 2026).

Clase	Carga	Pares	Mediana |ΔR2| (pp)	Mínimo–máximo (pp)	Dirección mediana	|I1| mediana (A pico)
LEVE	0 %	50	0.95	0.77–1.11	−80.4°	5.656
LEVE	25 %	50	0.82	0.48–1.32	−80.7°	5.987
LEVE	50 %	50	0.69	0.25–1.17	−81.6°	7.104
LEVE	75 %	50	0.65	0.31–0.88	−75.9°	7.756
INTERMEDIA	0 %	50	3.71	3.35–4.28	−82.4°	5.706
INTERMEDIA	25 %	50	3.16	2.70–3.41	−81.3°	6.114
INTERMEDIA	50 %	50	2.38	2.04–2.86	−78.6°	7.063
INTERMEDIA	75 %	50	2.14	1.93–2.45	−80.7°	7.725
SEVERA	0 %	50	7.10	6.00–7.74	−85.1°	5.539
SEVERA	25 %	50	6.99	6.44–7.38	−83.6°	6.102
SEVERA	50 %	50	5.57	4.80–6.10	−83.8°	7.025
SEVERA	75 %	47	5.21	4.66–5.49	−82.7°	7.739

Los datos muestran tres propiedades, visibles también en la Figura 62:

1. **La dirección de la firma es estable** en las tres severidades y las cuatro cargas: las medianas van de −75.9° a −85.1°. El detector usa −81.4°, la dirección media de la clase LEVE.
2. **La magnitud crece con la severidad y disminuye con la carga** dentro de cada clase. De 0 a 75 % de carga, la mediana pasa de 0.95 a 0.65 pp en LEVE, de 3.71 a 2.14 pp en INTERMEDIA y de 7.10 a 5.21 pp en SEVERA.
3. **Las clases no se traslapan en ninguna carga.** El máximo de LEVE (1.32 pp) queda por debajo del mínimo de INTERMEDIA (1.93 pp), y el máximo de INTERMEDIA (4.28 pp) por debajo del mínimo de SEVERA (4.66 pp).

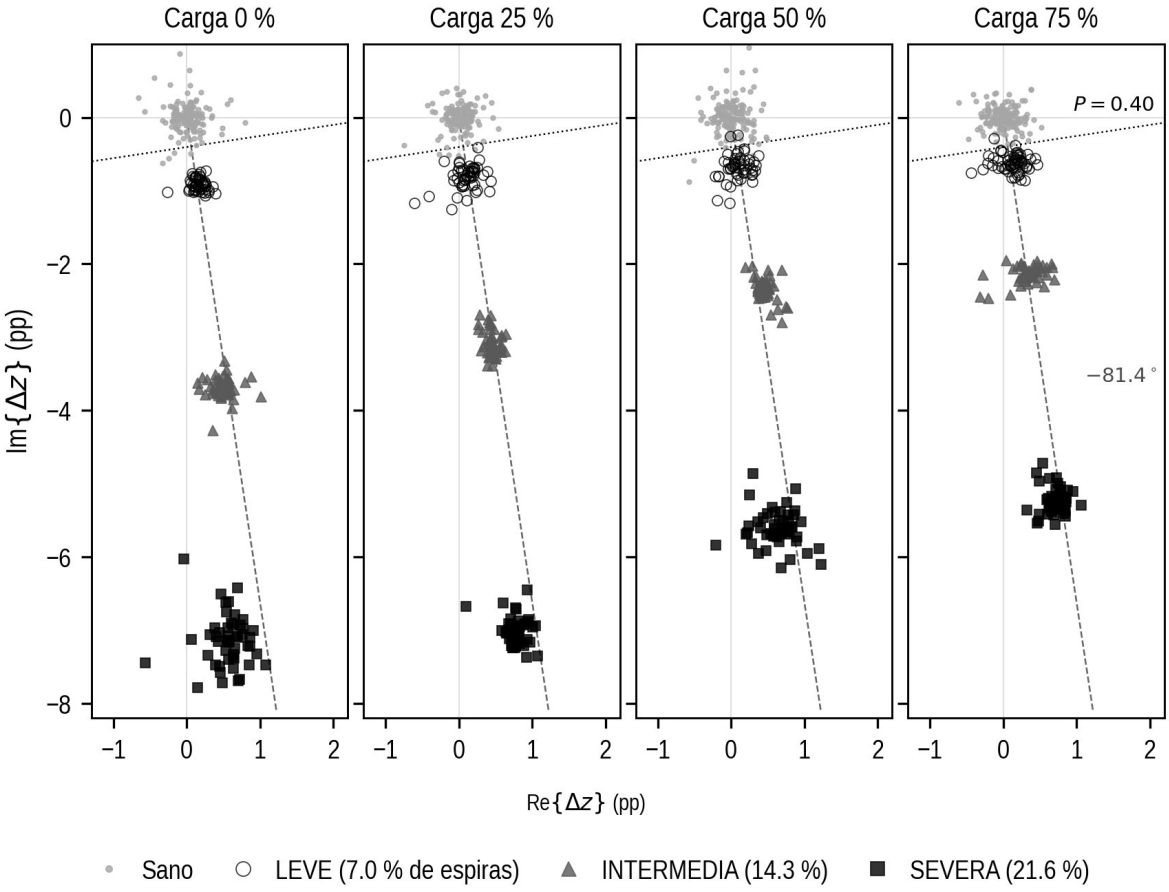


Figura 62. *Cambio de la firma de secuencia negativa Δz de cada par en el plano complejo, por clase y carga. La línea discontinua indica la dirección de falla de −81.4° y la punteada el umbral de detección P = 0.40 pp.*

Una repetición de la clase LEVE a 75 % en otra sesión, el 7 de octubre, dio una mediana de 0.67 pp con 10 pares, coincidente con los 0.65 pp del bloque principal; no se incluyó en el bloque principal y se reservó para la validación del clasificador de severidad. Todas las exclusiones se documentaron: en SEVERA a 0 % se excluyó un par sin inyección; en SEVERA a 25 % se excluyeron cinco pares porque la resistencia limitadora se soltó; en SEVERA a 75 % quedaron 47 pares porque el lazo se abrió cuatro veces; y en INTERMEDIA a 50 % se descartó un primer intento porque la corriente del lazo bajó a 3.3 A.

4.1.5 CLASIFICADOR DE SEVERIDAD

El clasificador de severidad es la etapa de aprendizaje automático del sistema. Su diseño, declarado antes de ver los resultados (*commit* `dcff8fe`), con los modelos M1, M2 y M3 y las validaciones V1 (dejando una carga fuera) y V2 (con la repetición de LEVE de otra sesión), se describe en la sección 3.3.3.2. Los resultados (Tabla 17) están **validados**.

Tabla 17. Resultados del clasificador de severidad con validación dejando una carga fuera (V1) y con la repetición de otra sesión (V2).

Modelo	Exactitud V1	IC 95 % (Wilson)	F1 macro	V2 (repetición)
M1: umbrales sobre |ΔR2|	**596/597 = 0.998**	0.991–1.000	0.998	10/10
M2: regresión logística [|ΔR2|, |I1|]	597/597 = 1.000	0.994–1.000	1.000	10/10
M3: árbol [|ΔR2|, proyección, |I1|]	596/597 = 0.998	0.991–1.000	0.998	10/10

Tabla 18. Matriz de confusión del modelo M1 con validación dejando una carga fuera (filas: clase real; columnas: clase predicha).

Real \ Predicha	LEVE	INTERMEDIA	SEVERA
LEVE	200	0	0
INTERMEDIA	0	199	1
SEVERA	0	0	197

El único error de M1 (Tabla 18) es un par INTERMEDIA a 0 % de carga con $|\Delta R_2|$ = 4.28 pp, clasificado como SEVERA porque, al dejar fuera esa carga, el umbral aprendido fue de 4.03 pp. La exactitud por carga dejada fuera fue de 0.993 a 0 % (n = 150) y de 1.000 a 25, 50 y 75 % (n = 150, 150 y 147). Entrenados con todo el bloque, los umbrales finales son **1.626 pp** entre LEVE e INTERMEDIA y **4.471 pp** entre INTERMEDIA y SEVERA. Se eligió M1 aunque M2 alcanza 597/597, porque es el más simple de embeber en el microcontrolador y de explicar, y la diferencia es un solo par. La limitación declarada es que cada severidad se capturó en un día distinto, de modo que la clase está confundida con la sesión de captura; la validación V2 solo descarta ese efecto para la clase LEVE.

4.1.6 DETECTOR DIRECCIONAL EMBEBIDO

El detector direccional del firmware final (`299a943`), con su enclavamiento y su severidad embebida, se describe en la sección 3.3.4: confirma la alarma cuando la proyección del cambio de la firma sobre la dirección de falla de −81.4° supera 0.40 pp de $|I_1|$ durante 3 bloques consecutivos (alrededor de 5 s).

Para verificar que el detector embebido hace lo que se diseñó, el *script* de validación recalcula en la computadora el detector sobre los fasores crudos y lo compara bloque a bloque con la decisión del microcontrolador. La concordancia fue total en cuatro sesiones (806/806, 382/382, 385/385 y 379/379 bloques) y de 254/257 en una quinta, donde las tres diferencias coinciden con el arranque del motor. La equivalencia entre la computadora y el firmware está **validada**.

4.1.7 VALIDACIÓN CIEGA EN VIVO

La validación en vivo sigue un plan aleatorio, generado con una semilla registrada, que mezcla pruebas reales (un pulso de 11 s del relevador) y simuladas (sin pulso). Se cuenta una detección si hay una alarma confirmada entre el inicio de la prueba y 2 s después del final del pulso. Desde el protocolo v5 (*commit* `7c5707f`) se puntúan además la severidad reportada por el microcontrolador, el enclavamiento y el tiempo de liberación. La Tabla 19 reúne los bloques realizados con el firmware final a 75 % de carga.

Tabla 19. Validación ciega en vivo con el firmware final a 75 % de carga (8 y 9 de octubre de 2026).

Bloque	Estado	Reales TP/FN	Simuladas TN/FP	Severidad correcta	Enclavadas	Liberación	Concordancia PC/firmware	Tiempo sin pulso	Falsas alarmas
SEVERA 08-oct 16:12	**Validado, completo**	**10/0**	**10/0**	**10/10**	**10/10**	5.1–6.1 s	385/385	4.1 min	0
INTER 09-oct 15:13	**Validado, completo**	**10/0**	**10/0**	**10/10**	**10/10**	5.1–6.1 s	379/379	3.6 min	0
SEVERA 08-oct 16:37	Incompleto (6/20; se apagó el motor)	2/0	4/0	2/2	2/2	5.4–5.9 s	117/117	1.3 min	0
INTER 08-oct 16:28	Incompleto (12/20; se apagó el motor)	5/0	6/1	5/5	5/5	5.1–5.9 s	232/232	2.4 min	1
INTER 09-oct 15:01	Incompleto (13/20; se apagó el motor)	6/1	6/0	6/6 detectadas	5	0.0–6.1 s	254/257	2.5 min	2 (arranque del motor)
LEVE 09-oct 15:26	Anulado: derivaciones mal conectadas	0/3	1/0	—	—	—	94/95	1.1 min	0
LEVE 09-oct 15:46	Contaminado: el lazo se degradó (1.7 → 1.3 A)	2/4	5/0	2/2	2/2	5.9 s	225/225	2.2 min	0

Los dos bloques completos con el firmware final, SEVERA e INTERMEDIA a 75 %, dieron sensibilidad de 10/10, especificidad de 10/10, severidad correcta en 10/10, enclavamiento en 10/10 y liberación en 5.1–6.1 s, sin falsas alarmas en las pruebas simuladas ni en los 4.1 y 3.6 min sin pulso. Los bloques LEVE a 75 % con el firmware final quedaron anulados o contaminados por fallas del lazo de inyección, no del detector. Para la clase LEVE a 75 % se dispone de los bloques anteriores al firmware final (Tabla 20), con la detección en la computadora o con el detector embebido sin enclavamiento ni severidad.

Tabla 20. Bloques de validación ciega LEVE a 75 % anteriores al firmware final (6 y 7 de octubre de 2026).

Bloque	Decisión	Estado	Reales TP/FN	Simuladas TN/FP	Concordancia	Tiempo sin pulso	Falsas alarmas
LEVE 06-oct 17:53	Computadora	Preliminar (interrumpido en la prueba 25 de 40)	10/0	15/0	—	—	Excursiones aisladas hasta 0.87 pp
LEVE 07-oct 14:04	Computadora, persistencia 3	**Validado para la detección**	**19/1**	**20/0**	—	11.3 min	0
LEVE 07-oct 14:50	Firmware `81657f4`	Contaminado: el lazo dejó de conducir en las últimas 5 pruebas reales	10/10	20/0	806/806	11.6 min	0
LEVE 07-oct 15:29	Firmware `81657f4`	**Validado para la detección**	**9/1**	**10/0**	382/382	3.9 min	0

La detección de LEVE a 75 % está **validada** con 19/20 pruebas reales y 20/20 simuladas en la computadora, y con 9/10 y 10/10 en el detector embebido sin enclavamiento. El falso negativo del bloque de las 14:04 se debió a un transitorio sano de alrededor de 0.45 pp en sentido contrario, justo antes del pulso, que contaminó la referencia. **No existe un bloque LEVE completo a 75 % con el firmware final**; la severidad LEVE del firmware final se verificó al reproducir la sesión del 7 de octubre y, en vivo, en las dos detecciones del 9 de octubre a las 15:46, ambas clasificadas como LEVE y enclavadas.

El 15 de octubre, dentro de la validación ciega 2, se validó por primera vez el firmware final a 0 y 25 % de carga (Tabla 21). Antes de cada sesión se hizo un pulso de prueba aislado con la pinza, fuera del análisis.

Tabla 21. Validación ciega en vivo con el firmware final a 0 y 25 % de carga (15 de octubre de 2026).

Bloque	Pinza (A)	Reales TP/FN	Simuladas TN/FP	Severidad reportada	Enclavadas	Liberación	Concordancia PC/firmware	Tiempo sin pulso
LEVE 0 % 11:40	1.7–1.8	**20/0**	17/3	13 LEVE, 7 INTER	20/20	5.1–282 s	936/936	12.7 min
INTER 0 % 12:15	3.7–4.0	**9/1**	10/0	5 INTER, 4 SEVERA	8	1.0–6.1 s	461/461	6.1 min
LEVE 25 % 12:43	1.7–2.0	**20/0**	20/0	**20/20 LEVE**	20/20	5.1–6.4 s	909/909	11.7 min
INTER 25 % 13:23	4.3–4.4	**10/0**	10/0	**10/10 INTER**	10/10	5.1–6.1 s	464/464	6.3 min

La **detección** del firmware final generaliza a 0 y 25 % de carga: 59 de 60 pulsos reales detectados, y la única falla no detectada es un pulso en que el lazo no condujo (prueba 12 del bloque INTER a 0 %). A 25 % todo fue correcto: detección, especificidad, severidad y enclavamiento. A 0 % aparecieron dos problemas de severidad. En el bloque LEVE, los 3 falsos positivos y las 7 severidades INTER provienen de un mismo incidente: a las 11:59:38 hubo un escalón espontáneo de la corriente de la fase A (3.71 → 3.79 A) con el motor sano, la alarma quedó enclavada (de ahí la liberación de 282 s) y los pulsos LEVE siguientes se sumaron a esa base y escalaron a INTER. En el bloque INTER, las 4 clasificaciones como SEVERA se explican porque la corriente del lazo fue mayor que en el conjunto de datos (3.7–4.0 A frente a 3.6–3.7 A) y porque la firma INTER a 0 % ya es la más cercana al umbral INTER/SEVERA (mediana de 3.71 pp, máximo de 4.28 pp, umbral de 4.471 pp). Por lo tanto, la detección está **validada** a 0, 25 y 75 %, y la severidad, a 25 y 75 %; a 0 % la severidad reportada puede subir un nivel cuando el lazo conduce más que en la captura.

Los ensayos en vivo dejaron registradas dos debilidades del firmware final, que se reportan como limitaciones sin modificar el firmware. La primera es una falsa alarma al arrancar el motor: el 9 de octubre a las 15:01, con el motor recién encendido, una alarma LEVE quedó enclavada y contaminó la primera prueba; como medida operativa se espera alrededor de 5 min después de encender el motor, aunque el calentamiento del firmware es de 60 s. La segunda es la alarma enclavada ante un escalón del motor sano: el 8 de octubre a las 16:34:49, un escalón sano de 0.7–0.9 pp en la dirección de falla provocó una alarma que no se liberó, porque el enclavamiento solo se libera cuando la firma cae contra la base congelada; el escalón espontáneo del 15 de octubre es la misma debilidad.

4.1.8 PRUEBA LARGA SIN FALLA

La tasa de falsas alarmas en operación continua se midió el 12 de octubre con un protocolo declarado antes de correr (*commit* `9e2968d`), el firmware final sin cambios y el lazo de inyección desconectado. Como el generador no permite cambiar la carga con el motor girando, una enmienda (`f8699ee`) dividió la prueba en cuatro tramos de 22.5 min (25, 50, 75 y 0 %), cada uno con 5 min de calentamiento previo; por la misma razón, los escalones de carga en caliente quedaron **sin evaluar**. Desde el tramo de 50 %, una regla adicional (`4f08c86`) registró como atascada toda alarma enclavada por más de 60 s y reinició el detector con el comando `r` mediante un *script* auxiliar. La Tabla 22 presenta el resultado, que está **validado** y es negativo.

Tabla 22. Prueba larga sin falla del 12 de octubre de 2026 (firmware final, lazo desconectado).

Tramo	Exposición	Falsas alarmas	Severidad máxima y duración
25 %	22.5 min	2	LEVE 5.1 s; INTER 972.6 s (abierta al final del tramo)
50 %	22.5 min	2	LEVE 10.2 s; LEVE 37.1 s
75 %	22.5 min	4	LEVE 8.4, 7.1, 63.3 (reiniciada con `r`) y 5.1 s
0 %	22.5 min	3	LEVE 11.5, 6.4 y 12.0 s

Tramo	Exposición	Falsas alarmas	Severidad máxima y duración
Total	**90 min**	**11**	**7.3/h (IC 95 % de Poisson exacto: 3.7–13.1/h)**

El criterio declarado era de **a lo sumo una falsa alarma por hora, y no se cumple**. La alarma de 972.6 s se enclavó después de un transitorio de desbalance de voltaje de la red y escaló a INTERMEDIA. En los registros de arranque, al 25 % la alarma ya estaba enclavada desde el inicio del registro, al 75 % hubo 2 alarmas LEVE que se liberaron solas y al 50 % y al 0 % no hubo ninguna.

Un análisis post hoc, no declarado, investigó si las falsas alarmas provenían del desbalance de la red. Primero se verificó que el port del detector a Python es fiel: su estado coincide con el del firmware en el 99.6–100 % de los bloques y reproduce los resultados de las siete sesiones en vivo. Después se probó compensar la firma con la corriente que el motor sano toma por el desbalance de voltaje ($R_{2c} = R_2 - Y_2 \cdot V_2$). La compensación no sirvió: el voltaje de secuencia negativa explica solo el 3.9 % de la varianza de la firma, las falsas alarmas se mantuvieron en 10, en INTERMEDIA los falsos positivos subieron de 0 a 4 y en LEVE los verdaderos positivos bajaron de 2 a 1. Además, en los bloques que suman racha hacia una alarma, $|\Delta V_2|$ está en el percentil 48, igual que en cualquier bloque. **Se retracta, por lo tanto, la hipótesis de que las falsas alarmas se deben principalmente al desbalance de la red.** El mecanismo que sí explican los datos es estadístico: la proyección del motor sano tiene una desviación estándar de 0.256 pp, de modo que el umbral de 0.40 pp queda a unas 1.6 desviaciones; la autocorrelación entre bloques consecutivos es de 0.53 (0.21 a dos bloques), y con bloques independientes la persistencia de 3 dejaría alrededor de 0.35 eventos en 90 min, frente a los 10–11 observados.

4.1.9 COMPARACIÓN CON APRENDIZAJE AUTOMÁTICO PARA LA DETECCIÓN

Se realizaron tres estudios con protocolos declarados para decidir si un modelo de aprendizaje automático debía sustituir a la regla física en la etapa de detección.

4.1.9.1 ESTUDIO 1: REGLA FRENTE A MODELOS EN OPERACIÓN CONTINUA

El protocolo se declaró antes de correr (*commit* `459bee9`). Los datos son 13 grupos (los 4 tramos de la prueba larga y 9 sesiones en vivo a 75 %), con 94 pulsos reales (60 LEVE, 22 INTERMEDIA y 12 SEVERA) y 2.76 h de motor sano, evaluados dejando un grupo fuera (Tabla 23 y Figura 63). El resultado es **preliminar**: el criterio declarado (intervalos de confianza de 95 % sin traslape) no se cumple.

Tabla 23. Estudio 1: detectores evaluados dejando un grupo fuera (13 grupos, 2.76 h de motor sano).

Detector	Falsas alarmas	FA/h (IC 95 %)	LEVE	INTER	SEVERA
Regla del firmware (0.40 pp, 3 bloques)	14	5.1 (2.8–8.5)	48/60	21/22	12/12
Random Forest (s = 0.55, 3 bloques)	5	1.8 (0.6–4.2)	48/60	22/22	12/12
Gradient boosting (s = 0.85, 2 bloques)	6	2.2 (0.8–4.7)	48/60	22/22	12/12

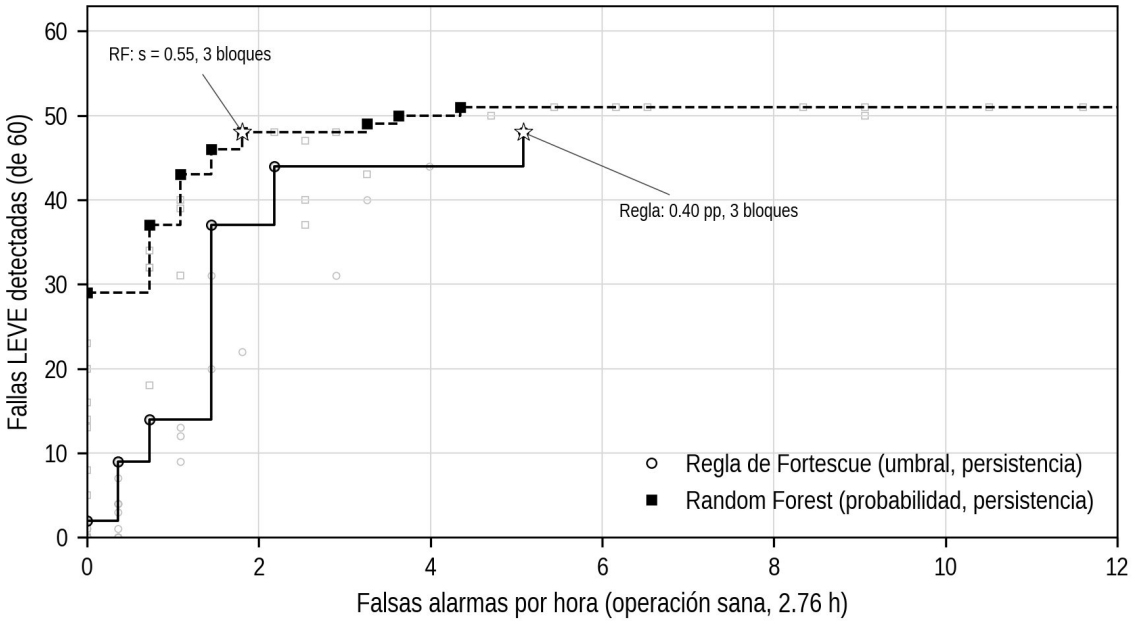


Figura 63. *Compromiso entre falsas alarmas por hora y detección de la falla LEVE en el estudio 1, para la regla de Fortescue y el Random Forest. Las estrellas indican los puntos de operación declarados.*

La mejora aparente del Random Forest proviene sobre todo de ΔI_1, porque $|I_1|$ aumenta durante la falla (medianas de +0.29 % en LEVE, +1.90 % en INTERMEDIA y +5.05 % en SEVERA); sin esa característica, el Random Forest da 11 falsas alarmas (análisis exploratorio). La Figura 64 muestra que ese aumento depende de la carga: en vacío, la falla LEVE prácticamente no modifica $|I_1|$.

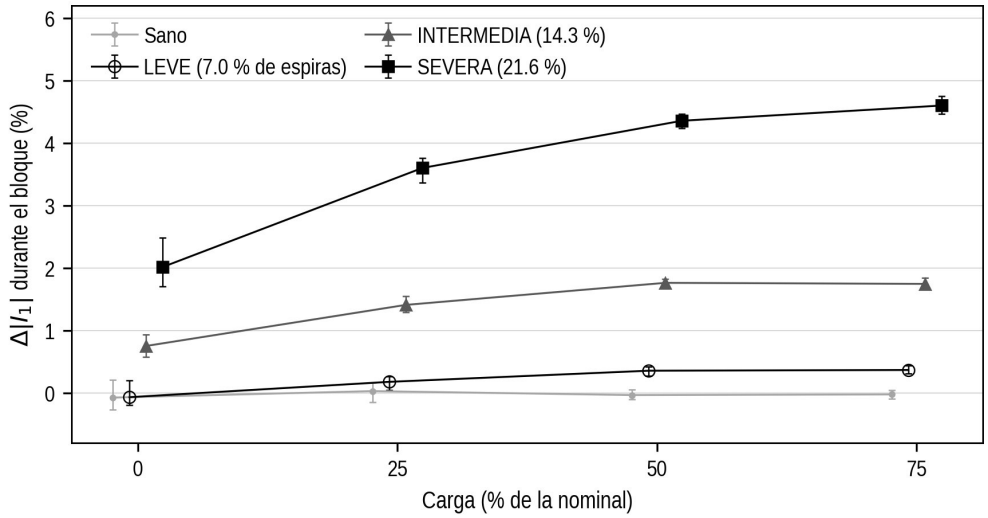


Figura 64. *Cambio relativo de |I1| durante el bloque con falla, por clase y carga (mediana y rango intercuartil). En vacío, la falla LEVE prácticamente no modifica |I1|.*

4.1.9.2 ESTUDIO 2: GENERALIZACIÓN ENTRE CARGAS

El protocolo (*commit* `61c09f1`) y su enmienda (`dd93b67`) se declararon antes de correr. Se usaron 607 pares del conjunto multiclase con dos esquemas: E1, entrenar a 75 % y probar en las otras cargas, y E2, dejar una carga fuera. El resultado está **validado** y es negativo para las características MCSA (Tabla 24).

Tabla 24. Estudio 2: generalización entre cargas con pares de bloques (607 pares).

Método	Resultado entre cargas
Regla de Fortescue	LEVE a 0 % 50/50; de 0 a 50 % detecta todas las fallas salvo 2 LEVE a 50 %
Random Forest con características de Fortescue	Igual que la regla: LEVE a 0 % 50/50 (prueba de McNemar sin pares discordantes); 448/450 fallas en E1
Random Forest con características MCSA absolutas	Entrenado a 75 %, detecta 0 de 450 fallas en las otras cargas; sin datos a 75 %, el motor sano a 75 % sale como falla en 150/150. Aprende el nivel de corriente
Random Forest con características MCSA relativas	Al nivel del azar incluso dentro del 75 % (exactitud balanceada 0.49–0.55; AUC 0.45–0.54)

Con las características MCSA que registra este sistema, absolutas o relativas, el Random Forest no detecta la falla entre cargas; es un resultado de este banco y esta adquisición, no del análisis MCSA en general. Con pares limpios, el Random Forest de Fortescue no mostró ningún atajo, aunque este estudio no habría podido revelarlo, porque en los pares la proyección por sí sola separa todas las clases.

4.1.9.3 VALIDACIÓN CIEGA 2: MODELO CONGELADO FRENTE A LA REGLA CON DATOS NUEVOS

La validación ciega 2 comparó, con datos nuevos y continuos, la regla con el Random Forest del estudio 1 congelado (verificado por su huella SHA). El protocolo se declaró el 14 de octubre, antes de ir al banco (*commit* `54ecef3`), con una enmienda el 15 de octubre antes del tramo de 0 % (`769cce9`), y el análisis se corrió una sola vez el 16 de octubre, después del último registro. Los datos comprenden las sesiones en vivo LEVE e INTERMEDIA a 0 y 25 % (Tabla 21) y 60 min de motor sano a 25, 50 y 75 % (15 de octubre) y a 0 % (16 de octubre), con el lazo desconectado: 4.906 h de exposición sana, sin exclusiones. El resultado está **validado**. La hipótesis H1 (el Random Forest reduce las falsas alarmas) se confirma (Tabla 25); la hipótesis H2 (el Random Forest conserva la detección de LEVE a baja carga) no se cumple: el Random Forest pierde (Tabla 26).

Tabla 25. Validación ciega 2, hipótesis H1: falsas alarmas en 4.906 h de operación sana.

Detector	Falsas alarmas	FA/h	IC 95 %
Regla, lógica del estudio 1 ($P \geq 0.40$ pp × 3)	21	4.28	2.65–6.54
Random Forest congelado ($s \geq 0.55$ × 3)	**2**	**0.41**	0.05–1.47

La prueba binomial condicional unilateral da $p = 3.3 \times 10^{-5}$.

Tabla 26. Validación ciega 2, hipótesis H2: detección de la falla LEVE a baja carga.

Carga	n	Regla	Random Forest	McNemar exacto
0 %	20	20	**4**	$p = 3.1 \times 10^{-5}$
25 %	20	20	**1**	$p = 3.8 \times 10^{-6}$

La lectura fijada en el protocolo para el caso «H1 sí y H2 con pérdida del Random Forest» es que **el Random Forest mejora la especificidad, pero no generaliza a baja carga, y se confirma el atajo** (Figura 65). Un diagnóstico post hoc explica el mecanismo: aplicando el mismo Random Forest a los pares con 10 semillas, sin las columnas de ΔI_1 detecta LEVE a 0 % en 50/50, y con ΔI_1 solo en 8–9/50. En vacío la falla LEVE no aumenta $|I_1|$, y el modelo, entrenado con datos continuos a 75 %, donde la proyección por sí sola no basta por el ruido, aprendió a usar «$|I_1|$ aumenta» como evidencia de falla. El atajo depende de con qué datos se entrena el modelo.

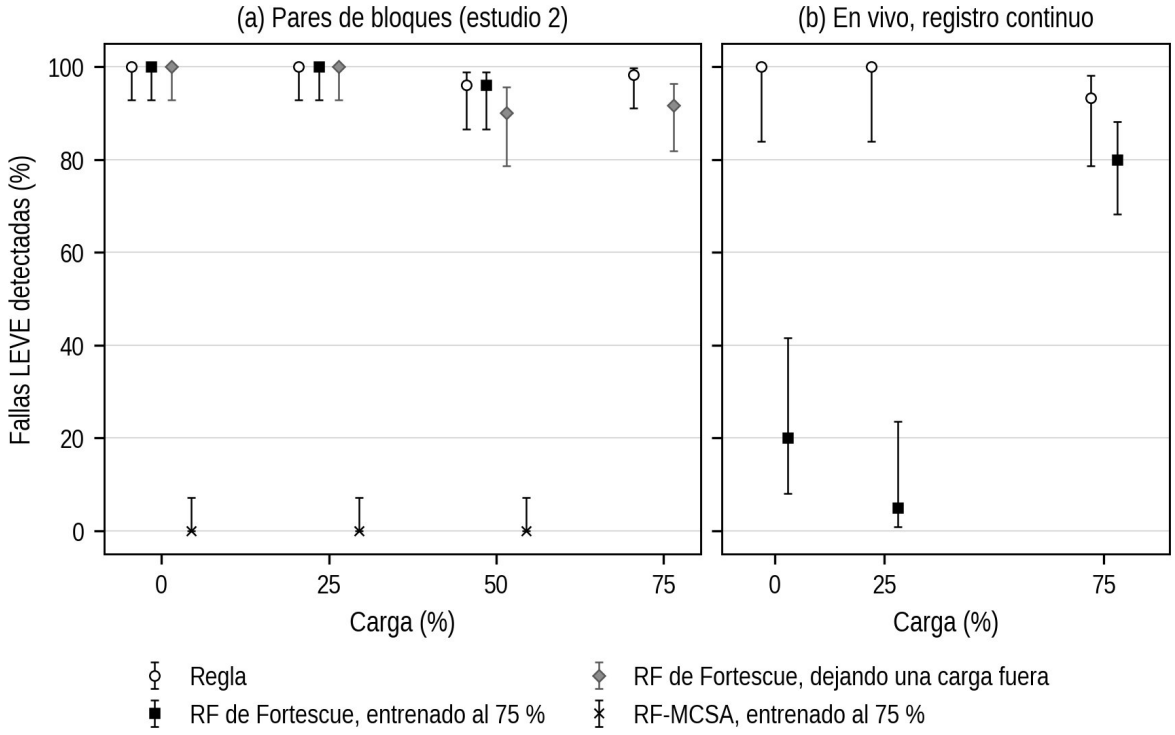


Figura 65. *Detección de la falla LEVE por carga, con intervalos de confianza de Wilson: (a) pares de bloques del estudio 2; (b) registro continuo en vivo, regla del firmware frente al Random Forest congelado entrenado a 75 %.*

La Tabla 27 reúne los resultados secundarios, descriptivos, y la Figura 66 compara las tasas de falsas alarmas de todos los detectores.

Tabla 27. Validación ciega 2, resultados secundarios (4.906 h de operación sana).

Detector	Falsas alarmas	FA/h (IC 95 %)	LEVE 0 % / 25 %	INTER 0 % / 25 %
Regla, lógica del estudio 1	21	4.28 (2.65–6.54)	20/20 · 20/20	8/10 · 10/10
Random Forest congelado	2	0.41 (0.05–1.47)	4/20 · 1/20	8/10 · 9/10
Regla completa (enclavamiento, reproducción sin `r`)	11	2.24 (1.12–4.01)	12/20 · 20/20	8/10 · 10/10
Variante: ángulo ≤ 35° y LEVE en 4 bloques	7	1.43 (0.57–2.94)	20/20 · 20/20	9/10 · 10/10
Firmware en vivo (con `r` automático)	20	4.08 (2.49–6.30)	Tabla 21	Tabla 21

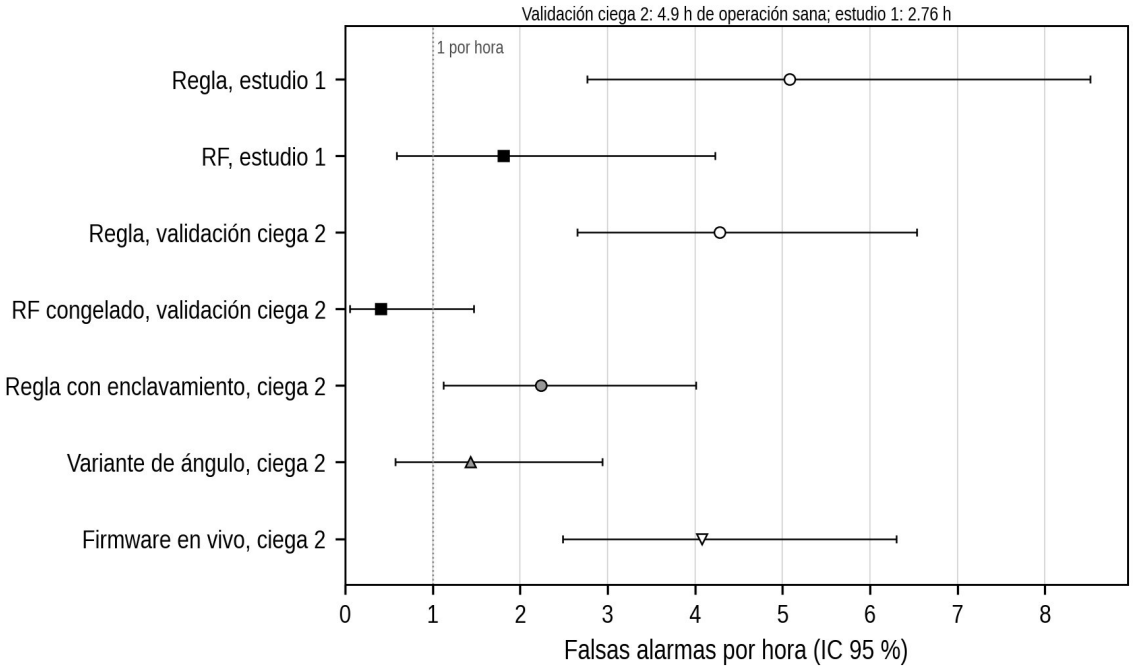


Figura 66. *Falsas alarmas por hora de operación sana, con intervalos de confianza de 95 %, en el estudio 1 y en la validación ciega 2. La línea punteada indica el criterio de una falsa alarma por hora.*

Por tramo de motor sano, las falsas alarmas de la regla, del Random Forest y del firmware fueron de 1/0/1 a 25 %, 0/0/0 a 50 %, 5/0/5 a 75 % y 13/1/12 a 0 %, y de 2/1/2 en los periodos sin pulso de las sesiones con falla. El firmware en vivo dio 20 falsas alarmas en 4.906 h (4.08/h, IC 2.49–6.30), que junto con las 11 del 12 de octubre suman 31 eventos en unas 6.4 h, alrededor de 4.8/h. Estas cifras no permiten atribuir la tasa a una carga: el 15 y 16 de octubre el reparto fue de 1 falsa alarma en unas 2 h a 25–50 %, 5 a 75 % y 12 a 0 %, mientras que el 12 de octubre fue más parejo (2/2/4/3 en 22.5 min por tramo). La regla completa pierde 8 detecciones LEVE a 0 % porque la reproducción sin `r` mantiene enclavada la alarma del escalón de corriente de las 11:59:38.

Observación post hoc, no declarada. 13 de las 21 falsas alarmas de la regla provienen del tramo sano a 0 %, donde la proyección oscila entre −0.8 y +1.3 pp. Es la carga en la que el Random Forest casi no detecta LEVE, de modo que buena parte de su ventaja en H1 es la otra cara de su insensibilidad a baja carga; no es una reducción de falsas alarmas «gratuita». Sin el tramo de 0 %, la regla da 8 falsas alarmas y el Random Forest 1.

La variante con compuerta de ángulo y persistencia de 4 bloques para LEVE tuvo en esta validación su primera prueba con datos nuevos: menos falsas alarmas que la regla completa (7 frente a 11) con la misma detección o mejor. Como los intervalos de confianza se traslapan, es **un indicio, no una confirmación**. Las cifras de 12, 14 y 21 falsas alarmas de esta sección y de la siguiente son reanálisis fuera de línea, útiles solo para comparar detectores entre sí sobre los mismos datos; las tasas del sistema real son las del firmware: 7.3/h el 12 de octubre y 4.08/h el 15 y 16 de octubre.

4.1.10 RUIDO DE LA FIRMA Y VARIANTES DEL DETECTOR

El análisis del ruido y de las variantes del detector se hizo fuera de línea con un protocolo declarado (*commit* `0518dd4`) y los mismos 13 grupos del estudio 1. El simulador, configurado como el firmware, coincide al 100 % en el estado de alarma y en la severidad con el port fiel. Sin reinicios `r`, la regla del firmware da 12 falsas alarmas

en 2.78 h (4.3/h, IC 2.2–7.5), con LEVE 47/60, INTERMEDIA 20/22 y SEVERA 12/12; esa es la línea base para comparar las variantes.

La naturaleza del ruido se caracterizó con la desviación de Allan de la proyección con el motor sano (Tabla 28). Promediar dos bloques no reduce el ruido: la pendiente es prácticamente nula en la primera octava y la autocorrelación a un bloque es de 0.53. Entre 3.4 y unos 50 s la pendiente es de alrededor de −0.25, entre la del ruido blanco y la del ruido de parpadeo: de 1.7 a 54 s el ruido baja 2.3 veces, cuando con ruido blanco bajaría 5.7 veces. Más allá aparece deriva, y a 0 % de carga desde unos 14 s. Alargar la referencia de 11 a 30 s solo baja la desviación de la proyección de 0.256 a 0.233 pp (−9 %). La relación señal a ruido está limitada por fluctuaciones lentas, cuyo origen (sensores, motor o red) no se separó; la mejor desviación de Allan alcanzable es de unos 0.07–0.08 pp, a 50–100 s.

Tabla 28. Desviación de Allan (pp) de la proyección de la firma con el motor sano, por carga y tiempo de promediado τ.

Carga	τ = 1.7 s	3.4 s	6.8 s	13.6 s	27.2 s	54.4 s	108.8 s
25 %	0.179	0.172	0.137	0.115	0.081	0.065	0.067
50 %	0.141	0.139	0.118	0.098	0.089	0.075	0.066
75 %	0.159	0.145	0.123	0.110	0.097	0.078	0.081
0 %	0.156	0.152	0.124	0.115	0.130	0.171	0.229

Se evaluó además una rejilla de 1904 configuraciones del detector, eligiendo cada una dejando un grupo fuera (Tabla 29). El firmware ya está en el óptimo de su familia de umbral y persistencia. La configuración formalmente «ganadora», con media móvil, referencia de 20 s y una compuerta absoluta $|Q| \leq 0.3$, no sirve: anula SEVERA a baja carga (pares SEVERA a 0 % detectados en 23/43 frente a 42/43), porque esa firma mide 7 pp y está girada −3.8°; en vivo no se vio porque todo se probó a 75 %. Es el mismo tipo de atajo de carga que el del Random Forest, y no se recomienda validarla; esta desviación del criterio declarado quedó documentada. Los umbrales de liberación de 0.20, 0.30 y 0.40 pp tuvieron un efecto pequeño, y la alarma atascada de 973 s persiste en los tres casos.

Tabla 29. Variantes del detector evaluadas fuera de línea (rejilla de 1904 configuraciones, selección dejando un grupo fuera).

Ingredientes permitidos	Falsas alarmas fuera de muestra	LEVE / INTER / SEVERA
Firmware (0.40 pp × 3, referencia de 11 s)	12	47 / 20 / 12
Solo umbral y persistencia	12 (elige el mismo punto)	47 / 20 / 12
Con longitud de referencia	10	45 / 20 / 12
Media móvil, referencia de 20 s, compuerta $\|Q\| \leq 0.3$	2	48 / 20 / 12
CUSUM	9	42 / 21 / 12

Una exploración post hoc, no declarada, probó una compuerta por ángulo ($|Q| \leq \tan 35° \cdot P$), que conserva INTERMEDIA y SEVERA a todas las cargas en los pares, combinada con una persistencia de 4 bloques solo para LEVE. La combinación dio 3 falsas alarmas frente a 12, con INTERMEDIA 21/22, SEVERA 12/12, LEVE 43/60 frente a 47 y un retardo de 6.4 s frente a 5.4 s. Más persistencia resultó insegura, porque la referencia móvil de 6–11 s atrás se contamina con la falla a unos 8 s: con 3 bloques más, ni una falla LEVE permanente se confirmaría. Congelar la referencia en el primer bloque sospechoso empeoró el resultado (19 falsas alarmas). Como la combinación se eligió mirando los mismos datos, no es un resultado sino la hipótesis que se probó después en la validación ciega 2 (7 frente a 11 falsas alarmas, con intervalos traslapados).

4.1.11 APLICACIÓN DE ESCRITORIO Y ASISTENTE RAG

La aplicación MotorGuard-ITF (versión 1.0.0, licencia GPL-3.0) muestra y registra lo que reporta el firmware final, que no se modificó, y el asistente se presenta como apoyo y no como diagnóstico. Sus resultados se agrupan en cuatro rubros.

Pruebas de software (validado). La aplicación cuenta con 119 funciones de prueba automática. Entre otras cosas, verifican el intérprete de la telemetría contra registros reales (el registro INTERMEDIA del 9 de octubre da 385 tramas, 65 con alarma confirmada); los bloqueos de inyección y el envío del comando de apagado al cerrar o desconectar, con un puerto virtual y un simulador del firmware que solo se usa en pruebas; la equivalencia de los formatos de adquisición con 1214 filas reales de 13 archivos; el plano de la firma, cuya proyección recalculada coincide con la del firmware en 1685 de 1696 tramas reales, y la severidad contra la base despejada en 220/220; el asistente con un servidor de modelo simulado; el contraste del texto (al menos 4.5:1); el botón «Liberar alarma» y el zoom de las gráficas. En las pruebas de resistencia acelerada, con registros reales reproducidos sin pausas, la aplicación procesó 50 818 tramas en 24 h simuladas (percentil 99 de 4.42 ms por trama, +9.7 MB de memoria y 0 errores) y 101 654 tramas en 48 h simuladas (4.50 ms, +13.4 MB y 0 errores). En tiempo real, durante 4 h contra el simulador el 13 de octubre, recibió 8400 de 8405 tramas (99.9 %), con 479 pulsos, 11 reconexiones y 0 errores. Los dos paquetes de instalación definitivos se probaron en contenedores limpios (Tabla 30).

Tabla 30. Paquetes de instalación de la aplicación probados el 17 de octubre de 2026.

Paquete	Tamaño	Distribuciones probadas
Ligero, `motorguard-itf_1.0.0_all.deb` (usa las bibliotecas de la distribución)	1.1 MB	Ubuntu 26.04 y Debian 13, sin avisos de lintian
Autónomo, `motorguard-itf-autonomo_1.0.0_amd64.deb` (PyInstaller)	64 MB	Ubuntu 22.04, 24.04 y 26.04; Debian 12 y 13

En cada distribución se comprobaron la instalación con apt, el arranque sin pantalla y con X11 reproduciendo un registro real, la ausencia de errores en el registro de la aplicación y la desinstalación limpia. Ninguno de los dos paquetes incluye los modelos de lenguaje ni los documentos, que se descargan con la función «Preparar asistente».

Prueba con el banco real (parcial). El 12 de octubre la aplicación recibió la telemetría en vivo, registró 2 alarmas LEVE del firmware y se recuperó de 2 pérdidas de conexión. El 13 de octubre se ejecutó el protocolo de prueba en vivo de la aplicación, con un veredicto de **no aprobada completa con los criterios declarados**. Se cumplieron los pasos 1, 2, 3 (en el segundo intento), 4, 5 (en el segundo intento), 7 (repetido a 75 % de carga), 8a y 8b: funcionaron la confirmación humana, el paro, el cierre seguro y los bloqueos. El paso 6 falló porque la pinza marcó 4.0 A en el lazo INTERMEDIA, fuera del criterio de 3.3–3.7 A. El paso 9 falló por un error en el registro de la aplicación causado por la reconexión del paso 3, mientras el microcontrolador se enumeraba (después se corrigió y ahora es un reintento), y porque falta el mensaje de apagado del pulso del paso 8b, ya que la aplicación había cerrado el puerto; el apagado se verificó en el banco.

En el primer intento del paso 7, a 0 % de carga, la tercera alarma salió SEVERA con las derivaciones INTERMEDIA (severidad 4.915 pp > 4.471 pp), una limitación del detector ya descrita en la sección 4.1.7; a 75 % todas salieron INTERMEDIA.

El 16 de octubre se usó la aplicación en sesiones con pulsos INTERMEDIA y SEVERA. En la sesión de las 13:48, una alarma LEVE se enclavó a las 14:13:34 y permaneció activa 567 s; los pulsos INTERMEDIA que llegaron durante ese intervalo escalaron la severidad de la alarma, y el firmware no la liberó sino hasta las 14:23:01 (Figura 67). Es la misma debilidad descrita en la sección 4.1.7 y el motivo por el que se agregó el botón

«Liberar alarma». En la sesión de las 19:28, con carga de 0 % (deducido de las corrientes: 3.81, 3.94 y 3.80 A con el motor sano, iguales a las de las sesiones a 0 % del 15 y 16 de octubre y distintas de las de 25 %, 4.18, 4.40 y 4.16 A), las 3 inyecciones con derivaciones INTERMEDIA salieron SEVERA (severidad de 4.61–4.75 pp) y las 3 siguientes, con derivaciones SEVERA, dieron 9.46–9.88 pp (Figura 68); que INTERMEDIA salga SEVERA a 0 % es la limitación descrita en la sección 4.1.7.

En los registros del 12 al 16 de octubre hay en total dos reinicios espontáneos del microcontrolador, de causa no identificada: el del 13 de octubre a las 14:44:56, con el motor apagado, y otro el 16 de octubre a las 13:57:12, con el motor en marcha (registros seriales `sesion_20261013_142900` y `sesion_20261016_134802` de la aplicación); los demás arranques del firmware correspondieron a desconexiones del cable.

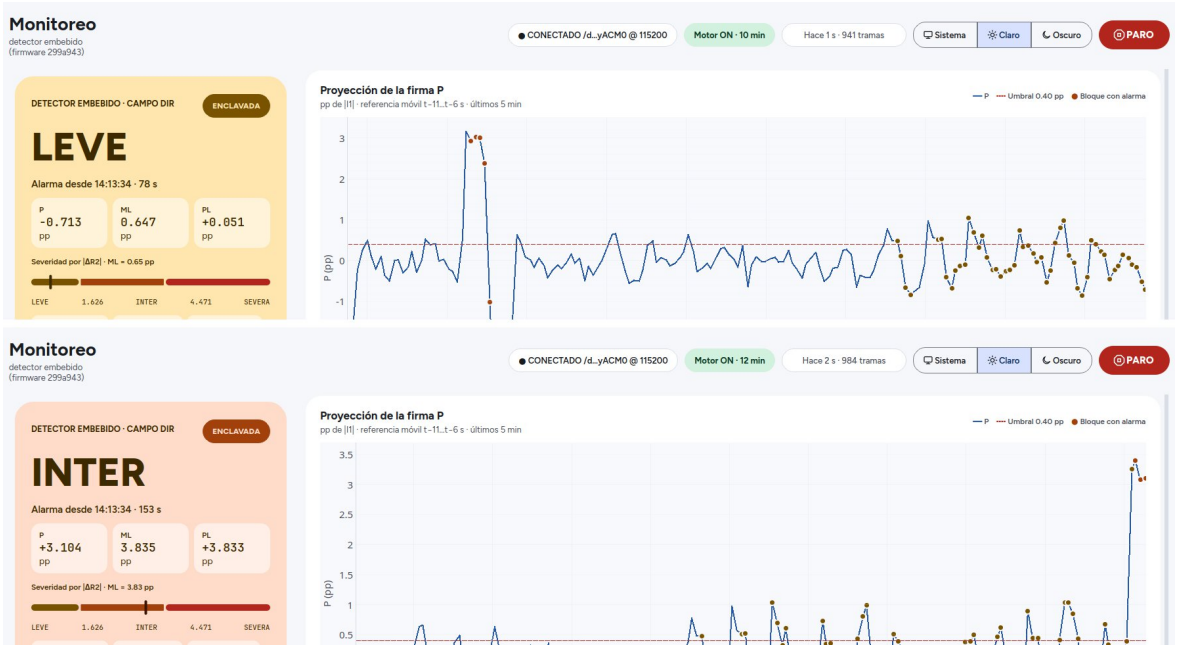


Figura 67. *Sección Monitoreo de la aplicación durante la sesión del 16 de octubre de 2026: alarma LEVE enclavada desde las 14:13:34 (arriba, a los 78 s) que escaló a INTERMEDIA con los pulsos siguientes (abajo, a los 153 s).*



Figura 68. *Proyección de la firma en la sesión del 16 de octubre a las 19:28: tres pulsos con derivaciones INTERMEDIA, clasificados SEVERA (4.61–4.75 pp), seguidos de tres pulsos con derivaciones SEVERA (9.46–9.88 pp).*

Quedan **pendientes** con el banco el paso 8c del protocolo y una captura de adquisición con el motor. El botón «Liberar alarma» se probó con el simulador y con el microcontrolador real (prueba del autor, confirmada el 17 de octubre de 2026).

Latencia de la alarma y de la recomendación. Son dos tiempos distintos. La alarma la emite el firmware sin esperar al asistente, y la aplicación la muestra en cuanto llega la trama. En 28 pulsos con alarma registrados por la aplicación el 13 y el 16 de octubre, la latencia desde el encendido del relevador hasta la confirmación del firmware tuvo una mediana de 4.5 s y un rango de 0.2 a 6.4 s. Cuatro de esos valores (0.2, 0.3, 1.2 y 1.3 s) son menores que el mínimo físico de una confirmación de 3 bloques de 1.70 s y corresponden a pulsos en que la alarma venía del pulso anterior; tres de ellos ocurrieron durante la alarma enclavada de 567 s del 16 de octubre a las 14:13:34. Sin esos cuatro, los 24 pulsos restantes dan una mediana de 5.0 s y un rango de 4.1 a 6.4 s. La recomendación del asistente local llega después. El criterio declarado antes de medir era de 90 s como máximo, y se hicieron 5 corridas de 3 consultas cada una (Tabla 31).

Tabla 31. Tiempo de la recomendación del asistente local (Qwen3.5-4B con el prefijo del mensaje precalentado) al confirmarse una alarma.

Corrida	Mensaje	Tiempos (s)	Condiciones registradas
13-oct 03:48	Inicial	77.5, 76.0, 71.7	No registradas
13-oct 11:16	Ajustado	80.6, 106.3, 109.3	No registradas
13-oct 11:29	Ajustado	104.4, 106.5, 98.9	Con cargador, 78–81 °C, 3.2 tokens/s
13-oct 11:41 (comparación A/B)	Inicial	104.9, 102.5, 91.7	Con cargador, 78–80 °C, 3.1–3.2 tokens/s
17-oct 15:41 (versión final)	Final (3–4 puntos)	**122.0, 132.2, 138.6**	Sin cargador, 86–98 °C, 1153–1198 MHz, 2.1–2.2 tokens/s

Solo 3 de las 15 consultas cumplen el criterio, y **la versión final no lo cumple: 122–139 s**, con estrangulamiento térmico del procesador. No se midió si el tiempo bajaría con el equipo en otras condiciones térmicas. Con un modelo externo por API, el tiempo mediano por pregunta fue de 3.3 s. En la versión final, el precalentado del prefijo tarda 192.5 s al arrancar la aplicación, con un prefijo de 1774 tokens; al alarmar se reutilizan 1758 tokens y se procesan 167. Sin precalentado, una medición del 13 de octubre dio 169.9 s y 5.7 GB de memoria para el modelo de 4B, y 74.2 s y 2.5 GB para el de 2B, ambos con el 100 % de las líneas con cita.

Evaluación del asistente. Se compararon primero, el 13 de octubre y de forma **preliminar**, los modelos de 4B y 2B parámetros con el borrador de las preguntas (19 con fuente y 4 sin fuente) y una calificación propuesta, no revisada (Tabla 32). Esa comparación motivó la elección del modelo de 4B.

Tabla 32. Comparación preliminar de los modelos locales (13 de octubre de 2026, borrador de 23 preguntas, calificación propuesta).

Métrica	Qwen3.5-4B	Qwen3.5-2B
Recall@5 (página esperada recuperada)	17/19	17/19
La respuesta cita la página esperada	17/19	17/19
Respuestas correctas, con fuente	16/19	14/19
Abstención correcta, sin fuente	3/4	2/4
Respuestas con alguna afirmación inventada	2/23	5/23
Tiempo mediano por pregunta libre	120.6 s	53.1 s

Ese mismo día hubo una segunda evaluación preliminar del sistema original con las preguntas revisadas (21 con fuente y 4 sin fuente): *recall*@5 de 19/21, 16/21 respuestas correctas del modelo de 4B, 4/4 abstenciones correctas y 3/25 respuestas con algo inventado. El 17 de octubre se ajustó la recuperación usando esas mismas preguntas y se llegó a 24/24; **ese resultado no se reporta**, porque se obtuvo con las preguntas empleadas para

el ajuste. Por eso se construyó un set de prueba independiente (set 2), con 24 preguntas (20 con fuente y 4 sin fuente) que se redactaron sin correr el sistema y se corrieron una sola vez con el sistema congelado e índice de 936 fragmentos. La calificación a mano es una propuesta del asistente de programación Claude, porque el autor delegó la revisión. La Tabla 33 compara el modelo local con un modelo externo por API que recibió los mismos fragmentos. El resultado se considera **preliminar** por esa calificación y por el tamaño del set.

Tabla 33. Evaluación del asistente con el set de prueba 2 (24 preguntas, 17 de octubre de 2026, calificación propuesta por Claude).

Métrica	Qwen3.5-4B (local)	Claude Sonnet 5.5 (API, esfuerzo bajo)
Recall@5	18/20	18/20
La respuesta cita la página esperada	18/20	18/20
Respuestas correctas, con fuente	17/20	18/20
Abstención correcta, sin fuente	4/4	4/4
Respuestas con alguna afirmación inventada	2/24	1/24
Tiempo mediano por pregunta	139.5 s	3.3 s

La Figura 69 muestra una respuesta del asistente local a una pregunta que los documentos no cubren: el asistente indica, punto por punto, que no hay fuente, y lista los fragmentos que consultó.

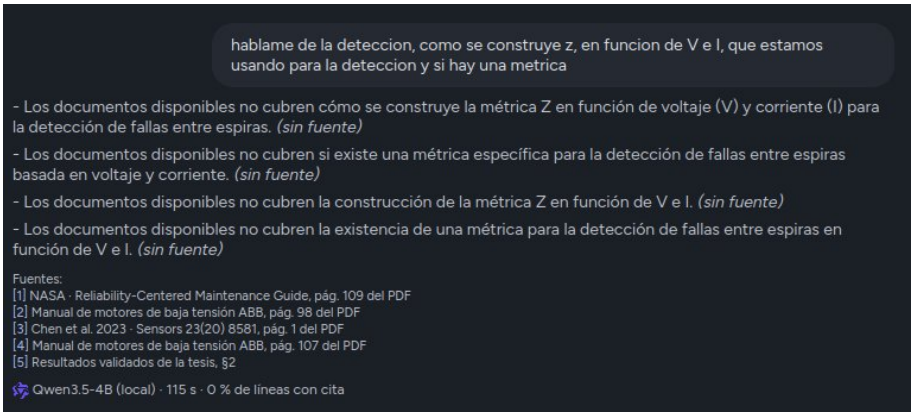


Figura 69. *Respuesta del asistente local (Qwen3.5-4B) a una pregunta sobre la construcción de la firma, no cubierta por los documentos: cada punto se marca «sin fuente» y se listan los fragmentos consultados.*

Las dos fallas de recuperación, comunes a ambos modelos, son una pregunta sobre el desbalance de 5 % de la guía de la NASA (página 104) y otra sobre una limitación del artículo de Ruzimov et al. (página 18). Las afirmaciones inventadas fueron, en el modelo local, un desbalance «aceptable» mayor de 10 % atribuido a la NASA y el uso de ultrasonido para detectar cortocircuitos; en el modelo externo, la atribución de las cifras del artículo de Chen et al. al banco de esta tesis.

Recomendación ante alarmas de un registro real. El 17 de octubre se reprodujo en la aplicación el registro serial de una sesión INTERMEDIA a 75 % de carga del 9 de octubre, con el asistente local, la recomendación automática al confirmarse la alarma y sin precarga de la consulta (Figura 70). El asistente respondió a las dos alarmas INTERMEDIA confirmadas por el firmware (firmas de 2.25 y 2.53 pp) con el 100 % de las líneas citadas, en 283 s la primera, que fue la primera consulta después de arrancar el modelo, y en 139 s la segunda. Estos tiempos no son una medición declarada; el segundo coincide con el rango de la Tabla 31. Las respuestas, el contexto que recibió el modelo y el *script* de la captura quedaron registrados junto con la figura. La figura

corresponde a una reproducción; la función «Recomendar» se probó además con el microcontrolador conectado en operación en vivo (prueba del autor, confirmada el 28 de octubre de 2026).

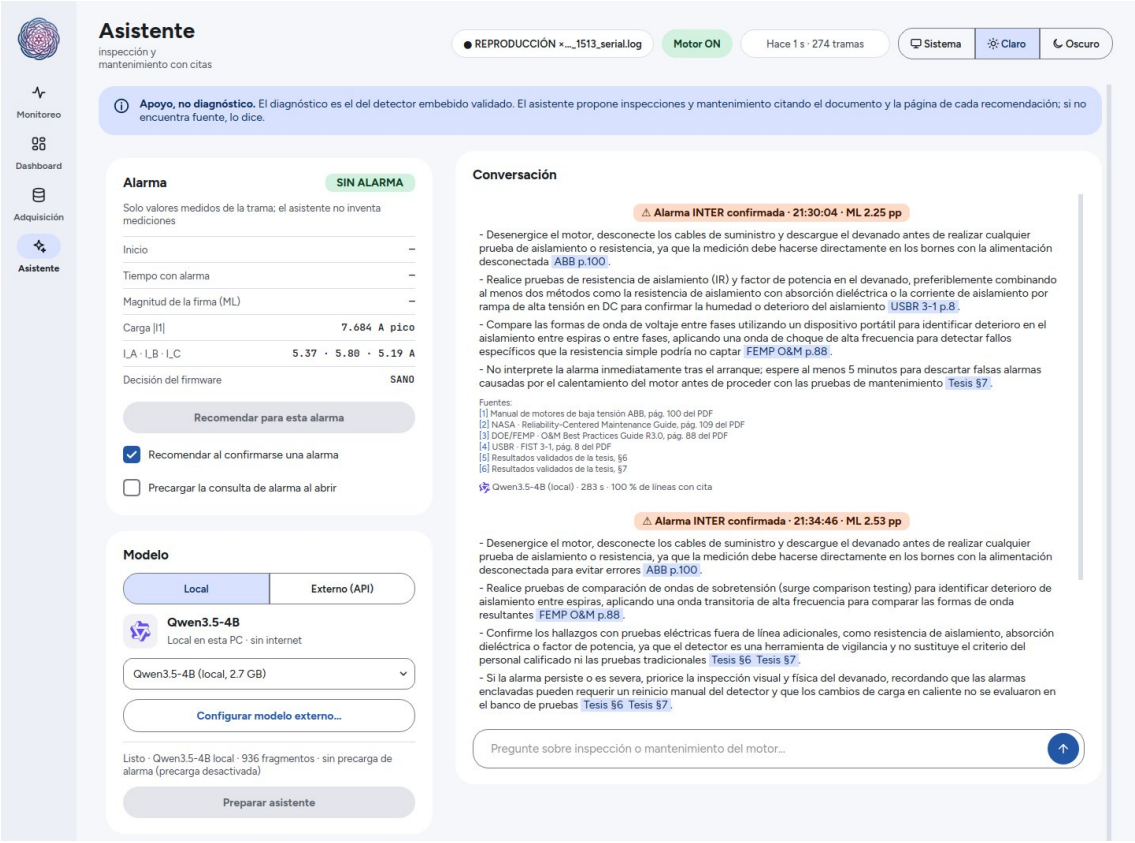


Figura 70. *Sección Asistente durante la reproducción de un registro real del banco (sesión INTER al 75 % del 9 de octubre de 2026). El asistente local (Qwen3.5-4B) respondió de forma automática a dos alarmas INTER confirmadas por el firmware (2.25 y 2.53 pp), con el 100 % de las líneas citadas. Las horas de las tarjetas son las de la reproducción; el panel de alarma ya muestra la alarma liberada.*

En una corrida previa idéntica de la misma reproducción, la primera respuesta afirmó que «la alarma INTER persistió y no se liberó sola», aunque el contexto que recibió el modelo indicaba «Tiempo con la alarma activa: 0 s». Es una afirmación no sustentada por el contexto, a pesar de llevar cita. La Figura 48 (sección 3.3.6.8) muestra otro caso, en el modo de demostración: una respuesta con todas sus líneas citadas que propone métodos de caracterización del banco como pruebas para confirmar la falla.

4.1.12 CUMPLIMIENTO DE OBJETIVOS E IMPACTO

EVALUACIÓN DE OBJETIVOS TÉCNICOS

El objetivo general registrado dice: «Desarrollar un sistema de diagnóstico predictivo para la detección temprana de cortocircuitos entre espiras en el estator de motores de inducción trifásicos, mediante análisis de firma de corriente (MCSA), procesamiento digital de señales y modelos de Machine Learning (XGBoost), con validación experimental en banco de pruebas». Se cumple **en lo esencial, no literalmente**: el sistema de diagnóstico con detección temprana, análisis MCSA, procesamiento digital de señales y validación experimental

en el banco existe y está validado, pero XGBoost se evaluó para la detección y no generalizó, por lo que no forma parte del sistema final. El aprendizaje automático del sistema está en la severidad, y la inteligencia artificial generativa, en la recomendación. La Tabla 34 contrasta los objetivos específicos del Capítulo 1 con los resultados, y la Tabla 35, los objetivos específicos del registro con lo realizado.

Tabla 34. Comparativa de los objetivos específicos planteados y los resultados obtenidos.

Objetivo específico	Resultado obtenido	Estatus
Construir y caracterizar el banco con inyección de fallas en tres severidades	Banco operativo; derivaciones caracterizadas (7.0, 14.3 y 21.6 % de espiras); cuatro niveles de carga; corriente del lazo medida por clase	Cumplido
Implementar en el ESP32-S3 la adquisición de seis canales, características MCSA y residuo de secuencia negativa	Muestreo de 6 canales a 2403.85 Hz en bloques de 1.70 s, FFT, componentes simétricas y firma de secuencia negativa en tiempo real	Cumplido
Implementar un detector y evaluar detección, latencia y falsas alarmas	Detector direccional embebido; detección validada en vivo a 0, 25 y 75 % (59/60 a 0 y 25 %; 10/10 SEVERA e INTER a 75 %); latencia de unos 5 s; falsas alarmas de 7.3/h y 4.08/h frente al criterio de 1/h	Parcial: no se cumple el criterio de falsas alarmas
Evaluar XGBoost y Random Forest para la detección y desarrollar un clasificador de severidad	Estudios con protocolo declarado (estudio 1, estudio 2 y validación ciega 2); clasificador de severidad validado (596/597 dejando una carga fuera) y embebido	Cumplido
Desarrollar una aplicación de monitoreo en tiempo real con un asistente de mantenimiento que cite sus fuentes	Aplicación MotorGuard-ITF 1.0.0 con 119 pruebas automáticas, paquetes de instalación y asistente RAG; prueba con el banco parcial	Cumplido, con la prueba en el banco incompleta

Tabla 35. Objetivos específicos del registro frente a lo realizado.

Objetivo registrado	Lo realizado	¿Se podría cumplir literalmente con lo que hay?
1. Modelo matemático del motor con fallas de 1, 5, 10 y 25 % de espiras	No se modeló; se midieron en el banco severidades de 7.0, 14.3 y 21.6 %	Sí, como modelo de régimen permanente calibrado con las 12 mediciones; faltan los parámetros del circuito equivalente
2. Pipeline MCSA (armónicos, RMS, factor de cresta, curtosis, secuencia)	Pipeline embebido con FFT, MCSA de alta resolución, RMS, THD, desbalances y Fortescue; sin factor de cresta ni curtosis	Casi: factor de cresta y curtosis requieren formas de onda crudas
3. XGBoost con exactitud ≥ 95 %, *recall* ≥ 98 % en la clase crítica y F1 ≥ 0.96	Las cifras se cumplen con el clasificador de severidad (99.8 %, SEVERA 100 %, F1 macro 0.998), pero no con XGBoost ni con la clase SANO	Sí, con un XGBoost de cuatro clases sobre el conjunto de octubre dejando una carga fuera y protocolo declarado

ESTADO PREVIO FRENTE AL ACTUAL

Al inicio del proyecto se disponía de un motor con derivaciones cuya documentación resultó incorrecta y de un plan basado en un clasificador entrenado con datos simulados. El estado actual es un sistema de tres etapas con un detector físico embebido validado en vivo a tres cargas, un clasificador de severidad aprendido y embebido, una aplicación de escritorio empaquetada y un asistente de mantenimiento local con citas a sus

fuentes, junto con una caracterización honesta de sus límites: la tasa de falsas alarmas en operación continua y la dependencia de la severidad respecto de la corriente del lazo.

ANÁLISIS DEL IMPACTO

- **Impacto económico:** la electrónica adquirida para el sistema costó $1,807 MXN, un orden de magnitud accesible para monitorear motores de baja y mediana potencia.
- **Impacto tecnológico:** se generaron un firmware, una aplicación empaquetada, un conjunto de datos multiclase y protocolos declarados con su evidencia de fecha, que pueden reutilizarse en otros motores del laboratorio.
- **Impacto académico:** el trabajo documenta, además de los resultados positivos, los resultados negativos y las hipótesis retractadas: la falta de generalización de los modelos de detección, el atajo de carga y la causa de las falsas alarmas, que no era la red. Esto lo convierte en una referencia útil para trabajos posteriores.

4.2 DISCUSIÓN DE RESULTADOS

El resultado central de este trabajo es una decisión de arquitectura respaldada por evidencia: la detección quedó a cargo de una regla física y no de un modelo de aprendizaje automático. No es una preferencia, sino la conclusión de cuatro observaciones independientes en las que los modelos de detección generalizaron mal fuera de las condiciones en que se entrenaron. En septiembre, XGBoost y el Random Forest no transfirieron entre campañas de captura. En el estudio 2, el Random Forest con características MCSA aprendió el nivel de corriente, es decir, la carga. En la validación ciega 2, el Random Forest de Fortescue entrenado con fallas a 75 % no detectó LEVE a 0 y 25 % en operación continua. Y en la rejilla de variantes, la compuerta absoluta que formalmente ganaba anulaba SEVERA a baja carga. Que un modelo funcione dentro de la distribución con que se entrenó no basta: hay que validarlo en las condiciones en que va a operar (Kaufman, Rosset y Perlich, 2011; Roberts et al., 2017).

La firma de secuencia negativa, en cambio, tiene una base física general. Un cortocircuito entre espiras rompe la simetría del devanado y aparece como corriente de secuencia negativa (Kliman et al., 1996; Bouzid y Champenois, 2013), y en este banco esa firma resultó estable en dirección en las tres severidades y las cuatro cargas, con clases que no se traslapan en magnitud. Por eso la inteligencia artificial se usa donde aporta y se puede verificar: en la severidad, con umbrales aprendidos y validados dejando una carga fuera, y en la recomendación, con un asistente que cita sus fuentes. El matiz que debe quedar escrito es que el Random Forest de Fortescue sí redujo las falsas alarmas (21 frente a 2, $p = 3.3 \times 10^{-5}$). Parte de esa ventaja coincide con su ceguera a LEVE en vacío, según una observación post hoc, pero un modelo entrenado con fallas a todas las cargas y datos continuos podría conservar la reducción sin el atajo; eso no se probó.

La limitación principal del sistema es la tasa de falsas alarmas en operación continua: 7.3/h el 12 de octubre y 4.08/h el 15 y 16 de octubre, frente al criterio declarado de una por hora. Se reporta como un resultado negativo validado. La hipótesis inicial de que la causa era el desbalance de la red se retractó con datos: la compensación con el voltaje de secuencia negativa no cambió las falsas alarmas y el desbalance en los bloques críticos es igual al de cualquier bloque. El mecanismo que sí explican los datos es que el umbral de 0.40 pp está a unas 1.6 desviaciones estándar de un ruido lento y correlacionado, que no se reduce promediando bloques ni alargando la referencia, y el firmware ya está en el óptimo de su familia de umbral y persistencia. Para un sistema desatendido, que necesitaría del orden de una falsa alarma por semana, la brecha es de 100 a 1000 veces, y ni ajustar la regla ni la inteligencia artificial la cierran por sí solos; si el ruido proviene de los sensores, atacar la

causa requeriría mejorar la adquisición. Para demostrar a lo sumo una falsa alarma por hora con 95 % de confianza harían falta 3.0 h de operación sin eventos.

La severidad embebida es correcta a 25 y 75 %, pero no siempre a 0 %, donde a veces clasifica INTERMEDIA como SEVERA. La razón es que la firma no depende solo del número de espiras en cortocircuito, sino también de la corriente que circula por el lazo, que varía con los contactos del relevador y de las derivaciones. En el conjunto de datos, la corriente INTERMEDIA fue de 3.6–3.7 A a baja carga; llegó a 4.0 A tanto en la validación del 15 de octubre como en la prueba de la aplicación, y la firma INTER a 0 % ya es la más cercana al umbral de 4.471 pp. Los umbrales de severidad son, por lo tanto, válidos para la corriente de falla con la que se capturaron. Un modelo simplificado del banco, $|\Delta I_2| \approx \mu \cdot I_f/3$, donde μ es la fracción de espiras e I_f la corriente del lazo, reproduce el orden de magnitud de la firma medida (cociente medido/modelo de 0.74–0.96), pero es un cálculo derivado post hoc, no declarado ni validado, y no permite afirmar un límite de detección.

La dirección de la firma también tiene una consecuencia que no se pudo verificar. Todas las fallas se inyectaron en la fase C. La teoría de componentes simétricas predice que una falla en las fases A o B rotaría la firma alrededor de ±120°, y el detector, que proyecta sobre −81.4°, tal como está no la confirmaría. Es una predicción teórica sin verificación en el banco, y una de las limitaciones más importantes para aplicar el sistema a otros motores o fases. La dirección depende además del orden de fases de la alimentación y del orden y la polaridad de los sensores, de modo que en otro motor o en otra instalación tendría que volver a medirse.

Finalmente, la aplicación y el asistente cumplen su papel sin intervenir en la decisión. La alarma la emite el firmware en unos 5 s y no depende del asistente; la recomendación local tarda de 122 a 139 s en la computadora del proyecto, fuera del criterio de 90 s, mientras que con un modelo externo tarda unos 3 s. En el set de prueba independiente, el asistente local recuperó la página esperada en 18 de 20 preguntas, se abstuvo correctamente en las 4 sin fuente e inventó algo en 2 de 24 respuestas. Que una de esas invenciones atribuya una cifra a una fuente que no la contiene confirma la decisión de mostrar cada cita con su fragmento original y de presentar al asistente como apoyo y no como diagnóstico. La reproducción del registro del 9 de octubre dio otro ejemplo: en una de sus corridas, el asistente afirmó que la alarma había persistido sin liberarse cuando el contexto indicaba 0 s con la alarma activa, y la afirmación llevaba cita. Que una línea esté citada no garantiza que esté sustentada; por eso la alarma y la severidad las decide el firmware, y la recomendación debe revisarla el personal de mantenimiento.

CAPÍTULO 5. CONCLUSIONES

Este trabajo permite concluir que es técnicamente viable detectar en línea un cortocircuito entre espiras en el estator de un motor de inducción trifásico con un sistema embebido de bajo costo, siempre que la detección se apoye en un indicador con fundamento físico, que cada modelo aprendido se valide en las condiciones en que va a operar y que la cadena de medición se trate como parte del sistema de diagnóstico. El banco construido, con un motor ABB de 1.5 kW, un generador como carga y un lazo de inyección controlada en la fase C, permitió reproducir de forma reversible fallas de tres severidades, y el ESP32-S3 resultó suficiente para adquirir seis canales, calcular la FFT y las componentes simétricas, decidir la alarma y clasificar su severidad en tiempo real.

El sistema final tiene tres etapas. La detección es una regla física sobre la corriente de secuencia negativa: la proyección del cambio de la firma sobre la dirección de falla de −81.4°, con un umbral de 0.40 pp durante 3 bloques y enclavamiento de la alarma. La severidad la estima un clasificador aprendido y embebido en el firmware. La recomendación la propone un asistente de mantenimiento local, con recuperación de documentos, integrado en la aplicación de escritorio MotorGuard-ITF. La detección quedó a cargo de la regla y no de un modelo de aprendizaje automático porque los modelos de detección evaluados no generalizaron entre cargas, y esa decisión se apoya en protocolos declarados antes de obtener los datos.

El capítulo resume primero el cumplimiento de los objetivos y de la hipótesis, después el papel de la inteligencia artificial y las lecciones del proyecto, y termina con las limitaciones y el trabajo futuro priorizado.

5.1 CUMPLIMIENTO DE LOS OBJETIVOS Y DE LA HIPÓTESIS

El objetivo general se cumple en lo esencial. Existe un sistema de diagnóstico que detecta tempranamente la falla con análisis de la corriente, componentes simétricas y procesamiento digital de señales, que usa inteligencia artificial en la severidad y en la recomendación, y que se validó experimentalmente en el banco. Respecto del objetivo registrado, el cumplimiento no es literal: XGBoost se evaluó para la detección, no generalizó y no forma parte del sistema final (sección 4.1.12).

En cuanto a los objetivos específicos (Tabla 34):

- **Banco de pruebas: cumplido.** La caracterización por voltaje inducido mostró que la documentación disponible del motor no correspondía a sus derivaciones reales y permitió definir clases de falla de aproximadamente 7.0, 14.3 y 21.6 % de las espiras de la fase C, con la corriente del lazo medida por clase.
- **Adquisición y procesamiento embebidos: cumplido.** El firmware adquiere los seis canales a 2403.85 Hz en bloques de 1.70 s y calcula en tiempo real la FFT, las características MCSA, las componentes simétricas y la firma de secuencia negativa.
- **Detector: cumplido en detección y latencia, no en falsas alarmas.** Con el firmware final, la validación ciega en vivo detectó 10/10 fallas SEVERA y 10/10 INTERMEDIA a 75 % de carga, y 59/60 fallas LEVE e INTERMEDIA a 0 y 25 %; la alarma se confirma en una mediana de 5.0 s. La tasa de falsas alarmas en operación continua fue de 7.3/h el 12 de octubre y de 4.08/h el 15 y 16 de octubre, frente al criterio declarado de una por hora.
- **Modelos de aprendizaje automático: cumplido.** Se compararon la regla, el Random Forest y XGBoost con protocolos declarados, y el clasificador de severidad acertó en 596 de 597 muestras dejando una carga fuera y quedó embebido en el firmware.
- **Aplicación de monitoreo y asistente: cumplido, con la prueba en el banco incompleta.** La aplicación MotorGuard-ITF 1.0.0 cuenta con 119 pruebas automáticas, paquetes de instalación y el

asistente de mantenimiento, y se usó con el banco real; su protocolo de prueba en vivo del 13 de octubre no se aprobó completo.

La hipótesis (sección 1.4) se confirma en su parte de detección y se rechaza en la de falsas alarmas. La falla LEVE, de aproximadamente 7 % de las espiras, se detectó en línea con un sistema embebido a 0, 25 y 75 % de carga: 20/20 y 20/20 a 0 y 25 % con el firmware final, y 19/20 en la computadora y 9/10 en el microcontrolador sin enclavamiento a 75 %. En cambio, el sistema no opera sin falsas alarmas durante la operación sana: es el resultado negativo principal de la tesis y se reporta sin atenuar. Tampoco se sostuvo la compensación contra el desbalance de la red prevista en la hipótesis: el voltaje de secuencia negativa explica solo el 3.9 % de la varianza de la firma y compensarlo no cambió las falsas alarmas, por lo que la hipótesis de que estas provenían de la red se retractó.

5.2 EL PAPEL DE LA INTELIGENCIA ARTIFICIAL

La conclusión metodológica central es que la exactitud de un modelo dentro de la distribución con que se entrenó no mide su capacidad de diagnóstico. Cuatro observaciones independientes lo muestran. En septiembre, los clasificadores sobre características espectrales superaron el 95 % de exactitud con partición aleatoria, pero el Random Forest bajó a 49.78 % al evaluarlo con otra campaña. En el estudio 2, el Random Forest con características MCSA, entrenado a 75 %, no detectó ninguna de las 450 fallas de las otras cargas, porque había aprendido la carga a través del nivel de corriente. En la validación ciega 2, el Random Forest con características de Fortescue entrenado con fallas a 75 % detectó solo 4/20 y 1/20 fallas LEVE a 0 y 25 %, frente a 20/20 y 20/20 de la regla ($p = 3.1 \times 10^{-5}$ y 3.8×10^{-6}), porque se apoyaba en el aumento de $|I_1|$ que la falla produce a carga alta y no en vacío. Y en la rejilla de 1904 variantes del detector, la configuración formalmente ganadora anulaba SEVERA a baja carga.

Por eso, en este sistema la inteligencia artificial se limita a las tareas en las que su desempeño se pudo comprobar. En la severidad, los umbrales aprendidos sobre la magnitud de la firma se validaron dejando una carga fuera y en vivo. En la recomendación, el asistente cita la página de cada fuente, se abstiene cuando los documentos no cubren la pregunta y no interviene en la decisión de la alarma. Aun así, el Random Forest de Fortescue sí redujo las falsas alarmas (2 frente a 21, $p = 3.3 \times 10^{-5}$). Según una observación post hoc, esa reducción se debe en parte a que casi no detecta la falla LEVE en vacío; un modelo entrenado con fallas a todas las cargas podría conservarla sin ese atajo. Esa posibilidad no se probó y es una de las líneas de trabajo futuro.

La aplicación y el asistente completan el sistema sin comprometer su seguridad. La aplicación solo puede encender y apagar el relevador, hacer consultas de lectura y reiniciar el detector; cada inyección exige confirmación humana y pasa por bloqueos que la impiden sin telemetría reciente, en calentamiento, con el relevador encendido o antes del enfriamiento. El asistente local funciona sin conexión a internet y, en el set de prueba independiente, recuperó la página esperada en 18 de 20 preguntas, respondió correctamente 17 de 20, se abstuvo correctamente en las 4 sin fuente e incluyó algo inventado en 2 de 24 respuestas; esta evaluación es preliminar por el tamaño del set y porque la calificación no es independiente.

Otro aprendizaje importante fue que el límite de sensibilidad lo fijó durante buena parte del proyecto la instrumentación y no el algoritmo. Con la protoboard original, las fluctuaciones de corriente por las conexiones eran del mismo orden que la firma de la falla; al rehacer las conexiones, la firma LEVE se midió por primera vez de forma reproducible. Con el sistema final, lo que limita las falsas alarmas sigue siendo el ruido lento de la firma, cercano al umbral de detección. En sistemas de bajo costo con sensores de salida ratiométrica, el montaje y la adquisición deben diseñarse y verificarse con el mismo cuidado que el procesamiento. Finalmente, declarar cada protocolo antes de obtener los datos, con un *commit* como evidencia de fecha, permitió distinguir los resultados validados de los análisis post hoc y reportar como tales los resultados negativos y las hipótesis retractadas.

5.3 LIMITACIONES

Del detector y de su validación

- **Falsas alarmas en operación continua.** Es la limitación principal: 7.3/h (IC 95 %: 3.7–13.1/h) el 12 de octubre y 4.08/h (IC 95 %: 2.49–6.30/h) el 15 y 16 de octubre, frente al criterio de una por hora. Su origen es un ruido lento y correlacionado de la firma, no el desbalance de la red, y la brecha con un sistema desatendido es de dos a tres órdenes de magnitud (sección 4.2).
- **Falsa alarma al arrancar el motor.** Con el motor recién encendido, el calentamiento de 60 s del firmware no basta: el 9 de octubre una alarma LEVE quedó enclavada al arranque. La medida operativa fue esperar unos 5 min después de encender el motor.
- **Alarmas enclavadas por escalones del motor sano.** Un escalón espontáneo de la corriente con el motor sano, en la dirección de la falla, deja la alarma enclavada, porque la liberación se compara con la base congelada; las fallas que llegan después se suman a esa base y escalan la severidad. En las pruebas largas se atendió con un reinicio automático del detector desde un *script* externo, y en la aplicación con el botón «Liberar alarma». Liberar una alarma real que persiste deja al detector ciego a esa falla, porque la nueva referencia ya la incluye; la aplicación lo advierte con una casilla obligatoria.
- **Severidad dependiente de la corriente del lazo.** A 0 % de carga, algunas inyecciones INTERMEDIA se clasificaron como SEVERA, porque la firma crece también con la corriente del lazo; los umbrales solo valen para la corriente de falla con la que se capturó el conjunto de datos.
- **Condiciones no evaluadas.** El generador no permite cambiar la carga con el motor girando, por lo que los escalones de carga en caliente no se evaluaron. Al 50 % de carga solo hay registros con el motor sano en las validaciones en vivo, y al 75 % no hay un bloque LEVE completo con el firmware final.
- **Fases A y B.** Todas las fallas se inyectaron en la fase C. Una falla en otra fase giraría la dirección de la firma, y el detector, tal como está, podría no confirmarla; es una predicción teórica sin verificación en el banco (sección 4.2).
- **Un solo motor y un solo banco.** Los resultados corresponden al motor ABB M1AA 145T-D4, con fallas inducidas mediante derivaciones fijas y una resistencia limitadora de 5 Ω; no representan la degradación natural del aislamiento. La dirección de −81.4° depende del orden de fases y de la polaridad de los sensores y tendría que volver a medirse en otro motor o instalación.
- **Lazo de inyección e instrumentación.** Los contactos del relevador, de la resistencia y de las derivaciones se degradaron durante las capturas y anularon o contaminaron algunos bloques de validación. Toda la instrumentación se montó en protoboard; no se construyó una placa soldada, y la verificación de los seis canales contra osciloscopio con el firmware final está pendiente.
- **Análisis post hoc.** La variante «ángulo ≤ 35° + LEVE en 4 bloques», el desglose de falsas alarmas por tramo de carga y el modelo $|\Delta I_2| \approx \mu \cdot I_f/3$ no estaban declarados; son indicios, no resultados validados.

De la aplicación y del asistente

- **Latencia de la recomendación.** Con el procesador estrangulado térmicamente, el asistente local tardó de 122 a 139 s, por encima del criterio declarado de 90 s; la alarma no depende de él.
- **Evaluación del asistente.** Los sets de preguntas son pequeños (24 preguntas en el set independiente) y la calificación es una propuesta del asistente de programación, no una calificación

independiente. Además, una respuesta citada puede contener afirmaciones que el contexto no sustenta, como ocurrió en la reproducción de una alarma del 9 de octubre.

- **Prueba con el banco.** El protocolo de prueba en vivo de la aplicación no se aprobó completo: la pinza marcó 4.0 A en el lazo INTERMEDIA, fuera del criterio. Además, en los registros del 12 al 16 de octubre hubo dos reinicios espontáneos del microcontrolador, de causa no identificada.
- **Plataforma y fuentes.** La versión 1.0.0 se distribuye solo para Ubuntu, y el manual de ABB no se puede redistribuir, por lo que cada usuario debe descargarlo del fabricante.

5.4 TRABAJO FUTURO

La Tabla 36 ordena las mejoras del detector por la ganancia que se espera de ellas frente a su costo. Cualquier campaña nueva en el banco debería diseñarse para servir también como conjunto de datos de la mejora 4, con registro continuo a todas las cargas.

Tabla 36. Trabajo futuro priorizado para reducir las falsas alarmas del detector.

Prioridad	Mejora	Ganancia esperada	Costo estimado	¿Firmware?
1	Confirmar la variante «ángulo ≤ 35° + LEVE en 4 bloques» con una prueba declarada de al menos 6 h nuevas de motor sano a todas las cargas	De 1.5 a 4 veces menos falsas alarmas, si se confirma	1–2 días de banco	No
2	Adquisición con ADC de muestreo simultáneo y transformadores de corriente, ya que el ACS712-30A usa cerca de 1/6 de su rango y el ADC del ESP32 es multiplexado	Atacar la causa raíz del ruido, si proviene de los sensores	Rediseño, recalibración y nueva captura	Sí
3	Firmware v2: referencia de dos velocidades, calentamiento de unos 3 min, liberación relativa al ruido o reinicio automático de alarmas atascadas, y ángulo de I_1 en la telemetría	Menos falsas alarmas y alarmas atascadas	Moderado	Sí
4	Detector aprendido entrenado con fallas a todas las cargas y datos continuos, sin ΔI_1 o con características invariantes a la carga, como segunda opinión en la aplicación	Conservar la reducción de falsas alarmas sin el atajo de carga	1–2 semanas, con protocolo declarado y datos nuevos	Sí (telemetría)

Además de esas mejoras, quedan las siguientes líneas de trabajo:

- **Validación ampliada:** escalones de carga en caliente con una carga que pueda conmutarse con el motor girando, un bloque LEVE completo a 75 % con el firmware final, fallas en las fases A y B, y otros motores y condiciones de alimentación, incluido un variador de frecuencia.
- **Montaje definitivo de la instrumentación:** una placa soldada en lugar de la protoboard, la verificación de los seis canales contra osciloscopio y la repetición de la prueba de provocación y de las sesiones de base.

- **Aplicación:** una versión para Windows, que requiere adaptar el acceso al puerto serie, el motor de inferencia local y su prioridad, las rutas y el instalador, y validarla con el banco; y el cierre de los pasos pendientes de la prueba en vivo.
- **Asistente:** medir su latencia en condiciones térmicas controladas o en un equipo más rápido, y ampliar la evaluación con más preguntas y calificadores independientes.
- **Monitoreo remoto:** un servidor de telemetría con FastAPI y validación con Pydantic, comunicación HTTP y WebSocket y almacenamiento de series de tiempo en PostgreSQL con TimescaleDB, para el registro histórico de varios motores (previsto en el plan inicial y fuera del alcance de esta tesis).

En síntesis, el sistema desarrollado detecta y clasifica en vivo fallas de cortocircuito entre espiras desde aproximadamente el 7 % de las espiras de una fase, a distintas cargas, con un costo bajo y con su lógica de decisión verificable; su principal pendiente para operar desatendido es reducir la tasa de falsas alarmas en dos o tres órdenes de magnitud.

REFERENCIAS

ABB. (2022). *Low voltage motors: Installation, operation, maintenance and safety manual* (3GZF500730-85 Rev H) [Manual]. ABB. Recuperado el 10 de octubre de 2026, de https://search.abb.com

Allan, D. W. (1966). Statistics of atomic frequency standards. *Proceedings of the IEEE, 54*(2), 221–230. https://doi.org/10.1109/PROC.1966.4634

Allegro MicroSystems. (2024). *ACS712: Fully integrated, Hall-effect-based linear current sensor IC with 2.1 kVRMS isolation and a low-resistance current conductor* (ACS712-DS, Rev. 22) [Hoja de datos]. Recuperado el 4 de octubre de 2026, de https://www.allegromicro.com/-/media/files/datasheets/acs712-datasheet.pdf

Arduino. (s. f.). *Nano ESP32*. Arduino Docs. Recuperado el 4 de octubre de 2026, de https://docs.arduino.cc/hardware/nano-esp32/

Arkan, M., Kostic-Perovic, D., y Unsworth, P. J. (2001). Online stator fault diagnosis in induction motors. *IEE Proceedings - Electric Power Applications, 148*(6), 537–547. https://doi.org/10.1049/ip-epa:20010588

Arkan, M., Kostic-Perovic, D., y Unsworth, P. J. (2005). Modelling and simulation of induction motors with inter-turn faults for diagnostics. *Electric Power Systems Research, 75*(1), 57–66. https://doi.org/10.1016/j.epsr.2004.08.015

Bellini, A., Filippetti, F., Tassoni, C., y Capolino, G.-A. (2008). Advances in diagnostic techniques for induction machines. *IEEE Transactions on Industrial Electronics, 55*(12), 4109–4126. https://doi.org/10.1109/TIE.2008.2007527

Benbouzid, M. E. H. (2000). A review of induction motors signature analysis as a medium for faults detection. *IEEE Transactions on Industrial Electronics, 47*(5), 984–993. https://doi.org/10.1109/41.873206

Bonnett, A. H., y Soukup, G. C. (1992). Cause and analysis of stator and rotor failures in three-phase squirrel-cage induction motors. *IEEE Transactions on Industry Applications, 28*(4), 921–937. https://doi.org/10.1109/28.148460

Bouzid, M. B. K., y Champenois, G. (2013). New expressions of symmetrical components of the induction motor under stator faults. *IEEE Transactions on Industrial Electronics, 60*(9), 4093–4102. https://doi.org/10.1109/TIE.2012.2235392

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Chen, L., Shen, J., Xu, G., Chi, C., Feng, Q., Zhou, Y., Deng, Y., y Wen, H. (2023). Induction motor stator winding inter-tern short circuit fault detection based on start-up current envelope energy. *Sensors, 23*(20), 8581. https://doi.org/10.3390/s23208581

Chen, T., y Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

Cooley, J. W., y Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation, 19*(90), 297–301. https://doi.org/10.1090/S0025-5718-1965-0178586-1

Cormack, G. V., Clarke, C. L. A., y Buettcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. En *Proceedings of the 32nd International ACM SIGIR Conference on*

Research and Development in Information Retrieval (pp. 758–759). ACM. https://doi.org/10.1145/1571941.1572114

Espressif Systems. (2026). *ESP32-S3 series datasheet* (Versión 2.2) [Hoja de datos]. Recuperado el 4 de octubre de 2026, de https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf

Fortescue, C. L. (1918). Method of symmetrical co-ordinates applied to the solution of polyphase networks. *Transactions of the American Institute of Electrical Engineers, 37*(2), 1027–1140. https://doi.org/10.1109/T-AIEE.1918.4765570

Fotek Controls. (s. f.). *Solid state relay SSR series* [Hoja de datos].

Gangsar, P., y Tiwari, R. (2020). Signal based condition monitoring techniques for fault detection and diagnosis of induction motors: A state-of-the-art review. *Mechanical Systems and Signal Processing, 144*, 106908. https://doi.org/10.1016/j.ymssp.2020.106908

Gerganov, G. (s. f.). *llama.cpp* [Software]. GitHub. Recuperado el 17 de octubre de 2026, de https://github.com/ggml-org/llama.cpp

Grandke, T. (1983). Interpolation algorithms for discrete Fourier transforms of weighted signals. *IEEE Transactions on Instrumentation and Measurement, 32*(2), 350–355. https://doi.org/10.1109/TIM.1983.4315077

Harris, F. J. (1978). On the use of windows for harmonic analysis with the discrete Fourier transform. *Proceedings of the IEEE, 66*(1), 51–83. https://doi.org/10.1109/PROC.1978.10837

Henao, H., Capolino, G.-A., Fernandez-Cabanas, M., Filippetti, F., Bruzzese, C., Strangas, E., Pusca, R., Estima, J., Riera-Guasp, M., y Hedayati-Kia, S. (2014). Trends in fault diagnosis for electrical machines: A review of diagnostic techniques. *IEEE Industrial Electronics Magazine, 8*(2), 31–42. https://doi.org/10.1109/MIE.2013.2287651

Hinkley, D. V. (1971). Inference about the change-point from cumulative sum tests. *Biometrika, 58*(3), 509–523. https://doi.org/10.1093/biomet/58.3.509

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., y Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys, 55*(12), 1–38. https://doi.org/10.1145/3571730

Kaufman, S., Rosset, S., y Perlich, C. (2011). Leakage in data mining: Formulation, detection, and avoidance. En *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 556–563). ACM. https://doi.org/10.1145/2020408.2020496

Kliman, G. B., Premerlani, W. J., Koegl, R. A., y Hoeweler, D. (1996). A new approach to on-line turn fault detection in AC motors. En *Conference Record of the 1996 IEEE Industry Applications Conference, 31st IAS Annual Meeting* (Vol. 1, pp. 687–693). IEEE. https://doi.org/10.1109/IAS.1996.557113

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., y Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. En *Advances in Neural Information Processing Systems 33* (pp. 9459–9474). Curran Associates. https://arxiv.org/abs/2005.11401

Lorden, G. (1971). Procedures for reacting to a change in distribution. *The Annals of Mathematical Statistics, 42*(6), 1897–1908. https://doi.org/10.1214/aoms/1177693055

Nandi, S., Toliyat, H. A., y Li, X. (2005). Condition monitoring and fault diagnosis of electrical motors—A review. *IEEE Transactions on Energy Conversion, 20*(4), 719–729. https://doi.org/10.1109/TEC.2005.847955

National Aeronautics and Space Administration. (2008). *Reliability-centered maintenance guide for facilities and collateral equipment*. NASA. Recuperado el 10 de octubre de 2026, de https://www.nasa.gov/wp-content/uploads/2023/06/nasa-rcmguide.pdf

Niculescu-Mizil, A., y Caruana, R. (2005). Predicting good probabilities with supervised learning. En *Proceedings of the 22nd International Conference on Machine Learning* (pp. 625–632). ACM. https://doi.org/10.1145/1102351.1102430

Page, E. S. (1954). Continuous inspection schemes. *Biometrika, 41*(1/2), 100–115. https://doi.org/10.2307/2333009

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. En A. J. Smola, P. Bartlett, B. Schölkopf y D. Schuurmans (Eds.), *Advances in large margin classifiers* (pp. 61–74). MIT Press.

Qingxian Zeming Langxi Electronic. (s. f.). *ZMPT101B current-type voltage transformer* [Hoja de datos]. http://www.micro-transformer.com

Qwen Team. (s. f.-a). *Qwen3.5-4B* [Modelo de lenguaje]. Hugging Face. Recuperado el 17 de octubre de 2026, de https://huggingface.co/Qwen/Qwen3.5-4B

Qwen Team. (s. f.-b). *Qwen3-Embedding-0.6B* [Modelo de embeddings]. Hugging Face. Recuperado el 17 de octubre de 2026, de https://huggingface.co/Qwen/Qwen3-Embedding-0.6B

Riley, W. J. (2008). *Handbook of frequency stability analysis* (NIST Special Publication 1065). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.1065

Roberts, D. R., Bahn, V., Ciuti, S., Boyce, M. S., Elith, J., Guillera-Arroita, G., Hauenstein, S., Lahoz-Monfort, J. J., Schröder, B., Thuiller, W., Warton, D. I., Wintle, B. A., Hartig, F., y Dormann, C. F. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography, 40*(8), 913–929. https://doi.org/10.1111/ecog.02881

Robertson, S., y Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval, 3*(4), 333–389. https://doi.org/10.1561/1500000019

Ruzimov, S., Zhang, J., Huang, X., y Aziz, M. S. (2025). Detection of inter-turn short-circuit faults for inverter-fed induction motors based on negative-sequence current analysis. *Sensors, 25*(15), 4844. https://doi.org/10.3390/s25154844

Siddique, A., Yadava, G. S., y Singh, B. (2005). A review of stator fault monitoring techniques of induction motors. *IEEE Transactions on Energy Conversion, 20*(1), 106–114. https://doi.org/10.1109/TEC.2004.837304

Sullivan, G. P., Pugh, R., Melendez, A. P., y Hunt, W. D. (2010). *Operations & maintenance best practices: A guide to achieving operational efficiency* (Release 3.0). Pacific Northwest National Laboratory para el U.S. Department of Energy, Federal Energy Management Program.

Tallam, R. M., Habetler, T. G., y Harley, R. G. (2002). Transient model for induction machines with stator winding turn faults. *IEEE Transactions on Industry Applications, 38*(3), 632–637. https://doi.org/10.1109/TIA.2002.1003411

Tallam, R. M., Lee, S. B., Stone, G. C., Kliman, G. B., Yoo, J., Habetler, T. G., y Harley, R. G. (2007). A survey of methods for detection of stator-related faults in induction machines. *IEEE Transactions on Industry Applications, 43*(4), 920–933. https://doi.org/10.1109/TIA.2007.900448

Thomson, W. T., y Fenger, M. (2001). Current signature analysis to detect induction motor faults. *IEEE Industry Applications Magazine, 7*(4), 26–34. https://doi.org/10.1109/2943.930988

Thorsen, O. V., y Dalva, M. (1995). A survey of faults on induction motors in offshore oil industry, petrochemical industry, gas terminals, and oil refineries. *IEEE Transactions on Industry Applications, 31*(5), 1186–1196. https://doi.org/10.1109/28.464536

U.S. Bureau of Reclamation. (1991a). *Testing solid insulation of electrical equipment* (Facilities Instructions, Standards, and Techniques, Vol. 3-1). Recuperado el 10 de octubre de 2026, de https://www.usbr.gov/power/data/fist/fist3_1/vol3-1.pdf

U.S. Bureau of Reclamation. (1991b). *Keeping motor windings dry* (Facilities Instructions, Standards, and Techniques, Vol. 3-4). Recuperado el 10 de octubre de 2026, de https://www.usbr.gov/power/data/fist/fist3_4/vol3-4.pdf

U.S. Bureau of Reclamation. (2026). *Maintenance scheduling for electrical equipment* (Facilities Instructions, Standards, and Techniques, Vol. 4-1B). Recuperado el 10 de octubre de 2026, de https://www.usbr.gov/power/data/fist/FIST_4-1B_(01-2026).pdf

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., y Polosukhin, I. (2017). Attention is all you need. En *Advances in Neural Information Processing Systems 30* (pp. 5998–6008). Curran Associates. https://arxiv.org/abs/1706.03762

Waide, P., y Brunner, C. U. (2011). *Energy-efficiency policy opportunities for electric motor-driven systems* (IEA Energy Papers, No. 2011/07). OECD Publishing. https://doi.org/10.1787/5kgg52gb9gjd-en

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., . . . Qiu, Z. (2025). *Qwen3 technical report* (arXiv:2505.09388). arXiv. https://arxiv.org/abs/2505.09388

Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., Xie, P., Yang, A., Liu, D., Lin, J., Huang, F., y Zhou, J. (2025). *Qwen3 Embedding: Advancing text embedding and reranking through foundation models* (arXiv:2506.05176). arXiv. https://arxiv.org/abs/2506.05176

ANEXOS

Los anexos reúnen el código fuente de las partes del sistema que sustentan los resultados de esta tesis. Los fragmentos se copiaron textualmente de los archivos de trabajo, por lo que conservan sus comentarios originales; se indica en cada caso el archivo y las líneas de origen. El código completo, junto con los registros de ensayo y los *scripts* que generan las figuras, se publica en el repositorio público de GitHub que se indica abajo; las rutas de los listados son relativas a la raíz de ese repositorio, y los números de línea corresponden a sus archivos.

https://github.com/juanemmanuel2220-star/motorguard-itf-tesis

ANEXO A. FIRMWARE DEL ESP32-S3

Este anexo contiene el detector direccional con enclavamiento y clasificación de severidad (firmware 299a943), el cálculo de las componentes simétricas, la llamada al detector en el lazo principal y las protecciones del lazo de inyección. El CUSUM del Capítulo 4 sigue compilado en el firmware solo como indicador informativo, por lo que no se reproduce.

Listado 5. *Detector direccional: constantes y estado (firmware/PRUEBA_MODELO_1/detector_dir.h, líneas 1–70).*

```
#ifndef DETECTOR_DIR_H
#define DETECTOR_DIR_H

#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

// ==================== DETECTOR DIRECCIONAL DE FALLA ENTRE ESPIRAS ====================
// Port del detector validado en vivo el 07-oct-2026 (validacion_vivo_direccional.py, commit
39d45ed):
//   z(t)   = R2(t) / |I1(t)| * 100                      (pp de |I1|)
//   dz(t)  = z(t) - mediana(z en [t - 11 s, t - 6 s])     (referencia por componente)
//   proy   = Re{ dz * exp(-j*DIR_ANG_FALLA) }
//   alarma confirmada cuando DIR_PERSISTENCIA bloques seguidos tienen proy >= DIR_UMBRAL
// Parametros ajustados con el bloque LEVE del 06-oct-2026 (200 pares, 4 cargas). No modificar sin
// repetir la validacion ciega.
constexpr float DIR_ANG_FALLA = -81.4f;      // grados, direccion de la firma respecto a arg(V1)
constexpr float DIR_UMBRAL = 0.40f;          // pp de |I1|
constexpr uint32_t DIR_REF_DESDE_MS = 11000; // la referencia usa bloques entre t-11 s ...
constexpr uint32_t DIR_REF_HASTA_MS = 6000;  // ... y t-6 s
constexpr uint32_t DIR_HIST_MS = 30000;      // historial conservado
constexpr int DIR_PERSISTENCIA = 3;          // bloques consecutivos sobre el umbral (~5 s)
#define DIR_HIST_N 32                        // capacidad del historial (~1.7 s por bloque)

// ==================== ENCLAVAMIENTO Y SEVERIDAD (08-oct-2026) ====================
// Al confirmar la alarma se congela la referencia pre-falla (z_base). Mientras dura la alarma, la
// firma se mide contra esa base, asi una falla sostenida no se "absorbe" en la referencia movil.
// La alarma se libera cuando la proyeccion contra la base cae por debajo de
// max(DIR_UMBRAL/2, DIR_LIBERA_FRAC * proyeccion de confirmacion) durante DIR_PERSISTENCIA bloques.
constexpr float DIR_LIBERA_FRAC = 0.35f;
// Severidad = umbrales sobre |dz| (pp), entrenados con 597 pares el 08-oct-2026
// (entrenar_severidad.py, modelo M1: 0.998 dejando una carga fuera). Se clasifica con la mediana
// de los 3 bloques de confirmacion y solo puede escalar mientras la alarma siga enclavada.
constexpr float DIR_SEV_U1 = 1.626f;         // LEVE < U1 <= INTER
constexpr float DIR_SEV_U2 = 4.471f;         // INTER < U2 <= SEVERA

typedef struct {
    uint32_t t_ms[DIR_HIST_N];
    float z_re[DIR_HIST_N];
    float z_im[DIR_HIST_N];
    int n;              // bloques validos en el historial (ordenados por tiempo)
```

```
    int racha;          // bloques consecutivos con proy >= DIR_UMBRAL (referencia movil)
    int ref_ok;         // 1 si el ultimo bloque tuvo referencia suficiente (>= 2 bloques)
    float d_re, d_im;   // dz del ultimo bloque contra la referencia movil (pp)
    float proy;         // proyeccion del ultimo bloque sobre la direccion de falla (pp)
    float mag3[DIR_PERSISTENCIA];  // |dz| de los ultimos bloques (para la severidad al confirmar)
    int confirmada;     // 1 mientras la alarma esta enclavada
    // Estado enclavado
    float base_re, base_im;  // referencia pre-falla congelada al confirmar (pp)
    float proy_conf;    // proyeccion en el bloque de confirmacion (pp)
    float proy_lat;     // proyeccion contra la base congelada (pp)
    float mag_lat;      // |z - base| (pp): magnitud de la firma mientras dura la alarma
    float maglat3[DIR_PERSISTENCIA];
    int nlat;           // bloques transcurridos desde la confirmacion
    int bajos;          // bloques consecutivos bajo el nivel de liberacion
    int sev;            // 0 sin alarma, 1 LEVE, 2 INTER, 3 SEVERA
} DetectorDir;

void dir_init(DetectorDir* st);

// Procesa un bloque y devuelve la severidad enclavada (0 = sin alarma, 1..3). r2_re/r2_im en A
pico,
// i1_mag = |I1| en A pico (los mismos fasores de la telemetria RAW).
int dir_step(DetectorDir* st, uint32_t t_ms, float r2_re, float r2_im, float i1_mag);

#ifdef __cplusplus
}
#endif

#endif // DETECTOR_DIR_H
```

Listado 6. *Detector direccional: implementación completa (firmware/PRUEBA_MODELO_1/detector_dir.cpp, líneas 1–124).*

```
#include "detector_dir.h"
#include <math.h>
#include <string.h>

void dir_init(DetectorDir* st) {
    memset(st, 0, sizeof(DetectorDir));
}

// Mediana como statistics.median de Python: promedio de los dos centrales si n es par
static float mediana(float* v, int n) {
    for (int i = 1; i < n; i++) {
        float x = v[i];
        int j = i - 1;
        while (j >= 0 && v[j] > x) { v[j + 1] = v[j]; j--; }
        v[j + 1] = x;
    }
    return (n % 2) ? v[n / 2] : 0.5f * (v[n / 2 - 1] + v[n / 2]);
}

static int clasificar_sev(float mag) {
    return (mag >= DIR_SEV_U2) ? 3 : (mag >= DIR_SEV_U1) ? 2 : 1;
}

static float proyectar(float re, float im) {
    const float a = DIR_ANG_FALLA * 0.017453293f;
    return re * cosf(a) + im * sinf(a);
}

static void empujar(float* v, float x) {
    for (int i = 0; i < DIR_PERSISTENCIA - 1; i++) v[i] = v[i + 1];
    v[DIR_PERSISTENCIA - 1] = x;
}

int dir_step(DetectorDir* st, uint32_t t_ms, float r2_re, float r2_im, float i1_mag) {
    if (i1_mag < 1e-6f) return st->sev;
```

```
    float z_re = r2_re / i1_mag * 100.0f;
    float z_im = r2_im / i1_mag * 100.0f;

    // Referencia con los bloques previos dentro de [t - 11 s, t - 6 s]
    float ref_re[DIR_HIST_N], ref_im[DIR_HIST_N];
    int m = 0;
    for (int i = 0; i < st->n; i++) {
        uint32_t edad = t_ms - st->t_ms[i];
        if (edad <= DIR_REF_DESDE_MS && edad >= DIR_REF_HASTA_MS) {
            ref_re[m] = st->z_re[i];
            ref_im[m] = st->z_im[i];
            m++;
        }
    }

    // Agregar el bloque actual y descartar los de mas de 30 s
    int k = 0;
    for (int i = 0; i < st->n; i++) {
        if (t_ms - st->t_ms[i] <= DIR_HIST_MS) {
            st->t_ms[k] = st->t_ms[i]; st->z_re[k] = st->z_re[i]; st->z_im[k] = st->z_im[i]; k++;
        }
    }
    if (k == DIR_HIST_N) {  // historial lleno: se descarta el mas antiguo
        memmove(st->t_ms, st->t_ms + 1, (k - 1) * sizeof(uint32_t));
        memmove(st->z_re, st->z_re + 1, (k - 1) * sizeof(float));
        memmove(st->z_im, st->z_im + 1, (k - 1) * sizeof(float));
        k--;
    }
    st->t_ms[k] = t_ms; st->z_re[k] = z_re; st->z_im[k] = z_im;
    st->n = k + 1;

    // Detector con referencia movil (sin referencia suficiente no se modifica la racha)
    st->ref_ok = (m >= 2);
    float rre = 0.0f, rim = 0.0f;
    if (st->ref_ok) {
        rre = mediana(ref_re, m);
        rim = mediana(ref_im, m);
        st->d_re = z_re - rre;
        st->d_im = z_im - rim;
        st->proy = proyectar(st->d_re, st->d_im);
        st->racha = (st->proy >= DIR_UMBRAL) ? st->racha + 1 : 0;
        empujar(st->mag3, sqrtf(st->d_re * st->d_re + st->d_im * st->d_im));
    }

    if (!st->confirmada) {
        // Confirmacion: se congela la referencia pre-falla y se clasifica la severidad
        if (st->ref_ok && st->racha >= DIR_PERSISTENCIA) {
            float tmp[DIR_PERSISTENCIA];
            memcpy(tmp, st->mag3, sizeof(tmp));
            st->confirmada = 1;
            st->base_re = rre;
            st->base_im = rim;
            st->proy_conf = st->proy;
            st->proy_lat = st->proy;
            st->mag_lat = mediana(tmp, DIR_PERSISTENCIA);
            st->sev = clasificar_sev(st->mag_lat);
            memset(st->maglat3, 0, sizeof(st->maglat3));
            st->nlat = 0;
            st->bajos = 0;
        }
        return st->sev;
    }

    // Alarma enclavada: firma contra la base congelada
    float dl_re = z_re - st->base_re, dl_im = z_im - st->base_im;
    st->proy_lat = proyectar(dl_re, dl_im);
    st->mag_lat = sqrtf(dl_re * dl_re + dl_im * dl_im);
    empujar(st->maglat3, st->mag_lat);
    if (++st->nlat >= DIR_PERSISTENCIA) {
        // La severidad solo escala (una falla entre espiras no se repara sola)
```

```
        float tmp[DIR_PERSISTENCIA];
        memcpy(tmp, st->maglat3, sizeof(tmp));
        int s = clasificar_sev(mediana(tmp, DIR_PERSISTENCIA));
        if (s > st->sev) st->sev = s;
    }

    float nivel = DIR_LIBERA_FRAC * st->proy_conf;
    if (nivel < 0.5f * DIR_UMBRAL) nivel = 0.5f * DIR_UMBRAL;
    st->bajos = (st->proy_lat < nivel) ? st->bajos + 1 : 0;
    if (st->bajos >= DIR_PERSISTENCIA) {
        // Falla retirada: se libera la alarma; la racha se reinicia para no re-disparar
        st->confirmada = 0;
        st->sev = 0;
        st->racha = 0;
        st->bajos = 0;
    }
    return st->sev;
}
```

Listado 7. *Cálculo de fasores y componentes simétricas (firmware/PRUEBA_MODELO_1/secuencia.cpp, líneas 135–289).*

```
void calcular_secuencia(
    const int16_t samp_buf[6][SEC_NBIG],
    float f0,
    const float cal_v[3],
    const float cal_i[3],
    const float skew_us[6],
    ComplexFloat Y2_comp,
    ComplexFloat I2_0,
    SecuenciaResultados* res
) {
    if (!samp_buf || !res) return;
    memset(res, 0, sizeof(SecuenciaResultados));
    res->f0 = f0;

    // 1. Calcular media (offset DC) de cada uno de los 6 canales
    float mean[6];
    for (int c = 0; c < 6; c++) {
        double s = 0.0;
        for (int i = 0; i < SEC_NBIG; i++) {
            s += (double)samp_buf[c][i];
        }
        mean[c] = (float)(s / (double)SEC_NBIG);
    }

    // 2. Oscilador para carrier e^(-j * 2pi * f0 * n / Fs)
    float dtheta = TWO_PI_F * f0 / SEC_FS;
    float step_re = cosf(-dtheta);
    float step_im = sinf(-dtheta);
    float z_re = 1.0f, z_im = 0.0f;

    // Oscilador para ventana Hann periódica: w[n] = 0.5 - 0.5*cos(2pi*n / N)
    float dphi = TWO_PI_F / (float)SEC_NBIG;
    float wstep_re = cosf(dphi);
    float wstep_im = sinf(dphi);
    float wz_re = 1.0f, wz_im = 0.0f;

    // Acumuladores de fasores para los 6 canales
    float acc_re[6] = {0}, acc_im[6] = {0};

    for (int i = 0; i < SEC_NBIG; i++) {
        float w = 0.5f - 0.5f * wz_re;
        float wz_real = w * z_re;
        float wz_imag = w * z_im;

        for (int c = 0; c < 6; c++) {
            float val = (float)samp_buf[c][i] - mean[c];
            acc_re[c] += val * wz_real;
```

```
            acc_im[c] += val * wz_imag;
        }

        // Paso del oscilador carrier
        float nz_re = z_re * step_re - z_im * step_im;
        float nz_im = z_re * step_im + z_im * step_re;
        z_re = nz_re; z_im = nz_im;

        // Paso del oscilador Hann
        float nwz_re = wz_re * wstep_re - wz_im * wstep_im;
        float nwz_im = wz_re * wstep_im + wz_im * wstep_re;
        wz_re = nwz_re; wz_im = nwz_im;

        // Renormalización cada 256 muestras para prevenir deriva numérica
        if ((i & 255) == 255) {
            float norm_z = 1.0f / sqrtf(z_re * z_re + z_im * z_im);
            z_re *= norm_z; z_im *= norm_z;
            float norm_w = 1.0f / sqrtf(wz_re * wz_re + wz_im * wz_im);
            wz_re *= norm_w; wz_im *= norm_w;
        }
    }

    // Normalización: factor 2 / sum(w) = 2 / (SEC_NBIG / 2) = 4 / SEC_NBIG = 1 / 1024
    float scale_factor = 4.0f / (float)SEC_NBIG;
    ComplexFloat X[6];
    for (int c = 0; c < 6; c++) {
        X[c].re = acc_re[c] * scale_factor;
        X[c].im = acc_im[c] * scale_factor;
    }

    // 3. Compensación de retardo de muestreo secuencial (SKEW_US)
    // Para muestras tomadas en t + tau, se retarda la fase multiplicando por exp(-j * 2pi * f0 *
tau)
    const float* skew = skew_us ? skew_us : DEFAULT_SKEW_US;
    for (int c = 0; c < 6; c++) {
        float tau = skew[c] * 1e-6f;
        float angle = -TWO_PI_F * f0 * tau;
        ComplexFloat rot = {cosf(angle), sinf(angle)};
        X[c] = c_mul(X[c], rot);
    }

    // 4. Aplicación de factores de calibración física
    ComplexFloat Va = c_scale(X[0], cal_v ? cal_v[0] : 1.0f);
    ComplexFloat Vb = c_scale(X[1], cal_v ? cal_v[1] : 1.0f);
    ComplexFloat Vc = c_scale(X[2], cal_v ? cal_v[2] : 1.0f);

    ComplexFloat Ia = c_scale(X[3], cal_i ? cal_i[0] : 1.0f);
    ComplexFloat Ib = c_scale(X[4], cal_i ? cal_i[1] : 1.0f);
    ComplexFloat Ic = c_scale(X[5], cal_i ? cal_i[2] : 1.0f);

    // 5 y 6. Operador de Fortescue: a = exp(j * 2pi/3) = -0.5 + j * sqrt(3)/2
    ComplexFloat a_op   = {-0.5f,  SQRT3_DIV_2};
    ComplexFloat a2_op  = {-0.5f, -SQRT3_DIV_2};

    // V1 = (Va + a*Vb + a^2*Vc) / 3
    ComplexFloat V1_raw = c_scale(c_add(Va, c_add(c_mul(a_op, Vb), c_mul(a2_op, Vc))), 1.0f / 3.0f);

    // Referencia angular respecto a V1 (phi_ref = arg(V1))
    float phi_ref = atan2f(V1_raw.im, V1_raw.re);
    ComplexFloat rot_ref = {cosf(-phi_ref), sinf(-phi_ref)};

    Va = c_mul(Va, rot_ref);
    Vb = c_mul(Vb, rot_ref);
    Vc = c_mul(Vc, rot_ref);
    Ia = c_mul(Ia, rot_ref);
    Ib = c_mul(Ib, rot_ref);
    Ic = c_mul(Ic, rot_ref);
```

```
    // Diagnóstico por fase: ángulo de cada corriente respecto a su voltaje (≈ -30..-50° en un
motor)
    {
        ComplexFloat Vk[3] = {Va, Vb, Vc}, Ik[3] = {Ia, Ib, Ic};
        for (int k = 0; k < 3; k++) {
            ComplexFloat z = c_mul(Ik[k], c_conj(Vk[k]));
            res->ang_iv[k] = atan2f(z.im, z.re) * 57.29578f;
            res->mag_i[k] = c_abs(Ik[k]);
        }
    }

    // Recalcular componentes simétricas referenciadas
    ComplexFloat V1 = c_scale(c_add(Va, c_add(c_mul(a_op, Vb), c_mul(a2_op, Vc))), 1.0f / 3.0f);
    ComplexFloat V2 = c_scale(c_add(Va, c_add(c_mul(a2_op, Vb), c_mul(a_op, Vc))), 1.0f / 3.0f);

    ComplexFloat I1 = c_scale(c_add(Ia, c_add(c_mul(a_op, Ib), c_mul(a2_op, Ic))), 1.0f / 3.0f);
    ComplexFloat I2 = c_scale(c_add(Ia, c_add(c_mul(a2_op, Ib), c_mul(a_op, Ic))), 1.0f / 3.0f);


    res->V1 = V1;
    res->V2 = V2;
    res->I1 = I1;
    res->I2 = I2;

    float mag_V1 = c_abs(V1);
    float mag_V2 = c_abs(V2);
    float mag_I1 = c_abs(I1);
    float mag_I2 = c_abs(I2);

    res->u_V = (mag_V1 > 1e-6f) ? (mag_V2 / mag_V1) : 0.0f;
    res->u_I = (mag_I1 > 1e-6f) ? (mag_I2 / mag_I1) : 0.0f;

    // 7. Residuo compensado: e_2 = |I2 - Y2*V2 - I2_0| / |I1|
    ComplexFloat Y2_V2 = c_mul(Y2_comp, V2);
    ComplexFloat diff = c_sub(c_sub(I2, Y2_V2), I2_0);
    res->R2 = diff;
    float mag_diff = c_abs(diff);
    res->e_2 = (mag_I1 > 1e-6f) ? (mag_diff / mag_I1) : 0.0f;

    // 8. Guardia de dominio (NEMA MG-1: u_V <= 0.02)
    res->dom = (res->u_V <= SEC_VUF_MAX) ? 1 : 0;
}
```

Listado 8. *Decisión principal en el lazo del firmware: llamada al detector direccional (firmware/PRUEBA_MODELO_1/PRUEBA_MODELO_1.ino, líneas 747–753).*

```
    // 4. Decision principal: detector direccional sobre el residuo crudo R2 (independiente del
CUSUM).
    //    Devuelve la severidad enclavada: 0 sano, 1 LEVE, 2 INTER, 3 SEVERA.
    float i1_dir = sqrtf(sec_res.I1.re * sec_res.I1.re + sec_res.I1.im * sec_res.I1.im);
    pred = dir_step(&det_dir, millis(), sec_res.R2.re, sec_res.R2.im, i1_dir);
    // Confianza = 1 con la alarma enclavada; si no, avance de la persistencia (0, 1/3, 2/3)
    conf = det_dir.confirmada ? 1.0f
         : (float)(det_dir.racha < DIR_PERSISTENCIA ? det_dir.racha : DIR_PERSISTENCIA) /
(float)DIR_PERSISTENCIA;
```

Listado 9. *Protecciones del lazo de inyección: activación y desactivación del relevador (firmware/PRUEBA_MODELO_1/PRUEBA_MODELO_1.ino, líneas 769–821).*

```
void ssrOff(bool forzado) {
  bool alguno = false;
  for (int i = 0; i < 3; i++) {
    if (ssr[i]) { ssr[i] = false; alguno = true; }
    uint8_t pin = (i == 0) ? SSR_F1 : (i == 1) ? SSR_F2 : SSR_F3;
    gpio_num_t gpio = (gpio_num_t)((i == 0) ? 8 : (i == 1) ? 7 : 9);
    digitalWrite(pin, relayInactiveLevel(i));
    gpio_set_level(gpio, relayInactiveLevel(i));
```

```
  }
  if (alguno) {
    t_on_ms = millis()-t_on_ini;
    t_last_off = millis();
    Serial.printf("[DEBUG SSR] Apagado. Forzado: %d | T_ON: %lu ms\n", forzado, t_on_ms);
    addLog(forzado ? "SSR OFF forzado (watchdog)" : "SSR DESACTIVADO");
  }
}
bool ssrOn(int n) {
  Serial.printf("[DEBUG SSR] Solicitud F%d | I_rms=%.2fA | Cooldown=%lu ms\n",
                n+1, I_rms, (millis() - t_last_off));
  if (n < 0 || n > 2) return false;
  if (!motor_on) {
    Serial.println("[DEBUG SSR] Rechazado: Motor apagado. Peligro de Inrush.");
    addLog("Motor off: aborta SSR");
    return false;
  }
  if (ssr[0] || ssr[1] || ssr[2]) {
    Serial.println("[DEBUG SSR] Rechazado: Ya hay un SSR activo.");
    addLog("SSR ya activo");
    return false;
  }
  if (millis()-t_last_off < T_COOLDOWN) {
    Serial.printf("[DEBUG SSR] Rechazado: Cooldown activo (%lu/%lu ms).\n", (millis()-t_last_off),
T_COOLDOWN);
    addLog("Cooldown activo, ignora /on");
    return false;
  }
  if (I_rms > I_UMBRAL) {
    Serial.printf("[DEBUG SSR] Rechazado: I_rms=%.2fA supera umbral %.1fA\n", I_rms, I_UMBRAL);
    char b[40]; snprintf(b, sizeof(b), "Aborto: I_rms=%.1fA", I_rms);
    addLog(b);
    return false;
  }
  ssr[n] = true; t_on_ini = millis(); act_count++;
  uint8_t pin = (n == 0) ? SSR_F1 : (n == 1) ? SSR_F2 : SSR_F3;
  gpio_num_t gpio = (gpio_num_t)((n == 0) ? 8 : (n == 1) ? 7 : 9);
  digitalWrite(pin, relayActiveLevel(n));
  gpio_set_level(gpio, relayActiveLevel(n));
  Serial.printf("[DEBUG SSR] Exito: GPIO %d puesto en %s (%dV)\n",
                gpio, relayActiveLevel(n) ? "HIGH" : "LOW", relayActiveLevel(n) ? 3 : 0);
  char b[40]; snprintf(b, sizeof(b), "SSR F%d ACTIVADO (%s)", n+1, namePred(n+1));
  addLog(b);
  return true;
}
```

Listado 10. *Supervisión del relevador en el lazo principal (tiempo máximo y sobrecorriente) (firmware/PRUEBA_MODELO_1/PRUEBA_MODELO_1.ino, líneas 1407–1413).*

```
  static uint8_t ovCnt = 0;
  if (ssr[0]||ssr[1]||ssr[2]) {
    uint32_t ton = millis()-t_on_ini;
    if (ton >= T_MAX_ON) ssrOff(true);
    else if (ton > INRUSH_BLANK_MS && I_fast > I_UMBRAL) { if (++ovCnt >= 3) { ovCnt = 0;
ssrOff(true); } }
    else ovCnt = 0;
  } else ovCnt = 0;
```

ANEXO B. ADQUISICIÓN Y PROTOCOLO DE ENSAYO

Scripts que se ejecutan en la computadora durante los ensayos: el registro de sesiones de base sana y pulsos de falla, y la prueba de provocación de las conexiones.

Listado 11. *Registro de una sesión de base sana seguida de pulsos de falla (adquisicion/sesion_larga_base_pulsos.py, líneas 1–133).*

```
#!/usr/bin/env python3
"""
MotorGuard-ITF: sesion larga de adquisicion (base sana + tren de pulsos).

Registra TODAS las lineas seriales con marca de tiempo y etiqueta de fase
(WAIT / WARMUP / BASE / Pk_ON / Pk_OFF), para diseñar el detector offline
con los fasores crudos (campo RAW de la trama [MCSA]).

Uso:
  python3 sesion_larga_base_pulsos.py --falla LEVE --carga 50% --base-min 30 --pulsos 10
"""
import argparse
import sys
import time
from datetime import datetime

import serial

ap = argparse.ArgumentParser()
ap.add_argument("--falla", required=True, help="Etiqueta de los taps conectados
(LEVE/INTER/SEVERA)")
ap.add_argument("--carga", required=True, help="Carga mecanica (p.ej. 50%%)")
ap.add_argument("--base-min", type=float, default=30.0, help="Minutos de base sana tras el warmup")
ap.add_argument("--pulsos", type=int, default=10)
ap.add_argument("--t-on", type=float, default=6.0, help="Duracion de cada pulso (s)")
ap.add_argument("--t-off", type=float, default=30.0, help="Reposo tras cada pulso (s)")
ap.add_argument("--port", default="/dev/ttyACM0")
ap.add_argument("--espera-min", type=float, default=15.0, help="Minutos maximos esperando el
encendido del motor")
args = ap.parse_args()

ts = datetime.now().strftime("%Y%m%d_%H%M%S")
carga_tag = args.carga.replace("%", "")
out_path = f"/home/luna/sesion_{args.falla}_{carga_tag}carga_{ts}.log"
out = open(out_path, "w")
out.write(f"# Sesion larga: falla={args.falla} carga={args.carga} base={args.base_min} min "
          f"pulsos={args.pulsos}x{args.t_on}s reposo={args.t_off}s inicio={ts}\n")
print(f"LOG: {out_path}", flush=True)

import os
print(f"Esperando puerto {args.port}...", flush=True)
_tp = time.time()
while True:
    if os.path.exists(args.port):
        try:
            s = serial.Serial(args.port, 115200, timeout=1)
            break
        except serial.SerialException:
            pass
    if time.time() - _tp > args.espera_min * 60:
        print("TIMEOUT: el puerto serial no aparecio", flush=True)
        sys.exit(1)
    time.sleep(1)
print("Puerto conectado", flush=True)
t0 = time.time()
fase = "WAIT"
EVENTOS = ("IA ->", "[RED]", "Motor ON", "Motor apagado", "Motor:OFF", "[DEBUG SSR]", "Zero SSR")
ultimo_off = [0.0]

def log_linea(l):
    out.write(f"{time.time() - t0:8.2f}s [{fase}] {l}\n")
    out.flush()

def leer(dur, eco=True):
    t = time.time()
    while time.time() - t < dur:
```

```
        l = s.readline().decode(errors="replace").rstrip()
        if not l:
            continue
        log_linea(l)
        if "Motor:OFF" in l and fase not in ("WAIT",):
            # Un corte de alimentacion invalida la sesion: avisar una vez cada 30 s
            if time.time() - ultimo_off[0] > 30:
                ultimo_off[0] = time.time()
                print(f"{time.time() - t0:7.1f}s [{fase}] ALERTA: Motor:OFF detectado", flush=True)
        elif eco and any(k in l for k in EVENTOS) and "MOTOR_DET" not in l:
            print(f"{time.time() - t0:7.1f}s [{fase}] {l[:120]}", flush=True)

def esperar(token, timeout):
    t = time.time()
    while time.time() - t < timeout:
        l = s.readline().decode(errors="replace").rstrip()
        if not l:
            continue
        log_linea(l)
        if any(tok in l for tok in (token if isinstance(token, tuple) else (token,))):
            return l
    return None

try:
    s.write(b"0\n")
    s.flush()
    print("Esperando motor encendido...", flush=True)
    l_on = esperar(("Motor ON", "Motor:WARMUP", "Motor:ON"), args.espera_min * 60)
    if not l_on:
        print(f"TIMEOUT: el motor no encendio en {args.espera_min} min", flush=True)
        sys.exit(1)
    if "Motor:ON" in l_on:
        print(f"{time.time() - t0:7.1f}s Motor ya en marcha (warmup completado)", flush=True)
    else:
        fase = "WARMUP"
        print(f"{time.time() - t0:7.1f}s Motor ON, warmup 60 s", flush=True)
        if not esperar("IA -> SANO", 120):
            print("AVISO: no se observo fin de warmup en 120 s; se continua", flush=True)

    fase = "BASE"
    print(f"{time.time() - t0:7.1f}s BASE {args.base_min} min (SSR OFF)", flush=True)
    t_base = time.time()
    while time.time() - t_base < args.base_min * 60:
        leer(min(300, args.base_min * 60 - (time.time() - t_base)))
        print(f"{time.time() - t0:7.1f}s [BASE] progreso {(time.time() - t_base) /
60:.1f}/{args.base_min} min", flush=True)

    for k in range(1, args.pulsos + 1):
        fase = f"P{k}_ON"
        s.reset_input_buffer()
        s.write(b"1\n")
        s.flush()
        leer(args.t_on, eco=False)
        s.write(b"0\n")
        s.flush()
        fase = f"P{k}_OFF"
        leer(args.t_off, eco=False)
        print(f"{time.time() - t0:7.1f}s pulso {k}/{args.pulsos} completado", flush=True)
    print("FIN SESION", flush=True)
finally:
    s.write(b"0\n")
    s.flush()
    time.sleep(0.2)
    s.close()
    out.close()
```

Listado 12. *Prueba de provocación de conexiones: lectura y análisis (adquisicion/prueba_provocacion.py, líneas 1–149).*

```
#!/usr/bin/env python3
```

```
"""
MotorGuard-ITF: prueba de provocación de conexiones (protoboard).

Mide la estabilidad de los 6 canales con el motor girando y el SSR apagado:
  1) REFERENCIA: 40 s sin tocar nada.
  2) Por cada módulo y por los rieles de la protoboard: 15 s moviendo con suavidad
     y dando golpecitos a sus cables, seguidos de 6 s de reposo.
Al final compara cada tramo contra la referencia y emite un veredicto.

Uso (en una terminal junto al banco, motor ya girando y fuera del warmup):
  python3 ~/prueba_provocacion.py
  python3 ~/prueba_provocacion.py --analizar ~/provocacion_YYYYMMDD_HHMMSS.log   # re-analizar un
log
"""
import argparse
import os
import re
import sys
import time
from datetime import datetime
from statistics import median

PORT = "/dev/ttyACM0"
BAUD = 115200
T_REF = 40.0
T_PROV = 15.0
T_REPOSO = 6.0
TRAMOS = [
    ("IA", "Cables del sensor de corriente fase A (VCC, GND, OUT)"),
    ("IB", "Cables del sensor de corriente fase B"),
    ("IC", "Cables del sensor de corriente fase C"),
    ("VA", "Cables del sensor de voltaje fase A"),
    ("VB", "Cables del sensor de voltaje fase B"),
    ("VC", "Cables del sensor de voltaje fase C"),
    ("RIELES", "Rieles de 5 V, 3.3 V y GND de la protoboard y cables al Arduino"),
    ("GOLPES", "Golpecitos suaves sobre la mesa y la protoboard, sin tocar cables"),
]
CANALES = ["IA", "IB", "IC", "VA", "VB", "VC"]
# Umbral mínimo de desviación por tipo de canal (además de 5 veces la dispersión de la referencia)
PISO = {"I": 0.04, "V": 1.0, "R2": 0.50}

RX_RAW = re.compile(r"RAW: R2=(-?[\d.]+),(-?[\d.]+) I1=([\d.]+)")

def parse(line):
    if "[MCSA] Motor:ON" not in line:
        return None
    d = {}
    for k in CANALES:
        m = re.search(rf"\b{k}=([\d.]+)", line)
        if not m:
            return None
        d[k] = float(m.group(1))
    m = RX_RAW.search(line)
    if m and float(m.group(3)) > 0:
        i1 = float(m.group(3))
        d["R2"] = complex(float(m.group(1)), float(m.group(2))) / i1 * 100.0
    d["alarma"] = "| LEVE |" in line
    return d

def mad(xs):
    m = median(xs)
    return median([abs(x - m) for x in xs]) * 1.4826

def analizar(path):
    orden = []  # (tramo, datos) en el orden del log
    for l in open(path, encoding="utf-8", errors="replace"):
        m = re.match(r"\s*[\d.]+s \[(\w+)\] (.*)", l)
        if not m:
            continue
        d = parse(m.group(2))
```

```
        if d:
            orden.append((m.group(1), d))
    # Señal diferencial: cada canal menos el promedio de sus dos compañeros del mismo tipo.
    # Un cambio de red o de carga mueve las tres fases juntas y se cancela; un mal contacto no.
    for _, d in orden:
        for grupo in (CANALES[:3], CANALES[3:]):
            for k in grupo:
                otros = [d[j] for j in grupo if j != k]
                d["d" + k] = d[k] - sum(otros) / len(otros)
    ref = [d for t, d in orden if t == "REF"]
    if len(ref) < 10:
        print(f"[ERROR] Solo hay {len(ref)} tramas de referencia (se necesitan >= 10). ¿Motor
girando y fuera del warmup?")
        return 2
    # Ruido de referencia: dispersión robusta de los cambios bloque a bloque
    ruido = {}
    for k in CANALES:
        xs = [d["d" + k] for d in ref]
        ruido[k] = mad([b - a for a, b in zip(xs, xs[1:])])
    r2ref = [d["R2"] for d in ref if "R2" in d]
    ruido["R2"] = mad([abs(b - a) for a, b in zip(r2ref, r2ref[1:])]) if len(r2ref) > 2 else 0.0

    def umbral(k):
        tipo = "R2" if k == "R2" else ("I" if k.startswith("I") else "V")
        piso = {"I": 0.04, "V": 3.5, "R2": 0.6}[tipo]  # calibrados con logs reales del 01-oct-2026
        return max(5 * ruido[k], piso)

    print("\n" + "=" * 88)
    print("  PRUEBA DE PROVOCACIÓN")
    print("  Cada tramo se compara con los 6 bloques previos. Los canales se miden contra el
promedio")
    print("  de sus dos compañeros, así que los cambios de red o de carga no cuentan como saltos.")
    print("=" * 88)
    print(f"  {'Tramo':<8} {'n':>3}  " + "  ".join(f"{k:>7}" for k in CANALES) + f"  {'R2(pp)':>7}
Veredicto")
    fallas = []
    for nombre, _ in TRAMOS:
        idx = [i for i, (t, _) in enumerate(orden) if t in (nombre, nombre + "_R")]
        if not idx:
            print(f"  {nombre:<8} sin datos")
            continue
        previos = [orden[i][1] for i in range(max(0, idx[0] - 6), idx[0])]
        if len(previos) < 3:
            print(f"  {nombre:<8} sin bloques previos suficientes")
            continue
        ds = [orden[i][1] for i in idx]
        cols, malos = [], []
        for k in CANALES:
            base = median([d["d" + k] for d in previos])
            dev = max(abs(d["d" + k] - base) for d in ds)
            cols.append(dev)
            if dev > umbral(k) + 1e-9:
                malos.append(k)
        r2p = [d["R2"] for d in previos if "R2" in d]
        r2dev = float("nan")
        if r2p:
            c0 = complex(median([z.real for z in r2p]), median([z.imag for z in r2p]))
            r2dev = max(abs(d["R2"] - c0) for d in ds if "R2" in d)
            if r2dev > umbral("R2") + 1e-9:
                malos.append("R2")
        if malos:
            fallas.append((nombre, malos))
        ver = "OK" if not malos else "SALTO en " + ", ".join(malos)
        print(f"  {nombre:<8} {len(ds):>3}  " + "  ".join(f"{c:>7.3f}" for c in cols) + f"
{r2dev:>7.2f}  {ver}")
    alarmas = sum(d["alarma"] for _, d in orden if _ != "ESPERA")
    print("-" * 88)
    print("  Columnas: mayor cambio diferencial del canal en el tramo (A para corrientes, V para
voltajes).")
    print(f"  Umbrales: corrientes {max(umbral(k) for k in CANALES[:3]):.3f} A · voltajes
{max(umbral(k) for k in CANALES[3:]):.2f} V · residuo {umbral('R2'):.2f} pp")
    print(f"  Tramas en alarma del firmware durante la prueba: {alarmas}")
```

```
    print("=" * 88)
    if fallas:
        print("  RESULTADO: NO APROBADA. Revisa y fija estas conexiones antes de la sesión:")
        for n, ks in fallas:
            print(f"    · Al mover {n}: saltó {', '.join(ks)}")
        return 1
    print("  RESULTADO: APROBADA. Las conexiones son estables; puedes iniciar la sesión.")
    return 0
```

ANEXO C. MODELOS DE INTELIGENCIA ARTIFICIAL Y CLASIFICADOR DE SEVERIDAD

Funciones de la etapa exploratoria (Random Forest binario con validación cruzada agrupada y su exportador a lenguaje C) y del clasificador de severidad por umbrales M1 cuyos valores se incrustaron en el firmware.

Listado 13. *Carga de datos por bloques y validación cruzada agrupada (modelos/validacion_agrupada.py, líneas 65–195).*

```
def load_dataset(file_list, block_size=20):
    dfs = []
    eps = 1e-6
    for fn in file_list:
        p = os.path.join(DATASET_DIR, fn)
        if not os.path.exists(p):
            raise FileNotFoundError(f"No se encontro archivo de datos: {p}")
        d = pd.read_csv(p)
        if d.shape[1] == 15:
            d.columns = TRUE_COLS
        d['origen'] = fn
        d['r_55'] = d['A_55'] / (d['A_fund'] + eps)
        d['r_65'] = d['A_65'] / (d['A_fund'] + eps)
        d['r_50'] = d['A_50'] / (d['A_fund'] + eps)
        d['r_70'] = d['A_70'] / (d['A_fund'] + eps)

        # Generacion de grupo temporal: bloques contiguos de block_size muestras
        n_blocks = len(d) // block_size
        d['block_id'] = [f"{fn}_b{i // block_size}" for i in range(len(d))]
        dfs.append(d)

    df = pd.concat(dfs, ignore_index=True)
    df['clase'] = pd.to_numeric(df['clase'], errors='coerce').astype(int)
    for c in ['V_rms', 'I_rms', 'A_55', 'A_65', 'A_50', 'A_70', 'A_fund', 'I_neg', 'THD',
              'r_55', 'r_65', 'r_50', 'r_70']:
        df[c] = pd.to_numeric(df[c], errors='coerce')
    df = df.dropna(subset=['r_55', 'r_65', 'r_50', 'r_70', 'I_neg']).reset_index(drop=True)
    return df

def get_base_clf(seed=42):
    return RandomForestClassifier(
        n_estimators=45,
        max_depth=6,
        min_samples_split=4,
        min_samples_leaf=2,
        random_state=seed,
        n_jobs=-1
    )

def compute_ece(y_true, y_prob, n_bins=10):
    """Calcula el Expected Calibration Error (ECE)."""
    bin_limits = np.linspace(0.0, 1.0, n_bins + 1)
    ece = 0.0
    n = len(y_true)
    for i in range(n_bins):
        low, high = bin_limits[i], bin_limits[i + 1]
        mask = (y_prob >= low) & (y_prob < high if i < n_bins - 1 else y_prob <= high)
        bin_count = np.sum(mask)
        if bin_count > 0:
```

```
            bin_acc = np.mean(y_true[mask])
            bin_conf = np.mean(y_prob[mask])
            ece += (bin_count / n) * np.abs(bin_acc - bin_conf)
    return ece

def bootstrap_group_ci(df, group_col, metric_fn, B=2000, seed=42):
    """Intervalo de confianza al 95% por bootstrap de grupos."""
    rng = np.random.default_rng(seed)
    unique_groups = df[group_col].unique()
    n_groups = len(unique_groups)
    boot_vals = []

    for _ in range(B):
        sample_groups = rng.choice(unique_groups, size=n_groups, replace=True)
        # Tomar los registros correspondientes
        boot_df = pd.concat([df[df[group_col] == g] for g in sample_groups], ignore_index=True)
        try:
            val = metric_fn(boot_df)
            boot_vals.append(val)
        except Exception:
            continue

    if not boot_vals:
        return 0.0, 0.0
    low = np.percentile(boot_vals, 2.5)
    high = np.percentile(boot_vals, 97.5)
    return float(low), float(high)

def evaluate_cv(X, y, groups, cv_type='group', n_splits=5, seed=42):
    if cv_type == 'group':
        splitter = StratifiedGroupKFold(n_splits=n_splits)
        splits = list(splitter.split(X, y, groups=groups))
    else:
        splitter = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
        splits = list(splitter.split(X, y))

    acc_list, bal_acc_list, auc_list, f1_list = [], [], [], []
    all_y_true = []
    all_y_pred = []
    all_y_prob = []

    for train_idx, val_idx in splits:
        X_tr, y_tr = X.iloc[train_idx], y.iloc[train_idx]
        X_val, y_val = X.iloc[val_idx], y.iloc[val_idx]

        clf = get_base_clf(seed=seed)
        clf.fit(X_tr, y_tr)
        probs = clf.predict_proba(X_val)[:, 1]
        preds = (probs >= 0.50).astype(int)

        acc_list.append(accuracy_score(y_val, preds))
        bal_acc_list.append(balanced_accuracy_score(y_val, preds))
        try:
            auc_list.append(roc_auc_score(y_val, probs))
        except Exception:
            auc_list.append(0.5)
        f1_list.append(f1_score(y_val, preds, zero_division=0))

        all_y_true.extend(y_val.tolist())
        all_y_pred.extend(preds.tolist())
        all_y_prob.extend(probs.tolist())

    return {
        'accuracy_mean': float(np.mean(acc_list)),
        'accuracy_std': float(np.std(acc_list)),
        'bal_accuracy_mean': float(np.mean(bal_acc_list)),
        'bal_accuracy_std': float(np.std(bal_acc_list)),
        'auc_mean': float(np.mean(auc_list)),
        'auc_std': float(np.std(auc_list)),
```

```
        'f1_mean': float(np.mean(f1_list)),
        'f1_std': float(np.std(f1_list)),
        'confusion_matrix': confusion_matrix(all_y_true, all_y_pred).tolist(),
        'y_true': all_y_true,
        'y_pred': all_y_pred,
        'y_prob': all_y_prob
    }
```

Listado 14. *Exportación del Random Forest a tablas planas en C (modelos/export_rf_flat.py, líneas 18–172).*

```
def export_flat_trees():
    print(f"[EXPORTADOR] Cargando modelo: {MODEL_PATH}")
    rf = joblib.load(MODEL_PATH)
    n_trees = len(rf.estimators_)
    print(f"[EXPORTADOR] Número de estimadores: {n_trees}")

    all_nodes = []
    tree_offsets = []

    for t_idx, estimator in enumerate(rf.estimators_):
        tree = estimator.tree_
        offset = len(all_nodes)
        tree_offsets.append(offset)

        for i in range(tree.node_count):
            feat = tree.feature[i]
            if feat >= 0:
                left = offset + tree.children_left[i]
                right = offset + tree.children_right[i]
                thr = np.float32(tree.threshold[i])
            else:
                left = -1
                right = -1
                val = tree.value[i][0]
                thr = np.float32(val[1] / np.sum(val)) # Fracción de clase 1 (LEVE)
            all_nodes.append((feat, left, right, thr))

    total_nodes = len(all_nodes)
    print(f"[EXPORTADOR] Total de nodos exportados: {total_nodes}")
    bytes_nodes = total_nodes * 12 # 2 + 2 + 2 + 2(pad) + 4
    print(f"[EXPORTADOR] Tamaño estimado en Flash: {bytes_nodes} bytes ({bytes_nodes / 1024:.2f}
KB)")

    # 1. Generar modelo_rf_flat.h
    h_code = f"""#ifndef MODELO_RF_FLAT_H
#define MODELO_RF_FLAT_H

#include <stdint.h>

#ifdef __cplusplus
extern "C" {{
#endif

#define RF_FLAT_N_TREES {n_trees}
#define RF_FLAT_TOTAL_NODES {total_nodes}
#define RF_FLAT_N_FEATURES 5

// Nodo compacto de 12 bytes alineado en Flash (PROGMEM)
typedef struct {{
    int16_t feat;           // Feature index (0..4), o -1 para nodo hoja
    int16_t left;           // Índice absoluto hijo izquierdo (o -1 si hoja)
    int16_t right;          // Índice absoluto hijo derecho (o -1 si hoja)
    int16_t pad;            // Relleno de alineación a 32 bits
    float thr_or_value;     // Umbral de corte en nodo split, o P(LEVE) en hoja
}} RfNode;

extern const RfNode g_rf_flat_nodes[RF_FLAT_TOTAL_NODES];
```

```
extern const int16_t g_rf_flat_offsets[RF_FLAT_N_TREES];

// Inferencia iterativa en float puro (sin recursión ni emulación double)
float rf_flat_predict_proba(const float* features);
int rf_flat_predict_clase(const float* features, float threshold, float* out_proba);

#ifdef __cplusplus
}}
#endif

#endif // MODELO_RF_FLAT_H
"""
    with open(OUT_H, 'w') as f:
        f.write(h_code)
    print(f"[EXPORTADOR] Guardado header: {OUT_H}")

    # 2. Generar modelo_rf_flat.cpp
    cpp_lines = [
        '#include "modelo_rf_flat.h"',
        '',
        f'const int16_t g_rf_flat_offsets[RF_FLAT_N_TREES] = {{',
        '    ' + ', '.join(str(o) for o in tree_offsets),
        '};',
        '',
        f'const RfNode g_rf_flat_nodes[RF_FLAT_TOTAL_NODES] = {{'
    ]

    for feat, left, right, thr in all_nodes:
        # Formatear float asegurando sufijo 'f'
        thr_str = f"{thr:.9g}f" if '.' in f"{thr:.9g}" or 'e' in f"{thr:.9g}" else f"{thr:.1f}f"
        cpp_lines.append(f'    {{{feat}, {left}, {right}, 0, {thr_str}}},')

    cpp_lines.append('};')
    cpp_lines.append('')
    cpp_lines.append("""float rf_flat_predict_proba(const float* features) {
    if (!features) return 0.0f;
    float sum_p = 0.0f;
    for (int t = 0; t < RF_FLAT_N_TREES; t++) {
        int idx = g_rf_flat_offsets[t];
        while (g_rf_flat_nodes[idx].feat >= 0) {
            int feat_idx = g_rf_flat_nodes[idx].feat;
            if (features[feat_idx] <= g_rf_flat_nodes[idx].thr_or_value) {
                idx = g_rf_flat_nodes[idx].left;
            } else {
                idx = g_rf_flat_nodes[idx].right;
            }
        }
        sum_p += g_rf_flat_nodes[idx].thr_or_value;
    }
    return sum_p * (1.0f / (float)RF_FLAT_N_TREES);
}

int rf_flat_predict_clase(const float* features, float threshold, float* out_proba) {
    float p = rf_flat_predict_proba(features);
    if (out_proba) *out_proba = p;
    return (p >= threshold) ? 1 : 0;
}
""")

    with open(OUT_CPP, 'w') as f:
        f.write('\n'.join(cpp_lines))
    print(f"[EXPORTADOR] Guardado fuente: {OUT_CPP}")

    # 3. Validación de equivalencia con predict_proba en todo el dataset de entrenamiento
    files = [
        '/home/luna/tesis_itf/dataset/cap_ENTRELAZADA_LEVE_25%_20260914_1252.csv',
        '/home/luna/tesis_itf/dataset/cap_ENTRELAZADA_LEVE_50%_20260914_1438.csv',
        '/home/luna/tesis_itf/dataset/cap_ENTRELAZADA_LEVE_75%_20260915_1205.csv'
    ]
```

```
    dfs = [pd.read_csv(f) for f in files]
    df_all = pd.concat(dfs, ignore_index=True)
    X = np.column_stack([
        df_all['A_55'] / df_all['A_fund'],
        df_all['A_65'] / df_all['A_fund'],
        df_all['A_50'] / df_all['A_fund'],
        df_all['A_70'] / df_all['A_fund'],
        df_all['I_neg']
    ]).astype(np.float32)

    y_sk = rf.predict_proba(X)[:, 1]

    def predict_sim(x_row):
        s = 0.0
        for off in tree_offsets:
            cur = off
            while all_nodes[cur][0] >= 0:
                ft = all_nodes[cur][0]
                cur = all_nodes[cur][1] if x_row[ft] <= all_nodes[cur][3] else all_nodes[cur][2]
            s += float(all_nodes[cur][3])
        return s / n_trees

    y_flat = np.array([predict_sim(row) for row in X])
    diff = np.abs(y_sk - y_flat)
    max_d = np.max(diff)
    print(f"[EXPORTADOR] Verificación de equivalencia sobre {len(X)} muestras:")
    print(f"             Máxima diferencia absoluta: {max_d:.2e} (Criterio: < 1e-6)")
    assert max_d < 1e-6, f"Diferencia {max_d} supera 1e-6"
    print("[EXPORTADOR] Éxito: 100% de muestras verificadas con equivalencia estricta.")
```

Listado 15. *Clasificador de severidad M1 por dos umbrales sobre $|\Delta R_2|$ (modelos/entrenar_severidad.py, líneas 79–97).*

```
class Umbrales:
    """M1: dos umbrales sobre mag."""

    def fit(self, X, y):
        mag = X[:, 0]
        self.u = []
        for a, b in [(0, 1), (1, 2)]:
            lo, hi = mag[y == a], mag[y == b]
            if lo.max() < hi.min():
                self.u.append(0.5 * (lo.max() + hi.min()))
            else:
                cand = np.sort(np.concatenate([lo, hi]))
                acc = [((lo < t).sum() + (hi >= t).sum()) for t in cand]
                self.u.append(float(cand[int(np.argmax(acc))]))
        return self

    def predict(self, X):
        mag = X[:, 0]
        return (mag >= self.u[0]).astype(int) + (mag >= self.u[1]).astype(int)
```

ANEXO D. ANÁLISIS DE LAS SESIONES DE OCTUBRE

Scripts que procesan los registros del 1 de octubre de 2026 y producen la firma de la falla LEVE y la evaluación del detector direccional de escalón reportadas en el Capítulo 4.

Listado 16. *Análisis de una sesión larga: ruido de base y firma de los pulsos (analisis/analisis_sesion_larga.py, líneas 1–49).*

```
#!/usr/bin/env python3
"""Analisis offline de una sesion larga (base + pulsos) con fasores crudos RAW."""
import re, sys, numpy as np
f = sys.argv[1]
RX = re.compile(r'RAW: R2=(-?[\d.]+),(-?[\d.]+) I1=([\d.]+) V2=(-?[\d.]+),(-?[\d.]+) V1=([\d.]+)')
```

```
B = []
for l in open(f):
    if '[MCSA] Motor:ON' not in l: continue
    m = RX.search(l); fase = re.search(r'\[(BASE|P\d+_ON|P\d+_OFF)\]', l)
    if not m or not fase: continue
    g = lambda k: float(re.search(rf'\b{k}=(-?[\d\.]+)', l)[1])
    i1 = float(m[3]); v1 = float(m[6])
    B.append(dict(t=float(l.split('s ', 1)[0]), fase=fase[1],
                  r=complex(float(m[1]), float(m[2])) / i1 * 100, v2=complex(float(m[4]),
float(m[5])) / v1 * 100,
                  i1=i1, ia=g('IA'), ib=g('IB'), ic=g('IC'), va=g('VA'), e2=g('E2')))
t = np.array([b['t'] for b in B]); r = np.array([b['r'] for b in B]); v2 = np.array([b['v2'] for b
in B])
fase = np.array([b['fase'] for b in B]); base = fase == 'BASE'
print(f'Bloques: {len(B)} (BASE {base.sum()}, {(t[base][-1]-t[base][0])/60:.1f} min)')
rb = r[base]
print('\n== 1. Ruido del residuo en BASE ==')
print(f'r = R2/|I1|: media {rb.mean():.2f} pp (|r|={abs(rb.mean()):.2f}, ang
{np.degrees(np.angle(rb.mean())):.0f} deg)')
d1 = np.diff(rb)   # ruido rapido bloque a bloque
u = rb.mean() / abs(rb.mean())
rad = (d1 * np.conj(u)).real; tan = (d1 * np.conj(u)).imag
print(f'Cambio bloque a bloque: sd radial {rad.std():.3f}  sd tangencial {tan.std():.3f} pp
(rotacion de referencia => domina tangencial)')
for w in (1, 5, 35, 175):
    m = np.convolve(rb, np.ones(w)/w, 'valid'); print(f'  promedio {w:3d} bloques ({w*1.7:5.0f} s):
sd |desv| = {np.std(abs(m-m.mean())):.3f}  sd comp re {m.real.std():.3f} im {m.imag.std():.3f}')
X = np.array([[b['ia'], b['ib'], b['ic'], b['va']] for b in B])[base]
for i, n in enumerate(['IA', 'IB', 'IC', 'VA']):
    print(f'  corr(re r, {n})={np.corrcoef(rb.real, X[:,i])[0,1]:+.2f}  corr(im r,
{n})={np.corrcoef(rb.imag, X[:,i])[0,1]:+.2f}')
print(f'  corr(r, V2): re {np.corrcoef(rb.real, v2[base].real)[0,1]:+.2f} im {np.corrcoef(rb.imag,
v2[base].imag)[0,1]:+.2f}')

print('\n== 2. Firma de los pulsos (media >=3 s tras ON menos media 10 s previos) ==')
sig = []
for k in range(1, 20):
    on = np.where(fase == f'P{k}_ON')[0]
    if len(on) == 0: break
    t_on = t[on[0]] - 0.5
    pre = (t < t_on) & (t > t_on - 12) & ~np.char.endswith(fase.astype(str), '_ON')
    dur = on[t[on] - t_on >= 3.0]
    if len(dur) == 0 or pre.sum() == 0: continue
    s = r[dur].mean() - r[pre].mean(); sig.append(s)
    print(f'P{k}: |dr|={abs(s):.2f} pp  ang={np.degrees(np.angle(s)):6.1f} deg  (n_dur={len(dur)},
n_pre={pre.sum()})')
sig = np.array(sig)
if len(sig):
    th = np.angle(sig.mean())
    print(f'Media: |dr|={abs(sig.mean()):.2f} pp ang={np.degrees(th):.1f} deg | dispersion ang
±{np.degrees(np.std(np.angle(sig*np.exp(-1j*th)))):.1f} deg | |dr| sd {abs(sig).std():.2f}')
np.savez('/tmp/claude-1000/-home-luna/b6437c26-0f05-4c91-8ee8-1400b33559a8/scratchpad/sesion.npz',
t=t, r=r, v2=v2, fase=fase.astype(str), sig=sig)
```

Listado 17. *Detector direccional de escalón evaluado fuera de línea (analisis/detector_escalon_oct01.py, líneas 1–61).*

```
#!/usr/bin/env python3
"""Detector direccional de escalon (fuera de linea) sobre los logs del 01-oct-2026.
Reproduce el analisis de la sesion de trabajo del 01-oct (22:05): proyeccion del
residuo R2/|I1| sobre la direccion de la firma LEVE (160.6 deg), referencia de
A..G bloques atras, media de N bloques, umbral T = maximo en la base de 10 min
(sesion SWAP, 0 falsas alarmas por construccion), retorno en T/2.
Uso: python3 detector_escalon_oct01.py
"""
import re
import numpy as np

BASE = '/home/luna/sesion_SWAP_AC_sensores_50carga_20261001_154157.log'
PULS = '/home/luna/sesion_LEVE_postconexion_50carga_20261001_155559.log'
RX = re.compile(r'RAW: R2=(-?[\d.]+),(-?[\d.]+) I1=([\d.]+)')
```

```
TH = np.radians(160.6)
A, G, N = 20, 8, 2

def load(f):
    t, r, fa = [], [], []
    for l in open(f):
        if '[MCSA] Motor:ON' not in l:
            continue
        m = RX.search(l); ph = re.search(r'\[(BASE|P\d+_ON|P\d+_OFF)\]', l)
        if not m or not ph:
            continue
        t.append(float(l.split('s ', 1)[0]))
        r.append(complex(float(m[1]), float(m[2])) / float(m[3]) * 100)
        fa.append(ph[1])
    return np.array(t), (np.array(r) * np.exp(-1j * TH)).real, np.array(fa)

def run(x, T, Toff):
    al, ref, out = False, None, []
    for i in range(len(x)):
        if i < A:
            out.append(False); continue
        cur = x[i - N + 1:i + 1].mean()
        if not al:
            ref_i = x[i - A:i - G].mean()
            if cur - ref_i > T:
                al, ref = True, ref_i
        elif cur - ref < Toff:
            al = False
        out.append(al)
    return np.array(out)

_, xb, fb = load(BASE)
xb = xb[fb == 'BASE']
T = max(xb[i - N + 1:i + 1].mean() - xb[i - A:i - G].mean() for i in range(A, len(xb)))
t, x, fa = load(PULS)
al = run(x, T, T / 2)
on = [i for i in range(1, len(al)) if al[i] and not al[i - 1]]
det = sorted({int(re.search(r'\d+', fa[i])[0]) for i in on if fa[i].endswith('_ON')})
extra = [fa[i] for i in on if not fa[i].endswith('_ON')]
lat = [t[i] - t[np.where(fa == fa[i])[0][0]] + 0.5 for i in on if fa[i].endswith('_ON')]
print(f'Base sana: {len(xb)} bloques ({len(xb)*1.7/60:.1f} min) -> umbral T = {T:.2f} pp, ruido sd =
{xb.std():.3f} pp')
print(f'Pulsos detectados durante ON: {len(det)}/10 {det} | latencia media {np.mean(lat):.1f} s')
print(f'Alarmas fuera de pulso: {extra}')
```

ANEXO E. APLICACIÓN DE ESCRITORIO Y ASISTENTE RAG

Fragmentos de MotorGuard-ITF 1.0.0: el registro de eventos de alarma a partir de la telemetría y la búsqueda híbrida del asistente (coseno y BM25 combinados por fusión de rangos recíprocos, con límites por página y por documento).

Listado 18. *Registro de alarmas: apertura y cierre de eventos a partir de cada trama (app/motorguard_dashboard/telemetry/alarmas.py, líneas 84–121).*

```
    def procesar(self, obj) -> Optional[EventoAlarma]:
        """Procesa una línea parseada; devuelve la alarma que se abrió, actualizó o cerró (o
None)."""
        if isinstance(obj, EventoSSR):
            if obj.tipo != "solicitud":
                self.anotar(obj.t, "SSR", obj.texto)
            return None
        if not isinstance(obj, Trama):
            return None
```

```
        if self._ignorar_hasta is not None:
            t_obj = obj.t if obj.t is not None else time.time()
            if obj.dir.enclavada and obj.motor != "OFF" and t_obj < self._ignorar_hasta:
                self._hubo_trama = True
                return None                    # aún llegan tramas de antes de la 'r'
            self._ignorar_hasta = None
        ev = self._abierta
        if ev is not None and (not obj.dir.enclavada or obj.motor == "OFF"):
            motivo = "motor apagado" if obj.motor == "OFF" else "liberada por el firmware"
            return self._cerrar(obj.t, motivo)
        if ev is None and obj.dir.enclavada and obj.motor != "OFF":
            self._n_eventos += 1
            ev = EventoAlarma(
                id=self._n_eventos, t_inicio=obj.t, sev_inicial=obj.dir.sev,
                ml_inicial=obj.dir.ml, p_inicial=obj.dir.p, i1=obj.i1,
                ia=obj.ia, ib=obj.ib, ic=obj.ic, ya_activa=not self._hubo_trama,
            )
            self.eventos.append(ev)
            if len(self.eventos) > MAX_EVENTOS_MEMORIA:
                del self.eventos[:-MAX_EVENTOS_MEMORIA]
            self._abierta = ev
        self._hubo_trama = True
        if ev is not None:
            ev.n_tramas += 1
            ev.sev_max = max(ev.sev_max, obj.dir.sev)
            ev.ml_max = max(ev.ml_max, obj.dir.ml)
            ev.t_ultima = obj.t
        return ev
```

Listado 19. *Búsqueda híbrida del asistente RAG (app/motorguard_dashboard/rag/indice.py, líneas 195–254).*

```
    def bm25(self, consulta_tokens: Sequence[str], k1: float = 1.5, b: float = 0.75) -> np.ndarray:
        puntos = np.zeros(len(self.fragmentos))
        q = Counter(consulta_tokens)
        for i, tf in enumerate(self._tf):
            s = 0.0
            for w in q:
                f = tf.get(w)
                if f:
                    s += self._idf[w] * f * (k1 + 1) / (f + k1 * (1 - b + b * self._largo[i] /
self._largo_medio))
            puntos[i] = s
        return puntos


    def buscar(self, consulta: str, vec_consulta: np.ndarray, k: int = 5,
               docs: Optional[Sequence[str]] = None) -> List[Resultado]:
        """Los k mejores fragmentos (RRF de coseno y BM25), con tope por página y diversidad por
documento."""
        if not self.fragmentos:
            return []
        v = vec_consulta.astype(np.float32).ravel()
        v /= max(float(np.linalg.norm(v)), 1e-9)
        cos = self.emb @ v
        q_tk = ampliar_consulta(consulta)
        bm = self.bm25(q_tk)
        permitido = np.ones(len(self.fragmentos), bool)
        if docs is not None:
            permitido = np.array([f.doc in docs for f in self.fragmentos])
        docs_mencionados = {d for w in q_tk if w in AUTOR_DOC for d in AUTOR_DOC[w]}
        rrf: Dict[int, float] = {}
        for puntos in (cos, bm):
            orden = [i for i in np.argsort(-puntos) if permitido[i]]
            for r, i in enumerate(orden):
                if puntos is bm and bm[i] <= 0:
                    break
                rrf[int(i)] = rrf.get(int(i), 0.0) + 1.0 / (K_RRF + r + 1)
        if docs_mencionados:
            bono = 1.0 / (K_RRF + 5)
            for i in list(rrf):
                if self.fragmentos[i].doc in docs_mencionados:
                    rrf[i] += bono
```

```
        salida, por_pagina, por_doc, diferidos = [], Counter(), Counter(), []
        for i in sorted(rrf, key=rrf.get, reverse=True):
            f = self.fragmentos[i]
            if por_pagina[(f.doc, f.pag)] >= MAX_POR_PAGINA:
                continue
            if docs is None and f.doc not in docs_mencionados and por_doc[f.doc] >= MAX_POR_DOC:
                diferidos.append(i)
                continue
            por_pagina[(f.doc, f.pag)] += 1
            por_doc[f.doc] += 1
            salida.append(Resultado(f, rrf[i], float(cos[i]), float(bm[i])))
            if len(salida) == k:
                break
        for i in diferidos:
            if len(salida) == k:
                break
            f = self.fragmentos[i]
            if por_pagina[(f.doc, f.pag)] < MAX_POR_PAGINA:
                por_pagina[(f.doc, f.pag)] += 1
                salida.append(Resultado(f, rrf[i], float(cos[i]), float(bm[i])))
        return salida
```